BÜ-Reporting HSB4 K

Logik, Datenfluss und vollständiger Makro-Quellcode

Master-Datei Version 12.93 · Stand 26.10.2026

Zweck dieses Dokuments: Grundlage für den Nachbau des Cockpits in Celonis auf Basis der beiden Rohtabellen BUE OPAG und BUE Verbund. Teil A beschreibt fachliche Logik, Datenfluss und Sonderregeln, Teil B die Power-Query-Abfragen, Teil C enthält eine Übersicht aller Prozeduren und den vollständigen VBA-Quellcode der Master-Datei. Anhang D zeigt den Python-Spiegel der Ladelogik (gleiche Rechnung wie das VBA, kompakter lesbar).

Alle Zahlenbeispiele stammen aus den Abzügen vom 22. und 23.09.2026 (oe_nr = HSB4 K).

Inhalt

A. Fachliche Logik und Datenfluss **3**

A.1 Datenfluss im Überblick . . . 3

A.2 Rohtabellen und Spalten . . . 3

A.3 Schlüssel, OPAG und Verbund . . . 3

A.4 Kennzahlen . . . 4

A.5 Sparten, Filter und Standard-Auswahl . . . 4

A.6 Sonderregeln und versteckte Logik . . . 5

A.7 Blätter der Master-Datei . . . 5

B. Power Query **6**

C. VBA-Makros **7**

C.1 Prozeduren je Modul . . . 7

C.2 Quellcode . . . 26

modLaden.bas . . . 27

modCockpit.bas . . . 92

modTagesablauf.bas . . . 103

modWerkzeuge.bas . . . 114

modStandard.bas . . . 123

modPPT.bas . . . 126

modFolienauswahl.bas . . . 131

modFortschritt.bas . . . 134

Cockpit.cls . . . 138

Kalender.cls . . . 139

Rohdatenblatt.cls . . . 140

D. Anhang: Python-Spiegel der Ladelogik **141**

A. Fachliche Logik und Datenfluss

A.1 Datenfluss im Überblick

	Schritt	Was passiert
1	Abzug	Das Team speichert je Arbeitstag den Rückstandsabzug OPAG und den Rückstandsabzug Verbund aus MIS-O als Excel-Datei in den Ordner „Rohdaten“ neben der Master-Datei (Dateiname beliebig).
2	Erkennen	„Neue Daten laden“ öffnet jede neue Datei, erkennt je Blatt Stichtag (Zeile „Rückstandsstichtag = …“ in den ersten 15 Zeilen) und Quelle (Kopfzeile mit „Themenpostkorb“; mit „OPS_AUFTR_ID“ = OPAG, sonst Verbund).
3	Vergleichen	Die Schlüssel des Stands werden mit den Schlüsseln des zuletzt geladenen Stands derselben Quelle verglichen (Blatt Snapshot_OPAG bzw. Snapshot_Verbund). Daraus entstehen Eingänge und Abgänge.
4	Verdichten	Je Stichtag, Quelle, Sparte, Verwaltungssystemschlüssel, Themenpostkorb und GeVo entsteht eine Zeile im Blatt „Fakten“ mit Rückstand, Altersklassen, Eingängen, Abgängen und dem höchsten Rückstandsalter.
5	Rechnen	Cockpit, Verlauf, Wochenwerte, Rohdatenblatt und PPT rechnen über Formeln aus „Fakten“ (gefiltert nach den Chips Themenpostkorb, GeVo, Sparte); die Makros setzen nur Filterzellen und Diagrammbereiche.
6	Ablegen	Mit Dateipfad (lokal, Netzlaufwerk, OneDrive-Verknüpfung) benennt das Makro die Datei einheitlich und legt sie in Rohdaten\JJJJ\MM_Monat ab (z. B. 20260923_BUe_OPAG.xlsx). Auf SharePoint ohne Pfad bleibt sie liegen.
7	Celonis	Jede geladene Rohdatei wird zusätzlich unter dem einheitlichen Namen in den Ordner aus Konfiguration B6 kopiert (Standard H:\Process-Mining-Sync\Upload); Celonis_Upload_Log.txt verhindert doppelte Übergaben.
8	Anwesenheit	Für die Produktivität liest „Anwesenheit laden“ die neueste Datei im Ordner „Anwesenheit“ (MAK-Liste je KW oder Telefonie- und Abwesenheitsplan) in die Blätter Mitarbeiter und Kalender.

A.2 Rohtabellen und Spalten

Beide Abzüge haben oben einige Kopfzeilen (Stichtag, Filter oe_nr, Altersgrenzen), dann die Spaltenköpfe, darunter eine Zeile je Datensatz und am Ende eine Summenzeile („Anzahl: 13.831“), die übersprungen wird. Spaltenreihenfolge egal.

Spalte	Quelle	Bedeutung und Verwendung
`Verwaltungssystemschlüssel`	beide	Verwaltungssystem (KV, LV, SV, KR …). Bestimmt die Sparte (Tabelle A.5).
`Ordnungsbegriff`	beide	Vertrags-/Ordnungsnummer im Verwaltungssystem.
`VORG_VERBU_LNR`	beide	Laufende Nummer des Vorgangs im Vorgangsverbund. Mit VSS und Ordnungsbegriff der Vorgangsschlüssel.
`oe_nr / OE_NR`	beide	Organisationseinheit (hier „HSB4 K“). Im HS-Reporting nur Filter des Abzugs; im Makler-Reporting bestimmt sie die Einheit der Zeile (OE-Ebene 4).
`Themenpostkorb`	beide	Postkorb, in dem der Vorgang liegt (z. B. „KO 23724“); Leerzeichen werden zusammengefasst, leer = „(ohne Postkorb)“.
`Rückstandsalter_Tage`	beide	Alter in Tagen am Stichtag (Abschnitt A.4).
`GEVO_SL`	beide	Geschäftsvorfall „Nummer Text“ (z. B. „108 …“); die Nummer ist der GeVo-Typ für die Filter.
`OPS_AUFTR_ID`	nur OPAG	Auftrags-ID im OPAG-System. Ein Vorgang kann mehrere Aufträge haben.
`OPS_ARBGUT_LNR`	nur OPAG	Laufende Nummer des Arbeitsguts innerhalb des Auftrags (1, 2, 3 …). Eine OPAG-Zeile = ein Arbeitsgut.
`Anzahl`	beide	Immer 1 je Zeile (wird nicht gebraucht).

Die Bedeutung von OPS_AUFTR_ID, OPS_ARBGUT_LNR und OE_NR ist aus Namen und Daten abgeleitet, wie das Reporting sie verwendet; eine fachliche Feldbeschreibung aus MIS-O liegt nicht vor.

A.3 Schlüssel, OPAG und Verbund

- **Verbund** zählt **Vorgänge**: Schlüssel = Verwaltungssystemschlüssel + Ordnungsbegriff + VORG_VERBU_LNR. Im Abzug ist jeder Schlüssel eindeutig.
- **OPAG** zählt **Arbeitsgüter**: Schlüssel = derselbe Vorgangsschlüssel + OPS_AUFTR_ID + OPS_ARBGUT_LNR.

- Verknüpfung: Über den Vorgangsschlüssel (VSS + Ordnungsbegriff + VORG_VERBU_LNR) passen beide Tabellen zusammen. Am 23.09.2026: 13.831 Verbund-Vorgänge, 29.525 OPAG-Arbeitsgüter zu 13.808 Vorgängen (Ø 2,14 Arbeitsgüter je Vorgang); 13.796 Vorgänge in beiden Tabellen, 12 nur in OPAG, 35 nur in Verbund.
- Das Reporting verknüpft die beiden Tabellen nicht zeilenweise, sondern rechnet jede Quelle für sich und stellt sie nebeneinander (Kennzahl „Verhältnis Verbund zu OPAG“ = OPAG-Arbeitsgüter je Verbund-Vorgang).

A.4 Kennzahlen

Kennzahl	Definition
Rückstand	Anzahl Zeilen des Abzugs am Stichtag in der gewählten Auswahl (OPAG Arbeitsgüter, Verbund Vorgänge).
Eingänge	Schlüssel, die heute in der Auswahl sind und im zuletzt geladenen Stand derselben Quelle nicht in der Auswahl waren: neu im Abzug oder aus einem anderen Postkorb/GeVo hereingewechselt. Zugeordnet nach den Merkmalen von heute.
Abgänge	Schlüssel, die im vorigen Stand in der Auswahl waren und heute nicht mehr: erledigt, an die Sparte weitergeleitet, an den 72-Stunden-BOT übergeben oder in einen Postkorb/GeVo außerhalb der Auswahl gewechselt. Zugeordnet nach den Merkmalen des Vortags.
Wechsel in der Auswahl	Weder Eingang noch Abgang (z. B. MV nach KO, beide gewählt). Dadurch gilt für jede Auswahl: Rückstand heute = Rückstand voriger Stand + Eingänge − Abgänge. Wechsel stehen im Blatt Umbuchungen.
Erster Stand	Der erste Stand je Quelle liefert nur den Rückstand; Eingänge und Abgänge gibt es ab dem zweiten.
Lücken	Wochenende oder fehlender Tag: Vergleich immer gegen den letzten geladenen Stand, die Bewegungen zählen zum neuen Stichtag.
Rückstandsalter	Spalte Rückstandsalter_Tage unverändert aus dem Abzug. Altersklassen 0–7, 8–30, 31–90, über 90 Tage; „Ältester Vorgang“ = Stichtag minus höchstes Alter in der Auswahl.
Wochenwert	Rückstand einer KW = Stand vom Sonntag der KW (Montag gehört zur neuen KW); fehlt der Sonntag, der letzte Stand der KW (mit Stern). Eingänge/Abgänge der KW = Summe der Bewegungen der Stände dieser KW.
SLA	Verbund: 5.500 Vorgänge (Konfiguration F3). OPAG: Wert in F4, leer = Vorschlag SLA Verbund × Ø-Verhältnis OPAG/Verbund der letzten 14 Tage.
Produktivität	Abgänge je MAK (nicht Teil des Celonis-Nachbaus); MAK aus Mitarbeiter/Kalender.

Rückstandsalter in den Daten (22. → 23.09.2026)

- Alle Schlüssel, die an beiden Tagen im Abzug stehen, haben am 23.09. genau einen Tag mehr: 13.327 von 13.327 (Verbund), 28.605 von 28.605 (OPAG). Ein Zurücksetzen während der Bearbeitung kommt in diesem Vergleich nicht vor.
- Neue Schlüssel am 23.09.: Verbund 205 mit Alter 0, daneben viele mit Alter größer 0 (1, 2, 13, 16, 19 …); OPAG 446 mit Alter 0. Alter 0 heißt also „am Stichtag entstanden“; ein Eingang im Sinne des Reportings kann aber älter sein (z. B. in den Abzug hineingewechselt). Eingänge deshalb immer über den Schlüsselvergleich bestimmen, nicht über Alter = 0.

Die Aussage stützt sich auf zwei aufeinanderfolgende Tage; über längere Zeiträume ist das Verhalten nicht geprüft.

A.5 Sparten, Filter und Standard-Auswahl

Verwaltungssystemschlüssel	Sparte
KV	KV
LV, LF, LG	Leben
SV, KR, DA, HA, AS, UV, GV, TR, IT	Komposit
alle übrigen	Sonstige

Zuordnung in Konfiguration A11:B40 (änderbar).

- **Themenpostkörbe**: Standard = KO, MV (ohne 72-Stunden-BOT, BÜ, GR und AZ; BE, WR, 00 fachlich offen). Festgelegt in Konfiguration H:L bzw. Knopf „Standard festlegen“. Das Rohdatenblatt (Wochenreport für das HS-Dashboard) rechnet immer mit dem Standard.
- **GeVo**: „Alle GeVo“ (kein Filter), „Nur BÜ-Schlüssel“ (GeVo 108, 469 Komposit/Leben; 2015, 6065 KV; Konfiguration D11:D20) oder manuelle Auswahl der 13 häufigsten GeVo-Typen plus „Alle übrigen“.
- **Sparten-Chips**: Komposit, Leben, KV, Sonstige grenzen alle Kennzahlen ein.

- Eingänge und Abgänge hängen von der Auswahl ab (Wechsel über die Auswahlgrenze zählen). Für Celonis heißt das: Bewegungen je Auswahl aus den Schlüsseln beider Stände neu bestimmen, nicht aus Summen ableiten.

A.6 Sonderregeln und versteckte Logik

- Reihenfolge: Stände werden nach Stichtag sortiert verarbeitet; ein Stand, der älter ist als der zuletzt geladene seiner Quelle, wird übersprungen.
- Ein Stand (Quelle + Stichtag) wird nur einmal geladen. Kommt ein neuer Abzug für einen schon geladenen Tag, fragt das Tool: übernehmen ersetzt den Stand und rechnet Eingänge/Abgänge dieses und aller späteren Stände neu (alte Datei …_ersetzt, verworfene …_verworfen).
- Sammeldateien: mehrere Blätter (z. B. „OPAG 22.09.“, „Verbund 22.09.“) werden je Blatt erkannt.
- Neue Themenpostkörbe anderer Bereiche (Postkorb nicht auf 23724 endend): Karte nach dem Laden fragt, welche nicht betrachtet werden.
- Ausgeblendete Blätter: Snapshot_OPAG / Snapshot_Verbund (Schlüssel des letzten Stands, Grundlage des Vergleichs), Wirksam (MAK je Person und KW), SP_Liste (Power-Query-Ergebnisse, im Tool ausgeblendete SharePoint-Dateien), Celonis_Log, Diagrammdaten (7 Perioden je Diagramm).
- Fakten: eine Zeile je Stichtag, Quelle, Sparte, VSS, Postkorb, GeVo mit Rückstand, Altersklassen, Eingängen, Abgängen, höchstem Alter und Schlüssel des ältesten Datensatzes. Cockpit, Verlauf und Wochenwerte rechnen über Formeln darauf.
- Rohdatenblatt: Zeilen 67–90 (7VA–7OL) = Rückstand zum letzten Stichtag der KW, Zu-/Abgänge der KW, immer Standard-Auswahl, „Gesamt“ einschließlich Sonstige; Zeile 160/161 = Datum des verwendeten Stands. Daraus liest das HR-Sales-Dashboard per Verknüpfung.
- Manuelle Schritte: Abzüge aus MIS-O ziehen und ablegen, Anwesenheit pflegen (Kalender), ggf. Standard und SLA festlegen.
- Bereich MAKLER (zweite Master-Datei): mehrere Einheiten je Tag, Vergleich je Einheitengruppe, Einheit aus OE_NR (Konfiguration Q11:R40). Zeilen der HS-Einheit (OE HSB4 K → HSB) zählen nicht; reine HS-Abzüge werden nicht geladen und heißen danach …_HS-ignoriert (12.91).
- Jeder Stand (Quelle, Stichtag, Einheit) wird nur einmal geladen. Als geladen gilt eine Zeile im Ladeprotokoll mit leerem Hinweis, „erster Stand …“ oder der Angabe zum Verkettungskennzeichen (StandGueltig, 12.92). Bis 12.91 las „Neue Daten laden“ mit OneDrive-Verknüpfung den letzten Stand jeder Quelle beim nächsten Laden noch einmal ein (Rückstand doppelt); das erste Laden mit 12.92 entfernt diese Doppel aus Fakten und Ladeprotokoll (DoppelteStaendeBereinigen).
- Neue Version ohne Neuladen (12.93, modUpdate): „Ältere Datei aktualisieren …“ übernimmt aus der zusätzlich geöffneten älteren Datei Fakten, Verlauf, Ladeprotokoll, Umbuchungen, Snapshots, Dateiinhalt, Mitarbeiter, Kalender, Konfiguration und Filter und speichert die neue Version unter Name und Ort der alten (Sicherung vorher in Protokolle\Sicherung).

A.7 Blätter der Master-Datei

Blatt	Inhalt
Cockpit	Kennzahlen, Chips, Diagramme
PPT	Kacheln für die Folie
Verlauf	eine Zeile je Stichtag (gefiltert)
KW	Abgänge, MAK, Produktivität je KW
Wochenwerte	Nachschlagetabelle je KW (Schlüssel Jahr*100+KW)
Rohdatenblatt	Nachbau des Wochenreports (Zeilen 1–157), BÜ-Zeilen automatisch
Fakten	verdichtete Sätze (siehe A.6)
Umbuchungen	Wechsel von Postkorb/GeVo/Sparte zwischen zwei Ständen
Ladeprotokoll	je Stand Datei, Blatt, Quelle, Stichtag, Zeilen, Bewegungen
Filter	Zellen der Chips und Hilfsgrößen
Konfiguration	Pfade, Sparten, BÜ-Schlüssel, Standard, SLA
Mitarbeiter, Kalender	Anwesenheit und Arbeitszeitmodell
Anleitung	Bedienung und Kennzahlen (Grundlage dieses Teils A)

B. Power Query

Power Query wird nur benutzt, wenn die Master-Datei aus SharePoint ohne Dateipfad geöffnet ist („In Desktop-App öffnen“). Mit Dateipfad (lokal, Netzlaufwerk, OneDrive-Verknüpfung) liest VBA die Ordner direkt; dann gibt es keine Abfragen. Die Abfragen werden zur Laufzeit von VBA erzeugt und in der Datei gespeichert (ThisWorkbook.Queries); ihr M-Code entsteht in den Funktionen unten (vollständig in Teil C, modLaden).

VBA-Funktion	Abfrage	Status
SpFormel	Auflistung eines SharePoint-Ordners samt Ablage JJJJ\MM_Monat: Spalten Rel (relativer Pfad), Geaendert, Groesse. Navigiert mit SharePoint.Contents, ersatzweise SharePoint.Files.	aktiv
SpTextFormel	Zeilen einer Textdatei im Ordner (Celonis_Upload_Log.txt, Ladelog) als Tabelle; prüft, dass der Ordner der Master-Datei gemeint ist.	aktiv
SpAblageFormel	Umbenennen/Verschieben über die SharePoint-Schnittstelle (_api, POST). Power Query sendet POST nur anonym; deshalb seit 12.81 abgeschaltet.	nicht aktiv
SpTabelle	Schreibt/aktualisiert die Abfrage, lädt sie in das Blatt SP_Liste und gibt die Werte an VBA zurück.	aktiv

Beispiel: erzeugter M-Code der Ordnerauflistung (SpFormel)

```
let
    Seite = "https://<host>/sites/<Seite>",
    Teile = {"<Bibliothek>", "<Ordner>", "Rohdaten"},
    Ordner = "https://<host>/sites/<seite>/<bibliothek>/<ordner>/rohdaten/",
    Groesse = (a) => try a[Size] otherwise null,
    IstOrdner = (x) => Value.Is(x, type table),
    Ebene = (t as table, rel as text, tiefe as number) as table =>
        let
            Dateien = Table.SelectRows(t, each not IstOrdner([Content])),
            Eigene = Table.FromColumns({List.Transform(Dateien[Name], each rel & _), Dateien[Date modified],
                                        List.Transform(Dateien[Attributes], Groesse)}, {"Rel", "Geaendert", "Groesse"}),
            Unter = Table.SelectRows(t, each IstOrdner([Content]) and (if tiefe = 2
                        then Text.Length([Name]) = 4 and Text.Length(Text.Select([Name], {"0".."9"})) = 4
                        else Text.Length([Name]) > 3 and Text.Length(Text.Select(Text.Start([Name], 2), {"0".."9"})) = 2
                             and Text.Middle([Name], 2, 1) = "_")),
            Tiefer = if tiefe <= 0 then {} else List.Transform(Table.ToRecords(Unter),
                        each @Ebene([Content], rel & [Name] & "\", tiefe - 1))
        in
            Table.Combine({Eigene} & Tiefer),
    Navigiert = try List.Accumulate(Teile, SharePoint.Contents(Seite, [ApiVersion = 15]),
                    (s, n) => s{[Name = n]}[Content]) otherwise null,
    Ersatz = () =>
        let
            Alle = SharePoint.Files(Seite, [ApiVersion = 15]),
            Treffer = Table.SelectRows(Alle, each Text.StartsWith(Text.Lower([Folder Path]), Ordner)),
            Rel = List.Transform(Table.ToRecords(Treffer),
                    each Text.Replace(Text.Middle([Folder Path], Text.Length(Ordner)), "/", "\") & [Name])
        in
            Table.FromColumns({Rel, Treffer[Date modified], List.Transform(Treffer[Attributes], Groesse)},
                              {"Rel", "Geaendert", "Groesse"}),
    Ergebnis = if Navigiert = null then Ersatz() else Ebene(Navigiert, "", 2)
in
    Ergebnis
```

C. VBA-Makros

Die Master-Datei enthält 11 Module mit zusammen 9448 Zeilen. Einstieg für das Laden ist `modLaden.NeueDatenLaden` (Knopf „Neue Daten laden“ über `modTagesablauf.LadenStarten`). Alle Karten und Dialoge sind Formen auf dem Blatt (keine UserForms). Die Rechnung selbst steckt in modLaden; die übrigen Module steuern Oberfläche, Filter und Folie.

Modul	Zeilen	Aufgabe
`modLaden.bas`	5605	Laden der Rohdaten (OPAG, Verbund), Erkennung, Vergleich, Fakten, Ablage, Celonis-Übergabe, Anwesenheit, SharePoint/Power Query
`modCockpit.bas`	959	Knöpfe und Chips des Cockpits, Filter, Ansicht (Tag/Aktuelle KW/Woche), Diagrammbereiche
`modTagesablauf.bas`	927	Geführter Tagesablauf „Neue Daten laden“ (drei Karten)
`modWerkzeuge.bas`	701	Werkzeug-Karten: SLA anpassen, Rohdaten löschen, Zurücksetzen, Ältester Vorgang
`modStandard.bas`	240	Karte „Standard festlegen“ (Themenpostkörbe, GeVo-Typen)
`modPPT.bas`	407	PowerPoint-Folie aus dem Blatt „PPT“
`modFolienauswahl.bas`	229	Karte „Welche Kacheln auf die Folie?“
`modFortschritt.bas`	323	Fortschritts-Overlay und Hinweiskarten
`Cockpit.cls`	26	Blattmodul Cockpit (Ereignisse)
`Kalender.cls`	11	Blattmodul Kalender (Ereignisse)
`Rohdatenblatt.cls`	20	Blattmodul Rohdatenblatt (Sprung zur aktuellen Woche)

C.1 Prozeduren je Modul

modLaden.bas

modLaden – Rohdaten (OPAG- und Verbund-Rueckstandsabzuege) laden, Anwesenheitsliste importieren Ablauf "Neue Daten laden": 1. Ordner "Rohdaten" (relativ zur Master-Datei, Konfiguration!B3) nach Excel-Dateien durchsuchen 2. In jeder Datei jedes Blatt pruefen: Stichtag aus "Rückstandsstichtag = ...", Quelle aus den Spaltenkoepfen (OPS_AUFTR_ID vorhanden = OPAG, sonst Verbund) 3. Alle gefundenen Staende nach Stichtag und Quelle sortieren, bereits geladene ueberspringen 4. Je Stand: Rueckstand je Sparte/VSS/Themenpostkorb/GeVo mit Altersklassen zaehlen, Eingaenge (Schluessel neu gegenueber dem letzten Stand) und Abgaenge (Schluessel weg) bestimmen, Zeilen an "Fakten" anhaengen, Snapshot der Schluessel ersetzen, Zeile in "Verlauf" ergaenzen 5. Ladeprotokoll fortschreiben, neu berechnen Liegt im Ordner Rohdaten ein neuer Abzug fuer einen schon geladenen Stand, bleibt die Datei liegen und eine Karte fragt (modTagesablauf.ErsatzFrageZeigen): Ersetzen loescht den alten Stand und rechnet ihn aus der neuen Datei neu – Eingaenge und Abgaenge gegen den Stand davor (aus dessen Datei gelesen), danach alle spaeteren Staende dieser Quelle aus ihren Dateien. Die alte Datei heisst danach ..._ersetzt, eine nicht uebernommene neue ..._verworfen; beide werden nicht mehr gelesen. Erkannte Dateien werden als JJJJMMTT_BUe_<OPAG|Verbund> in die Ablage Rohdaten\JJJJ\MM_Monat verschoben (Sammeldatei: JJJJMMTT-JJJJMMTT, beide Quellen: OPAG_Verbund); scheitert das Umbenennen, bleibt der Name und das Log vermerkt es. Fremde Dateien werden ohne Makros, ohne Dialoge und schreibgeschuetzt geoeffnet (QuelleO

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
`LadeModusEin`	Sub	Private	147	Lademodus (Sperre, Zustand sichern und zurueckgeben)
`LadeModusAus`	Sub	Private	158	
`LadenLaeuft`	Function	Public	170	
`LogSchreiben`	Sub	Public	174	
`NeueDatenLaden`	Sub	Public	188	Einstieg
`StaendeText`	Function	Private	722	"1 Stand" / "3 Stände" – Zaehlwort mit korrektem Singular und Plural
`ZielDateiname`	Function	Private	734	Einheitliche Dateinamen und Ablage Ein Stand: 2026\09_September\20260923_BUe_OPAG.xlsx; Sammeldatei: 2026\09_September\20260921-20260923_BUe_OPAG_Verbund.xlsx; mit Einheit (MAKLER): 20260923_BUe_Verbund_ODPB-D.xlsx (relativer Pfad unterhalb des Rohdaten-Ordners; der Monatsordner richtet sich nach dem ersten Stichtag der Datei)
`MonatsOrdner`	Function	Private	757	
`SchonAbgelegt`	Function	Private	764	
`OrdnerAnlegen`	Sub	Private	776	
`DateienSammeln`	Function	Private	787	

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
DateienChronologisch	Sub	Private	848	Dateien aelteste zuerst: Datum am Anfang des Namens (JJJJMMTT_... wie in der Ablage und in den MIS-O-Abzuegen), sonst das Aenderungsdatum (SharePoint-Auflistung bzw. Dateisystem); gleiches Datum nach Name (OPAG vor Verbund). Nur fuer die Reihenfolge von Schritt 1 – verarbeitet wird ohnehin nach dem Stichtag im Blatt.
DatumAusName	Function	Private	868	" · 03.10.2026", wenn der Dateiname mit JJJJMMTT beginnt (sonst leer) – fuer die Statuszeile von Schritt 1
DateiSortSchluessel	Function	Private	876	
DateienEinesOrdners	Function	Private	897	
AbgelegtUndGeladen	Function	Private	911	
FreierDateiname	Function	Private	928	Ist der Name schon vergeben (zweiter Abzug zum selben Stand), wird _2, _3 ... angehaengt
IstExcelDatei	Function	Private	942	Fremde Datei sicher oeffnen
StandAusName	Function	Private	951	Stand(e) aus einem einheitlichen Dateinamen (siehe NameZerlegen). Nur Einzeldateien; Sammeldateien (JJJJMMTT-JJJJMMTT_...) und Staende, die schon geladen sind (dann greift die Ersatz-Logik mit dem Inhalt der Datei), werden geoeffnet, im Bereich MAKLER jede Datei (die Gruppe ergibt sich aus dem Inhalt, siehe unten).
QuelleOeffnenRel	Function	Private	972	Rohdatei oeffnen. SharePoint: beim ersten Mal ueber https und sofort als lokale Kopie in %TEMP% sichern; weitere Zugriffe in diesem Laden (Verarbeiten, Celonis) nehmen die Kopie – eine Datei wird so nur einmal ueber das Netz geholt.
CacheVon	Function	Private	983	
CacheAblegen	Sub	Private	991	
CacheAufraeumen	Sub	Private	1010	lokale Kopien am Ende des Ladens loeschen
QuelleOeffnen	Function	Private	1023	
BlattErkennen	Function	Private	1061	Blatt erkennen straenge (optional): Einheit -> Zeilen. HS: eine leere Einheit; MAKLER mit Spalte OE_NR: jede Einheit aus Konfiguration!R11:R40, Zeilen anderer OEs zaehlen zur Haupteinheit des Blatts (HauptAusZaehlung); ohne Spalte OE_NR: die Gruppe aus der Kopfzeile.
StichtagAusText	Function	Public	1113	
StandDatenLesen	Function	Private	1150	Stand verarbeiten Blatt eines Stands oeffnen und die Datenzeilen unter der Kopfzeile lesen. sp(1..8) = Spalten VSS, Ordnungsbegriff, VORG_VERBU_LNR, Themenpostkorb, Rueckstandsalter, GEVO_SL, OPS_AUFTR_ID, OPS_ARBGUT_LNR, sp(9) = OE_NR, sp(10) = Verkettungskennzeichen (0 = fehlt). Ergebnis: Anzahl Zeilen. Fehlt das Blatt unter s...
LesenVergessen	Sub	Private	1238	
DatensatzSchluessel	Function	Private	1245	Schluessel eines Datensatzes (Verbund: VSS + Ordnungsbegriff + LNR; OPAG zusaetzlich Auftrag und Arbeitsgut)
Feld	Function	Private	1253	
StandVerarbeiten	Sub	Private	1257	
StaendeVerarbeiten	Sub	Private	1267	Staende einer Quelle und eines Stichtags gemeinsam verarbeiten: b(0..m-1), je Einheit (Strang) einer, aus einer oder mehreren Dateien. Jede Einheit wird mit ihrem eigenen Vortag (Snapshot) verglichen. Wechselt ein Vorgang zwischen zwei Einheiten dieses Stichtags, ist das eine Umbuchung (Blatt Umbuchungen), kein Abgang und Eingan...
StapelText	Function	Private	1488	Einheiten eines Stapels fuer Statuszeile und Log ("MSUE+ODPB"), leer im Bereich HS
UmbuchungenSchreiben	Sub	Private	1498	Umbuchungen eines Stands an das Blatt Umbuchungen anhaengen: je Paar (vorher, heute) eine Zeile mit Anzahl; die Formelspalten K..P (Filterkennzeichen vorher/heute) werden aus Zeile 2 fortgeschrieben – Spalte fuer Spalte wie in Fakten; Q Einheit heute, R Gruppe, S Einheit vorher.

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
UmbuchungenEntfernen	Sub	Private	1533	Umbuchungen einer Quelle (und Gruppe, Spalte R) ab einem Stichtag entfernen (beim Ersetzen eines Stands), Rest rueckt auf
QuelleBewegungenWaehlen	Sub	Private	1565	Kachel „Eingänge und Abgänge“: Standard ist Verbund. Liegt der letzte Verbund-Stand mehr als 2 Tage hinter dem letzten OPAG-Stand (Verbund liefert laenger nichts), zeigt die Kachel nach dem Laden OPAG. Die Chips oben in der Kachel schalten danach wie gewohnt um.
StandErsetzen	Function	Private	1582	Stand ersetzen Ersetzt einen geladenen Stand durch die neue Datei st. Vorab (ohne etwas zu aendern) wird geprueft und gelesen: der Stand davor (Vergleichsbasis fuer Eingaenge/Abgaenge, aus seiner Datei) und die Dateien aller spaeteren Staende dieser Quelle. Dann: Fakten der Quelle ab dem Stichtag entfernen, alte Protokollzeilen ...
GeladeneStaende	Function	Private	1670	Gueltig geladene Staende einer Quelle (und Einheit) aus dem Ladeprotokoll (Hinweis leer oder "erster Stand ..."), nach Stichtag sortiert
GeladenLesen	Function	Private	1699	Geladene Staende (Quelle|Stichtag) mit Zeitpunkt, Datei, Datensaetzen und Rueckstand des Ladens – fuer Ersatz-Frage und Ersetzen
StandDatei	Function	Private	1728	Rohdatei eines geladenen Stands finden: Eintrag im Protokoll, sonst der Standardname in der Monatsablage oder im Ordner selbst
FaktenEntfernen	Sub	Private	1752	Fakten-Zeilen einer Quelle (und Gruppe, Spalte S) ab einem Stichtag entfernen: der Rest rueckt auf, die frei gewordenen Zeilen am Ende werden geleert (Werte und Formelspalten). Keine Zeilen loeschen – die Namen F_* bleiben so unveraendert.
ProtokollErsetztMarkieren	Sub	Private	1787	Gueltige Protokollzeilen einer Quelle ab einem Stichtag als ersetzt markieren (zaehlen danach nicht mehr als geladen)
AlteDateiAusmustern	Sub	Private	1803	Alte Rohdatei nach dem Ersetzen umbenennen (..._ersetzt), damit sie nach einem Zuruecksetzen nicht wieder geladen wird
DateiTraegtGueltigenStand	Function	Private	1837	Steht die Datei im Ladeprotokoll noch bei einem gueltigen Stand (Sammeldatei mit weiteren Stichtagen)?
NeuerAbzugZuGeladen	Function	Private	1853	Datei ist ein anderer Abzug zu einem geladenen Stand: liegt direkt im Ordner Rohdaten oder ist ein Doppel (..._2) in der Ablage, und das Ladeprotokoll nennt fuer diesen Stand eine andere Datei
DoppelZuGeladen	Function	Private	1862	Doppel in der Ablage (JJJJMMTT_BUe_<Quelle>_n) zu einem Stand, der aus einer anderen Datei geladen ist – muss geoeffnet werden
IstDoppel	Function	Private	1874	
DateiName	Function	Private	1881	
AusgemustertName	Function	Private	1886	ersetzte oder nicht uebernommene Rohdateien und im Bereich MAKLER ignorierte HS-Abzuege (12.91) werden beim Laden nicht mehr gelesen
ErsatzDaten	Function	Private	1895	Angaben fuer die Ersatz-Frage: 0 Quelle, 1 Stichtag, 2 geladen am (leer, wenn unbekannt), 3 Datensaetze geladen, 4 Rueckstand geladen, 5 Name der neuen Datei, 6 Zeilen der neuen Datei (rund: Blattende minus Kopfzeile), 7 Einheit (MAKLER)
ZahlDE	Function	Public	1902	ganze Zahl mit Punkt als Tausendertrenner, unabhaengig von der Spracheinstellung von Windows
CelonisEinmal	Sub	Private	1917	Celonis (Process Mining): Kopie jeder Datei, aus der geladen wird Ziel: Konfiguration!B6 (z. B. H:\Process-Mining-Sync\Upload), leer = aus. Dateiname wie in der Ablage, ohne Doppel-Endung _2, damit ein ersetzter Stand die alte Kopie ueberschreibt. Fehler halten das Laden nicht auf, sie stehen in Ladelog.txt und in der Abschlussm...
CelonisOffenMerken	Sub	Private	1945	Gescheiterte Uebergaben (Celonis-Ordner nicht erreichbar, Kopie fehlgeschlagen) im Blatt Celonis_Log, Spalten F:I (Rohdatei relativ, einheitlicher Name, seit, „ja“ = Ersatz: ueberschreibt die alte Kopie). Jedes Laden versucht sie vor der Verarbeitung erneut (CelonisOffenNachholen) – die Rohdatei wird ja nicht noch einmal geladen...

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
CelonisOffenErledigt	Sub	Private	1965	
CelonisOffenUmbenennen	Sub	Private	1976	Rohdatei umbenannt und abgelegt (NeueDatenLaden, Schritt 1): eine offene Uebergabe – etwa eines Kollegen im SharePoint-Modus, der die Datei unter ihrem MIS-O-Namen vermerkt hat – auf den neuen Pfad umschreiben, damit CelonisOffenNachholen sie wiederfindet
CelonisOffenSuchen	Function	Private	1991	Rohdatei einer offenen Uebergabe, die nicht mehr unter ihrem Pfad liegt (ein Kollege hat sie umbenannt und abgelegt): unter dem einheitlichen Namen im Monatsordner suchen (JJJJ\MM_Monat aus dem Datum am Namensanfang, wie ZielDateiname), auch die Doppel _2 ... _9; gibt es mehrere, das zuletzt abgelegte (hoechstes _n). Ergebnis: rel...
CelonisAliasAblegen	Sub	Private	2014	Beim Ablegen (lokaler Lauf): Wurde die Datei unter ihrem bisherigen Namen schon an Celonis uebergeben (SharePoint-Modus bis 12.64), den einheitlichen Namen sofort dauerhaft als Alias ins Protokoll schreiben. Sonst kennt nur dieser Lauf die Zuordnung, und nach einem Neuaufbau oder mit einer neuen Master-Datei ginge die Datei erne...
CelonisOffenZeile	Function	Private	2032	
CelonisOffenNachholen	Function	Private	2045	Offene Uebergaben erneut versuchen; Ergebnis: Anzahl nachgeholter Dateien (nur wirklich kopierte). Liegt die Rohdatei nicht mehr unter dem vermerkten Pfad, wird sie unter dem einheitlichen Namen gesucht (CelonisOffenSuchen); ist sie auch dort nicht, steht das in der Abschlussmeldung und die Zeile entfaellt (sonst bliebe sie stil...
CelonisBlatt	Function	Private	2084	Blatt Celonis_Log (ausgeblendet): A:D Protokoll der Uebergaben, F:I offene Uebergaben; wird bei Bedarf angelegt
CelonisFehlerMerken	Sub	Private	2101	
CelonisSenden	Function	Private	2106	
DateiGroesse	Function	Private	2180	
UrlKopieren	Function	Private	2196	Datei von SharePoint als Kopie in einen lokalen Ordner sichern (FileCopy geht mit https nicht); Ergebnis leer = ok
CelonisLogPfade	Function	Private	2215	Celonis-Protokoll an drei Stellen, damit es einen Wechsel der Master-Datei (neue Version, Neuaufbau, SharePoint) uebersteht: Celonis_Upload_Log.txt im Ordner der Master-Datei (lokal) bzw. in %LOCALAPPDATA%\BUE_Reporting, dort immer zusaetzlich, und das ausgeblendete Blatt Celonis_Log in der Master-Datei. Gelesen wird die Vereini...
CelonisAb	Function	Private	2227	Konfiguration!F8: Staende vor diesem Stichtag gehen nicht an Celonis (0 = keine Grenze)
CelonisLogLesen	Function	Private	2235	Protokoll lesen: Dateiname (klein) -> Groesse der zuletzt uebergebenen Fassung. Zeilen: Zeitpunkt TAB Datei TAB Bytes TAB Herkunft
LogText	Function	Private	2275	Feld des Protokolls: Tabulatoren und Anfuehrungszeichen ersetzen (Speichern als Text wuerde das Feld sonst in "..." setzen)
LogSchluessel	Function	Private	2280	Dateiname aus einer Protokollzeile (Feld 2), klein, ohne umschliessende Anfuehrungszeichen; leer, wenn die Zeile keins hat
TextFeld	Function	Private	2288	Feld einer als Text gespeicherten Zeile: ohne umschliessende Anfuehrungszeichen, "" -> " (so speichert Excel Felder mit Komma)
LogGroesse	Function	Private	2296	Groesse aus einer Protokollzeile (Feld 3)
CelonisSpLesen	Sub	Private	2304	Celonis_Upload_Log.txt neben der Master-Datei auf SharePoint lesen (celonisSp bleibt Nothing, wenn das nicht gelingt – dann wird dort auch nichts zurueckgeschrieben, damit fremde Zeilen nicht verloren gehen)

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
CelonisSpZurueckschreiben	Function	Private	2328	Neue Zeilen an Celonis_Upload_Log.txt auf SharePoint anhaengen: Datei aus gelesenen und neuen Zeilen neu aufbauen und ueber Excel als Text (Tabulator, Windows-1252 – wie Print #) unter derselben Adresse speichern. Ergebnis leer = ok, sonst Fehlertext.
CelonisLogSchreiben	Sub	Private	2405	
NeuePostkoerbeAnzahl	Function	Public	2425	Neue Themenpostkoerbe ausserhalb HS (Karte modStandard nach dem Laden)
IstNeuerPostkorb	Function	Public	2429	
NeuePostkoerbeText	Function	Public	2433	
NeuePostkoerbeErledigt	Sub	Public	2443	
ErsatzFrageOffen	Function	Public	2448	Ersatz-Frage (Karte in modTagesablauf)
ErsatzAnzahl	Function	Public	2453	
ErsatzWert	Function	Public	2458	Angabe k (siehe ErsatzDaten) des i-ten offenen Stands (1 = erster)
ErsatzFrageText	Function	Public	2467	Klartext fuer das Ersatzfenster ohne Formen (MsgBox)
ErsatzAusAblauf	Function	Public	2477	
ErsatzEntscheiden	Sub	Public	2482	Antwort auf die Frage: True = neue Datei ersetzt den Stand, False = alter Stand bleibt; danach laedt der Aufrufer erneut
ErsatzAbbrechen	Sub	Public	2492	
Nz	Function	Private	2498	Hilfsfunktionen Verarbeitung
Sauber	Function	Private	2502	
Verdichten	Function	Public	2507	
PostkorbNorm	Function	Public	2514	
GevoTyp	Sub	Public	2521	
AlterKlasse	Function	Private	2543	
SpartenZuordnung	Function	Private	2557	
RohdatenPfad	Function	Public	2571	Ordner
RohdatenOrdner	Function	Private	2581	Rohdaten-Ordner. Ist er nur als SharePoint-Adresse erreichbar (Master-Datei aus dem Browser in der App geoeffnet), wird zuerst ein lokaler Ordner dazu gesucht (OneDrive-Verknuepfung oder Synchronisierung, auch wenn die Verknuepfung nur auf „Rohdaten“ oder auf die ganze Bibliothek „Dokumente“ gesetzt ist) – dann kann das Makro um...
LokalFuerOrdner	Function	Private	2605	Lokaler Ordner zu einer SharePoint-Ordneradresse: Zuordnung des OneDrive-Clients (Registry), sonst Suche unter den OneDrive Ordnern und ihren Unterordnern (eine Verknuepfung heisst wie der Ordner, auf den sie zeigt: „Rohdaten“, „BÜ - HS“, „Dokumente“ ...) und unter den Bibliotheksordnern im Benutzerprofil. Jedes Ende der Adresse w...
SyncPfad	Function	Private	2633	Registry des OneDrive-Clients: laengste passende Bibliotheksadresse -> lokaler Ordner (ohne Pruefung)
SuchWurzeln	Function	Private	2660	Orte, unter denen eine OneDrive-Verknuepfung oder synchronisierte Bibliothek liegen kann
PasstZurListe	Function	Private	2682	Liegen im lokalen Ordner dieselben Dateien wie laut SharePoint-Auflistung? (mindestens drei bzw. alle, wenn es weniger sind)
OrdnerAufloesen	Function	Private	2698	
IstUrl	Function	Public	2723	
PfadVon	Function	Private	2728	Ordner + relativer Pfad (Trenner \); bei einer https-Adresse mit /
RohdatenAufSharePoint	Function	Public	2737	Liegen die Rohdaten auf SharePoint ohne Dateipfad (Laden ueber https)? Fuer Karten und Knoepfe.
ProtokollPfad	Function	Private	2743	Textdatei des Tools im Ordner Protokolle neben der Master-Datei (lokal bzw. synchronisiert): Ordner anlegen, eine Datei vom bisherigen Ort (neben der Master-Datei) beim ersten Zugriff dorthin umziehen. Geht das Anlegen nicht: bisheriger Ort.

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
LokalerProtokollPfad	Function	Private	2758	Ladelog.txt: Protokolle neben der lokal erreichbaren Master-Datei, sonst %LOCALAPPDATA%\BUE_Reporting
SpTextSpeichern	Function	Private	2768	SharePoint ohne Dateipfad: Mappe als Text (xlTextWindows) in <Master-Ordner>/Protokolle speichern; fehlt der Ordner (Excel kann ueber https keinen anlegen), neben die Master-Datei wie bisher. Ergebnis: Adresse; ein Fehler am bisherigen Ort geht weiter.
LokalerOrdner	Function	Private	2790	Ordner fuer Logs, wenn die Master-Datei nur ueber https erreichbar ist: %LOCALAPPDATA%\BUE_Reporting (sonst %TEMP%)
AppDatenOrdner	Function	Private	2799	%LOCALAPPDATA%\BUE_Reporting (sonst %TEMP%\BUE_Reporting) – bleibt bei einem Wechsel der Master-Datei erhalten
BasisOrdner	Function	Public	2809	
SyncZuordnung	Function	Private	2859	Synchronisierte SharePoint-/OneDrive-Bibliothek: HKCU\Software\SyncEngines\Providers\OneDrive\<id> mit UrlNamespace (Adresse der Bibliothek) und MountPoint (lokaler Ordner). Die laengste passende Adresse gewinnt.
WebDavPfad	Function	Private	2891	SharePoint als WebDAV-Pfad: https://host/sites/... -> \\host@SSL\DavWWWRoot\sites\... Nur wenn der Windows-Dienst WebClient laeuft – sonst blockiert schon die Pruefung des Pfads (Windows sucht ihn erst als Netzwerkfreigabe). Microsoft hat WebClient im November 2023 abgekuendigt, er startet nicht mehr von selbst; deshalb kein Startve...
WebClientLaeuft	Function	Private	2902	
SharePointOrdnerUrl	Function	Private	2915	Eintrag in der Konfiguration als SharePoint-Adresse (Adresszeile des Browsers) -> Ordneradresse https://host/sites/.../Ordner Erkennt .../Forms/AllItems.aspx?id=/sites/...&viewid=..., Freigabe-Adressen /:f:/r/... und die direkte Ordneradresse.
WebDavVorschlag	Function	Public	2939	
ProfilSuche	Function	Private	2952	OneDrive-Ordner (Umgebungsvariablen) und Bibliotheksordner unter dem Benutzerprofil (z. B. C:\Users\x\ERGO\PV Dashboard - Dokumente): das Ende der Adresse (ab beliebigem Teil) darunter suchen, auch eine und zwei Ebenen tiefer
UnterOrdner	Function	Private	2984	
UrlDekodieren	Function	Private	2998	
OrdnerVorhanden	Function	Private	3041	
DateiVorhanden	Function	Private	3048	
SpListe	Function	Private	3061	SharePoint ohne Dateipfad (https) Excel kann einen SharePoint-Ordner nicht mit Dir() lesen, Power Query aber schon – mit der Anmeldung des Office-Kontos, ohne OneDrive und ohne WebDAV. SpListe schreibt die Abfrage BUE_SharePoint_Liste fuer den Ordner (SharePoint.Contents, Ordner fuer Ordner; findet sie den Ordner so nicht, Share...
SpTabelle	Function	Private	3089	Power-Query-Abfrage schreiben, in das Blatt SP_Liste ab zelle laden und als Werte (1-basiert, 2D) liefern; Empty = keine Zeilen. Fehler (Anmeldung abgebrochen, Adresse falsch) werden an den Aufrufer weitergegeben.
SpSchluessel	Function	Private	3121	
SpEintrag	Function	Private	3129	Eintrag einer Datei (https-Adresse) aus der Auflistung ihres Ordners: Array(rel, geaendert, groesse), sonst Empty
SpBlatt	Function	Private	3141	
SpFormel	Function	Private	3159	Power-Query-Formel fuer einen Ordner https://host/sites/<Seite>/<Bibliothek>/<Ordner>...: Dateien des Ordners und der Ablage JJJJ\MM_Monat (Spalten Rel, Geaendert, Groesse)
SpKopf	Function	Private	3192	Gemeinsamer Anfang der Abfragen: Seite (https://host/sites/<Seite>), Teile (Bibliothek und Ordner) und Ordner (Adresse klein, mit /)

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
SpTextFormel	Function	Private	3218	Abfrage: Zeilen einer Textdatei im SharePoint-Ordner (Windows-1252 wie Print #), keine Datei = keine Zeilen. Ein Abruffehler (Netz, Drosselung, Anmeldung) wird NICHT als „Datei fehlt“ verschluckt, sondern laesst die Abfrage scheitern – sonst wuerde das Protokoll beim Zurueckschreiben nur mit den neuen Zeilen ueberschrieben. unte…
MText	Function	Private	3250	
SpHinweis	Function	Private	3255	Meldung, wenn der SharePoint-Ordner nicht aufgelistet werden konnte
SpAusblenden	Sub	Private	3267	SharePoint: Datei im Tool ausblenden (geloescht oder nicht uebernommen) – wird beim Laden und in der Liste uebergangen Ausgeblendet wird eine bestimmte Datei: SP_Liste!J Name, K Groesse (seit 12.89). MIS-O vergibt jeden Tag dieselben Namen („Rueckstand_Verbund.xlsx“) – nur der Name traf sonst auch den neuen Abzug des naechsten T…
SpAusgeblendet	Function	Private	3281	groesse < 0: unbekannt. Ein Eintrag mit Groesse gilt nur fuer eine Datei derselben Groesse; ein Eintrag ohne Groesse (vor 12.89) nur fuer Namen, die das Tool vergeben hat (JJJJMMTT_BUe_…, je Stichtag eindeutig), nicht fuer MIS-O-Namen.
SpZerlegen	Function	Private	3311	Seite (https://host/sites/X) und server-relativer Pfad (/sites/X/Bibliothek/…) einer https-Adresse, ohne / am Ende
SpAblageFormel	Function	Private	3331	Abfrage fuer Schritte auf SharePoint, je Schritt Array(Art, A, B) mit server-relativen Pfaden: „ordner“ legt A an (gibt es ihn schon, antwortet SharePoint trotzdem mit 200), „verschieben“ verschiebt die Datei A nach B (ohne Ueberschreiben; erst MoveToUsingPath, sonst das aeltere moveto). Jeder Schritt wartet auf das Ergebnis des…
SpAblageAusfuehren	Function	Private	3377	Schritte ausfuehren (Blatt SP_Liste ab T1); Ergebnis leer = alle Schritte mit Status 2xx, sonst der erste Fehler
SpAblagePruefen	Sub	Private	3398	Einmal je Laden: geht die Ablage auf SharePoint? (Ordner Rohdaten „anlegen“ – gibt es ihn, aendert sich nichts)
SpVerschieben	Function	Private	3413	Datei rel (relativ zum Ordner, Trenner \) nach neu verschieben, Ordner davor anlegen. Ergebnis leer = verschoben (laut frischer Auflistung: neu da, rel weg)
ProtokollDateiUmschreiben	Function	Private	3449	Ladeprotokoll Spalte B: alle Zeilen mit der Datei alt auf neu umschreiben (nach dem Ablegen einer schon geladenen Datei, damit Nachrechnen, „Rohdatei öffnen“ und die Ersatz-Logik die Datei am neuen Ort finden). Liefert die Zahl der Zeilen.
CacheUmziehen	Sub	Private	3468	Lokale Kopie (Cache) einer verschobenen Datei auch unter der neuen Adresse finden
DateiUmbenennen	Function	Private	3475	Rohdatei umbenennen bzw. in die Ablage verschieben (rel und neu relativ zum Ordner); Ergebnis leer = gelungen
SpUnsortiert	Function	Private	3487	Rohdateien, die noch direkt im Ordner Rohdaten liegen (nicht in JJJJ\MM_Monat, nicht ausgeblendet)
SyncAnbieten	Sub	Private	3503	Einmal je Sitzung anbieten, den Ordner mit OneDrive zu synchronisieren (nicht im geführten Tagesablauf: dort steht es in der Meldung)
OrdnerSynchronisieren	Sub	Public	3518	Ordner einbinden (SharePoint-Modus) Power Query darf auf SharePoint nur lesen (POST mit Anmeldung lehnt es ab, Test am 09.10.2026). Verlaesslich ist die Verknuepfung „Verknüpfung zu OneDrive hinzufügen“ beim Ordner der Master-Datei: BasisOrdner findet den Ordner danach unter dem OneDrive-Ordner (ProfilSuche), auch wenn die Datei…
DateiSchonGeladen	Function	Private	3539	SharePoint: Wurde aus dieser Datei schon ein Stand geladen, ersetzt oder uebersprungen? (jede Zeile im Ladeprotokoll) Seit 12.89 bei MIS-O-Namen (nicht JJJJMMTT_BUe_…) nur, wenn auch die Groesse stimmt (Blatt Dateiinhalt): MIS-O vergibt jeden Tag dieselben Namen, ein neuer Abzug unter altem Namen (auf SharePoint ersetzt) wurde s…
ZahlOder	Function	Private	3555	
ZeitWert	Function	Private	3564	Zeitwert aus der Auflistung (Datum als Zahl oder Text) fuer den Vergleich

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
SnapshotBlatt	Function	Private	3576	Snapshot (Schluessel des letzten Stands je Quelle und Einheit) Blatt Snapshot_<Quelle>: A..E Schluessel und Merkmale, F "Gruppe|Einheit" (HS: "|"). Stichtag und Anzahl: H2/H3 fuer die leere Gruppe, J2:L41 (Gruppe, Stichtag, Schluessel) fuer jede Gruppe im Bereich MAKLER. Parameter einheit = Gruppe.
SnapshotLesen	Function	Private	3580	
SnapshotSchreiben	Sub	Private	3600	Schluessel einer Einheit ersetzen (die anderen Einheiten bleiben stehen); stichtag 0 = Einheit ohne Stand (leeren)
SnapshotErsetzen	Sub	Private	3608	Schluessel mehrerer Einheiten in einem Schreibvorgang ersetzen: je = Einheit -> Dictionary der heutigen Schluessel
SnapGruppe	Function	Private	3652	Spalte F des Snapshots: "Gruppe|Einheit" (aeltere Zeilen ohne "|": nur Gruppe)
SnapEinheit	Function	Private	3658	
StandVermerken	Sub	Private	3664	
VerlaufGrenzenNeu	Sub	Public	3695	Verlauf!AL/AM (erste/letzte Position der Fakten-Zeilen je Stichtag) aus den Fakten neu bestimmen – nach dem Ersetzen, wenn Zeilen aufgerueckt sind und ein Stichtag nicht mehr am Stueck steht
LetzterStand	Function	Private	3733	Stichtag des zuletzt geladenen Stands einer Quelle und Einheit (0 = noch keiner)
LetzterStandQuelle	Function	Private	3747	juengster Stand einer Quelle ueber alle Einheiten
VerlaufErgaenzen	Sub	Private	3760	Verlauf, Filterliste, Protokoll
PostkoerbeRegistrieren	Sub	Private	3803	
StandGueltig	Function	Public	3847	Hinweis einer Ladeprotokoll-Zeile: gilt der Stand als geladen? Ja bei leerem Hinweis (bis 12.68), „erster Stand …“ und der Verkettungs-Angabe, die seit 12.69 jeder normal geladene Stand traegt („Verkettungskennzeichen: …“, „ohne Spalte Verkettungskennzeichen: …“, ggf. mit „· … Zeilen der HS-Einheit … ignoriert“). Nein bei „übers...
DoppelteStaendeBereinigen	Function	Private	3862	Doppelt geladene Staende (12.92) Bis 12.91 galt ein Stand nur mit leerem Hinweis oder „erster Stand …“ als geladen, seit 12.69 traegt aber jeder normal geladene Stand die Verkettungs-Angabe. Mit Dateipfad (OneDrive-Verknuepfung, Dateien in der Ablage) las „Neue Daten laden“ deshalb den zuletzt geladenen Stand jeder Quelle beim n...
DoppeltMeldung	Function	Private	3971	Zusatz fuer die Abschlussmeldung (einmal, danach geleert)
ProtokollLesen	Function	Private	3978	
Protokoll	Sub	Private	4006	
InhaltBlatt	Function	Private	4021	Dateiinhalt (12.88) Blatt Dateiinhalt: je geprueftem Blatt einer Rohdatei und je Einheit eine Zeile – A Datei, B Kennung (Groesse in Byte, seit 12.89; SharePoint und OneDrive liefern dieselbe), C Blatt, D Quelle, E Stichtag, F Einheit (Strang), G Zeilen, H Einheiten der Datei, I geprueft am. Zum Nachschauen, was in welcher Datei...
InhaltFingerabdruck	Function	Private	4026	
DateiKennung	Function	Private	4040	
InhaltLaden	Sub	Private	4057	Index aus dem Blatt: "datei|kennung" (klein) -> Collection von Array(Blatt, Quelle, Stichtag, Einheit, Zeilen, Einheiten der Datei)
InhaltBekannt	Function	Private	4082	Blaetter und Einheiten einer schon geprueften Datei (gleiche Kennung) aus dem Index; False = unbekannt, Datei oeffnen
InhaltMerken	Sub	Private	4112	Ergebnis der Pruefung einer geoeffneten Datei vermerken (je Blatt und Einheit eine Zeile)
InhaltUmbenennen	Sub	Private	4144	Datei wurde umbenannt bzw. abgelegt: Eintraege folgen ihr (die Kennung bleibt, die Groesse aendert sich nicht)
AlteGruppenStraenge	Function	Private	4160	Bereich MAKLER: stammen die geladenen Staende noch aus der Zeit vor 12.88 (Strang = Gruppe mehrerer Einheiten, „MSUE+ODPB“)?

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
IstMakler	Function	Public	4177	Bereich und Einheiten Konfiguration!F7: HS (ein Strang je Quelle) oder MAKLER (je Quelle ein Strang je Einheit, siehe Kopf des Moduls)
IstHsEinheit	Function	Public	4185	Bereich MAKLER (12.91): gehoert eine Einheit (EinheitAusOE) zum HS-Reporting (OE HSB4 K -> HSB)? Deren Zeilen ignoriert der Makler-Master. Nur in Wegen des Bereichs MAKLER gefragt (je Datenzeile, deshalb ohne IstMakler).
HsDatenGeladen	Function	Private	4191	Bereich MAKLER: HS-Einheit, deren Zeilen schon in den Fakten stehen (vor 12.91 mitgeladene HS-Abzuege), sonst leer
HsAbzugAusmustern	Sub	Private	4204	Bereich MAKLER: reiner HS-Abzug im Ordner Rohdaten oder in der Ablage – nicht laden, nicht an Celonis; die Datei bleibt, wo sie liegt, und heisst danach ..._HS-ignoriert (wird nicht mehr gelesen). SharePoint ohne Dateipfad: im Tool ausgeblendet.
HsMeldung	Function	Private	4227	Hinweis fuer die Abschlussmeldung: ignorierte HS-Abzuege und HS-Zeilen dieses Ladens (leer, wenn keine)
HsText	Function	Private	4250	Zusatz fuer das Ladeprotokoll eines Strangs
Strang	Function	Public	4255	Anzeige- und Sortiername eines Strangs: "Verbund" bzw. "Verbund · ODPB D"
StandSchluessel	Function	Private	4260	Schluessel eines Stands: Quelle[|Einheit]|Stichtag (ohne Einheit wie bisher Quelle|Stichtag)
NamensSchluessel	Function	Private	4269	Schluessel aus dem abgelegten Dateinamen: "@Quelle[_Einheitteil]|Stichtag"
NamensSchluesselDatei	Function	Private	4275	Namensschluessel aus dem Dateinamen einer Protokollzeile (seit 12.88 traegt der Name die Einheiten der Datei, die Zeile nur ihre Einheit): passt der Stichtag im Namen zur Zeile, gilt die abgelegte Datei damit als geladen. Sonst leer.
EinheitDateiteil	Function	Private	4283	Einheit als Teil eines Dateinamens: Leerzeichen und Unterstriche werden zu "-", unzulaessige Zeichen fallen weg
NameZerlegen	Function	Private	4298	Abgelegter Name JJJJMMTT_BUe_<OPAG|Verbund|OPAG_Verbund>[_Einheit][_n]: Stichtag, Quellen und Einheit-Teil. Sammeldateien (JJJJMMTT-JJJJMMTT_...) nennen nicht jeden Stichtag: False (werden geoeffnet).
GruppeErkennen	Function	Private	4329	Gruppe eines Abzugs (Bereich MAKLER): die Einheiten aller Zeilen (Spalte OE_NR, siehe EinheitAusOE), sortiert und mit "+" verbunden, z. B. "MSUE+ODPB+ODPP". Ohne Spalte OE_NR: die Kopfzeile "oe_nr = '...'" bzw. "oe_nr in (...)", sonst leer. zaehl: Zeilen je Einheit der Gruppe, rest: Zeilen anderer OEs, mitOE: Spalte OE_NR vorhanden ...
GruppenEinheit	Function	Private	4385	Zaehlt eine Einheit fuer die Gruppe? Nur die in Konfiguration!R11:R40 genannten Einheiten (ODPB, ODPP, MSUE, KBI ...), damit einzelne Zeilen anderer OEs (z. B. „ODP D“ selbst) die Gruppe nicht von Tag zu Tag aendern. Leere Tabelle: jede Einheit. Die Liste wird je Laden einmal gelesen (gruppenListe; wird fuer jede Datenzeile gefrag...
HauptAusZaehlung	Function	Private	4404	Haupteinheit eines Blatts: die Einheit mit den meisten Zeilen (gleich viele: alphabetisch die erste); leer, wenn keine
HauptStrang	Function	Private	4414	dieselbe Regel auf den gelesenen Datenzeilen (Spalte OE_NR, ohne Fusszeilen „Anzahl ...“)
ZeileStrang	Function	Private	4430	Strang einer Datenzeile (Bereich MAKLER, Spalte OE_NR): ihre Einheit, wenn sie in Konfiguration!R11:R40 steht, sonst die Haupteinheit des Blatts. Ohne Spalte OE_NR entscheidet der Aufrufer (ein Strang je Blatt).
NichtFuehrend	Function	Private	4441	Verkettungskennzeichen „verkettet, nicht führend“ -> Zeile zaehlt nicht (wie im MIS-O-Bericht). Tolerant geschrieben: Gross/Klein, Bindestrich/Unterstrich, ohne Umlaut („nicht fuehrend“, „nicht-führend“, „Verkettet_nicht_fuehrend“)
VerkettungHinweis	Function	Private	4454	Hinweis fuer das Ladeprotokoll: ob der Abzug die Spalte Verkettungskennzeichen hat und wie viele verkettete, nicht fuehrende Vorgaenge ausgelassen wurden. Ohne Spalte zaehlen sie mit – MIS-O-Berichte mit Verkettungsfilter zeigen dann weniger.

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
VerkettungText	Function	Private	4458	
SortiertVerbunden	Function	Private	4468	
ZeilenEinheit	Function	Private	4481	Einheit einer Datenzeile (Bereich MAKLER, Spalte OE_NR), sonst leer
EinheitAusOE	Function	Public	4488	OE_NR -> Einheit (OE-Ebene 4): laengster passender Anfang aus Konfiguration!Q11:Q40 -> R; sonst die fuehrenden Buchstaben ohne angehaengtes "HH" (ODPB1 D -> ODPB, ODPBHH -> ODPB, MSUE2 M -> MSUE). Ergebnisse werden je Laden zwischengespeichert.
EinheitenNamenSetzen	Sub	Public	4518	Namen der Einheiten fuer die Sicht "Einheit" der Rueckstandsdiagramme (Filter!L22:L25): bis zu vier Einheiten alphabetisch, bei mehr als vier die ersten drei und "Übrige"
EinheitChipsSetzen	Sub	Private	4534	Einheiten-Chips im Cockpit (Bereich Makler, 12.86): Filter!X5:X10 die ersten sechs Einheiten der Fakten (sortiert), Y = im Filter. Bekannte Einheiten behalten ihren Zustand, neue kommen mit WAHR; leere Zeilen stehen auf WAHR (ohne Chip, ohne Wirkung). Ab der siebten Einheit gibt es keinen Chip – solche Einheiten zaehlen immer (n...
EinheitNorm	Function	Private	4560	"'ODPB D'" / "('ODPB D', 'ODPP D')" -> "ODPB D" / "ODPB D+ODPP D"
EinheitenListe	Function	Public	4572	Einheiten in den geladenen Fakten (Spalte R), sortiert – fuer die Einheiten-Auswahl (modWerkzeuge.EinheitZeigen, Dateien vor 12.70)
ToolLeeren	Sub	Public	4600	Tool leeren (Knopf "Zurücksetzen" im Cockpit) Leert alle geladenen Staende: Fakten, Umbuchungen, Verlauf (bis auf die erste Formelzeile), Ladeprotokoll, Snapshots, die Postkorb-Liste im Filter und die Standard-Auswahl der Postkoerbe. Mit anwesenheit zusaetzlich Mitarbeiter und Kalender. Die Rohdateien bleiben im Ordner; "Neue Da...
AnwesenheitLeeren	Sub	Private	4656	Mitarbeiter (Stammdaten A:N ab MA_START) und Kalender (Eintraege ab Zeile 7) leeren, Formeln bleiben
NutzerName	Function	Public	4677	Beladung läuft (orange Box oben rechts im Cockpit) Filter!L28 = Name, L29 = Beginn. Gesetzt beim Start von „Neue Daten laden“, geleert am Ende (auch nach Fehler, Ersatz-Frage, leerem Ordner). Die Datei wird jeweils gespeichert, damit Kollegen auf SharePoint die Box sehen bzw. danach die neuen Werte. Wer die Datei schon vorher of...
BeladungAndererText	Function	Public	4688	Laeuft gerade die Beladung eines anderen? Ergebnis: Hinweistext, sonst leer. Quellen: L28/L29 der eigenen Kopie und die Sperrdatei (siehe oben). Meldet die Sperrdatei „frei“ mit einem Ende nach dem Beginn in L28/L29, ist jene Beladung vorbei (die Datei wurde danach nicht mehr gespeichert, etwa nach einem Abbruch beim Ersetzen).
BeladungHinweis	Function	Private	4711	Hinweistext zu einer Beladung (Name, Beginn als Zahl); leer, wenn keine, die eigene (etwa nach einem Absturz von Excel) oder aelter als 3 Stunden (gilt dann als abgebrochen)
BeladungDatei	Function	Private	4720	Name der Sperrdatei: je Master-Datei (HS und Makler koennen im selben Ordner liegen)
BeladungDateiLesen	Function	Private	4728	erste Zeile der Sperrdatei; leer = keine Datei, leer oder nicht lesbar (dann gilt nur L28/L29)
BeladungDateiSchreiben	Sub	Private	4756	Sperrdatei schreiben (an: Beginn, aus: „frei“ mit Ende). Fehler halten das Laden nicht auf, sie stehen im Ladelog.
ZeitAusText	Function	Private	4796	"2026-10-08 09:15:00" -> Zeitpunkt als Zahl (unabhaengig von der Spracheinstellung), sonst 0
BeladungMarkieren	Sub	Private	4808	Sperre setzen/aufheben: L28/L29, Sperrdatei, dann die Master-Datei speichern (speichern = False: nur Sperre aufheben, etwa nach einem Abbruch beim Ersetzen). Vor dem Speichern wird das Ladefenster (LADE_*) entfernt, sonst laege es in der gespeicherten Datei ueber dem Cockpit und fienge die Klicks ab.

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
AeltesterVorgang	Function	Public	4832	Ältester Vorgang (Knopf an der Kachel im Cockpit) Faktenzeile mit dem hoechsten Rueckstandsalter am letzten Stichtag der Quelle nach aktivem Filter – dieselbe Rechnung wie die Kachel (Filter!O31/O32 Stichtag, O47/O48 Alter). Ergebnis Array: 0 Quelle, 1 Stichtag, 2 Alter, 3 Sparte, 4 VSS, 5 Postkorb, 6 GeVo, 7 Einheit, 8 Gruppe, ...
VorgangMarkieren	Function	Public	4874	Rohdatei des aeltesten Vorgangs sichtbar oeffnen (schreibgeschuetzt) und die Zeile gelb markieren. Gesucht wird ueber den Schluessel (VSS, Ordnungsbegriff, LNR ...); fehlt er: die Zeile mit VSS, Postkorb und GeVo und dem hoechsten Alter. Ergebnis leer = markiert, sonst Hinweistext.
RohdateienListe	Function	Public	4965	Rohdateien (Knopf "Rohdaten löschen" im Cockpit) Alle Excel-Dateien im Rohdaten-Ordner und in der Ablage (relative Pfade, neueste zuerst nach Dateiname/Datum)
RohdateiGeladen	Function	Public	4989	Wurde aus dieser Datei (relativer Pfad) ein Stand geladen? (Ladeprotokoll, gueltige Zeilen)
RohdateiGroesse	Function	Public	5003	
RohdateiLoeschen	Function	Public	5009	Datei endgueltig loeschen; Ergebnis leer = geloescht, sonst Fehlertext. Auf SharePoint kann Excel nicht loeschen: die Datei wird im Tool ausgeblendet (beim Laden und in dieser Liste uebergangen), endgueltig loescht man sie im Browser.
AnwesenheitLaden	Sub	Public	5034	Anwesenheitsliste (MAK je Mitarbeiter und Kalenderwoche) importieren Zwei Formate: Enthaelt die Datei ein Blatt "Verwaltung" (Telefonie- und Abwesenheitsplan, eine Zeile je Tag, eine Spalte je Person), uebernimmt PlanUebernehmen die Tage direkt (siehe dort). Sonst gilt die MAK-Liste je KW: Erwartet im Ordner "Anwesenheit" (Konfi...
AnwNichtUebernommen	Sub	Private	5303	Import versucht, aber nicht gelungen: Meldung fuer Laden und Tagesablauf (der Kalender bleibt, wie er ist)
AnwesenheitImLaden	Function	Private	5311	Anwesenheit innerhalb von „Neue Daten laden“ (vor Neuberechnung und Speichern, siehe AnwesenheitAutomatisch). Rueckgabe: Meldung fuer die Abschlusskarte (leer = nicht versucht).
AnwesenheitAutomatisch	Function	Public	5331	Uebernimmt „Neue Daten laden“ die Anwesenheit mit? Ja, solange noch niemand im Blatt Mitarbeiter steht (erster Import, beide Formate), und immer, wenn sie aus dem Telefonie- und Abwesenheitsplan kommt (Mitarbeiter!W7 = "Plan"; im Bereich Makler auch bei leerem W7 – Dateien vor 12.70): der Plan wird laufend gepflegt, Abwesenheite...
AnwesenheitAusPlan	Function	Public	5346	Ergebnis des letzten Imports fuer den Tagesablauf: aus dem Plan? heutige Anwesenheit laut Plan, Fehlertext (leer = kein Fehler)
AnwesenheitHeute	Function	Public	5350	
AnwesenheitFehltHeute	Function	Public	5354	
AnwesenheitMeldung	Function	Public	5358	
AnwesenheitFehler	Function	Public	5362	
AnwesenheitDatei	Function	Public	5366	
AnwesenheitStatusLeeren	Sub	Public	5372	Beginn des Tagesablaufs: Ergebnis eines frueheren Imports dieser Sitzung vergessen
AnwSetzen	Sub	Private	5378	Fortschritt des Anwesenheits-Imports; innerhalb von „Neue Daten laden“ auf dem Stueck zwischen Laden (0,9) und Neuberechnung (0,92)
wsM_Stand	Sub	Private	5383	
PlanUebernehmen	Function	Private	5404	Telefonie- und Abwesenheitsplan (Blatt "Verwaltung", z. B. ODPB3D + KBIG2D) uebernehmen: Kopf: Zeile mit "Datum" (Spalte der Tage), Zeile mit "Mitarbeiter" (1 = zaehlt mit), Zeile mit "Team ..." (Beginn je Team), Namen ueber diesen Zeilen; Spalte "Arbeitsfrei" (1 = Feiertag). Personen: jede Spalte mit 1 in der Mitarbeiter-Zeile ...

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
ZusatzMitarbeiterSetzen	Function	Private	5561	Bereich MAKLER: die Bearbeiter aus ZUSATZ_MA ins Blatt Mitarbeiter, falls sie fehlen (Einheit = ZUSATZ_EINHEIT, damit die Einheiten-Chips sie zuordnen; Sparte Komposit, Kapazitaet 1, aktiv). Vorhandene Zeilen bleiben, wie sie gepflegt sind; ein Austritt im Blatt Mitarbeiter nimmt eine Person heraus. Kalender: ohne Eintrag = anwe...
MitarbeiterAktiv	Function	Private	5580	Zaehlt die Person heute mit? Aktiv (G) nicht FALSCH und kein Austritt (F) vor heute
MitarbeiterZeile	Function	Private	5595	

modCockpit.bas

modCockpit – Schaltflaechen des Cockpits: Filter setzen, Ansicht aktualisieren Die Chips im Cockpit sind Zellen mit HYPERLINK auf ChipHover (Rollover: Filter!Q1 = Chip unter der Maus); der Klick landet ueber Worksheet_SelectionChange (Blattmodul Cockpit) in ChipSchalten und schaltet Filter!C (Themenpostkoerbe, Zeilen PK_VON..PK_BIS), Filter!H (GeVo-Typen, Zeilen 5..18) bzw. Filter!T (Sparten, Zeilen 5..8) um. Alle Kennzahlen und Diagramme rechnen ueber Formeln; DiagrammeAnpassen setzt nur veraenderte Bereiche. Diagramme: 1/2 Rueckstand OPAG/Verbund, 5 MAK, 6 Altersstruktur, 7/8 Produktivitaet Verbund/OPAG (feste Zeilen Diagrammdaten!50:56, Achse J bzw. I, Tag/KW/Woche nach Ansicht), 9 Rueckstand je Themenpostkorb; Kachel Eingaenge/Abgaenge: BW_Gesamt, BW_Komposit, BW_Leben, BW_KV (Quelle Verbund/OPAG nach Filter!L13, umgeschaltet ueber die Bookmark-Chips BWQ_Verbund/BWQ_OPAG). PPT: 10/11 Rueckstand, 12 MAK, 13/14 Bewegungen, 15 Altersstruktur. Ansicht Filter!L9: "Tag" (letzte 7 Stichtage), "KW" (Montag bis Sonntag der KW des Bezugstags, Knopf "Aktuelle KW"), "Woche" (letzte 7 KW). Achse je Quelle (Diagrammdaten!BR OPAG, BS Verbund): in der Wochensicht steht unter der KW das Datum des verwendeten Stands (Sonntag der KW, sonst letzter Stand; * = abgeschlossene KW ohne Sonntagsstand); Bewegungen nutzen dieselbe Achse ohne Stern (BT OPAG, BU Verbund).

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
FilterAllePostkoerbe	Sub	Public	26	
FilterAktiveBearbeitung	Sub	Public	31	
StandardPostkoerbeSetzen	Sub	Private	38	
StandardAnwenden	Sub	Public	53	Knopf "Auf Standard zurücksetzen": Themenpostkoerbe und GeVo-Typen wie in der Standard-Auswahl (Konfiguration H:L), alle Sparten. Sind alle GeVo-Typen "ja", gilt der Modus "Alle GeVo", sonst "Manuelle Auswahl" mit den Kaestchen.
FilterAlleGeVo	Sub	Public	71	
FilterNurBUE	Sub	Public	77	
FilterZuruecksetzen	Sub	Public	83	
FilterAlleSparten	Sub	Public	88	
FilterAlleEinheiten	Sub	Public	94	
EinheitNur	Sub	Private	101	Knopf „nur“ neben einem Einheiten-Chip: nur diese Einheit im Filter, alle anderen aus (kein Umschalten)
EinheitNur1	Sub	Public	112	
EinheitNur2	Sub	Public	116	
EinheitNur3	Sub	Public	120	
EinheitNur4	Sub	Public	124	
EinheitNur5	Sub	Public	128	
EinheitNur6	Sub	Public	132	
Aktualisieren	Sub	Public	136	
Hilfe	Sub	Public	144	
SetzePostkoerbe	Sub	Private	166	
ChipHover	Function	Public	178	Chips (Filterschaltflaechen)
PostkoerbeSortieren	Sub	Public	205	Themenpostkoerbe (Filter!B5:B44) nach Rueckstand am letzten Stand absteigend sortieren (Verbund + OPAG, Filter!D:E; bei Gleichstand nach Saetzen in Fakten, Filter!U). Die zehn Chips im Cockpit zeigen damit die groessten Postkoerbe der geladenen Daten (Postkoerbe ohne Daten erscheinen nicht). Filterkennzeichen (C) und Standard (K...
Zahl	Function	Private	250	
ChipSchalten	Sub	Public	255	

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
AuswahlWegPlanen	Sub	Public	288	
AuswahlWeg	Sub	Public	296	
AktualisierungPlanen	Sub	Public	306	
FilterAnzeigen	Sub	Public	317	
AnsichtTag	Sub	Public	337	
AnsichtWoche	Sub	Public	343	
AnsichtKW	Sub	Public	352	Knopf "Aktuelle KW" unter der Tagessicht: Tagessicht von Montag bis Sonntag der KW des Bezugstags (Filter!O40, ohne frueheren Bezugstag der neueste Stand, also die laufende Woche). Tage ohne Stand und nach dem Bezugstag bleiben leer, ihre Beschriftung steht trotzdem an der Achse; zurueck/vor/Neuester Stand verschieben den Bezugs...
WochensichtBeimOeffnen	Sub	Public	359	Beim Oeffnen (Workbook_Open) immer Wochensicht – nur schreiben, wenn noetig (sonst fragt Excel beim Schliessen ohne Grund)
BezugFormelSetzen	Sub	Public	369	
BezugZurueck	Sub	Public	376	
BezugVor	Sub	Public	380	
BezugNeuester	Sub	Public	384	
BezugSetzen	Sub	Private	390	
QuelleVerbund	Sub	Public	420	
QuelleOPAG	Sub	Public	426	
BookmarkFaerben	Sub	Private	432	
DeltaBoxenRunden	Sub	Private	456	
ChipFaerben	Sub	Private	475	
DiagrammeAnpassen	Sub	Public	498	Diagrammbereiche nachziehen
PlanungOeffnen	Sub	Public	588	Navigation
AlphabetischSortieren	Sub	Public	598	Mitarbeiter und ihre Kalenderzeilen alphabetisch sortieren. Mitarbeiter!A5:N64 (Eingaben inkl. Arbeitszeitmodell) und Kalender!D7:... gehoeren zeilengleich zusammen (Kalender-Zeile = Mitarbeiter-Zeile + 2) und werden mit derselben Reihenfolge umgestellt; Kalender!A:C, Mitarbeiter!O:T und Wirksam folgen ueber Formeln. Leere Zeile...
Vorher	Function	Private	647	a vor b? Leere Namen immer ans Ende
CockpitOeffnen	Sub	Public	654	
ZurAktuellenKW	Sub	Public	660	
KalenderSpalteHeute	Function	Public	667	Spalte des heutigen Tags im Kalender (am Wochenende der naechste Montag); 0 = ausserhalb des Kalenders
KalenderZuHeute	Sub	Public	681	Kalender (muss aktiv sein): "frei"/"FT" nachtragen, vergangene Wochen einklappen und so blaettern, dass heute sichtbar ist
KalenderVorbelegen	Sub	Public	697	Traegt im Kalender ein, was ohnehin feststeht: "FT" an Feiertagen (Konfiguration!N11:N60) und ein kursives "A" (abwesend) an Wochentagen, an denen die Person laut Arbeitszeitmodell nicht arbeitet (Mitarbeiter!J:N = 0). Angepasst werden nur leere Zellen, "FT" und die kursiven "A" des Tools – eigene Eintraege (U, K, A, Anteil) ble...
KalenderEinklappen	Function	Public	759	Alle Tage vor dem Montag der aktuellen Woche als eine Gliederungsgruppe zusammenfassen und zuklappen ("+" oben klappt auf). Rueckgabe: Spalte des Montags der aktuellen Woche (erste sichtbare Tagesspalte).
SetzeReihen	Sub	Private	782	
EtikettenUeberSaeule	Sub	Public	794	Gruppierte Saeulen: passt der Wert nicht in die Saeule (Saeule niedriger als das Etikett), steht er dunkel ueber der Saeule, sonst weiss innen am Ende

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
EtikettenAuslagern	Sub	Public	823	Etiketten wie in think-cell: passt der Wert nicht ins Segment, steht er mit Fuehrungslinie neben der Saeule (abwechselnd links und rechts, ohne Ueberlappung) bzw. ueber dem Balken.
RechnenPruefen	Sub	Public	954	Beim Oeffnen: steht Excel auf manuell, auf automatisch stellen und alles neu berechnen

modTagesablauf.bas

modTagesablauf – gefuehrter Tagesablauf in drei Schritten (Karten als Formen, ohne UserForm) Start (Knopf "Neue Daten laden" im Cockpit) 1. Rohdaten: "Liegen die neuen Abzuege im Ordner Rohdaten?" Ja -> modLaden.NeueDatenLaden (dessen Abschlussmeldung faengt modFortschritt.HinweisZeigen ab und reicht sie hierher weiter) 2. Anwesenheit: "Ist heute jemand nicht da?" Ja -> Blatt Kalender beim heutigen Tag, die Spalte von heute blinkt kurz auf; eine Karte am rechten Rand bleibt offen, waehrend eingetragen wird (U, K, A oder Anteil), "Fertig" schliesst den Schritt ab. Nein -> direkt zu 3. Kommt die Anwesenheit aus dem Telefonie- und Abwesenheitsplan (das Laden hat ihn eben uebernommen), fragt der Schritt nicht, sondern zeigt, wer heute laut Plan fehlt (PlanZeigen); "Im Kalender ansehen" oeffnet den Kalender wie oben. 3. Erfolg: "Alles fuer heute erledigt" – optional "PowerPoint-Folie erzeugen". Filter!O42 haelt den Zeitpunkt des letzten abgeschlossenen Tagesablaufs (Anzeige im Cockpit-Kopf). Klicks auf eine Karte loeschen die Karte nie im eigenen Klick, sondern per OnTime gleich danach (wie modFortschritt.HinweisSchliessen) – das Loeschen der geklickten Form im eigenen Makro kann Excel abstuerzen lassen.

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
TagesablaufVersion	Function	Public	48	
WartetAufLaden	Function	Public	53	Wird von modFortschritt.HinweisZeigen gefragt: laeuft gerade das Laden aus dem Tagesablauf?
LadeErgebnis	Sub	Public	58	Abschlussmeldung von NeueDatenLaden (oder eines Fehlers darin) uebernehmen und Schritt 2 planen
Start	Sub	Public	71	Einstieg
Schritt1Zeigen	Sub	Private	91	
KlickLaden	Sub	Public	114	Klicks Schritt 1
KlickOhneLaden	Sub	Public	118	
OhneLadenWeiter	Sub	Public	124	ohne neue Rohdaten: einen laufend gepflegten Anwesenheitsplan trotzdem frisch uebernehmen (MAK des Tages), dann Schritt 2
KlickOrdner	Sub	Public	131	
KlickAbbrechen	Sub	Public	144	
Nichts	Sub	Public	150	
LadenStarten	Sub	Public	154	
NachDemLaden	Sub	Public	174	
NachStandardWeiter	Sub	Public	191	nach der Karte „Neue Themenpostkörbe“ im Tagesablauf mit Schritt 2 weiter
FehlerNachLadenZeigen	Sub	Private	195	
ErsatzFrageZeigen	Sub	Public	217	Ersatz-Frage: neuer Abzug fuer einen schon geladenen Stand
Flaeche	Sub	Private	279	Flaeche (abgerundet, ohne Rand) als Hintergrund einer Tabelle
Linie	Sub	Private	290	duenne Trennlinie zwischen Tabellenzeilen
Marke	Sub	Private	299	Marke der Quelle (OPAG rot, Verbund petrol) – Pille mit weisser, fetter Schrift
KurzName	Function	Private	321	Dateiname kurz (ohne Ordner, hoechstens 34 Zeichen)
KlickErsetzen	Sub	Public	326	
KlickBehalten	Sub	Public	331	
KlickErsatzAbbrechen	Sub	Public	336	
ErsatzWeiter	Sub	Public	346	nach der Antwort erneut laden – im Tagesablauf mit dessen Fortsetzung (Schritt 2), sonst direkt
Schritt2Zeigen	Sub	Public	356	Schritt 2: Anwesenheit heute
PlanZeigen	Sub	Private	411	Schritt 2, wenn die Anwesenheit aus dem Telefonie- und Abwesenheitsplan kommt: nichts eintragen, nur zeigen, wer heute fehlt
KlickAlleDa	Sub	Public	458	
KlickAbwesend	Sub	Public	462	

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
AbwesenheitOeffnen	Sub	Public	466	
HeuteFrei	Function	Private	518	Namen, die heute laut Arbeitszeitmodell (Mitarbeiter!J:N = 0) nicht arbeiten
KlickZeigen	Sub	Public	537	
Hervorheben	Sub	Public	542	Die Spalte von heute (alle Mitarbeiterzeilen) und ihr Kopf blinken dreimal auf und verschwinden wieder
Rahmen	Sub	Private	577	
KlickFertig	Sub	Public	592	
AbwesenheitFertig	Sub	Public	596	
VorbelegungZuruecknehmen	Sub	Private	619	(frueher: vorbelegtes Datum im Blatt Abwesenheit zuruecknehmen – im Kalender gibt es keine Vorbelegung)
Schritt3Zeigen	Sub	Public	624	Schritt 3: Erfolg
KlickPPT	Sub	Public	666	
PPTStarten	Sub	Public	670	
KlickSchliessen	Sub	Public	676	
KartenEntfernenUndZurueck	Sub	Public	681	
Planen	Sub	Private	689	Bausteine
KartenEntfernen	Sub	Public	695	
KarteModal	Sub	Public	707	abgedunkelter Hintergrund ueber dem sichtbaren Bereich und Karte in der Mitte
Karte	Sub	Public	723	
Kopf	Sub	Private	741	Kopf der Karte: "TAGESABLAUF · SCHRITT n VON 3", Fortschritt in drei Segmenten mit Beschriftung
Beschriftung	Sub	Public	771	
Feld	Sub	Private	793	Feld mit Pfad (grau hinterlegt, eine Zeile, links gekuerzt, wenn zu lang)
Pille	Sub	Public	818	hinterlegter Hinweisblock (Ergebnis des Ladens, Zusammenfassung)
Knopf	Sub	Public	839	art: 1 = primaer (petrol), 2 = sekundaer (weiss mit Rand), 3 = Textknopf (grau), 4 = PowerPoint (rot)
Haken	Sub	Private	871	gruener Kreis mit weissem Haken (Linienzug, keine Schriftzeichen)
PilleHoehe	Function	Private	894	Hoehe eines Hinweisblocks: je Absatz und je ca. 85 Zeichen (9 pt auf 480 pt Breite) eine Zeile zu 14 pt, plus Rand
KurzText	Function	Private	906	Lademeldung einzeilig mit "Rohdaten:" davor (mehrere Meldungen werden mit Leerzeichen verbunden)
WochentagLang	Function	Private	914	
ErsatzFrage1	Sub	Private	919	Fallback ohne Formen (z. B. geschuetztes Blatt): klassische Abfrage

modWerkzeuge.bas

modWerkzeuge – Karten (Formen, ohne UserForm) fuer die Werkzeug-Knoepfe im Cockpit SLA anpassen ... Konfiguration!F3 (SLA Verbund, 0 = keine Linie) und F4 (SLA OPAG, leer = Vorschlag) eingeben Einheit ... nur Bereich MAKLER, nur Dateien vor 12.70 (Knopf BTN_Einheit, seither entfernt): Gesamt (Standard) oder eine Einheit (Filter!L16); Fakten!P und Umbuchungen!K/L zaehlen dann nur Zeilen dieser Einheit Rohdaten löschen ... Dateien im Ordner Rohdaten (und der Monatsablage) auswaehlen und endgueltig loeschen; auf Wunsch danach das Tool aus den uebrigen Dateien neu aufbauen Zurücksetzen ... alle geladenen Staende leeren (modLaden.ToolLeeren), optional auch die Anwesenheit Sparte | Einheit nur Bereich MAKLER: Chips in der Kopfleiste „Rückstand OPAG“; Filter!L19 schaltet die Stapel beider Rueckstandsdiagramme (Cockpit und Folie) zwischen Sparten und Einheiten (Namen Filter!L22:L25) um Bausteine (Karte, Beschriftung, Knopf, Pille) kommen aus modTagesablauf; alle Formen tragen den Praefix TAGW_ und verschwinden mit modTagesablauf.KartenEntfernen. Klicks loeschen die geklickte Form nie im eigenen Makro (OnTime).

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
WerkzeugeVersion	Function	Public	40	
Planen	Sub	Private	44	
Bereit	Function	Private	50	

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
Schliessen	Sub	Public	60	
KlickSchliessen	Sub	Public	64	
Ueberschrift	Sub	Private	68	
Zeile	Sub	Private	74	Rechteck mit Text und Makro (Zeilen der Dateiliste, Umschalter)
Eingabe	Sub	Private	106	Eingabefeld: Textfeld ohne Makro (Klick hinein, tippen)
FeldText	Function	Private	126	
Wert	Function	Private	132	
ZahlLesen	Function	Private	137	"5.500" / "5500" / "5 500" -> 5500; leer -> -1; keine Zahl -> -2
SlaZeigen	Sub	Public	149	SLA anpassen
KlickSlaSpeichern	Sub	Public	174	
SlaSpeichern	Sub	Public	178	
EinheitZeigen	Sub	Public	201	Einheit (Bereich MAKLER)
KlickEinheit	Sub	Public	232	
EinheitSetzen	Sub	Public	242	
EinheitKnopfBeschriften	Sub	Public	255	Text des Cockpit-Knopfs "Einheit: ..." nach Filter!L16 (nur Dateien vor 12.70; fehlt der Knopf, nur SichtFaerben)
SichtSparte	Sub	Public	266	Sicht Sparte | Einheit (Bereich MAKLER)
SichtEinheit	Sub	Public	270	
SichtSetzen	Sub	Private	274	
SichtFaerben	Sub	Public	283	Chips wie die Bookmark-Chips: aktiv = weiss mit Schrift in der Kopffarbe (OPAG rot), inaktiv = Kopffarbe mit weisser Schrift
RohdatenZeigen	Sub	Public	305	Rohdaten löschen
DateiAnzahl	Function	Private	322	
AnzahlGewaehlt	Function	Private	328	
RohdatenZeichnen	Sub	Private	335	
DateiZeile	Sub	Private	368	
KlickDatei	Sub	Public	392	
DateiUmschalten	Sub	Public	402	
KlickSeiteZurueck	Sub	Public	410	
KlickSeiteVor	Sub	Public	415	
RohdatenNeu	Sub	Public	420	
KlickLoeschenFragen	Sub	Public	426	
LoeschenFragen	Sub	Public	431	
KlickZurAuswahl	Sub	Public	464	
KlickLoeschenNur	Sub	Public	468	
KlickLoeschenNeu	Sub	Public	472	
LoeschenNur	Sub	Public	476	
LoeschenNeu	Sub	Public	484	
BeladungAnderer	Function	Private	496	Laeuft gerade die Beladung eines Kollegen? Dann Hinweis statt Loeschen oder Leeren – sonst waere das Tool leer und wuerde nicht neu aufgebaut, und gespeichert ueberschriebe der leere Stand die Datei, in die dort gerade geladen wird.
Loeschen	Function	Private	506	ausgewaehlte Dateien loeschen; Ergebnis als Meldungstext
ZurueckssetzenZeigen	Sub	Public	532	Zurücksetzen
ZurueckssetzenZeichnen	Sub	Private	538	
KlickAnwesenheit	Sub	Public	554	
ZurueckssetzenNeu	Sub	Public	559	
KlickLeeren	Sub	Public	565	

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
Leeren	Sub	Public	569	
AeltesterZeigen	Sub	Public	583	Ältester Vorgang (Knopf „Details“ an der Kachel) Je Quelle der Vorgang mit dem hoechsten Rueckstandsalter am letzten Stichtag nach aktivem Filter (wie die Kachel): Ordnungsbegriff, LNR, VSS und Sparte, GeVo, Themenpostkorb, Einheit und Rohdatei. „In Rohdatei markieren“ oeffnet die Rohdatei und markiert die Zeile gelb (modLaden.V...
Zeilentext	Sub	Private	623	
ZahlTage	Function	Private	628	
DateiKurz	Function	Private	632	
KlickMarkierenVerbund	Sub	Public	637	
KlickMarkierenOPAG	Sub	Public	641	
MarkierenVerbund	Sub	Public	645	
MarkierenOPAG	Sub	Public	649	
Markieren	Sub	Private	653	
UpdateZeigen	Sub	Public	664	
KlickUpdate	Sub	Public	688	
UpdateAusfuehren	Sub	Public	692	

modStandard.bas

modStandard – Karte "Was gehört zum Standard?" (Knopf "Standard festlegen ..." im Cockpit) Zeigt alle bekannten Themenpostkoerbe (Filter!B5:B44) und GeVo-Typen (Filter!G5:G18) als Chips; ein Klick schaltet "im Standard" an oder ab. Gespeichert wird als ja/nein in der Konfiguration (I11:I50 zeilengleich mit Filter!B5:B44, L11:L24 zeilengleich mit Filter!G5:G18). Der Standard gilt fuer den Wochenreport (Rohdatenblatt, Fakten!Q) und fuer "Auf Standard zurücksetzen". "Speichern und anwenden" stellt zusaetzlich die Chips im Cockpit auf den Standard. Zeichnet mit den Bausteinen aus modTagesablauf (Praefix TAGW_ zum Aufraeumen).

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
StandardVersion	Function	Public	25	
Zeigen	Sub	Public	30	Einstieg
ChipZeichnen	Sub	Private	108	Chip: gefuellt (petrol, weisse Schrift) = im Standard, weiss mit Rand = nicht im Standard
KlickChip	Sub	Public	153	Klicks
Umschalten	Sub	Public	166	
KlickAlleGeVo	Sub	Public	179	
AlleGeVo	Sub	Public	183	
KlickAbbrechen	Sub	Public	193	
NeueFertig	Sub	Public	201	
KlickSpeichern	Sub	Public	208	
KlickAnwenden	Sub	Public	212	
Speichern	Sub	Public	216	
SpeichernUndAnwenden	Sub	Public	223	
StandardSchreiben	Sub	Private	230	

modPPT.bas

modPPT – Kacheln des Blatts "PPT" als Formen auf eine Folie der ERGO-Vorlage kopieren Ablauf: Diagramme aktualisieren, PowerPoint starten, Vorlage (Vorlage\ERGO_Vorlage.pptx neben der Master-Datei) als unbenannte Kopie oeffnen (sonst leere Praesentation), Folie mit Layout "Nur Überschrift" (oder "Only Title") anlegen, Titel und Untertitel aus PPT!BA1:BA2 (Untertitel in die Kopfzeile ueber dem Titel, falls das Layout eine hat), je gewaehlter Kachel (Auswahl aus modFolienauswahl, "111111") die Kachel unter dem Titel aufbauen: Textfelder in PowerPoint nachbauen, Diagramme einzeln einfuegen (sonst als Bild), Kopie im Ordner "Praesentationen" sichern.

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
NachPowerPoint	Sub	Public	21	

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
KopieSichern	Function	Private	237	Kopie der Praesentation sichern: Ordner Praesentationen neben der Master-Datei. Auf SharePoint (https) zuerst dort hinein (Ordner muss bestehen), sonst in „Dokumente“ des Benutzers. Ergebnis: Pfad der Kopie.
KachelVon	Function	Private	262	Kachel (0..5) einer Form im Blatt PPT, -1 = keine Kachel. Jede Kachel hat genau ein Diagramm ("Chart 10" bis "Chart 15", Kachel = Nummer - 10). Ein Textfeld gehoert zu dem Diagramm, ueber dem es liegt (Mitte waagerecht im Diagramm) und dessen Kopfbereich oder Flaeche es senkrecht trifft. Gemessen an den echten Diagrammen in Exce...
EtikettenLeererPunkte	Sub	Private	289	Etiketten von Punkten ohne Wert (#NV) abschalten: Excel blendet sie aus, PowerPoint schreibt "#NV"
DiagrammSetzen	Sub	Private	306	Ein Diagramm des Blatts PPT einzeln kopieren und als PowerPoint-Diagramm einfuegen (#NV-Etiketten weg, Verknuepfung zur Master-Datei geloest). Kommt kein Diagramm an (Bild, eingebettetes Objekt), das Diagramm als PNG exportieren und als Bild setzen – dann ist es nicht bearbeitbar, aber sicher sichtbar.
TextfeldSetzen	Sub	Private	352	Ein Textfeld des Blatts PPT in PowerPoint nachbauen: Text, Schrift (Name, Groesse, fett, kursiv, Farbe), Fuellung, Ausrichtung, Innenraender. zeile2: zweite Zeile (Kennzahl der Kopfleiste), normal statt fett, beide Zeilen mittig
PPTModulVersion	Function	Public	405	

modFolienauswahl.bas

modFolienauswahl – Karte "Welche Kacheln sollen auf die Folie?" vor dem PowerPoint-Export Die sechs Kacheln des Blatts PPT stehen als Kaestchen in derselben Anordnung wie auf dem Blatt (3 x 2); ein Klick schaltet eine Kachel an oder ab. Die Auswahl steht als "111111" in Filter!O46 und gilt beim naechsten Mal wieder. "Folie erzeugen" ruft modPPT.NachPowerPoint mit der Auswahl auf; dort werden nur diese Kacheln kopiert und auf der Folie neu angeordnet (1-3 in einer Reihe, 4 als 2 x 2, 5-6 in zwei Reihen). Zeichnet mit den Bausteinen aus modTagesablauf (gleicher Stil, gleiches Praefix TAGW_ zum Aufraeumen).

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
FolienauswahlVersion	Function	Public	26	
Titel	Function	Private	30	
Farbe	Function	Private	35	
GespeicherteAuswahl	Function	Public	43	
Zeigen	Sub	Public	53	Einstieg (Knopf "PowerPoint-Folie erzeugen")
KachelnZeichnen	Sub	Private	79	Kaestchen je Kachel (3 x 2 wie auf dem Blatt PPT) und Zaehler darunter
Anordnung	Function	Private	92	
KachelZeichnen	Sub	Private	101	
KlickKachel	Sub	Public	180	Klicks
Umschalten	Sub	Public	192	
KlickAlle	Sub	Public	203	
AlleAn	Sub	Public	207	
KlickAbbrechen	Sub	Public	214	
KlickErzeugen	Sub	Public	218	
Erzeugen	Sub	Public	223	

modFortschritt.bas

modFortschritt – Lade-Overlay mit Fortschrittsbalken (ohne UserForm: Formen auf dem aktiven Blatt) FortschrittZeigen "Titel" -> abgedunkelte Flaeche, weisse Karte, Titel, Statuszeile, Balken, Prozent FortschrittSetzen 0.42, "Text"[, "Detail"] -> Balken, Statuszeile und kleine Zeile darunter (z. B. Dateiname) nachziehen (zeichnet auch bei ScreenUpdating = False); zu lange Texte werden auf die Breite gekuerzt FortschrittSchritt "Schritt 1 von 2 · ..." -> Zeile ueber dem Status (Abschnitt des Ablaufs), leer = keine FortschrittSchliessen -> alle Formen entfernen

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
FortschrittZeigen	Sub	Public	22	
FortschrittSetzen	Sub	Public	87	

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
FortschrittSchritt	Sub	Public	107	Abschnitt des Ablaufs ueber der Statuszeile, z. B. "SCHRITT 1 VON 2 · DATEIEN PRÜFEN"
TextPassend	Sub	Private	118	Text in ein Feld ohne Umbruch setzen und so weit kuerzen, dass er in die Breite des Felds passt (gemessen an der echten Textbreite). mitte = True kuerzt in der Mitte (Dateinamen: Anfang und Endung bleiben sichtbar), sonst am Ende.
Gekuerzt	Function	Private	141	
FortschrittSchliessen	Sub	Public	152	
Zeichnen	Sub	Private	171	
Textfeld	Sub	Private	179	
HinweisZeigen	Sub	Public	198	Hinweis-Karte (Ersatz fuer MsgBox): Titel, Text, OK-Knopf
HinweisZeichnen	Sub	Public	223	
HinweisSchliessen	Sub	Public	307	OK-Knopf und Schleier: nicht im eigenen Klick loeschen, sondern gleich danach per OnTime
HinweisEntfernen	Sub	Public	314	

Cockpit.cls

Blattmodul Cockpit: Der Hyperlink eines Chips zeigt auf Spalte A (oder H) der Chipzeile; dort steht in Spalte A die Chip-Nummer (1..99 Themenpostkoerbe, 101..114 GeVo-Typen, 201..204 Sparten, 301..306 Einheiten). Der Auswahlwechsel schaltet den Chip, danach wandert die Auswahl per OnTime auf H1, damit der naechste Klick wieder ein Auswahlwechsel ist.

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
Worksheet_SelectionChange	Sub	Private	5	Blattmodul Cockpit: Der Hyperlink eines Chips zeigt auf Spalte A (oder H) der Chipzeile; dort steht in Spalte A die Chip-Nummer (1..99 Themenpostkoerbe, 101..114 GeVo-Typen, 201..204 Sparten, 301..306 Einheiten). Der Auswahlwechsel schaltet den Chip, danach wandert die Auswahl per OnTime auf H1, damit der naechste Klick wieder e...
Worksheet_Change	Sub	Private	17	

Kalender.cls

Blattmodul Kalender: beim ersten Aufruf je Sitzung zum heutigen Tag springen. Danach bleibt die Position, wie der Nutzer sie verlaesst ("Zu heute" springt jederzeit zurueck).

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
Worksheet_Activate	Sub	Private	6	

Rohdatenblatt.cls

Blattmodul Rohdatenblatt: beim ersten Aufruf je Sitzung zur aktuellen Woche springen (Vorwoche links, damit das Delta zur aktuellen Woche sichtbar ist). Danach bleibt die Position, wie der Nutzer sie verlaesst.

Name	Art	Sicht	Zeile	Beschreibung (Kommentar im Code)
Worksheet_Activate	Sub	Private	6	

C.2 Quellcode

Vollständig und unverändert, mit Zeilennummern; lange Zeilen sind umbrochen (eingerückt fortgesetzt).

modLaden.bas

```
Option Explicit
' ================================================================================
' modLaden – Rohdaten (OPAG- und Verbund-Rueckstandsabzuege) laden, Anwesenheitsliste importieren
'
' Ablauf "Neue Daten laden":
'   1. Ordner "Rohdaten" (relativ zur Master-Datei, Konfiguration!B3) nach Excel-Dateien durchsuchen
'   2. In jeder Datei jedes Blatt pruefen: Stichtag aus "Rückstandsstichtag = …", Quelle aus den Spaltenkoepfen
'      (OPS_AUFTR_ID vorhanden = OPAG, sonst Verbund)
'   3. Alle gefundenen Staende nach Stichtag und Quelle sortieren, bereits geladene ueberspringen
'   4. Je Stand: Rueckstand je Sparte/VSS/Themenpostkorb/GeVo mit Altersklassen zaehlen,
'      Eingaenge (Schluessel neu gegenueber dem letzten Stand) und Abgaenge (Schluessel weg) bestimmen,
'      Zeilen an "Fakten" anhaengen, Snapshot der Schluessel ersetzen, Zeile in "Verlauf" ergaenzen
'   5. Ladeprotokoll fortschreiben, neu berechnen
' Liegt im Ordner Rohdaten ein neuer Abzug fuer einen schon geladenen Stand, bleibt die Datei liegen und eine Karte fragt
' (modTagesablauf.ErsatzFrageZeigen): Ersetzen loescht den alten Stand und rechnet ihn aus der neuen Datei neu – Eingaenge und
' Abgaenge gegen den Stand davor (aus dessen Datei gelesen), danach alle spaeteren Staende dieser Quelle aus ihren Dateien.
' Die alte Datei heisst danach …_ersetzt, eine nicht uebernommene neue …_verworfen; beide werden nicht mehr gelesen.
' Erkannte Dateien werden als JJJJMMTT_BUe_<OPAG|Verbund> in die Ablage Rohdaten\JJJJ\MM_Monat verschoben
' (Sammeldatei: JJJJMMTT-JJJJMMTT, beide Quellen: OPAG_Verbund); scheitert das Umbenennen, bleibt der Name und das Log vermerkt es.
' Fremde Dateien werden ohne Makros, ohne Dialoge und schreibgeschuetzt geoeffnet (QuelleOeffnen).
' Bereich MAKLER (Konfiguration!F7): Die Abzuege enthalten mehrere Einheiten (OE-Ebene 4: ODPB D, ODPP D, MSUE D …), auch verteilt auf
' mehrere Dateien am selben Tag. Je Zeile ergibt sich die Einheit aus OE_NR (ODPB1 D, ODPB2HH, ODPBHH -> ODPB; Zuordnung in
' Konfiguration!Q11:R40, sonst die fuehrenden Buchstaben ohne HH). Seit 12.88 ist jede Einheit aus Konfiguration!R11:R40 ein
' eigener Strang – unabhaengig davon, wie MIS-O die Einheiten auf Dateien verteilt: eigener Stand je Quelle und Sticht ag, eigener
' Snapshot, eigene Ersatz-Frage; Zeilen anderer OEs zaehlen zur Haupteinheit des Blatts. Alle Einheiten eines Stichtags werden
' gemeinsam verarbeitet (Wechsel zwischen ihnen = Umbuchung); ein Nachzuegler wird eingefuegt (StandErsetzen mit einfu egen).
' Fakten (R Einheit, S Strang), Umbuchungen (Q/S Einheit heute/vorher, R Strang), Ladeprotokoll (K Strang) und Snapsho t
' (F "Strang|Einheit") fuehren beides mit; der Dateiname traegt die Einheiten der Datei. Blatt Dateiinhalt merkt sich je Rohdatei
' Blaetter und Einheiten (InhaltMerken), damit sie nicht zum Erkennen erneut geoeffnet wird.
' Im Bereich HS bleiben Einheit und Gruppe leer (wie bisher).
' Seit 12.91 ignoriert der Bereich MAKLER Zeilen der HS-Einheit (OE HSB4 K -> Einheit HSB, HS_EINHEITEN): sie zaehlen weder als
' Rueckstand noch als Eingang. Ein reiner HS-Abzug im Ordner Rohdaten wird nicht geladen, nicht an Celonis uebergeben und heisst
' danach …_HS-ignoriert (SharePoint ohne Dateipfad: im Tool ausgeblendet). Stehen aus der Zeit davor schon HS-Zeilen in den
' Fakten, baut „Neue Daten laden“ das Tool einmal ohne sie neu auf (HsDatenGeladen, ToolLeeren).
' Seit 12.92 gilt ein Stand mit der Verkettungs-Angabe im Hinweis des Ladeprotokolls (seit 12.69 jeder normal geladene Stand) als
' geladen (StandGueltig). Bis 12.91 las „Neue Daten laden“ mit Dateipfad den zuletzt geladenen Stand jeder Quelle beim naechsten
' Laden deshalb erneut ein, sein Rueckstand stand doppelt; DoppelteStaendeBereinigen raeumt das beim ersten Laden auf.
' Waehrend des Ladens ist die Eingabe gesperrt (LadeModusEin/-Aus), Fehler raeumen immer auf.
' SharePoint ohne OneDrive (BasisOrdner liefert dann die https-Adresse): Dir, Name, MkDir und Kill gehen dort nicht. Die Ordner
' werden ueber Power Query aufgelistet (SpListe, Abfrage BUE_SharePoint_Liste, Anmeldung mit dem Office-Konto), die Dateien
' direkt ueber https geoeffnet. Seit 12.80 legt das Tool auch dort ab (JJJJ\MM_Monat, einheitlicher Name): ueber die
' SharePoint-Schnittstelle per Power Query mit derselben Anmeldung (SpAblagePruefen, SpVerschieben). Geht das nicht (Rechte,
' Richtlinie), bleibt es beim Folgenden: Ablage in JJJJ\MM_Monat und Umbenennen entfallen; schon geladene Dateien werden am Namen erkannt
' und nicht erneut geoeffnet. Logs liegen dann unter %LOCALAPPDATA%\BUE_Reporting.
' ================================================================================

Private Const FAKTEN_SPALTEN As Long = 16       ' A..O Daten, P = Aktiv (Formel), Q = Standard (Formel, Spalte 17)
Private Const STANDARD_VORSCHLAG As String = "|KO 23724|MV 23724|"   ' neue Postkoerbe mit diesem Namen kommen als "ja" in den Standard
Private Const FAKTEN_MAX As Long = 300000       ' Ende der benannten Bereiche F_* (Fakten!X2:X300000)
Private Const UMB_MAX As Long = 100000          ' Ende der benannten Bereiche U_* (Umbuchungen!X2:X100000)
Private Const VERLAUF_MAX As Long = 1000        ' Ende der Verlauf-Bezuege
Private Const MAX_KOPFZEILEN As Long = 15
Private Const SP_EINHEIT_F As Long = 18         ' Fakten!R: Einheit der Zeile (aus OE_NR, Bereich MAKLER)
Private Const SP_GRUPPE_F As Long = 19          ' Fakten!S: Gruppe = Einheiten der Rohdatei (Strang fuer den Vergleich)
Private Const SP_ALTER_F As Long = 20           ' Fakten!T: hoechstes Rueckstandsalter (Tage) der Zeile – KPI „Älteste r Vorgang“
Private Const SP_VORGANG_F As Long = 21         ' Fakten!U: Schluessel des aeltesten Datensatzes der Zeile (VSS|Ordnungsbegriff|LNR[|Auftrag|Arbeitsgut])
Private Const SP_ALTSUM_F As Long = 22          ' Fakten!V: Summe der Rueckstandsalter (Tage) der Zeile – Ø Alter = V / Rueckstand (seit 12.84)
Private Const SP_EINHEIT_U As Long = 17         ' Umbuchungen!Q: Einheit heute
Private Const SP_GRUPPE_U As Long = 18          ' Umbuchungen!R: Gruppe
Private Const SP_EINHEIT_VOR_U As Long = 19     ' Umbuchungen!S: Einheit vorher
Private Const SP_GRUPPE_L As Long = 11          ' Ladeprotokoll!K: Gruppe (Einheiten der Datei)
Private Const SNAP_EINHEITEN As Long = 40       ' Snapshot_*!J2:L41: Stichtag und Schluessel je Gruppe
Private Const TRENNER As String = "|"
' Bereich MAKLER: BUe-Bearbeiter der Vertriebsunterstuetzung MSUED (Mail K. Staudt, 13.10.2026), zusaetzlich zum Anwesenheitsplan
Private Const ZUSATZ_MA As String = "Hubertus Baumgarten|Melanie Möller|Claudia Plottke|Nicolo Schuchnelski|Renate Friedrich|Silke Hutter|Ira Püttjer|Rohit Singh Puri"
Private Const ZUSATZ_EINHEIT As String = "MSUE"
```

```
Public Const MA_START As Long = 5              ' erste Mitarbeiterzeile im Blatt Mitarbeiter
Public Const MA_ZEILEN As Long = 60            ' reservierte Mitarbeiterzeilen
Public Const PL_VERSATZ As Long = 2            ' Kalender-Zeile = Mitarbeiter-Zeile + 2 (Kalender beginnt in Zeile 7)
Public Const PK_VON As Long = 5                ' Filter!B5:B44 Themenpostkoerbe
Public Const PK_BIS As Long = 44

Private Type Stand
    Datei As String
    Blatt As String
    Quelle As String
    Gruppe As String                           ' nur Bereich MAKLER: Strang = Einheit (seit 12.88, z. B. "ODPB"), son
   st leer
    DateiGruppe As String                      ' nur Bereich MAKLER: alle Einheiten der Datei (z. B. "MSUE+ODPB"), fu
   er den Dateinamen
    Stichtag As Date
    Zeilen As Long                             ' Datensaetze laut Blatt (fuer die Ersatz-Frage)
    Ersetzen As Boolean                        ' schon geladener Stand, neue Datei soll ihn ersetzen
End Type

Private calcVorher As XlCalculation
Private logPfad As String                     ' Ladelog.txt neben der Master-Datei
Private importMeldung As String               ' Ergebnis des letzten Anwesenheits-Imports (fuer die gemeinsame Meldun
   g)
Private anwImLaden As Boolean                 ' AnwesenheitLaden laeuft innerhalb von NeueDatenLaden (dessen Lademodu
   s, keine eigene Neuberechnung)
Private anwPlan As Boolean                    ' letzter Import (diese Sitzung): Anwesenheitsplan (Blatt Verwaltung),
   gelungen
Private anwHeute As String                    ' Plan: heutige Anwesenheit als Text fuer den Tagesablauf
Private anwFehltHeute As Long                 ' Plan: heute abwesend laut Plan (-1 = heute arbeitsfrei, -2 = heute ni
   cht im Plan)
Private anwFehlschlag As Boolean              ' letzter Import (diese Sitzung) versucht, aber nicht gelungen
Private lademodusAktiv As Boolean
Private ersatzWahl As Object                  ' Quelle|Stichtag -> True (ersetzen) / False (alten behalten)
Private ersatzOffen As Object                 ' Quelle|Stichtag -> Beschreibung, solange die Frage offen ist
Private celonisAnzahl As Long                 ' an Celonis uebergebene Dateien (dieses Laden)
Private neuePK As Object                      ' neue Themenpostkoerbe ausserhalb HS (dieses Laden), Standard "ja", Fr
   age folgt
Private oeEinheit As Object                   ' OE_NR -> Einheit (Cache; Zuordnung Konfiguration!Q11:R40)
Private inhaltIndex As Object                 ' Blatt Dateiinhalt als Index (je Laden neu gelesen)
Private gruppenListe As Object                ' Einheiten aus Konfiguration!R11:R40 (Cache je Laden; leer = jede Einh
   eit)
Private lesenSchluessel As String             ' zuletzt gelesenes Blatt (Datei|Blatt|Quelle|Stichtag) – Straenge eine
   r Datei lesen sie nur einmal
Private lesenDaten As Variant
Private lesenSp() As Long
Private lesenZeilen As Long
Private basisCache As String                  ' aufgeloester Ordner der Master-Datei bei SharePoint-Adresse (je Sitzu
   ng)
Private basisCacheUrl As String
Private spListen As Object                    ' SharePoint: Ordneradresse (klein) -> Dictionary rel (Trenner \) -> Ar
   ray(rel, geaendert, groesse)
Private spFehler As String                    ' SharePoint: Fehlertext der letzten Auflistung
Private Const SP_ABFRAGE As String = "BUE_SharePoint_Liste"
Private Const SP_BLATT As String = "SP_Liste"
Private celonisLog As Object                  ' Celonis_Upload_Log.txt: Datei -> Groesse der uebergebenen Fassung
Private celonisVorhanden As Long              ' lagen schon im Celonis-Ordner (nicht erneut kopiert)
Private celonisFehler As String               ' Hinweis, wenn der Celonis-Ordner nicht erreichbar war
Private celonisAlt As Long                    ' vor „Celonis ab Stichtag“ (Konfiguration!F8), nicht uebergeben
Private celonisSp As Collection               ' SharePoint: Zeilen von Celonis_Upload_Log.txt neben der Master-Datei
   (Nothing = nicht gelesen)
Private celonisSpNeu As Collection            ' SharePoint: in diesem Laden neu uebergebene Dateien (werden dort ange
   haengt)
Private celonisNamen As Object                ' SharePoint: Rohdatei (klein) -> einheitlicher Name JJJJMMTT_BUe_<Quel
   le>… fuer die Celonis-Kopie
Private celonisAlias As Object                ' Rohdatei (klein, wie geladen) -> frueherer Name (vor dem Umbenennen b
   zw. MIS-O-Name) fuer den Abgleich
Private celonisSpFehler As String             ' SharePoint: Fehlertext, wenn das Protokoll nicht gelesen werden konnt
   e
Private Const CELONIS_LOG As String = "Celonis_Upload_Log.txt"
Private Const PROTOKOLL_ORDNER As String = "Protokolle"      ' Textdateien des Tools (Sperrdatei, Celonis-Protokoll, L
   adelog) neben der Master-Datei (12.90)
Private Const SP_TEXT_ABFRAGE As String = "BUE_SharePoint_Text"
Private Const SP_SPERRE_ABFRAGE As String = "BUE_SharePoint_Sperre"   ' Sperrdatei „Beladung läuft“ (Blatt SP_Liste ab
    N1)
Private ersatzImAblauf As Boolean             ' Frage kam aus dem Tagesablauf (danach dort weiter)
Private ersatzLaeuft As Boolean               ' StandErsetzen laeuft: alter Stand schon entfernt, ein Abbruch speiche
   rt nicht
Private ladeOrt As String                     ' aktueller Schritt beim Laden (fuer die Fehlermeldung)
Private ladeAnteil As Double                  ' aktueller Stand des Fortschrittsbalkens (Celonis-Meldung haelt ihn, s
   tatt zu springen)
Private ladeStatus As String                  ' aktuelle Statuszeile (Celonis-Meldung steht darunter)
Private cacheOrdner As String                 ' SharePoint: %TEMP%\BUE_Cache_… mit lokalen Kopien der Rohdateien (die
   ses Laden)
Private cacheDateien As Object                ' https-Adresse (klein) -> lokale Kopie
Private spAblage As Long                      ' SharePoint: Ablage ueber Power Query moeglich? 0 = ungeprueft, 1 = ja
   , -1 = nein (je Laden)
Private spAblageFehler As String              ' SharePoint: warum nicht (fuer Ladelog und Meldung)
Private spSyncGefragt As Boolean
Private rohLokalUrl As String                                ' SharePoint-Adresse des Rohdaten-Ordners, zu der rohLok
   al gefunden wurde
Private rohLokal As String                                   ' lokaler Ordner dazu (OneDrive-Verknuepfung/Sync), mit
   Trenner               ' „Ordner synchronisieren?“ in dieser Sitzung schon gefragt
Private Const SP_ABLAGE_ABFRAGE As String = "BUE_SharePoint_Ablage"     ' Ordner anlegen, Datei verschieben (Blatt SP_
   Liste ab T1)
' Bereich MAKLER (12.91): Abzuege des HS-Reportings (OE HSB4 K) gehoeren nicht ins Makler-Reporting
Private Const HS_EINHEITEN As String = "|HSB|HS|"      ' Einheiten (EinheitAusOE), deren Zeilen der Makler-Master igno
```

```
riert
Private Const HS_ENDUNG As String = "_HS-ignoriert"     ' Name eines reinen HS-Abzugs nach dem Laden (wird nicht mehr gelesen)
Private blattNurHs As Boolean                  ' BlattErkennen: Blatt hat nur Zeilen der HS-Einheit (kein Stand)
Private hsDateien As Collection                ' dieses Laden: reine HS-Abzuege, nicht geladen (Namen fuer die Meldung)
Private hsAusgeblendet As Long                 ' davon im SharePoint-Modus nur ausgeblendet (nicht umbenannt)
Private hsZeilen As Long                       ' dieses Laden: ignorierte HS-Zeilen in Makler-Abzuegen
Private hsNeuaufbau As Boolean                 ' dieses Laden: Tool ohne vorher mitgeladene HS-Zeilen neu aufgebaut
Private doppeltText As String                  ' bereinigte doppelt geladene Staende (12.92), bis zur naechsten Abschlussmeldung

' ---------------------------------------------------------------- Lademodus (Sperre, Zustand sichern und zurueckgeben)
Private Sub LadeModusEin(titel As String)
    If Not lademodusAktiv Then calcVorher = Application.Calculation
    lademodusAktiv = True
    modFortschritt.FortschrittZeigen titel
    Application.ScreenUpdating = False
    Application.EnableEvents = False            ' kein SelectionChange, keine Ereignisse fremder Dateien
    ' Application.Interactive wird bewusst nicht gesetzt: zusammen mit DoEvents und Workbooks.Open kann Excel abstuerzen.
    ' Die Schaltflaechen pruefen stattdessen LadenLaeuft() und tun waehrend des Ladens nichts.
    Application.Calculation = xlCalculationManual
End Sub

Private Sub LadeModusAus()
    On Error Resume Next
    modFortschritt.FortschrittSchliessen
    If calcVorher = 0 Then calcVorher = xlCalculationAutomatic
    Application.Calculation = calcVorher
    Application.EnableEvents = True
    Application.StatusBar = False
    Application.ScreenUpdating = True
    lademodusAktiv = False
    On Error GoTo 0
End Sub

Public Function LadenLaeuft() As Boolean
    LadenLaeuft = lademodusAktiv
End Function

Public Sub LogSchreiben(text As String)
    ' Ladelog.txt neben der Master-Datei: jeder Schritt eine Zeile (Datei wird je Zeile geoeffnet und geschlossen,
    ' damit nach einem Absturz von Excel der letzte Schritt sichtbar ist)
    Dim f As Integer
    On Error Resume Next
    If Len(logPfad) = 0 Then logPfad = LokalerProtokollPfad("Ladelog.txt")
    f = FreeFile
    Open logPfad For Append As #f
    Print #f, Format(Now, "yyyy-mm-dd hh:nn:ss") & "  " & text
    Close #f
    On Error GoTo 0
End Sub

' ---------------------------------------------------------------- Einstieg
Public Sub NeueDatenLaden()
    Dim ordner As String, datei As String, pfad As String
    Dim staende() As Stand, n As Long, i As Long, j As Long, st As Stand
    Dim wbQ As Workbook, ws As Worksheet
    Dim geladen As Long, uebersprungen As Long, meldung As String
    Dim dateien() As String, nDateien As Long, d As Long, protokolliert As Object
    Dim nVorher As Long, neuName As String, k As Long, umbenannt As Long, fehlerUmb As String, abgelegt As Long
    Dim geladenInfo As Object, schluessel As String, zurueckhalten As Boolean, verworfen As Long, alleVerworfen As Boolean
    Dim ersetzt As Long, nachgerechnet As Long, gesendet As Object, celonisNachgeholt As Long, sperre As String
    Dim anwText As String
    Dim nurAblegen As Boolean                                ' SharePoint (12.80): schon geladene Datei, wird nur noch abgelegt
    Dim nichtAbgelegt As Long, ablageFehler As String         ' SharePoint (12.81): Verschieben gescheitert (fuer die Meldung)
    Dim bl() As Stand, blS() As Object, nBl As Long, b As Long, u As Variant, straenge As Object   ' erkannte Blaetter und ihre Einheiten (12.88)
    Dim erledigt() As Boolean, stapel() As Stand, nStapel As Long, imStapel As Object, x As Long, nachtrag As Long
    Dim nHsBl As Long, hsEinheit As String                   ' Blaetter nur mit HS-Zeilen (12.91)
    Dim doppeltNeu As String                                 ' eben bereinigte doppelt geladene Staende (12.92)

    If lademodusAktiv Then Exit Sub
    sperre = BeladungAndererText()                           ' einmal lesen (im SharePoint-Modus ein Abruf der Sperrdatei)
    If Len(sperre) > 0 Then                                  ' laeuft gerade bei jemand anderem: warten statt doppelt laden
        ' im Tagesablauf als Fehler melden: dort folgt „Nochmal laden / Ohne Laden weiter“ statt „erledigt“
        modFortschritt.HinweisZeigen "Beladung läuft", sperre, modTagesablauf.WartetAufLaden()
        Exit Sub
    End If
    If AlteGruppenStraenge() Then
        ' bis 12.87 war im Bereich Makler die Gruppe der Datei der Strang; seit 12.88 ist es die Einheit – alte Staende passen nicht dazu
        modFortschritt.HinweisZeigen "Hinweis", "Seit Version 12.88 vergleicht das Tool jede Einheit einzeln mit ihrem Vortag – " & _
            "auch wenn MIS-O die Einheiten von Tag zu Tag anders auf die Dateien verteilt. Die bisher geladenen Stände sind noch nach " & _
            "Datei-Gruppen gespeichert (z. B. „MSUE+ODPB“)." & vbCrLf & vbCrLf & "Bitte einmal im Cockpit „Zurücksetzen …“ (ohne Anwesenheit) " & _
            "und danach „Neue Daten laden“ klicken: alle Abzüge aus dem Ordner Rohdaten werden neu eingelesen.", True
```

```
218         Exit Sub
219     End If
220     ersatzLaeuft = False
221     Set spListen = Nothing                                  ' SharePoint-Ordner bei jedem Laden frisch auflisten
222     On Error Resume Next
223     ordner = RohdatenOrdner()
224     If Err.Number <> 0 Then
225         modFortschritt.HinweisZeigen "Fehler", Err.Description, True
226         Exit Sub
227     End If
228     On Error GoTo 0
229     Application.StatusBar = IIf(IstUrl(ordner), "SharePoint-Ordner Rohdaten wird aufgelistet …", False)
230     If Not OrdnerVorhanden(ordner) Then
231         Application.StatusBar = False
232         If IstUrl(ordner) Then
233             modFortschritt.HinweisZeigen "Fehler", SpHinweis(ordner), True
234         Else
235             modFortschritt.HinweisZeigen "Fehler", "Der Rohdaten-Ordner wurde nicht gefunden:" & vbCrLf & ordner & vbCrLf & vbCrLf & _
236                     "Bitte den Ordner neben der Master-Datei anlegen oder den Pfad in 'Konfiguration' anpassen.", True
237         End If
238         Exit Sub
239     End If
240
241     ' Bereich MAKLER (12.91): vorher mitgeladene HS-Zeilen (Einheit HSB) in den Fakten – einmal leeren und ohne sie neu aufbauen
242     hsNeuaufbau = False
243     hsEinheit = HsDatenGeladen()
244     If Len(hsEinheit) > 0 Then
245         LogSchreiben "---- HS-Zeilen (Einheit " & hsEinheit & ") in den Fakten: Tool wird ohne sie neu aufgebaut"
246         ToolLeeren False
247         hsNeuaufbau = True
248     End If
249
250     BeladungMarkieren True                                  ' orange Box im Cockpit (auch fuer Kollegen: Datei wird gespeichert)
251     LogSchreiben "---- NeueDatenLaden Start, Ordner " & ordner
252     LadeModusEin "Rohdaten werden geladen"
253     Application.StatusBar = "Rohdaten werden gesucht …"
254     On Error GoTo Fehler
255
256     ' 12.92: bis 12.91 doppelt geladene Staende (Ladeprotokoll fuehrt einen Stand mehrfach) aus Fakten und Protokoll entfernen
257     ladeOrt = "Doppelt geladene Stände bereinigen"
258     doppeltNeu = DoppelteStaendeBereinigen()
259     If Len(doppeltNeu) > 0 Then doppeltText = doppeltNeu
260
261     ' 1. Dateien sammeln: Rohdaten selbst und die Ablage Rohdaten\JJJJ\MM_Monat (relative Pfade), nur echte Excel-Endungen
262     ladeOrt = "Ladeprotokoll lesen"
263     ladeStatus = ""
264     Set protokolliert = ProtokollLesen()
265     Set geladenInfo = GeladenLesen()
266     celonisAnzahl = 0: celonisVorhanden = 0: celonisFehler = "": celonisAlt = 0: Set celonisLog = Nothing
267     Set celonisSp = Nothing: Set celonisSpNeu = New Collection: Set celonisNamen = CreateObject("Scripting.Dictionary"): celonisSpFehler = "": Set celonisAlias = CreateObject("Scripting.Dictionary")
268     Set neuePK = CreateObject("Scripting.Dictionary")
269     Set hsDateien = New Collection: hsAusgeblendet = 0: hsZeilen = 0
270     Set oeEinheit = Nothing                                 ' Zuordnung OE -> Einheit frisch aus der Konfiguration
271     Set gruppenListe = Nothing
272     Set inhaltIndex = Nothing
273     LesenVergessen
274     If ersatzWahl Is Nothing Then Set ersatzWahl = CreateObject("Scripting.Dictionary")
275     Set ersatzOffen = CreateObject("Scripting.Dictionary")
276     ladeOrt = "Dateien im Ordner Rohdaten auflisten"
277     nDateien = DateienSammeln(ordner, dateien)
278     spAblage = 0
279     spAblageFehler = ""
280     ' SharePoint ohne Dateipfad: Excel kann dort nicht verschieben (Power Query schickt POST nur anonym, kein Power Automate) -
281     ' die Dateien bleiben liegen und werden abgelegt, sobald jemand mit OneDrive-Verknuepfung laedt (12.81; SpAblagePruefen bleibt
282     ' fuer Tenants, in denen es gehen sollte, wird aber nicht mehr aufgerufen)
283     If IstUrl(ordner) Then
284         spAblage = -1
285         spAblageFehler = "SharePoint ohne Dateipfad"
286         LogSchreiben "  SharePoint-Modus: Rohdateien bleiben liegen (Ablage durch jemanden mit OneDrive-Verknuepfung)"
287     End If
288     DateienChronologisch ordner, dateien, nDateien          ' aelteste zuerst, damit die Anzeige dem Datum folgt
289     ' 2. Staende erkennen: jede Datei kurz oeffnen und Stichtag und Quelle lesen (stehen im Blatt, nicht sicher im Namen)
290     modFortschritt.FortschrittSchritt "Schritt 1 von 2 · Dateien prüfen (Stichtag, Quelle)"
291     n = 0
292     For d = 0 To nDateien - 1
293         datei = dateien(d)
294         ladeAnteil = 0.05 + 0.25 * d / IIf(nDateien = 0, 1, nDateien)
295         modFortschritt.FortschrittSetzen ladeAnteil, "Datei " & (d + 1) & " von " & nDateien & DatumAusName(datei), DateiName(datei)
296         pfad = PfadVon(ordner, datei)
297         ' Bereits abgelegte Datei, deren Stand laut Ladeprotokoll geladen ist: nicht oeffnen (der Name traegt Stichtag und Quelle)
298         ' Ausnahme: ein Doppel (…_2) zu einem Stand, der aus einer anderen Datei geladen ist – das ist ein neuer Abzug zum selben Tag
299         If InStr(datei, Application.PathSeparator) > 0 And AbgelegtUndGeladen(datei, protokolliert) Then
300             If Not DoppelZuGeladen(datei, geladenInfo) Then
301                 abgelegt = abgelegt + 1
```

```
302                 GoTo NaechsteDatei
303             End If
304         End If
305         ' Im SharePoint-Modus (Kollege ohne OneDrive) ausgeblendet – ersetzt, nicht uebernommen oder „geloescht“: loka
    l wie dort
306         ' behandeln, also umbenennen (…_verworfen) und nicht laden
307         If Not IstUrl(ordner) Then
308             If SpAusgeblendet(datei, DateiGroesse(PfadVon(ordner, datei))) Then
309                 On Error Resume Next
310                 neuName = FreierDateiname(ordner, Left$(datei, InStrRev(datei, ".") - 1) & "_verworfen" & Mid$(datei,
    InStrRev(datei, ".")))
311                 Name ordner & datei As ordner & neuName
312                 If Err.Number = 0 Then LogSchreiben "  ausgeblendet (SharePoint), umbenannt: " & datei & " -> " & neuN
    ame
313                 Err.Clear
314                 On Error GoTo Fehler
315                 verworfen = verworfen + 1
316                 GoTo NaechsteDatei
317             End If
318         End If
319         ' SharePoint: Eine Datei, aus der schon geladen wurde, nicht erneut oeffnen – ausser sie liegt noch unsortiert
     im Ordner
320         ' Rohdaten und die Ablage geht (12.80): dann wird sie erkannt und wie mit Dateipfad abgelegt (der Stand selbst
     ist schon da)
321         nurAblegen = False
322         If IstUrl(ordner) Then
323             If DateiSchonGeladen(datei, ordner) Then
324                 If spAblage = 1 And InStr(datei, Application.PathSeparator) = 0 Then
325                     nurAblegen = True
326                     LogSchreiben "Schon geladen, liegt noch im Ordner Rohdaten - wird abgelegt: " & datei
327                 Else
328                     abgelegt = abgelegt + 1
329                     GoTo NaechsteDatei
330                 End If
331             End If
332         ElseIf InStr(datei, Application.PathSeparator) = 0 Then
333             ' Dateipfad (z. B. OneDrive-Verknuepfung): eine schon geladene Datei, die noch unsortiert im Ordner Rohdat
    en liegt
334             ' (von Kollegen im SharePoint-Modus geladen), nur ablegen – keine Ersatz-Frage, der Stand ist schon da
335             If DateiSchonGeladen(datei, ordner) Then
336                 nurAblegen = True
337                 LogSchreiben "Schon geladen, liegt noch im Ordner Rohdaten - wird abgelegt: " & datei
338             End If
339         End If
340         nVorher = n
341         zurueckhalten = False
342         alleVerworfen = True
343         ladeOrt = "Datei " & datei & " prüfen"
344         ' Schneller Weg: einheitlich benannte Datei aus der Ablage (JJJJMMTT_BUe_<Quelle>[_Einheit][_n]) – Stichtag un
    d Quelle stehen
345         ' im Namen, den das Tool selbst aus dem Inhalt vergeben hat. Die Datei wird erst zum Verarbeiten geoeffnet.
346         If StandAusName(datei, staende, n, geladenInfo) Then
347             alleVerworfen = False
348             LogSchreiben "Stand aus Dateiname (nicht geoeffnet): " & datei
349             GoTo NachErkennung
350         End If
351         ' Inhalt der Datei (Blaetter, Quelle, Stichtag, Einheiten): aus dem Blatt Dateiinhalt, wenn die Datei schon ei
    nmal geprueft
352         ' wurde (gleicher Name, gleiche Groesse) – dann wird sie hier nicht geoeffnet; sonst oeffnen und vermerken
353         nBl = 0
354         nHsBl = 0
355         If InhaltBekannt(ordner, datei, bl, blS, nBl) Then
356             LogSchreiben "Inhalt aus Blatt Dateiinhalt (nicht geoeffnet): " & datei
357         Else
358             LogSchreiben "Pruefe Datei " & datei
359             Set wbQ = QuelleOeffnenRel(ordner, datei)
360             LogSchreiben "  geoeffnet, Blaetter: " & wbQ.Worksheets.Count
361             For Each ws In wbQ.Worksheets
362                 If BlattErkennen(ws, st, straenge) Then
363                     st.Datei = datei
364                     st.Blatt = ws.Name
365                     ReDim Preserve bl(0 To nBl)
366                     ReDim Preserve blS(0 To nBl)
367                     bl(nBl) = st
368                     Set blS(nBl) = straenge
369                     nBl = nBl + 1
370                 ElseIf blattNurHs Then
371                     nHsBl = nHsBl + 1
372                 End If
373             Next ws
374             wbQ.Close SaveChanges:=False
375             Set wbQ = Nothing
376             LogSchreiben "  geschlossen"
377             InhaltMerken ordner, datei, bl, blS, nBl
378         End If
379         ' Bereich MAKLER (12.91): reiner HS-Abzug (alle Zeilen OE HSB4 K) – gehoert ins HS-Reporting, wird nicht gelad
    en
380         If nBl = 0 And nHsBl > 0 Then
381             HsAbzugAusmustern ordner, datei
382             GoTo NaechsteDatei
383         End If
384         ' Bereich MAKLER: je Einheit ein Stand (Strang), auch wenn mehrere Einheiten in einer Datei stehen
385         For b = 0 To nBl - 1
386             LogSchreiben "  erkannt: " & bl(b).Blatt & " = " & Strang(bl(b).Quelle, bl(b).DateiGruppe) & " " & Format(
    bl(b).Stichtag, "dd.mm.yyyy") & _
387                          IIf(blS(b).Count > 1, " (" & blS(b).Count & " Einheiten)", "")
```

```
388             For Each u In blS(b).Keys
389                 st = bl(b)
390                 st.Gruppe = CStr(u)
391                 st.Zeilen = CLng(blS(b).Item(u))
392                 st.Ersetzen = False
393                 ' Neue Datei (im Ordner Rohdaten oder Doppel …_2 in der Ablage) fuer einen Stand, der aus einer andere
    n Datei
394                 ' geladen ist: fragen, dann ersetzen oder verwerfen
395                 schluessel = StandSchluessel(st.Quelle, st.Gruppe, st.Stichtag)
396                 If nurAblegen Then
397                     ' schon geladen (Ladeprotokoll nennt diese Datei): keine Ersatz-Frage – die Entscheidung von damal
    s gilt,
398                     ' die Staende werden in Schritt 4 als vorhanden uebersprungen
399                     alleVerworfen = False
400                 ElseIf NeuerAbzugZuGeladen(datei, schluessel, geladenInfo) Then
401                     If ersatzWahl.Exists(schluessel) Then
402                         st.Ersetzen = ersatzWahl(schluessel)
403                         If st.Ersetzen Then alleVerworfen = False
404                     Else
405                         zurueckhalten = True
406                         If Not ersatzOffen.Exists(schluessel) Then ersatzOffen.Add schluessel, ErsatzDaten(st, geladen
    Info(schluessel))
407                     End If
408                 Else
409                     alleVerworfen = False
410                 End If
411                 ReDim Preserve staende(0 To n)
412                 staende(n) = st
413                 n = n + 1
414             Next u
415         Next b
416 NachErkennung:
417         ' SharePoint: die Datei behaelt ihren Namen; die Celonis-Kopie bekommt trotzdem den einheitlichen Namen wie in
     der Ablage
418         If n > nVorher And IstUrl(ordner) Then
419             celonisNamen(LCase$(datei)) = DateiName(ZielDateiname(staende, nVorher, n - 1, datei))
420             celonisAlias(LCase$(datei)) = DateiName(datei)      ' MIS-O-Name (so stand er bis 12.64 im Protokoll)
421         End If
422         ' Datei einheitlich benennen und in die Ablage JJJJ\MM_Monat verschieben (nur erkannte Dateien). Bei offener E
    rsatz-Frage
423         ' bleibt die Datei liegen, damit sie beim naechsten Laden wieder gefunden wird. Auf SharePoint (https) wird ni
    cht verschoben;
424         ' eine nicht uebernommene Datei wird dort im Tool ausgeblendet (statt …_verworfen).
425         If n > nVorher And Not zurueckhalten And alleVerworfen And IstUrl(ordner) And spAblage <> 1 Then
426             SpAusblenden datei, ordner
427             verworfen = verworfen + 1
428             LogSchreiben "  nicht uebernommen, ausgeblendet: " & datei
429         End If
430         If n > nVorher And Not zurueckhalten And (Not IstUrl(ordner) Or spAblage = 1) Then
431             neuName = ZielDateiname(staende, nVorher, n - 1, datei)
432             If alleVerworfen Then
433                 neuName = Left$(neuName, InStrRev(neuName, ".") - 1) & "_verworfen" & Mid$(neuName, InStrRev(neuName,
    "."))
434                 verworfen = verworfen + 1
435             End If
436             If Not SchonAbgelegt(datei, neuName) Then
437                 fehlerUmb = ""
438                 On Error Resume Next
439                 If Not IstUrl(ordner) Then OrdnerAnlegen ordner, Left$(neuName, InStrRev(neuName, Application.PathSepa
    rator) - 1)
440                 neuName = FreierDateiname(ordner, neuName)
441                 Err.Clear
442                 On Error GoTo Fehler
443                 fehlerUmb = DateiUmbenennen(ordner, datei, neuName)     ' SharePoint: ueber die Schnittstelle (legt di
    e Ordner an)
444                 If Len(fehlerUmb) = 0 Then
445                     For k = nVorher To n - 1
446                         staende(k).Datei = neuName
447                     Next k
448                     celonisAlias(LCase$(neuName)) = DateiName(datei)   ' Originalname (Uebergaben aus dem SharePoint-M
    odus)
449                     CelonisAliasAblegen ordner, DateiName(datei), neuName   ' dauerhaft, falls unter dem Originalnamen
     schon uebergeben
450                     CelonisOffenUmbenennen datei, neuName              ' offene Uebergabe (z. B. eines Kollegen) folgt
     der Datei
451                     If ProtokollDateiUmschreiben(datei, neuName) > 0 Then Set geladenInfo = GeladenLesen()   ' schon g
    eladen: Protokoll folgt
452                     umbenannt = umbenannt + 1
453                     LogSchreiben "  abgelegt: " & datei & " -> " & neuName
454                 Else
455                     LogSchreiben "  nicht abgelegt (" & fehlerUmb & "): " & datei
456                     nichtAbgelegt = nichtAbgelegt + 1
457                     If Len(ablageFehler) = 0 Then ablageFehler = fehlerUmb
458                     If IstUrl(ordner) And alleVerworfen Then        ' SharePoint: nicht verschoben – wie ohne Ablage a
    usblenden
459                         SpAusblenden datei, ordner
460                         LogSchreiben "  nicht uebernommen, ausgeblendet: " & datei
461                     End If
462                 End If
463             End If
464         End If
465 NaechsteDatei:
466     Next d
467
468     ' Offene Ersatz-Frage: nichts verarbeiten, Karte zeigen; die Antwort startet das Laden erneut
469     If ersatzOffen.Count > 0 Then
```

```
470          LogSchreiben "Ersatz-Frage: " & ersatzOffen.Count & " Stand/Staende schon geladen"
471          CacheAufraeumen
472          BeladungMarkieren False
473          LadeModusAus
474          ersatzImAblauf = modTagesablauf.WartetAufLaden()
475          On Error Resume Next
476          Application.OnTime Now + TimeSerial(0, 0, 1), "'" & ThisWorkbook.Name & "'!modTagesablauf.ErsatzFrageZeigen"
477          On Error GoTo 0
478          Exit Sub
479      End If
480
481      If n = 0 Then
482          ' kein neuer Abzug: die Anwesenheit (Plan) trotzdem auffrischen, damit MAK und Produktivitaet den heutigen Pla
         n zeigen
483          anwText = AnwesenheitImLaden()
484          If Len(anwText) > 0 Or Len(doppeltText) > 0 Then
485              ladeOrt = "Kennzahlen und Diagramme neu berechnen"
486              Application.Calculation = xlCalculationAutomatic
487              Application.Calculate
488              modCockpit.DiagrammeAnpassen True
489          End If
490          CacheAufraeumen
491          BeladungMarkieren False
492          LadeModusAus
493          If abgelegt + verworfen = 0 And hsDateien.Count > 0 Then
494              meldung = "Keine neuen Makler-Abzüge"
495          ElseIf abgelegt + verworfen > 0 Then
496              meldung = "Keine neuen Abzüge – alles im Ordner ist schon geladen" & IIf(verworfen > 0, " (" & verworfen &
          " ausgeblendet)", "")
497          Else
498              meldung = "Im Ordner " & ordner & " wurde kein Rückstandsabzug erkannt." & vbCrLf & _
499                        "Erwartet werden Blätter mit 'Rückstandsstichtag = …' in den ersten Zeilen und einer Kopfzeile m
         it 'Themenpostkorb'"
500          End If
501          If Len(HsMeldung()) > 0 Then meldung = meldung & "." & vbCrLf & HsMeldung()
502          meldung = meldung & DoppeltMeldung()
503          If Len(anwText) > 0 And Not modTagesablauf.WartetAufLaden() Then meldung = meldung & "." & vbCrLf & anwText
504          modFortschritt.HinweisZeigen "Hinweis", meldung & ".", False
505          Exit Sub
506      End If
507
508      ' 3. sortieren nach Stichtag, dann Quelle (und Einheit), dann Datei absteigend: liegt ein Stand zweimal vor (Grund
         datei und Doppel …_2,
509      '    etwa beim ersten Laden aus der fertigen Ablage), wird das Doppel als aktuellerer Abzug geladen, die Grunddate
         i uebersprungen
510      For i = 0 To n - 2
511          For j = i + 1 To n - 1
512              If staende(j).Stichtag < staende(i).Stichtag Or _
513                 (staende(j).Stichtag = staende(i).Stichtag And Strang(staende(j).Quelle, staende(j).Gruppe) < Strang(st
         aende(i).Quelle, staende(i).Gruppe)) Or _
514                 (staende(j).Stichtag = staende(i).Stichtag And Strang(staende(j).Quelle, staende(j).Gruppe) = Strang(st
         aende(i).Quelle, staende(i).Gruppe) And _
515                  StrComp(staende(j).Datei, staende(i).Datei, vbTextCompare) > 0) Then
516                  st = staende(i): staende(i) = staende(j): staende(j) = st
517              End If
518          Next j
519      Next i
520
521      ' 4. verarbeiten (Protokoll einmal lesen statt je Stand zu suchen). Jede Datei, aus der ein Stand geladen oder ers
         etzt wird,
522      '    geht einmal als Kopie an Celonis – auch beim ersten Laden einer leeren Datei aus der fertigen Ablage.
523      '    Streng nach Stichtag: Eingaenge und Abgaenge vergleichen jeden Stand mit dem vorigen derselben Quelle.
524      modFortschritt.FortschrittSchritt "Schritt 2 von 2 · Stände nach Datum verarbeiten"
525      Set protokolliert = ProtokollLesen()
526      Set gesendet = CreateObject("Scripting.Dictionary")
527      ladeOrt = "offene Celonis-Übergaben nachholen"
528      celonisNachgeholt = CelonisOffenNachholen(ordner)
529      ReDim erledigt(0 To n - 1)
530      For i = 0 To n - 1
531          If erledigt(i) Then GoTo NaechsterStand
532          schluessel = StandSchluessel(staende(i).Quelle, staende(i).Gruppe, staende(i).Stichtag)
533          If staende(i).Ersetzen And geladenInfo.Exists(schluessel) Then
534              ladeAnteil = 0.3 + 0.6 * i / n
535              ladeStatus = "Stand " & (i + 1) & " von " & n & ": " & Format$(staende(i).Stichtag, "dd.mm.yyyy") & " · "
         & Strang(staende(i).Quelle, staende(i).Gruppe) & " ersetzen"
536              modFortschritt.FortschrittSetzen ladeAnteil, ladeStatus, DateiName(staende(i).Datei)
537              LogSchreiben "Ersetze " & Strang(staende(i).Quelle, staende(i).Gruppe) & " " & Format(staende(i).Stichtag,
          "dd.mm.yyyy") & " durch " & staende(i).Datei
538              ladeOrt = "Stand " & Strang(staende(i).Quelle, staende(i).Gruppe) & " " & Format(staende(i).Stichtag, "dd.
         mm.yyyy") & " aus " & staende(i).Datei & " ersetzen"
539              ersatzLaeuft = True                                ' ab hier ist der alte Stand weg: ein Abbruch darf nic
         ht gespeichert werden
540              nachgerechnet = nachgerechnet + StandErsetzen(ordner, staende(i), geladenInfo, False)
541              ersatzLaeuft = False
542              ersetzt = ersetzt + 1
543              CelonisEinmal ordner, staende(i).Datei, gesendet, True, 0       ' Ersatz ueberschreibt die alte Kopie
544              If Not protokolliert.Exists(schluessel) Then protokolliert.Add schluessel, True
545              Set geladenInfo = GeladenLesen()
546          ElseIf protokolliert.Exists(schluessel) Then
547              uebersprungen = uebersprungen + 1
548          ElseIf staende(i).Stichtag < LetzterStand(staende(i).Quelle, staende(i).Gruppe) Then
549              ' Nachzuegler: dieser Stichtag kommt fuer diese Einheit erst nach spaeteren Tagen. Bereich MAKLER: einfueg
         en und die
550              ' spaeteren Staende der Einheit aus ihren Dateien neu rechnen (wie beim Ersetzen). Fehlt eine dieser Datei
         en, oder im
551              ' Bereich HS: uebersprungen wie bisher.
```

```
552             nachtrag = -1
553             If IstMakler() Then
554                 ladeAnteil = 0.3 + 0.6 * i / n
555                 ladeStatus = "Stand " & (i + 1) & " von " & n & ": " & Format$(staende(i).Stichtag, "dd.mm.yyyy") & "
        · " & Strang(staende(i).Quelle, staende(i).Gruppe) & " nachtragen"
556                 modFortschritt.FortschrittSetzen ladeAnteil, ladeStatus, DateiName(staende(i).Datei)
557                 LogSchreiben "Trage nach " & Strang(staende(i).Quelle, staende(i).Gruppe) & " " & Format(staende(i).St
    ichtag, "dd.mm.yyyy") & " aus " & staende(i).Datei
558                 ladeOrt = "Stand " & Strang(staende(i).Quelle, staende(i).Gruppe) & " " & Format(staende(i).Stichtag,
    "dd.mm.yyyy") & " aus " & staende(i).Datei & " nachtragen"
559                 ersatzLaeuft = True                             ' spaetere Staende der Einheit werden entfernt und neu
     gerechnet
560                 nachtrag = StandErsetzen(ordner, staende(i), geladenInfo, True)
561                 ersatzLaeuft = False
562             End If
563             If nachtrag >= 0 Then
564                 nachgerechnet = nachgerechnet + nachtrag
565                 geladen = geladen + 1
566                 CelonisEinmal ordner, staende(i).Datei, gesendet, False, staende(i).Stichtag
567                 Set geladenInfo = GeladenLesen()
568             Else
569                 Protokoll staende(i), 0, 0, 0, 0, "uebersprungen: Stichtag liegt vor dem zuletzt geladenen Stand (" & F
    ormat(LetzterStand(staende(i).Quelle, staende(i).Gruppe), "dd.mm.yyyy") & ")"
570                 uebersprungen = uebersprungen + 1
571             End If
572             protokolliert.Add schluessel, True
573         Else
574             ' alle normalen Staende dieser Quelle und dieses Stichtags gemeinsam (je Einheit ein Strang, auch aus mehr
    eren Dateien):
575             ' wechselt ein Vorgang zwischen zwei dieser Einheiten, ist das eine Umbuchung statt Abgang und Eingang
576             Set imStapel = CreateObject("Scripting.Dictionary")
577             nStapel = 0
578             For x = i To n - 1
579                 If staende(x).Quelle <> staende(i).Quelle Or CLng(staende(x).Stichtag) <> CLng(staende(i).Stichtag) Th
    en Exit For
580                 If Not erledigt(x) And Not imStapel.Exists(staende(x).Gruppe) Then
581                     schluessel = StandSchluessel(staende(x).Quelle, staende(x).Gruppe, staende(x).Stichtag)
582                     If Not (staende(x).Ersetzen And geladenInfo.Exists(schluessel)) And Not protokolliert.Exists(schlu
    essel) Then
583                         If Not staende(x).Stichtag < LetzterStand(staende(x).Quelle, staende(x).Gruppe) Then
584                             ReDim Preserve stapel(0 To nStapel)
585                             stapel(nStapel) = staende(x)
586                             nStapel = nStapel + 1
587                             imStapel.Add staende(x).Gruppe, True
588                             erledigt(x) = True
589                         End If
590                     End If
591                 End If
592             Next x
593             Application.StatusBar = "Verarbeite " & staende(i).Quelle & " " & Format(staende(i).Stichtag, "dd.mm.yyyy"
    ) & " …"
594             ladeAnteil = 0.3 + 0.6 * i / n
595             ladeStatus = "Stand " & (i + 1) & " von " & n & ": " & Format$(staende(i).Stichtag, "dd.mm.yyyy") & " · "
    & Strang(staende(i).Quelle, StapelText(stapel, nStapel))
596             modFortschritt.FortschrittSetzen ladeAnteil, ladeStatus, DateiName(staende(i).Datei)
597             LogSchreiben "Stand " & Strang(staende(i).Quelle, StapelText(stapel, nStapel)) & " " & Format(staende(i).S
    tichtag, "dd.mm.yyyy") & " aus " & staende(i).Datei & " / " & staende(i).Blatt
598             ladeOrt = "Stand " & Strang(staende(i).Quelle, StapelText(stapel, nStapel)) & " " & Format(staende(i).Stic
    htag, "dd.mm.yyyy") & " aus " & staende(i).Datei & " / " & staende(i).Blatt
599             StaendeVerarbeiten ordner, stapel, nStapel
600             LogSchreiben "  Stand fertig"
601             For x = 0 To nStapel - 1
602                 ladeOrt = "Celonis-Kopie von " & stapel(x).Datei
603                 CelonisEinmal ordner, stapel(x).Datei, gesendet, False, stapel(x).Stichtag
604                 schluessel = StandSchluessel(stapel(x).Quelle, stapel(x).Gruppe, stapel(x).Stichtag)
605                 If Not protokolliert.Exists(schluessel) Then protokolliert.Add schluessel, True
606                 geladen = geladen + 1
607             Next x
608         End If
609 NaechsterStand:
610     Next i
611
612     ' 4b. Anwesenheit (erster Import oder Anwesenheitsplan) vor der Neuberechnung und dem Speichern: MAK und Produktiv
    itaet
613     '     werden so mit dem heutigen Plan gerechnet und gespeichert
614     anwText = AnwesenheitImLaden()
615
616     ' 5. neu berechnen
617     ladeOrt = "Kennzahlen und Diagramme neu berechnen"
618     modFortschritt.FortschrittSchritt "Abschluss"
619     modFortschritt.FortschrittSetzen 0.92, "Kennzahlen und Diagramme werden neu berechnet"
620     LogSchreiben "Neuberechnung"
621     QuelleBewegungenWaehlen
622     EinheitenNamenSetzen
623     Application.Calculation = xlCalculationAutomatic
624     Application.Calculate
625     If geladen + ersetzt > 0 Then                       ' Chips im Cockpit: groesste Postkoerbe der geladenen Daten zue
    rst
626         modCockpit.PostkoerbeSortieren
627         Application.Calculate
628     End If
629     LogSchreiben "Diagramme anpassen"
630     modCockpit.DiagrammeAnpassen True
631     ladeOrt = "Celonis-Protokoll auf SharePoint fortschreiben"
632     fehlerUmb = CelonisSpZurueckschreiben()
633     If Len(fehlerUmb) > 0 Then
```

```
634          If InStr(fehlerUmb, CELONIS_LOG) = 0 Then fehlerUmb = "Protokoll " & CELONIS_LOG & " auf SharePoint nicht fort
       geschrieben (" & fehlerUmb & ")"
635          CelonisFehlerMerken fehlerUmb & " – die Einträge stehen lokal und werden beim nächsten Laden nachgetragen"
636      End If
637      CacheAufraeumen
638      LogSchreiben "Fertig: " & geladen & " geladen, " & ersetzt & " ersetzt, " & uebersprungen & " uebersprungen"
639      modFortschritt.FortschrittSetzen 0.98, "Datei speichern (orange Box für Kollegen ausblenden)"
640      BeladungMarkieren False                              ' entfernt zuerst das Ladefenster, dann wird gespeichert
641      LadeModusAus
642      ersatzWahl.RemoveAll
643      If geladen = 0 And ersetzt = 0 Then
644          meldung = "Keine neuen Stände geladen"
645      ElseIf geladen = 0 Then
646          meldung = StaendeText(ersetzt) & " ersetzt"
647      Else
648          meldung = StaendeText(geladen) & " geladen"
649          If ersetzt > 0 Then meldung = meldung & ", " & StaendeText(ersetzt) & " ersetzt"
650      End If
651      If nachgerechnet > 0 Then meldung = meldung & ", " & StaendeText(nachgerechnet) & " danach neu gerechnet"
652      If celonisAnzahl > 0 Then meldung = meldung & ", " & IIf(celonisAnzahl = 1, "1 Datei", celonisAnzahl & " Dateien")
       & " an Celonis übergeben" & _
653          IIf(celonisNachgeholt > 0, " (davon " & celonisNachgeholt & " nachgeholt)", "")
654      If celonisVorhanden > 0 Then meldung = meldung & " (" & IIf(celonisVorhanden = 1, "1 Datei war", celonisVorhanden
       & " Dateien waren") & " schon übergeben)"
655      If celonisAlt > 0 Then meldung = meldung & ", " & IIf(celonisAlt = 1, "1 Datei", celonisAlt & " Dateien") & " vor
       dem " & Format(CelonisAb(), "dd.mm.yyyy") & " nicht an Celonis (Konfiguration F8)"
656      If Len(celonisFehler) > 0 Then meldung = meldung & ". Celonis: " & celonisFehler
657      If NeuePostkoerbeAnzahl() > 0 Then meldung = meldung & ". Neue Themenpostkörbe (nicht HS) zählen zum Standard: " &
       NeuePostkoerbeText() & _
658          " – „Standard festlegen …“ nimmt einzelne heraus"
659      If verworfen > 0 Then meldung = meldung & ", " & IIf(verworfen = 1, "1 neue Datei", verworfen & " neue Dateien") &
       " nicht übernommen (alter Stand bleibt)"
660      If uebersprungen > 0 Then meldung = meldung & ", " & StaendeText(uebersprungen) & " bereits vorhanden oder überspr
       ungen"
661      If umbenannt > 0 Then meldung = meldung & ", " & IIf(umbenannt = 1, "1 Rohdatei", umbenannt & " Rohdateien") & " u
       mbenannt und in JJJJ\MM_Monat abgelegt"
662      If abgelegt > 0 Then meldung = meldung & ", " & IIf(abgelegt = 1, "1 abgelegte Datei", abgelegt & " abgelegte Date
       ien") & " bereits geladen"
663      If nichtAbgelegt > 0 Then
664          meldung = meldung & "." & vbCrLf & IIf(nichtAbgelegt = 1, "1 Rohdatei", nichtAbgelegt & " Rohdateien") & _
665                    " konnte" & IIf(nichtAbgelegt = 1, "", "n") & " nicht in JJJJ\MM_Monat abgelegt werden (" & Left$(ab
       lageFehler, 160) & "). Abhilfe (einmal je Person): auf " & _
666                    "SharePoint beim Ordner der Master-Datei „Verknüpfung zu OneDrive hinzufügen“ klicken"
667      ElseIf IstUrl(ordner) And spAblage = 1 And umbenannt > 0 Then
668          meldung = meldung & " (über SharePoint)"
669      End If
670      If IstUrl(ordner) And spAblage = -1 Then
671          k = SpUnsortiert(ordner)
672          If k > 0 Then
673              meldung = meldung & "." & vbCrLf & IIf(k = 1, "1 Rohdatei liegt", k & " Rohdateien liegen") & " noch unsor
       tiert im Ordner " & _
674                        "Rohdaten – sie werden umbenannt und abgelegt, sobald jemand lädt, bei dem der Ordner über OneDr
       ive auf dem Rechner liegt " & _
675                        "(„Verknüpfung zu OneDrive hinzufügen“ auf „BÜ - HS“ oder „Rohdaten“, OneDrive-Client angemeldet
       und synchronisiert)"
676          End If
677      End If
678      If hsNeuaufbau Then meldung = meldung & "." & vbCrLf & "Neu aufgebaut ohne HS-Daten: In den geladenen Ständen stan
       den Zeilen der HS-Einheit " & _
679          "(OE HSB4 K) aus HS-Abzügen, die im Ordner Rohdaten lagen. Seit 12.91 ignoriert das Makler-Reporting sie. Die
       Auswahl der " & _
680          "Standard-Postkörbe bitte einmal prüfen („Standard festlegen …“)"
681      If Len(HsMeldung()) > 0 Then meldung = meldung & "." & vbCrLf & HsMeldung()
682      meldung = meldung & DoppeltMeldung()
683      ' Ergebnis der Anwesenheit (4b) in die gemeinsame Meldung – im Tagesablauf zeigt sie Schritt 2 selbst
684      If Len(anwText) > 0 And Not modTagesablauf.WartetAufLaden() Then meldung = meldung & "." & vbCrLf & anwText
685      modFortschritt.HinweisZeigen "Fertig", meldung & ".", False
686      Exit Sub
687
688  Fehler:
689      ' Beschreibung zuerst sichern: jedes On Error in einer aufgerufenen Prozedur leert das Err-Objekt
690      Dim fehlertext As String
691      fehlertext = "Fehler " & Err.Number & ": " & Err.Description
692      If Len(ladeOrt) > 0 Then fehlertext = fehlertext & vbCrLf & "Schritt: " & ladeOrt
693      On Error Resume Next
694      LogSchreiben "FEHLER " & fehlertext
695      fehlertext = fehlertext & vbCrLf & "Protokoll: " & logPfad
696      If Not wbQ Is Nothing Then wbQ.Close SaveChanges:=False
697      Dim spText As String
698      spText = CelonisSpZurueckschreiben()                      ' schon uebergebene Dateien trotz Abbruch vermerken
699      If Len(spText) > 0 Then fehlertext = fehlertext & vbCrLf & "Celonis: " & spText
700      CacheAufraeumen
701      If Not ersatzWahl Is Nothing Then ersatzWahl.RemoveAll
702      VerlaufGrenzenNeu
703      Application.Calculation = xlCalculationAutomatic
704      Application.Calculate
705      Dim zusatz As String
706      zusatz = "Bis dahin geladen: " & StaendeText(geladen) & ", übersprungen: " & StaendeText(uebersprungen) & ". Berei
       ts geladene Stände bleiben erhalten."
707      If ersatzLaeuft Then
708          ' Abbruch mitten im Ersetzen: alter Stand entfernt, neuer nur teilweise gerechnet. Nicht speichern – sonst bli
       ebe die Luecke
709          ' dauerhaft (im SharePoint-Modus wird die Datei danach nicht mehr geoeffnet). Sperre und Ladefenster werden tr
       otzdem aufgehoben.
710          ersatzLaeuft = False
711          BeladungMarkieren False, False
```

```
712         zusatz = "Der Abbruch kam mitten im Ersetzen eines Stands (der alte Stand war schon entfernt). Die Datei wurde
    deshalb nicht " & _
713                 "gespeichert: bitte schließen, ohne zu speichern, neu öffnen und erneut laden."
714     Else
715         BeladungMarkieren False
716     End If
717     LadeModusAus
718     modFortschritt.HinweisZeigen "Fehler", "Fehler beim Laden: " & fehlertext & vbCrLf & vbCrLf & zusatz, True
719 End Sub
720
721 ' "1 Stand" / "3 Stände" – Zaehlwort mit korrektem Singular und Plural
722 Private Function StaendeText(n As Long) As String
723     If n = 1 Then
724         StaendeText = "1 Stand"
725     Else
726         StaendeText = n & " Stände"
727     End If
728 End Function
729
730 ' ---------------------------------------------------------------- Einheitliche Dateinamen und Ablage
731 ' Ein Stand: 2026\09_September\20260923_BUe_OPAG.xlsx; Sammeldatei: 2026\09_September\20260921-20260923_BUe_OPAG_Verbu
    nd.xlsx;
732 ' mit Einheit (MAKLER): 20260923_BUe_Verbund_ODPB-D.xlsx
733 ' (relativer Pfad unterhalb des Rohdaten-Ordners; der Monatsordner richtet sich nach dem ersten Stichtag der Datei)
734 Private Function ZielDateiname(staende() As Stand, von As Long, bis As Long, datei As String) As String
735     Dim i As Long, minD As Date, maxD As Date, opag As Boolean, verbund As Boolean, teil As String, quelle As String,
    einheit As String
736     minD = staende(von).Stichtag: maxD = minD
737     einheit = staende(von).DateiGruppe
738     For i = von To bis
739         If staende(i).Stichtag < minD Then minD = staende(i).Stichtag
740         If staende(i).Stichtag > maxD Then maxD = staende(i).Stichtag
741         If staende(i).Quelle = "OPAG" Then opag = True Else verbund = True
742         If staende(i).DateiGruppe <> einheit Then einheit = ""  ' Blaetter mit verschiedenen Einheiten: Name ohne Einh
    eit
743     Next i
744     teil = Format$(minD, "yyyymmdd")
745     If maxD > minD Then teil = teil & "-" & Format$(maxD, "yyyymmdd")
746     If opag And verbund Then
747         quelle = "OPAG_Verbund"
748     ElseIf opag Then
749         quelle = "OPAG"
750     Else
751         quelle = "Verbund"
752     End If
753     If Len(einheit) > 0 Then quelle = quelle & "_" & EinheitDateiteil(einheit)
754     ZielDateiname = MonatsOrdner(minD) & Application.PathSeparator & teil & "_BUe_" & quelle & LCase$(Mid$(datei, InSt
    rRev(datei, ".")))
755 End Function
756
757 Private Function MonatsOrdner(d As Date) As String
758     ' 2026\09_September – Monatsnamen fest auf Deutsch (unabhaengig von der Sprache von Windows/Office)
759     Dim namen As Variant
760     namen = Array("Januar", "Februar", "März", "April", "Mai", "Juni", "Juli", "August", "September", "Oktober", "Nove
    mber", "Dezember")
761     MonatsOrdner = CStr(Year(d)) & Application.PathSeparator & Format$(Month(d), "00") & "_" & namen(Month(d) - 1)
762 End Function
763
764 Private Function SchonAbgelegt(datei As String, ziel As String) As Boolean
765     ' Liegt die Datei schon unter dem Zielnamen – auch als Doppel mit _2, _3 …? Dann nicht erneut verschieben.
766     Dim basis As String, endung As String, rest As String
767     If StrComp(datei, ziel, vbTextCompare) = 0 Then SchonAbgelegt = True: Exit Function
768     basis = Left$(ziel, InStrRev(ziel, ".") - 1)
769     endung = Mid$(ziel, InStrRev(ziel, "."))
770     If StrComp(Left$(datei, Len(basis) + 1), basis & "_", vbTextCompare) <> 0 Then Exit Function
771     If StrComp(Right$(datei, Len(endung)), endung, vbTextCompare) <> 0 Then Exit Function
772     rest = Mid$(datei, Len(basis) + 2, Len(datei) - Len(basis) - 1 - Len(endung))
773     SchonAbgelegt = (Len(rest) > 0 And rest Like String(Len(rest), "#"))
774 End Function
775
776 Private Sub OrdnerAnlegen(ordner As String, rel As String)
777     ' legt rel (z. B. 2026\09_September) unterhalb von ordner Stufe fuer Stufe an
778     Dim teile As Variant, i As Long, pfad As String
779     teile = Split(rel, Application.PathSeparator)
780     pfad = ordner
781     For i = LBound(teile) To UBound(teile)
782         pfad = pfad & teile(i) & Application.PathSeparator
783         If Not OrdnerVorhanden(pfad) Then MkDir Left$(pfad, Len(pfad) - 1)
784     Next i
785 End Sub
786
787 Private Function DateienSammeln(ordner As String, dateien() As String) As Long
788     ' Excel-Dateien im Rohdaten-Ordner und in der Ablage JJJJ\MM_Monat als relative Pfade (Dir nicht verschachtelt: je
    Ebene
789     ' erst vollstaendig lesen, dann absteigen). Andere Unterordner (z. B. Archiv) bleiben aussen vor.
790     Dim n As Long, sep As String, jahre As Collection, monate As Collection, j As Variant, m As Variant, eintrag As St
    ring, rel As String
791     Dim liste As Object, k As Variant, t() As String, nm As String, ok As Boolean
792     sep = Application.PathSeparator
793     n = 0
794     If IstUrl(ordner) Then                                  ' SharePoint: aus der Power-Query-Auflistung (gleiche Reg
    eln wie unten)
795         Set liste = SpListe(ordner)
796         If liste Is Nothing Then Err.Raise vbObjectError + 13, , SpHinweis(ordner)
797         For Each k In liste.Keys
798             rel = CStr(liste(k)(0))
```

```
            t = Split(rel, sep)
            nm = t(UBound(t))
            ' VBA wertet And/Or immer ganz aus: t(1) erst pruefen, wenn es die Ebene gibt (sonst Laufzeitfehler 9)
            ok = (UBound(t) = 0)
            If UBound(t) = 2 Then ok = (t(0) Like "####" And t(1) Like "##_*")
            If ok Then
                If Left$(nm, 2) <> "~$" And InStr(nm, ".") > 0 Then
                    If IstExcelDatei(nm) Then
                        If Not AusgemustertName(nm) And Not SpAusgeblendet(rel, ZahlOder(liste(k)(2), -1)) Then
                            ReDim Preserve dateien(0 To n)
                            dateien(n) = rel
                            n = n + 1
                        End If
                    End If
                End If
            End If
        Next k
        DateienSammeln = n
        Exit Function
    End If
    n = DateienEinesOrdners(ordner, "", dateien, n)
    Set jahre = New Collection
    eintrag = Dir(ordner & "*", vbDirectory)
    Do While Len(eintrag) > 0
        If eintrag Like "####" Then
            If (GetAttr(ordner & eintrag) And vbDirectory) = vbDirectory Then jahre.Add eintrag
        End If
        eintrag = Dir()
    Loop
    For Each j In jahre
        Set monate = New Collection
        eintrag = Dir(ordner & j & sep & "*", vbDirectory)
        Do While Len(eintrag) > 0
            If eintrag Like "##_*" Then
                If (GetAttr(ordner & j & sep & eintrag) And vbDirectory) = vbDirectory Then monate.Add eintrag
            End If
            eintrag = Dir()
        Loop
        For Each m In monate
            rel = j & sep & m & sep
            n = DateienEinesOrdners(ordner, rel, dateien, n)
        Next m
    Next j
    DateienSammeln = n
End Function

' Dateien aelteste zuerst: Datum am Anfang des Namens (JJJJMMTT_… wie in der Ablage und in den MIS-O-Abzuegen), sonst das
' Aenderungsdatum (SharePoint-Auflistung bzw. Dateisystem); gleiches Datum nach Name (OPAG vor Verbund). Nur fuer die Reihenfolge
' von Schritt 1 – verarbeitet wird ohnehin nach dem Stichtag im Blatt.
Private Sub DateienChronologisch(ordner As String, dateien() As String, n As Long)
    Dim schl() As String, i As Long, j As Long, t As String, k As String
    If n < 2 Then Exit Sub
    ReDim schl(0 To n - 1)
    For i = 0 To n - 1
        schl(i) = DateiSortSchluessel(ordner, dateien(i))
    Next i
    For i = 1 To n - 1                                      ' Einfuegesortierung (wenige hundert Dateien)
        t = dateien(i): k = schl(i)
        j = i - 1
        Do While j >= 0
            If StrComp(schl(j), k, vbTextCompare) <= 0 Then Exit Do
            dateien(j + 1) = dateien(j): schl(j + 1) = schl(j)
            j = j - 1
        Loop
        dateien(j + 1) = t: schl(j + 1) = k
    Next i
End Sub

' " · 03.10.2026", wenn der Dateiname mit JJJJMMTT beginnt (sonst leer) – fuer die Statuszeile von Schritt 1
Private Function DatumAusName(rel As String) As String
    Dim nm As String
    nm = DateiName(rel)
    If nm Like "20######*" Then
        If Mid$(nm, 5, 2) Like "[01]#" And Mid$(nm, 7, 2) Like "[0-3]#" Then DatumAusName = " · " & Mid$(nm, 7, 2) & "." & Mid$(nm, 5, 2) & "." & Left$(nm, 4)
    End If
End Function

Private Function DateiSortSchluessel(ordner As String, rel As String) As String
    Dim nm As String, d As Double, e As Variant
    nm = DateiName(rel)
    If nm Like "20######*" Then
        If Mid$(nm, 5, 2) Like "[01]#" And Mid$(nm, 7, 2) Like "[0-3]#" Then DateiSortSchluessel = Left$(nm, 8) & "|" & LCase$(nm): Exit Function
    End If
    On Error Resume Next
    If IstUrl(ordner) Then
        e = SpEintrag(PfadVon(ordner, rel))
        If IsArray(e) Then d = ZeitWert(e(1))
    Else
        d = CDbl(FileDateTime(ordner & rel))
    End If
    On Error GoTo 0
    If d > 0 Then
```

```
891          DateiSortSchluessel = Format$(CDate(d), "yyyymmdd") & "|" & LCase$(nm)
892      Else
893          DateiSortSchluessel = "99999999|" & LCase$(nm)
894      End If
895  End Function
896
897  Private Function DateienEinesOrdners(ordner As String, rel As String, dateien() As String, n As Long) As Long
898      Dim datei As String
899      datei = Dir(ordner & rel & "*.xls*")
900      Do While Len(datei) > 0
901          If Left$(datei, 2) <> "~$" And IstExcelDatei(datei) And Not AusgemustertName(datei) Then
902              ReDim Preserve dateien(0 To n)
903              dateien(n) = rel & datei
904              n = n + 1
905          End If
906          datei = Dir()
907      Loop
908      DateienEinesOrdners = n
909  End Function
910
911  Private Function AbgelegtUndGeladen(rel As String, protokolliert As Object) As Boolean
912      ' Abgelegte Datei JJJJMMTT_BUe_<Quelle>[_n].xlsx: Stand aus dem Namen; True, wenn er im Ladeprotokoll steht.
913      ' Sammeldateien (JJJJMMTT-JJJJMMTT_…) werden geoeffnet, weil der Name nicht jeden Stichtag nennt.
914      ' Vergleich ueber die Namensschluessel "@Quelle[_Einheit]|Tag", die ProtokollLesen zu jedem Stand mitfuehrt
915      Dim tag As Date, quellen As Variant, teil As String, q As Variant, alle As Boolean
916      If Not NameZerlegen(rel, tag, quellen, teil) Then Exit Function
917      alle = True
918      For Each q In quellen
919          If Not protokolliert.Exists(NamensSchluessel(CStr(q), teil, tag)) Then alle = False
920      Next q
921      AbgelegtUndGeladen = alle
922      Exit Function
923  Nein:
924      AbgelegtUndGeladen = False
925  End Function
926
927  ' Ist der Name schon vergeben (zweiter Abzug zum selben Stand), wird _2, _3 … angehaengt
928  Private Function FreierDateiname(ordner As String, wunsch As String) As String
929      Dim basis As String, endung As String, i As Long, kand As String
930      basis = Left$(wunsch, InStrRev(wunsch, ".") - 1)
931      endung = Mid$(wunsch, InStrRev(wunsch, "."))
932      kand = wunsch
933      i = 1
934      Do While DateiVorhanden(ordner & kand)
935          i = i + 1
936          kand = basis & "_" & i & endung
937      Loop
938      FreierDateiname = kand
939  End Function
940
941  ' ---------------------------------------------------------------- Fremde Datei sicher oeffnen
942  Private Function IstExcelDatei(datei As String) As Boolean
943      Select Case LCase$(Mid$(datei, InStrRev(datei, ".") + 1))
944          Case "xlsx", "xlsm", "xlsb", "xls": IstExcelDatei = True
945      End Select
946  End Function
947
948  ' Stand(e) aus einem einheitlichen Dateinamen (siehe NameZerlegen). Nur Einzeldateien; Sammeldateien (JJJJMMTT-JJJJMMT
       T_…) und
949  ' Staende, die schon geladen sind (dann greift die Ersatz-Logik mit dem Inhalt der Datei), werden geoeffnet, im Bereic
       h MAKLER
950  ' jede Datei (die Gruppe ergibt sich aus dem Inhalt, siehe unten).
951  Private Function StandAusName(rel As String, staende() As Stand, n As Long, geladenInfo As Object) As Boolean
952      Dim tag As Date, quellen As Variant, teil As String, q As Variant, gruppe As String, st As Stand
953      ' Bereich MAKLER: immer oeffnen. Die Gruppe im Namen stammt aus der Zuordnung OE -> Einheit beim Ablegen; nach ein
       er Aenderung
954      ' (Konfiguration!Q11:R40) und Neuaufbau faende StandDatenLesen kein Blatt dieser Gruppe mehr und das Laden braeche
       ab.
955      If IstMakler() Then Exit Function
956      If Not NameZerlegen(rel, tag, quellen, teil) Then Exit Function
957      If Len(teil) > 0 Then Exit Function
958      For Each q In quellen
959          If geladenInfo.Exists(StandSchluessel(CStr(q), gruppe, tag)) Then Exit Function
960      Next q
961      For Each q In quellen
962          st.Datei = rel: st.Blatt = "": st.Quelle = CStr(q): st.Gruppe = gruppe: st.DateiGruppe = gruppe: st.Stichtag =
       tag: st.Zeilen = 0: st.Ersetzen = False
963          ReDim Preserve staende(0 To n)
964          staende(n) = st
965          n = n + 1
966      Next q
967      StandAusName = True
968  End Function
969
970  ' Rohdatei oeffnen. SharePoint: beim ersten Mal ueber https und sofort als lokale Kopie in %TEMP% sichern; weitere Zug
       riffe in
971  ' diesem Laden (Verarbeiten, Celonis) nehmen die Kopie – eine Datei wird so nur einmal ueber das Netz geholt.
972  Private Function QuelleOeffnenRel(ordner As String, rel As String) As Workbook
973      Dim url As String, wb As Workbook, lokal As String
974      If Not IstUrl(ordner) Then Set QuelleOeffnenRel = QuelleOeffnen(ordner & rel): Exit Function
975      url = PfadVon(ordner, rel)
976      lokal = CacheVon(url)
977      If Len(lokal) > 0 Then Set QuelleOeffnenRel = QuelleOeffnen(lokal): Exit Function
978      Set wb = QuelleOeffnen(url)
979      CacheAblegen wb, url
980      Set QuelleOeffnenRel = wb
```

```
End Function

Private Function CacheVon(url As String) As String
    On Error Resume Next
    If cacheDateien Is Nothing Then Exit Function
    If cacheDateien.Exists(LCase$(url)) Then
        If Len(Dir(CStr(cacheDateien(LCase$(url))))) > 0 Then CacheVon = CStr(cacheDateien(LCase$(url)))
    End If
End Function

Private Sub CacheAblegen(wb As Workbook, url As String)
    Dim lokal As String
    On Error GoTo Ende
    If cacheDateien Is Nothing Then Set cacheDateien = CreateObject("Scripting.Dictionary")
    If Len(cacheOrdner) = 0 Then
        cacheOrdner = Environ$("TEMP") & Application.PathSeparator & "BUE_Cache_" & Format$(Now, "yyyymmdd_hhnnss")
        If Len(Dir(cacheOrdner, vbDirectory)) = 0 Then MkDir cacheOrdner
    End If
    lokal = cacheOrdner & Application.PathSeparator & (cacheDateien.Count + 1) & "_" & DateiName(url)
    Application.DisplayAlerts = False
    wb.SaveCopyAs lokal
    Application.DisplayAlerts = True
    cacheDateien(LCase$(url)) = lokal
    Exit Sub
Ende:
    Application.DisplayAlerts = True
End Sub

' lokale Kopien am Ende des Ladens loeschen
Private Sub CacheAufraeumen()
    Dim k As Variant
    On Error Resume Next
    If Not cacheDateien Is Nothing Then
        For Each k In cacheDateien.Keys
            Kill CStr(cacheDateien(k))
        Next k
    End If
    If Len(cacheOrdner) > 0 Then RmDir cacheOrdner
    Set cacheDateien = Nothing
    cacheOrdner = ""
End Sub

Private Function QuelleOeffnen(pfad As String) As Workbook
    ' Ohne Makros (AutomationSecurity), ohne Dialoge, schreibgeschuetzt; bereits offene Datei wird nicht erneut geoeff
    net
    Dim sicherheit As MsoAutomationSecurity, nm As String, wb As Workbook
    Dim fehlerNr As Long, fehlerText As String, versuch As Long
    nm = Mid$(pfad, InStrRev(Replace(pfad, "/", Application.PathSeparator), Application.PathSeparator) + 1)
    If IstUrl(pfad) Then nm = UrlDekodieren(nm)
    On Error Resume Next
    Set wb = Workbooks(nm)
    On Error GoTo 0
    If Not wb Is Nothing Then Err.Raise vbObjectError + 8, , "Die Datei " & nm & " ist bereits in Excel geöffnet. Bitt
    e schließen und erneut laden."
    sicherheit = Application.AutomationSecurity
    Application.AutomationSecurity = msoAutomationSecurityForceDisable
    Application.DisplayAlerts = False
    ' SharePoint antwortet beim Massenladen gelegentlich kurz nicht (Fehler 1004 "kann nicht zugreifen"): bis zu drei
    ' weitere Versuche mit wachsender Pause, erst danach Abbruch
    For versuch = 1 To IIf(IstUrl(pfad), 4, 1)
        If versuch > 1 Then
            LogSchreiben "Öffnen von " & nm & " fehlgeschlagen (" & fehlerText & "), Versuch " & versuch & " von 4"
            Application.Wait Now + TimeSerial(0, 0, 3 * (versuch - 1))
        End If
        On Error Resume Next
        Err.Clear
        Set wb = Workbooks.Open(FileName:=pfad, UpdateLinks:=0, ReadOnly:=True, IgnoreReadOnlyRecommended:=True)
        fehlerNr = Err.Number: fehlerText = Err.Description
        On Error GoTo 0
        If fehlerNr = 0 Then Exit For
    Next versuch
    Application.DisplayAlerts = True
    Application.AutomationSecurity = sicherheit
    If fehlerNr <> 0 Then Err.Raise fehlerNr, , "Datei " & nm & " konnte nicht geöffnet werden: " & fehlerText & vbCrL
    f & _
        "(Aus dem Internet oder Teams heruntergeladene Dateien: Eigenschaften > 'Zulassen' setzen oder den Ordner als
    vertrauenswürdigen Speicherort eintragen.)"
    If wb Is Nothing Then Err.Raise vbObjectError + 9, , "Datei " & nm & " wurde in der geschützten Ansicht geöffnet u
    nd kann nicht gelesen werden."
    Set QuelleOeffnen = wb
End Function

' ------------------------------------------------------------- Blatt erkennen
' straenge (optional): Einheit -> Zeilen. HS: eine leere Einheit; MAKLER mit Spalte OE_NR: jede Einheit aus Konfigurat
    ion!R11:R40,
' Zeilen anderer OEs zaehlen zur Haupteinheit des Blatts (HauptAusZaehlung); ohne Spalte OE_NR: die Gruppe aus der Kop
    fzeile.
Private Function BlattErkennen(ws As Worksheet, ByRef st As Stand, Optional straenge As Object) As Boolean
    Dim r As Long, c As Long, t As String, kopf As Long, letzteSpalte As Long
    Dim stichtag As Date, gefunden As Boolean, kopfdaten As Variant
    Dim zaehl As Object, rest As Long, mitOE As Boolean, haupt As String, u As Variant, hs As Long
    BlattErkennen = False
    blattNurHs = False
    If ws.UsedRange.Rows.Count < 3 Then Exit Function
    letzteSpalte = ws.UsedRange.Column + ws.UsedRange.Columns.Count - 1
    kopfdaten = ws.Range(ws.Cells(1, 1), ws.Cells(MAX_KOPFZEILEN, letzteSpalte)).Value2     ' MAX_KOPFZEILEN >= 2 -> i
```

```
        mmer 2D
1070      kopf = 0
1071      For r = 1 To MAX_KOPFZEILEN
1072          For c = 1 To letzteSpalte
1073              t = Trim$(CStr(Nz(kopfdaten(r, c))))
1074              If Not gefunden And InStr(1, t, "stichtag", vbTextCompare) > 0 Then
1075                  stichtag = StichtagAusText(t)
1076                  If stichtag > 0 Then gefunden = True
1077              End If
1078              If t = "Themenpostkorb" Then kopf = r
1079          Next c
1080          If kopf > 0 Then Exit For
1081      Next r
1082      If kopf = 0 Or Not gefunden Then Exit Function
1083      st.Stichtag = stichtag
1084      st.Zeilen = ws.UsedRange.Row + ws.UsedRange.Rows.Count - 1 - kopf
1085      st.Quelle = "Verbund"
1086      For c = 1 To letzteSpalte
1087          If UCase$(Trim$(CStr(Nz(kopfdaten(kopf, c))))) = "OPS_AUFTR_ID" Then st.Quelle = "OPAG"
1088      Next c
1089      st.DateiGruppe = ""
1090      Set zaehl = CreateObject("Scripting.Dictionary")
1091      If IstMakler() Then st.DateiGruppe = GruppeErkennen(ws, kopfdaten, kopf, letzteSpalte, zaehl, rest, mitOE, hs)
1092      ' Bereich MAKLER (12.91): nur Zeilen der HS-Einheit (OE HSB4 K) – ein HS-Abzug, kein Stand fuer das Makler-Reporti
        ng
1093      If hs > 0 And rest = 0 And Len(st.DateiGruppe) = 0 Then
1094          blattNurHs = True
1095          Exit Function
1096      End If
1097      st.Gruppe = st.DateiGruppe
1098      Set straenge = CreateObject("Scripting.Dictionary")
1099      If mitOE And zaehl.Count > 0 Then
1100          haupt = HauptAusZaehlung(zaehl)
1101          For Each u In zaehl.Keys
1102              straenge.Add CStr(u), CLng(zaehl(u))
1103          Next u
1104          straenge(haupt) = straenge(haupt) + rest
1105      ElseIf mitOE Then
1106          straenge.Add "", IIf(hs > 0, rest, st.Zeilen)
1107      Else
1108          straenge.Add st.DateiGruppe, st.Zeilen
1109      End If
1110      BlattErkennen = True
1111  End Function
1112
1113  Public Function StichtagAusText(t As String) As Date
1114      ' "Rückstandsstichtag = Mittwoch, 23. September 2026" / "= 23. September 2026" / "= 23.09.2026" -> 23.09.2026
1115      Dim s As String, teile() As String, tag As Long, jahr As Long, monat As Long, i As Long
1116      Dim monate As Variant
1117      monate = Array("januar", "februar", "märz", "april", "mai", "juni", "juli", "august", "september", "oktober", "nov
        ember", "dezember")
1118      s = t
1119      If InStr(s, "=") > 0 Then s = Mid$(s, InStr(s, "=") + 1)
1120      If InStr(s, ",") > 0 Then s = Mid$(s, InStrRev(s, ",") + 1)
1121      s = Trim$(s)
1122      ' numerisches Datum dd.mm.yyyy
1123      teile = Split(s, ".")
1124      If UBound(teile) = 2 Then
1125          If IsNumeric(Trim$(teile(0))) And IsNumeric(Trim$(teile(1))) And IsNumeric(Trim$(teile(2))) Then
1126              If Len(Trim$(teile(2))) = 4 Then
1127                  StichtagAusText = DateSerial(CLng(teile(2)), CLng(teile(1)), CLng(teile(0)))
1128                  Exit Function
1129              End If
1130          End If
1131      End If
1132      s = Trim$(Replace(s, ".", " "))
1133      Do While InStr(s, "  ") > 0: s = Replace(s, "  ", " "): Loop
1134      teile = Split(s, " ")
1135      If UBound(teile) < 2 Then Exit Function
1136      If Not IsNumeric(teile(0)) Or Not IsNumeric(teile(2)) Then Exit Function
1137      tag = CLng(teile(0)): jahr = CLng(teile(2))
1138      For i = 0 To 11
1139          If LCase$(teile(1)) = monate(i) Then monat = i + 1
1140      Next i
1141      If monat = 0 Then Exit Function
1142      StichtagAusText = DateSerial(jahr, monat, tag)
1143  End Function
1144
1145  ' ------------------------------------------------------------------ Stand verarbeiten
1146  ' Blatt eines Stands oeffnen und die Datenzeilen unter der Kopfzeile lesen. sp(1..8) = Spalten VSS, Ordnungsbegriff,
1147  ' VORG_VERBU_LNR, Themenpostkorb, Rueckstandsalter, GEVO_SL, OPS_AUFTR_ID, OPS_ARBGUT_LNR, sp(9) = OE_NR, sp(10) = Ver
        kettungskennzeichen
1148  ' (0 = fehlt). Ergebnis: Anzahl Zeilen.
1149  ' Fehlt das Blatt unter seinem Namen (Datei neu abgelegt), wird es ueber Quelle und Stichtag gesucht.
1150  Private Function StandDatenLesen(ordner As String, st As Stand, ByRef daten As Variant, ByRef sp() As Long) As Long
1151      Dim wbQ As Workbook, ws As Worksheet, kopfdaten As Variant, w As Worksheet, pruef As Stand
1152      Dim kopf As Long, r As Long, c As Long, letzteZeile As Long, letzteSpalte As Long, letzte As Range
1153      Dim fehlerNr As Long, fehlerText As String, schl As String, pruefS As Object
1154      ' dasselbe Blatt eben schon gelesen (weitere Einheit derselben Datei, Ersetzen): nicht erneut oeffnen
1155      schl = LCase$(st.Datei) & "|" & st.Blatt & "|" & st.Quelle & "|" & CLng(st.Stichtag)
1156      If schl = lesenSchluessel And Len(lesenSchluessel) > 0 Then
1157          daten = lesenDaten
1158          sp = lesenSp
1159          StandDatenLesen = lesenZeilen
1160          Exit Function
1161      End If
```

```
    LesenVergessen
    ReDim sp(1 To 10)
    On Error GoTo Fehler
    LogSchreiben "  oeffne " & st.Datei
    Set wbQ = QuelleOeffnenRel(ordner, st.Datei)
    On Error Resume Next
    Set ws = wbQ.Worksheets(st.Blatt)
    On Error GoTo Fehler
    If ws Is Nothing Then
        For Each w In wbQ.Worksheets
            If BlattErkennen(w, pruef, pruefS) Then
                If pruef.Quelle = st.Quelle And CLng(pruef.Stichtag) = CLng(st.Stichtag) Then
                    If pruefS.Exists(st.Gruppe) Or pruef.DateiGruppe = st.Gruppe Then Set ws = w: Exit For
                End If
            End If
        Next w
        If ws Is Nothing Then Err.Raise vbObjectError + 10, , "Blatt für " & Strang(st.Quelle, st.Gruppe) & " " & Form
    at(st.Stichtag, "dd.mm.yyyy") & " nicht gefunden"
        st.Blatt = ws.Name                                  ' gefundenes Blatt merken (Ladeprotokoll)
    End If
    letzteSpalte = ws.UsedRange.Column + ws.UsedRange.Columns.Count - 1
    LogSchreiben "  Blatt " & ws.Name & ", Spalten bis " & letzteSpalte
    kopfdaten = ws.Range(ws.Cells(1, 1), ws.Cells(MAX_KOPFZEILEN, letzteSpalte)).Value2
    For r = 1 To MAX_KOPFZEILEN
        For c = 1 To letzteSpalte
            If Trim$(CStr(Nz(kopfdaten(r, c)))) = "Themenpostkorb" Then kopf = r
        Next c
        If kopf > 0 Then Exit For
    Next r
    If kopf = 0 Then Err.Raise vbObjectError + 1, , "Kopfzeile mit 'Themenpostkorb' fehlt"
    For c = 1 To letzteSpalte
        Select Case UCase$(Trim$(CStr(Nz(kopfdaten(kopf, c)))))
            Case "VERWALTUNGSSYSTEMSCHLÜSSEL": sp(1) = c
            Case "ORDNUNGSBEGRIFF": sp(2) = c
            Case "VORG_VERBU_LNR": sp(3) = c
            Case "THEMENPOSTKORB": sp(4) = c
            Case "RÜCKSTANDSALTER_TAGE": sp(5) = c
            Case "GEVO_SL": sp(6) = c
            Case "OPS_AUFTR_ID": sp(7) = c
            Case "OPS_ARBGUT_LNR": sp(8) = c
            Case "OE_NR": sp(9) = c
            Case Else
                If InStr(1, CStr(Nz(kopfdaten(kopf, c))), "verkett", vbTextCompare) > 0 Then sp(10) = c   ' Verkettung
    skennzeichen
        End Select
    Next c
    If Not IstMakler() Then sp(9) = 0                    ' Einheit je Zeile nur im Bereich MAKLER
    LogSchreiben "  Spalten: VSS " & sp(1) & ", Ordnungsbegriff " & sp(2) & ", LNR " & sp(3) & ", Postkorb " & sp(4) &
    ", Alter " & sp(5) & _
                 ", GeVo " & sp(6) & ", Auftrag " & sp(7) & ", Arbeitsgut " & sp(8) & ", OE " & sp(9) & ", Verkettung
    " & sp(10)
    If sp(1) = 0 Or sp(2) = 0 Or sp(3) = 0 Or sp(4) = 0 Or sp(5) = 0 Or sp(6) = 0 Then
        Err.Raise vbObjectError + 1, , "Spaltenköpfe unvollständig, es fehlt: " & Mid$( _
            IIf(sp(1) = 0, ", Verwaltungssystemschlüssel", "") & IIf(sp(2) = 0, ", Ordnungsbegriff", "") & IIf(sp(3) =
    0, ", VORG_VERBU_LNR", "") & _
            IIf(sp(4) = 0, ", Themenpostkorb", "") & IIf(sp(5) = 0, ", Rückstandsalter_Tage", "") & IIf(sp(6) = 0, ",
    GEVO_SL", ""), 3)
    End If
    ' letzte belegte Zeile auch bei ausgeblendeten/gefilterten Zeilen
    Set letzte = ws.Columns(sp(1)).Find(What:="*", LookIn:=xlValues, SearchOrder:=xlByRows, SearchDirection:=xlPreviou
    s)
    If letzte Is Nothing Then letzteZeile = kopf Else letzteZeile = letzte.Row
    If letzteZeile > kopf Then
        LogSchreiben "  lese Zeilen " & (kopf + 1) & " bis " & letzteZeile
        daten = ws.Range(ws.Cells(kopf + 1, 1), ws.Cells(letzteZeile, letzteSpalte)).Value2
        StandDatenLesen = letzteZeile - kopf
    End If
    wbQ.Close SaveChanges:=False
    Set wbQ = Nothing
    LogSchreiben "  Quelle geschlossen"
    lesenSchluessel = schl
    lesenDaten = daten
    lesenSp = sp
    lesenZeilen = StandDatenLesen
    Exit Function
Fehler:
    fehlerNr = Err.Number: fehlerText = Err.Description
    On Error Resume Next
    If Not wbQ Is Nothing Then wbQ.Close SaveChanges:=False
    On Error GoTo 0
    Err.Raise fehlerNr, "StandDatenLesen", fehlerText & " (" & st.Datei & " / " & st.Blatt & ")"
End Function

Private Sub LesenVergessen()
    lesenSchluessel = ""
    lesenDaten = Empty
    lesenZeilen = 0
End Sub

' Schluessel eines Datensatzes (Verbund: VSS + Ordnungsbegriff + LNR; OPAG zusaetzlich Auftrag und Arbeitsgut)
Private Function DatensatzSchluessel(daten As Variant, r As Long, sp() As Long, quelle As String) As String
    Dim key As String
    ' Fehlt eine OPAG-Spalte im Abzug (Spalte 0), bleibt ihr Teil leer statt Laufzeitfehler 9
    key = Sauber(daten(r, sp(1))) & TRENNER & Sauber(daten(r, sp(2))) & TRENNER & Sauber(daten(r, sp(3)))
    If quelle = "OPAG" Then key = key & TRENNER & Feld(daten, r, sp(7)) & TRENNER & Feld(daten, r, sp(8))
    DatensatzSchluessel = key
```

```
End Function

Private Function Feld(daten As Variant, r As Long, c As Long) As String
    If c > 0 Then Feld = Sauber(daten(r, c))
End Function

Private Sub StandVerarbeiten(ordner As String, st As Stand)
    Dim b(0 To 0) As Stand
    b(0) = st
    StaendeVerarbeiten ordner, b, 1
End Sub

' Staende einer Quelle und eines Stichtags gemeinsam verarbeiten: b(0..m-1), je Einheit (Strang) einer, aus einer oder mehreren
' Dateien. Jede Einheit wird mit ihrem eigenen Vortag (Snapshot) verglichen. Wechselt ein Vorgang zwischen zwei Einheiten dieses
' Stichtags, ist das eine Umbuchung (Blatt Umbuchungen), kein Abgang und Eingang. Zeilen einer Datei, deren Einheit nicht in b
' steht (anderer Strang, z. B. schon geladen), bleiben hier aussen vor.
Private Sub StaendeVerarbeiten(ordner As String, b() As Stand, m As Long)
    Dim daten As Variant, sp() As Long, r As Long, x As Long, y As Long
    Dim cVss As Long, cPk As Long, cAlt As Long, cGevo As Long
    Dim sparten As Object, alt As Object, altIdx As Object, neuE As Object, neuAlle As Object, agg As Object, umb As Object
    Dim erster As Object, zz As Object, gelesen As Object, dieseDatei As Object, snap As Object, vorher As Variant
    Dim key As String, vss As String, sparte As String, pk As String, gevo As String, typ As String, nr As Long, einheit As String
    Dim aggKey As String, umbKey As String, werte As Variant, k As Variant, sg As String, haupt As String, dk As String, z As Variant
    Dim quelle As String, stichtag As Date, nZeilen As Long, verk As Boolean, hinweis As String, hsBl As Long, ks As Variant

    quelle = b(0).Quelle: stichtag = b(0).Stichtag
    Set sparten = SpartenZuordnung()
    Set alt = CreateObject("Scripting.Dictionary")        ' Strang -> Schluessel des Vortags (leer beim ersten Stand)
    Set erster = CreateObject("Scripting.Dictionary")     ' Strang -> True beim ersten Stand dieser Einheit
    Set altIdx = CreateObject("Scripting.Dictionary")     ' Schluessel -> Strang am Vortag (ueber alle Straenge dieses Stichtags)
    Set neuE = CreateObject("Scripting.Dictionary")       ' Strang -> heutige Schluessel (neuer Snapshot)
    Set neuAlle = CreateObject("Scripting.Dictionary")    ' Schluessel -> Strang heute
    Set agg = CreateObject("Scripting.Dictionary")
    Set umb = CreateObject("Scripting.Dictionary")
    Set zz = CreateObject("Scripting.Dictionary")         ' Strang -> Array(Saetze, Rueckstand, Eingang, Abgang, nicht fuehrend, Spalte Verkettung, Blatt leer, HS-Zeilen)
    Set gelesen = CreateObject("Scripting.Dictionary")
    For x = 0 To m - 1
        sg = b(x).Gruppe
        Set snap = SnapshotLesen(quelle, sg)               ' Nothing beim ersten Stand dieser Quelle und Einheit
        If snap Is Nothing Then
            erster.Add sg, True
            Set snap = CreateObject("Scripting.Dictionary")
        End If
        alt.Add sg, snap
        For Each k In snap.Keys
            If Not altIdx.Exists(k) Then altIdx.Add k, sg
        Next k
        neuE.Add sg, CreateObject("Scripting.Dictionary")
        zz.Add sg, Array(0, 0, 0, 0, 0, False, True, 0)
    Next x

    ' Rueckstand, Altersklassen, Eingaenge (Schluessel der Fakten: Sparte|VSS|Postkorb|GeVo|Einheit|Strang)
    ' Verkettete, nicht fuehrende Vorgaenge zaehlen nicht (wie im MIS-O-Bericht: Verkettungskennzeichen = verkettet, fuehrend /
    ' nicht verkettet) — nur wenn der Abzug die Spalte Verkettungskennzeichen enthaelt
    For x = 0 To m - 1
        dk = LCase$(b(x).Datei) & "|" & b(x).Blatt
        If Not gelesen.Exists(dk) Then
            gelesen.Add dk, True
            Set dieseDatei = CreateObject("Scripting.Dictionary")     ' Straenge dieses Blatts in b
            For y = x To m - 1
                If LCase$(b(y).Datei) & "|" & b(y).Blatt = dk Then dieseDatei(b(y).Gruppe) = True
            Next y
            nZeilen = StandDatenLesen(ordner, b(x), daten, sp)
            If nZeilen > 0 Then
                cVss = sp(1): cPk = sp(4): cAlt = sp(5): cGevo = sp(6)
                haupt = HauptStrang(daten, sp)
                LogSchreiben "  zaehle " & Join(dieseDatei.Keys, ", ")
                For Each k In dieseDatei.Keys
                    z = zz(k): z(5) = (sp(10) > 0): z(6) = False: zz(k) = z
                Next k
                hsBl = 0
                For r = 1 To UBound(daten, 1)
                    vss = Sauber(daten(r, cVss))
                    If Len(vss) > 0 Then
                        If sp(9) = 0 Then
                            sg = b(x).Gruppe
                        ElseIf IstHsEinheit(ZeilenEinheit(daten, r, sp)) Then
                            sg = vbNullChar                           ' Zeile der HS-Einheit (OE HSB4 K, 12.91): gehoert ins HS-Reporting
                            hsBl = hsBl + 1
                        Else
                            sg = ZeileStrang(daten, r, sp, haupt)
                        End If
                        If Not dieseDatei.Exists(sg) Then
                            vss = ""
                        ElseIf NichtFuehrend(daten, r, sp) Then
```

```
                            vss = ""
                            z = zz(sg): z(4) = z(4) + 1: zz(sg) = z
                        End If
                    End If
                    If Len(vss) > 0 Then
                        key = DatensatzSchluessel(daten, r, sp, quelle)
                        If Not neuAlle.Exists(key) Then                ' derselbe Vorgang zweimal (zwei Dateien): einm
al zaehlen
                            z = zz(sg): z(0) = z(0) + 1: zz(sg) = z
                            If sparten.Exists(vss) Then sparte = sparten(vss) Else sparte = "Sonstige"
                            pk = PostkorbNorm(CStr(Nz(daten(r, cPk))))
                            gevo = Verdichten(CStr(Nz(daten(r, cGevo))))
                            einheit = ZeilenEinheit(daten, r, sp)
                            aggKey = sparte & TRENNER & vss & TRENNER & pk & TRENNER & gevo & TRENNER & einheit & TREN
NER & sg
                            If Not agg.Exists(aggKey) Then agg.Add aggKey, Array(0, 0, 0, 0, 0, 0, 0, 0, "", 0)
                            werte = agg(aggKey)
                            werte(0) = werte(0) + 1
                            werte(3 + AlterKlasse(daten(r, cAlt))) = werte(3 + AlterKlasse(daten(r, cAlt))) + 1
                            If IsNumeric(daten(r, cAlt)) And Not IsEmpty(daten(r, cAlt)) Then
                                If CDbl(daten(r, cAlt)) > werte(7) Or CStr(werte(8)) = "" Then werte(7) = CDbl(daten(r
, cAlt)): werte(8) = key   ' aeltester Datensatz
                                werte(9) = werte(9) + CDbl(daten(r, cAlt))           ' Summe fuer das Ø Alter (ohne Al
ter zaehlt der Satz mit 0 Tagen)
                            End If
                            vorher = Empty
                            If erster.Exists(sg) Then
                                ' erster Stand dieser Einheit: keine Eingaenge
                            ElseIf alt.Item(sg).Exists(key) Then
                                Set snap = alt.Item(sg)
                                vorher = snap.Item(key)
                            ElseIf altIdx.Exists(key) Then
                                Set snap = alt.Item(CStr(altIdx(key)))
                                vorher = snap.Item(key)                        ' gestern in einer anderen Einheit die
ses Stichtags
                            Else
                                werte(1) = werte(1) + 1
                            End If
                            ' Umbuchung: gestern in einem anderen Themenpostkorb, GeVo oder einer anderen Einheit (Bla
tt Umbuchungen, beruehrt Fakten nicht)
                            If IsArray(vorher) Then
                                If CStr(vorher(2)) <> pk Or CStr(vorher(3)) <> gevo Or CStr(vorher(4)) <> einheit Then
                                    umbKey = sparte & TRENNER & CStr(vorher(2)) & TRENNER & CStr(vorher(3)) & TRENNER
& pk & TRENNER & gevo & _
                                             TRENNER & CStr(vorher(4)) & TRENNER & einheit & TRENNER & sg
                                    If umb.Exists(umbKey) Then umb(umbKey) = umb(umbKey) + 1 Else umb.Add umbKey, 1
                                End If
                            End If
                            agg(aggKey) = werte
                            neuAlle.Add key, sg
                            Set snap = neuE.Item(sg)
                            snap.Add key, Array(sparte, vss, pk, gevo, einheit)
                        End If
                    End If
                Next r
                If hsBl > 0 Then                                       ' im Ladeprotokoll beim Hauptstrang des Blatts
vermerken
                    hsZeilen = hsZeilen + hsBl
                    If dieseDatei.Exists(haupt) Then
                        sg = haupt
                    Else
                        ks = dieseDatei.Keys
                        sg = CStr(ks(0))
                    End If
                    z = zz(sg): z(7) = z(7) + hsBl: zz(sg) = z
                    LogSchreiben "  " & hsBl & " Zeilen der HS-Einheit (OE HSB4 K) ignoriert"
                End If
            End If
            Erase sp
            daten = Empty
        End If
    Next x

    ' Abgaenge: Schluessel des Vortags, die heute in keiner Einheit dieses Stichtags mehr stehen – zugeordnet nach den
    ' Vortagsmerkmalen; nur fuer Einheiten mit Datensaetzen (ein leeres Blatt ist kein Abbau auf null)
    For x = 0 To m - 1
        sg = b(x).Gruppe
        z = zz(sg)
        If Not erster.Exists(sg) And z(0) > 0 Then
            Set snap = alt.Item(sg)
            For Each k In snap.Keys
                If Not neuAlle.Exists(k) Then
                    werte = snap.Item(k)
                    aggKey = werte(0) & TRENNER & werte(1) & TRENNER & werte(2) & TRENNER & werte(3) & TRENNER & werte
(4) & TRENNER & sg
                    If Not agg.Exists(aggKey) Then agg.Add aggKey, Array(0, 0, 0, 0, 0, 0, 0, 0, "", 0)
                    werte = agg(aggKey)
                    werte(2) = werte(2) + 1
                    agg(aggKey) = werte
                End If
            Next k
        End If
    Next x

    ' Fakten schreiben (ein Block), Formelspalten P/Q aus Zeile 2 fortschreiben, R Einheit, S Strang
    Dim wsF As Worksheet, start As Long, aus() As Variant, ausE() As Variant, i As Long, teile() As String, neuePostko
```

```
        erbe As Object
1423     Set wsF = ThisWorkbook.Worksheets("Fakten")
1424     Set neuePostkoerbe = CreateObject("Scripting.Dictionary")
1425     start = wsF.Cells(wsF.Rows.Count, 1).End(xlUp).Row + 1
1426     If agg.Count > 0 Then
1427         If start + agg.Count - 1 > FAKTEN_MAX Then Err.Raise vbObjectError + 2, , "Das Blatt Fakten ist voll (mehr als
         " & FAKTEN_MAX & " Zeilen). Bitte im Cockpit „Zurücksetzen …“ und die Abzüge neu laden."
1428         ReDim aus(1 To agg.Count, 1 To 15)
1429         ReDim ausE(1 To agg.Count, 1 To 5)
1430     End If
1431     i = 0
1432     For Each k In agg.Keys
1433         i = i + 1
1434         teile = Split(k, TRENNER)
1435         werte = agg(k)
1436         GevoTyp teile(3), typ, nr
1437         aus(i, 1) = stichtag: aus(i, 2) = quelle: aus(i, 3) = teile(0): aus(i, 4) = teile(1)
1438         aus(i, 5) = teile(2): aus(i, 6) = teile(3): aus(i, 7) = typ: aus(i, 8) = nr
1439         aus(i, 9) = werte(0): aus(i, 10) = werte(1): aus(i, 11) = werte(2)
1440         aus(i, 12) = werte(3): aus(i, 13) = werte(4): aus(i, 14) = werte(5): aus(i, 15) = werte(6)
1441         ausE(i, 1) = teile(4): ausE(i, 2) = teile(5): ausE(i, 3) = werte(7): ausE(i, 4) = werte(8): ausE(i, 5) = werte
        (9)
1442         z = zz(teile(5))
1443         z(1) = z(1) + werte(0): z(2) = z(2) + werte(1): z(3) = z(3) + werte(2)
1444         zz(teile(5)) = z
1445         If Not neuePostkoerbe.Exists(teile(2)) Then neuePostkoerbe.Add teile(2), True
1446     Next k
1447     LogSchreiben "  gezaehlt: " & agg.Count & " Faktenzeilen; schreibe ab Fakten-Zeile " & start
1448     ' Protokoll zuerst (je Einheit eine Zeile): ein Abbruch danach hinterlaesst keinen unprotokollierten Halbstand
1449     For x = 0 To m - 1
1450         sg = b(x).Gruppe
1451         z = zz(sg)
1452         verk = z(5)
1453         If z(6) Then
1454             Protokoll b(x), 0, 0, 0, 0, "keine Datensätze unter der Kopfzeile"
1455         ElseIf z(0) = 0 Then
1456             Protokoll b(x), 0, 0, 0, 0, "keine Datensätze mit Verwaltungssystemschlüssel" & VerkettungText(verk, CLng(
        z(4)), True) & HsText(CLng(z(7)))
1457         ElseIf erster.Exists(sg) Then
1458             If Len(sg) > 0 Then hinweis = "erster Stand dieser Einheit (" & sg & ")" Else hinweis = "erster Stand dies
        er Quelle"
1459             Protokoll b(x), CLng(z(0)), CLng(z(1)), 0, 0, hinweis & " – Eingänge und Abgänge ab dem nächsten Stichtag"
         & VerkettungText(verk, CLng(z(4)), True) & HsText(CLng(z(7)))
1460         Else
1461             Protokoll b(x), CLng(z(0)), CLng(z(1)), CLng(z(2)), CLng(z(3)), VerkettungText(verk, CLng(z(4)), False) &
        HsText(CLng(z(7)))
1462         End If
1463         LogSchreiben "  " & Strang(quelle, sg) & ": " & z(0) & " Saetze, Rueckstand " & z(1) & ", Ein " & z(2) & ", Ab
        g " & z(3)
1464     Next x
1465     If agg.Count = 0 Then Exit Sub
1466     wsF.Range(wsF.Cells(start, 1), wsF.Cells(start + agg.Count - 1, 15)).Value = aus
1467     wsF.Range(wsF.Cells(start, 1), wsF.Cells(start + agg.Count - 1, 1)).NumberFormat = "dd.mm.yyyy"
1468     wsF.Range(wsF.Cells(start, SP_VORGANG_F), wsF.Cells(start + agg.Count - 1, SP_VORGANG_F)).NumberFormat = "@"
1469     wsF.Range(wsF.Cells(start, SP_EINHEIT_F), wsF.Cells(start + agg.Count - 1, SP_ALTSUM_F)).Value = ausE
1470     LogSchreiben "  Fakten geschrieben, Formelspalte"
1471     wsF.Range(wsF.Cells(start, FAKTEN_SPALTEN), wsF.Cells(start + agg.Count - 1, FAKTEN_SPALTEN)).FormulaR1C1 = wsF.Ce
        lls(2, FAKTEN_SPALTEN).FormulaR1C1   ' relative Bezuege je Zeile
1472     wsF.Range(wsF.Cells(start, FAKTEN_SPALTEN + 1), wsF.Cells(start + agg.Count - 1, FAKTEN_SPALTEN + 1)).FormulaR1C1
        = wsF.Cells(2, FAKTEN_SPALTEN + 1).FormulaR1C1   ' Standard
1473     UmbuchungenSchreiben b(0), umb
1474     LogSchreiben "  Postkoerbe"
1475     PostkoerbeRegistrieren neuePostkoerbe
1476     LogSchreiben "  Snapshot"
1477     Set snap = CreateObject("Scripting.Dictionary")
1478     For x = 0 To m - 1
1479         z = zz(b(x).Gruppe)
1480         If z(0) > 0 Then snap.Add b(x).Gruppe, neuE.Item(b(x).Gruppe)
1481     Next x
1482     If snap.Count > 0 Then SnapshotErsetzen quelle, stichtag, snap
1483     LogSchreiben "  Verlauf"
1484     VerlaufErgaenzen stichtag, start - 1, start + agg.Count - 2
1485 End Sub
1486
1487 ' Einheiten eines Stapels fuer Statuszeile und Log ("MSUE+ODPB"), leer im Bereich HS
1488 Private Function StapelText(b() As Stand, m As Long) As String
1489     Dim x As Long
1490     For x = 0 To m - 1
1491         If Len(b(x).Gruppe) > 0 Then StapelText = StapelText & IIf(Len(StapelText) > 0, "+", "") & b(x).Gruppe
1492     Next x
1493 End Function
1494
1495 ' Umbuchungen eines Stands an das Blatt Umbuchungen anhaengen: je Paar (vorher, heute) eine Zeile mit Anzahl; die Form
       elspalten
1496 ' K..P (Filterkennzeichen vorher/heute) werden aus Zeile 2 fortgeschrieben – Spalte fuer Spalte wie in Fakten; Q Einhe
       it heute,
1497 ' R Gruppe, S Einheit vorher.
1498 Private Sub UmbuchungenSchreiben(st As Stand, umb As Object)
1499     Dim ws As Worksheet, start As Long, n As Long, aus() As Variant, k As Variant, teile() As String, i As Long, c As
        Long
1500     Dim typ As String, nr As Long, summe As Long, ausE() As Variant
1501     n = umb.Count
1502     If n = 0 Then
1503         LogSchreiben "  Umbuchungen: 0"
1504         Exit Sub
1505     End If
```

```
1506      Set ws = ThisWorkbook.Worksheets("Umbuchungen")
1507      start = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row + 1
1508      If start + n - 1 > UMB_MAX Then Err.Raise vbObjectError + 4, , "Das Blatt Umbuchungen ist voll (mehr als " & UMB_M
        AX & " Zeilen)."
1509      ReDim aus(1 To n, 1 To 10)
1510      ReDim ausE(1 To n, 1 To 3)
1511      For Each k In umb.Keys
1512          i = i + 1
1513          teile = Split(k, TRENNER)
1514          ausE(i, 1) = teile(6): ausE(i, 2) = teile(7): ausE(i, 3) = teile(5)      ' Einheit heute, Strang, Einheit vorh
        er
1515          aus(i, 1) = st.Stichtag: aus(i, 2) = st.Quelle: aus(i, 3) = teile(0)
1516          GevoTyp teile(2), typ, nr
1517          aus(i, 4) = teile(1): aus(i, 5) = typ: aus(i, 6) = nr
1518          GevoTyp teile(4), typ, nr
1519          aus(i, 7) = teile(3): aus(i, 8) = typ: aus(i, 9) = nr
1520          aus(i, 10) = umb(k)
1521          summe = summe + umb(k)
1522      Next k
1523      ws.Range(ws.Cells(start, 1), ws.Cells(start + n - 1, 10)).Value = aus
1524      ws.Range(ws.Cells(start, 1), ws.Cells(start + n - 1, 1)).NumberFormat = "dd.mm.yyyy"
1525      ws.Range(ws.Cells(start, SP_EINHEIT_U), ws.Cells(start + n - 1, SP_EINHEIT_VOR_U)).Value = ausE
1526      For c = 11 To 16
1527          ws.Range(ws.Cells(start, c), ws.Cells(start + n - 1, c)).FormulaR1C1 = ws.Cells(2, c).FormulaR1C1
1528      Next c
1529      LogSchreiben "  Umbuchungen: " & summe & " Faelle in " & n & " Zeilen"
1530  End Sub
1531
1532  ' Umbuchungen einer Quelle (und Gruppe, Spalte R) ab einem Stichtag entfernen (beim Ersetzen eines Stands), Rest rueck
        t auf
1533  Private Sub UmbuchungenEntfernen(quelle As String, einheit As String, abTag As Date)
1534      Dim ws As Worksheet, letzte As Long, blk As Variant, ein As Variant, aus() As Variant, ausE() As Variant, i As Lon
        g, m As Long, c As Long, ab As Double
1535      Set ws = ThisWorkbook.Worksheets("Umbuchungen")
1536      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
1537      If letzte < 2 Then Exit Sub
1538      ab = CDbl(CLng(abTag))
1539      blk = ws.Range(ws.Cells(2, 1), ws.Cells(letzte + 1, 10)).Value2          ' +1: immer zweidimensional
1540      ein = ws.Range(ws.Cells(2, SP_EINHEIT_U), ws.Cells(letzte + 1, SP_EINHEIT_VOR_U)).Value2       ' Q..S
1541      ReDim aus(1 To UBound(blk, 1), 1 To 10)
1542      ReDim ausE(1 To UBound(blk, 1), 1 To 3)
1543      For i = 1 To UBound(blk, 1) - 1
1544          If Not (Val(blk(i, 1)) >= ab And CStr(Nz(blk(i, 2))) = quelle And CStr(Nz(ein(i, 2))) = einheit) Then
1545              m = m + 1
1546              For c = 1 To 10
1547                  aus(m, c) = blk(i, c)
1548              Next c
1549              For c = 1 To 3
1550                  ausE(m, c) = ein(i, c)
1551              Next c
1552          End If
1553      Next i
1554      ws.Range(ws.Cells(2, 1), ws.Cells(letzte, 10)).ClearContents
1555      ws.Range(ws.Cells(2, SP_EINHEIT_U), ws.Cells(letzte, SP_EINHEIT_VOR_U)).ClearContents
1556      If m > 0 Then
1557          ws.Range(ws.Cells(2, 1), ws.Cells(m + 1, 10)).Value2 = aus
1558          ws.Range(ws.Cells(2, SP_EINHEIT_U), ws.Cells(m + 1, SP_EINHEIT_VOR_U)).Value2 = ausE
1559      End If
1560  End Sub
1561
1562  ' Kachel „Eingänge und Abgänge“: Standard ist Verbund. Liegt der letzte Verbund-Stand mehr als 2 Tage hinter dem letzt
        en
1563  ' OPAG-Stand (Verbund liefert laenger nichts), zeigt die Kachel nach dem Laden OPAG. Die Chips oben in der Kachel scha
        lten
1564  ' danach wie gewohnt um.
1565  Private Sub QuelleBewegungenWaehlen()
1566      Dim tagOpag As Date, tagVerbund As Date, q As String
1567      tagOpag = LetzterStandQuelle("OPAG"): tagVerbund = LetzterStandQuelle("Verbund")
1568      If tagOpag > 0 And (tagVerbund = 0 Or CLng(tagOpag) - CLng(tagVerbund) > 2) Then q = "OPAG" Else q = "Verbund"
1569      ThisWorkbook.Worksheets("Filter").Range("L13").Value = q
1570      LogSchreiben "Eingänge/Abgänge zeigen " & q & " (letzter Stand OPAG " & Format(tagOpag, "dd.mm.yyyy") & ", Verbund
        " & Format(tagVerbund, "dd.mm.yyyy") & ")"
1571  End Sub
1572
1573  ' ---------------------------------------------------------------- Stand ersetzen
1574  ' Ersetzt einen geladenen Stand durch die neue Datei st. Vorab (ohne etwas zu aendern) wird geprueft und gelesen: der
        Stand
1575  ' davor (Vergleichsbasis fuer Eingaenge/Abgaenge, aus seiner Datei) und die Dateien aller spaeteren Staende dieser Que
        lle.
1576  ' Dann: Fakten der Quelle ab dem Stichtag entfernen, alte Protokollzeilen als "ersetzt" markieren, Snapshot auf den St
        and
1577  ' davor setzen, den neuen Stand und danach die spaeteren Staende in zeitlicher Reihenfolge neu verarbeiten.
1578  ' Ergebnis: Anzahl der danach neu gerechneten spaeteren Staende.
1579  ' einfuegen (12.88, Nachzuegler im Bereich MAKLER): es gibt noch keinen Stand zu diesem Stichtag; derselbe Weg fuegt i
        hn ein und
1580  ' rechnet die spaeteren Staende der Einheit nach. Fehlt dabei eine Rohdatei, Ergebnis -1 (nichts geaendert, Aufrufer u
        eberspringt).
1581  ' Alles je Strang (Einheit): Zeilen anderer Einheiten derselben Dateien bleiben unberuehrt.
1582  Private Function StandErsetzen(ordner As String, st As Stand, geladenInfo As Object, einfuegen As Boolean) As Long
1583      Dim alle() As Stand, n As Long, i As Long, vorher As Long, spaeter As Long
1584      Dim basis As Object, daten As Variant, sp() As Long, sparten As Object, r As Long, key As String, vss As String, s
        parte As String
1585      Dim alteDatei As String, info As Variant, zeit As String, haupt As String, sg As String
1586
1587      ' geladene Staende dieser Quelle (gueltige Protokollzeilen), zeitlich sortiert
1588      n = GeladeneStaende(st.Quelle, st.Gruppe, alle)
```

```
1589      vorher = -1
1590      For i = 0 To n - 1
1591          If CLng(alle(i).Stichtag) < CLng(st.Stichtag) Then vorher = i
1592      Next i
1593      ' Dateien pruefen, bevor etwas geaendert wird
1594      For i = 0 To n - 1
1595          If CLng(alle(i).Stichtag) > CLng(st.Stichtag) Or i = vorher Then
1596              alle(i).Datei = StandDatei(ordner, alle(i))
1597              If Len(alle(i).Datei) = 0 And einfuegen Then
1598                  LogSchreiben "  nicht nachtragbar: Rohdatei von " & Strang(alle(i).Quelle, alle(i).Gruppe) & " " & For
        mat(alle(i).Stichtag, "dd.mm.yyyy") & " fehlt"
1599                  StandErsetzen = -1
1600                  Exit Function
1601              End If
1602              If Len(alle(i).Datei) = 0 Then Err.Raise vbObjectError + 11, , "Ersetzen nicht möglich: die Rohdatei des S
        tands " & _
1603                  Strang(alle(i).Quelle, alle(i).Gruppe) & " " & Format(alle(i).Stichtag, "dd.mm.yyyy") & " liegt nicht
        mehr im Ordner Rohdaten (oder der Ablage). Der alte Stand bleibt."
1604          End If
1605      Next i
1606      LogSchreiben "  pruefe neue Datei"
1607      If StandDatenLesen(ordner, st, daten, sp) = 0 Then Err.Raise vbObjectError + 12, , "Ersetzen nicht möglich: die ne
        ue Datei " & st.Datei & " enthält keine Datensätze. Der alte Stand bleibt."
1608      Erase daten
1609      ' Vergleichsbasis: Schluessel des Stands davor
1610      If vorher >= 0 Then
1611          LogSchreiben "  lese Vergleichsbasis " & Format(alle(vorher).Stichtag, "dd.mm.yyyy") & " aus " & alle(vorher).
        Datei
1612          Set sparten = SpartenZuordnung()
1613          Set basis = CreateObject("Scripting.Dictionary")
1614          If StandDatenLesen(ordner, alle(vorher), daten, sp) > 0 Then
1615              haupt = HauptStrang(daten, sp)
1616              For r = 1 To UBound(daten, 1)
1617                  vss = Sauber(daten(r, sp(1)))
1618                  If Len(vss) > 0 And sp(9) > 0 Then                ' nur die Zeilen dieser Einheit
1619                      sg = ZeileStrang(daten, r, sp, haupt)
1620                      If sg <> st.Gruppe Then vss = ""
1621                  End If
1622                  If Len(vss) > 0 Then
1623                      If NichtFuehrend(daten, r, sp) Then vss = ""
1624                  End If
1625                  If Len(vss) > 0 Then
1626                      key = DatensatzSchluessel(daten, r, sp, st.Quelle)
1627                      If sparten.Exists(vss) Then sparte = sparten(vss) Else sparte = "Sonstige"
1628                      If Not basis.Exists(key) Then basis.Add key, Array(sparte, vss, PostkorbNorm(CStr(Nz(daten(r, sp(4
        ))))), Verdichten(CStr(Nz(daten(r, sp(6))))), _
1629                                                                          ZeilenEinheit(daten, r, sp))
1630                  End If
1631              Next r
1632          End If
1633          Erase daten
1634      End If
1635      If geladenInfo.Exists(StandSchluessel(st.Quelle, st.Gruppe, st.Stichtag)) And Not einfuegen Then
1636          info = geladenInfo(StandSchluessel(st.Quelle, st.Gruppe, st.Stichtag))
1637          alteDatei = CStr(info(1))
1638      End If
1639
1640      ' ab hier wird geaendert
1641      zeit = Format(Now, "dd.mm.yyyy hh:nn")
1642      FaktenEntfernen st.Quelle, st.Gruppe, st.Stichtag
1643      UmbuchungenEntfernen st.Quelle, st.Gruppe, st.Stichtag
1644      If einfuegen Then
1645          ProtokollErsetztMarkieren st.Quelle, st.Gruppe, st.Stichtag, "neu gerechnet am " & zeit & " (Stand " & Format(
        st.Stichtag, "dd.mm.yyyy") & " nachgereicht)"
1646      Else
1647          ProtokollErsetztMarkieren st.Quelle, st.Gruppe, st.Stichtag, "ersetzt am " & zeit & " (neue Datei " & st.Datei
        & ")"
1648      End If
1649      If vorher >= 0 Then
1650          SnapshotSchreiben st.Quelle, st.Gruppe, alle(vorher).Stichtag, basis
1651      Else
1652          SnapshotSchreiben st.Quelle, st.Gruppe, 0, CreateObject("Scripting.Dictionary")
1653      End If
1654      StandVerarbeiten ordner, st
1655      For i = 0 To n - 1
1656          If CLng(alle(i).Stichtag) > CLng(st.Stichtag) Then
1657              modFortschritt.FortschrittSetzen ladeAnteil, "Danach neu rechnen: " & Format$(alle(i).Stichtag, "dd.mm.yyy
        y") & " · " & Strang(alle(i).Quelle, alle(i).Gruppe), DateiName(alle(i).Datei)
1658              LogSchreiben "  rechne nach: " & Strang(alle(i).Quelle, alle(i).Gruppe) & " " & Format(alle(i).Stichtag, "
        dd.mm.yyyy") & " aus " & alle(i).Datei
1659              StandVerarbeiten ordner, alle(i)
1660              spaeter = spaeter + 1
1661          End If
1662      Next i
1663      VerlaufGrenzenNeu
1664      ' alte Rohdatei als ersetzt kennzeichnen (nur Einzeldateien; Sammeldateien enthalten weitere Staende)
1665      AlteDateiAusmustern ordner, alteDatei, st.Datei
1666      StandErsetzen = spaeter
1667  End Function
1668
1669  ' Gueltig geladene Staende einer Quelle (und Einheit) aus dem Ladeprotokoll (Hinweis leer oder "erster Stand …"), nach
         Stichtag sortiert
1670  Private Function GeladeneStaende(quelle As String, einheit As String, alle() As Stand) As Long
1671      Dim ws As Worksheet, letzte As Long, daten As Variant, r As Long, n As Long, hinweis As String, i As Long, j As Lo
        ng, t As Stand
1672      Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
```

```
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    If letzte < 5 Then Exit Function
    daten = ws.Range(ws.Cells(5, 1), ws.Cells(letzte + 1, SP_GRUPPE_L)).Value2       ' +1: immer zweidimensional
    For r = 1 To UBound(daten, 1)
        If IsNumeric(daten(r, 5)) And Not IsEmpty(daten(r, 5)) And CStr(Nz(daten(r, 4))) = quelle And CStr(Nz(daten(r, SP_GRUPPE_L))) = einheit Then
            hinweis = CStr(Nz(daten(r, 10)))
            If StandGueltig(hinweis) Then
                ReDim Preserve alle(0 To n)
                alle(n).Datei = CStr(Nz(daten(r, 2)))
                alle(n).Blatt = CStr(Nz(daten(r, 3)))
                alle(n).Quelle = quelle
                alle(n).Gruppe = einheit
                alle(n).Stichtag = CDate(daten(r, 5))
                n = n + 1
            End If
        End If
    Next r
    For i = 0 To n - 2
        For j = i + 1 To n - 1
            If alle(j).Stichtag < alle(i).Stichtag Then t = alle(i): alle(i) = alle(j): alle(j) = t
        Next j
    Next i
    GeladeneStaende = n
End Function

' Geladene Staende (Quelle|Stichtag) mit Zeitpunkt, Datei, Datensaetzen und Rueckstand des Ladens — fuer Ersatz-Frage und Ersetzen
Private Function GeladenLesen() As Object
    Dim ws As Worksheet, letzte As Long, daten As Variant, r As Long, d As Object, hinweis As String, k As String, kk As Variant
    Set d = CreateObject("Scripting.Dictionary")
    Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    If letzte >= 5 Then
        daten = ws.Range(ws.Cells(5, 1), ws.Cells(letzte + 1, SP_GRUPPE_L)).Value2
        For r = 1 To UBound(daten, 1)
            If IsNumeric(daten(r, 5)) And Not IsEmpty(daten(r, 5)) And Len(CStr(Nz(daten(r, 4)))) > 0 Then
                hinweis = CStr(Nz(daten(r, 10)))
                If StandGueltig(hinweis) Then
                    ' Schluessel des Stands und Namensschluessel (Dateiname JJJJMMTT_BUe_Quelle[_Einheit]) fuer abgelegte Doppel
                    For Each kk In Array(StandSchluessel(CStr(daten(r, 4)), CStr(Nz(daten(r, SP_GRUPPE_L))), CDate(daten(r, 5))), _
                                         NamensSchluessel(CStr(daten(r, 4)), EinheitDateiteil(CStr(Nz(daten(r, SP_GRUPPE_L)))), CDate(daten(r, 5))), _
                                         NamensSchluesselDatei(CStr(Nz(daten(r, 2))), CStr(daten(r, 4)), CDate(daten(r, 5))))
                        k = CStr(kk)
                        If Len(k) = 0 Then GoTo NaechsterSchluessel
                        If d.Exists(k) Then d.Remove k
                        d.Add k, Array(daten(r, 1), CStr(Nz(daten(r, 2))), Val(daten(r, 6)), Val(daten(r, 7)))
NaechsterSchluessel:
                    Next kk
                End If
            End If
        Next r
    End If
    Set GeladenLesen = d
End Function

' Rohdatei eines geladenen Stands finden: Eintrag im Protokoll, sonst der Standardname in der Monatsablage oder im Ordner selbst
Private Function StandDatei(ordner As String, st As Stand) As String
    Dim sep As String, basis As String, kand As Variant, endung As Variant, nm As String, nm2 As Variant
    sep = Application.PathSeparator
    If Len(st.Datei) > 0 Then
        If DateiVorhanden(ordner & st.Datei) Then StandDatei = st.Datei: Exit Function
        nm = Mid$(st.Datei, InStrRev(st.Datei, sep) + 1)
        If DateiVorhanden(ordner & MonatsOrdner(st.Stichtag) & sep & nm) Then StandDatei = MonatsOrdner(st.Stichtag) & sep & nm: Exit Function
        If DateiVorhanden(ordner & nm) Then StandDatei = nm: Exit Function
    End If
    basis = MonatsOrdner(st.Stichtag) & sep & Format$(st.Stichtag, "yyyymmdd") & "_BUe_"
    If Len(st.Gruppe) > 0 Then
        kand = Array(st.Quelle & "_" & EinheitDateiteil(st.Gruppe), "OPAG_Verbund_" & EinheitDateiteil(st.Gruppe))
    Else
        kand = Array(st.Quelle, "OPAG_Verbund")
    End If
    For Each nm2 In kand
        For Each endung In Array(".xlsx", ".xlsm", ".xlsb", ".xls")
            If DateiVorhanden(ordner & basis & nm2 & endung) Then StandDatei = basis & nm2 & endung: Exit Function
        Next endung
    Next nm2
End Function

' Fakten-Zeilen einer Quelle (und Gruppe, Spalte S) ab einem Stichtag entfernen: der Rest rueckt auf, die frei geworden Zeilen
' am Ende werden geleert (Werte und Formelspalten). Keine Zeilen loeschen — die Namen F_* bleiben so unveraendert.
Private Sub FaktenEntfernen(quelle As String, einheit As String, abTag As Date)
    Dim ws As Worksheet, letzte As Long, kopf As Variant, ein As Variant, i As Long, i0 As Long, blk As Variant, aus() As Variant, ausE() As Variant
    Dim m As Long, c As Long, ab As Double
    Set ws = ThisWorkbook.Worksheets("Fakten")
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    If letzte < 2 Then Exit Sub
```

```
    ab = CDbl(CLng(abTag))
    kopf = ws.Range(ws.Cells(2, 1), ws.Cells(letzte + 1, 2)).Value2              ' +1: immer zweidimensional
    ein = ws.Range(ws.Cells(2, SP_EINHEIT_F), ws.Cells(letzte + 1, SP_ALTSUM_F)).Value2          ' R Einheit, S Gruppe, T Alter max, U Vorgang, V Alter Summe
    For i = 1 To UBound(kopf, 1) - 1
        If Val(kopf(i, 1)) >= ab And CStr(Nz(kopf(i, 2))) = quelle And CStr(Nz(ein(i, 2))) = einheit Then i0 = i: Exit For
    Next i
    If i0 = 0 Then Exit Sub
    blk = ws.Range(ws.Cells(i0 + 1, 1), ws.Cells(letzte + 1, 15)).Value2
    ReDim aus(1 To UBound(blk, 1), 1 To 15)
    ReDim ausE(1 To UBound(blk, 1), 1 To 5)
    For i = 1 To UBound(blk, 1) - 1
        If Not (Val(blk(i, 1)) >= ab And CStr(Nz(blk(i, 2))) = quelle And CStr(Nz(ein(i0 - 1 + i, 2))) = einheit) Then
            m = m + 1
            For c = 1 To 15
                aus(m, c) = blk(i, c)
            Next c
            ausE(m, 1) = ein(i0 - 1 + i, 1): ausE(m, 2) = ein(i0 - 1 + i, 2): ausE(m, 3) = ein(i0 - 1 + i, 3): ausE(m, 4) = ein(i0 - 1 + i, 4)
            ausE(m, 5) = ein(i0 - 1 + i, 5)
        End If
    Next i
    If m > 0 Then
        ws.Range(ws.Cells(i0 + 1, 1), ws.Cells(i0 + m, 15)).Value2 = aus
        ws.Range(ws.Cells(i0 + 1, SP_EINHEIT_F), ws.Cells(i0 + m, SP_ALTSUM_F)).Value2 = ausE
    End If
    If i0 + m + 1 <= letzte Then ws.Range(ws.Cells(i0 + m + 1, 1), ws.Cells(letzte, SP_ALTSUM_F)).ClearContents
    LogSchreiben "  Fakten entfernt: " & (letzte - 1 - (i0 - 1) - m) & " Zeilen " & Strang(quelle, einheit) & " ab " & Format(abTag, "dd.mm.yyyy")
End Sub

' Gueltige Protokollzeilen einer Quelle ab einem Stichtag als ersetzt markieren (zaehlen danach nicht mehr als geladen)
Private Sub ProtokollErsetztMarkieren(quelle As String, einheit As String, abTag As Date, text As String)
    Dim ws As Worksheet, letzte As Long, r As Long, h As String, v As Variant
    Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    For r = 5 To letzte
        v = ws.Cells(r, 5).Value2
        If IsNumeric(v) And Not IsEmpty(v) And CStr(ws.Cells(r, 4).Value) = quelle And CStr(ws.Cells(r, SP_GRUPPE_L).Value) = einheit Then
            If CLng(v) >= CLng(abTag) Then
                h = CStr(ws.Cells(r, 10).Value)
                If StandGueltig(h) Then ws.Cells(r, 10).Value = text
            End If
        End If
    Next r
End Sub

' Alte Rohdatei nach dem Ersetzen umbenennen (…_ersetzt), damit sie nach einem Zurueecksetzen nicht wieder geladen wird
Private Sub AlteDateiAusmustern(ordner As String, alteDatei As String, neueDatei As String)
    Dim nm As String, ziel As String
    If Len(alteDatei) = 0 Or StrComp(alteDatei, neueDatei, vbTextCompare) = 0 Then Exit Sub
    nm = Mid$(alteDatei, InStrRev(alteDatei, Application.PathSeparator) + 1)
    Dim fehler As String, ausblenden As Boolean
    If IstUrl(ordner) Then
        ' SharePoint ohne Ablage (oder Datei noch unter ihrem MIS-O-Namen): im Tool ausblenden (Blatt SP_Liste, Spalte J).
        ' Kollegen mit synchronisiertem Ordner benennen ausgeblendete Dateien beim naechsten Laden in …_verworfen um.
        ausblenden = (spAblage <> 1)
        If Not nm Like "########_BUe_*" Then ausblenden = True
        If ausblenden Then
            If Not DateiTraegtGueltigenStand(alteDatei) Then
                SpAusblenden alteDatei, ordner
                LogSchreiben "  alte Datei ausgeblendet (SharePoint): " & alteDatei
            End If
            Exit Sub
        End If
    End If
    If Not nm Like "########_BUe_*" Then Exit Sub
    If Not DateiVorhanden(ordner & alteDatei) Then Exit Sub
    If DateiTraegtGueltigenStand(alteDatei) Then Exit Sub     ' traegt noch Staende anderer Einheiten oder Stichtage
    ziel = FreierDateiname(ordner, Left$(alteDatei, InStrRev(alteDatei, ".") - 1) & "_ersetzt" & Mid$(alteDatei, InStrRev(alteDatei, ".")))
    fehler = DateiUmbenennen(ordner, alteDatei, ziel)
    If Len(fehler) = 0 Then
        LogSchreiben "  alte Datei ausgemustert: " & alteDatei & " -> " & ziel
    Else
        LogSchreiben "  alte Datei nicht umbenannt (" & fehler & "): " & alteDatei
        If IstUrl(ordner) Then
            If Not DateiTraegtGueltigenStand(alteDatei) Then SpAusblenden alteDatei, ordner   ' wie ohne Ablage
        End If
    End If
End Sub

' Steht die Datei im Ladeprotokoll noch bei einem gueltigen Stand (Sammeldatei mit weiteren Stichtagen)?
Private Function DateiTraegtGueltigenStand(rel As String) As Boolean
    Dim ws As Worksheet, letzte As Long, a As Variant, r As Long, h As String
    Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    If letzte < 5 Then Exit Function
    a = ws.Range(ws.Cells(5, 1), ws.Cells(letzte + 1, 10)).Value2
    For r = 1 To UBound(a, 1) - 1
        If StrComp(CStr(Nz(a(r, 2))), rel, vbTextCompare) = 0 Then
            h = CStr(Nz(a(r, 10)))
```

```
1846              If StandGueltig(h) Then DateiTraegtGueltigenStand = True: Exit Function
1847          End If
1848      Next r
1849  End Function
1850
1851  ' Datei ist ein anderer Abzug zu einem geladenen Stand: liegt direkt im Ordner Rohdaten oder ist ein Doppel (…_2) in d
        er Ablage,
1852  ' und das Ladeprotokoll nennt fuer diesen Stand eine andere Datei
1853  Private Function NeuerAbzugZuGeladen(datei As String, schluessel As String, geladenInfo As Object) As Boolean
1854      Dim info As Variant
1855      If Not geladenInfo.Exists(schluessel) Then Exit Function
1856      info = geladenInfo(schluessel)
1857      If StrComp(DateiName(CStr(info(1))), DateiName(datei), vbTextCompare) = 0 Then Exit Function
1858      NeuerAbzugZuGeladen = (InStr(datei, Application.PathSeparator) = 0 Or IstDoppel(datei))
1859  End Function
1860
1861  ' Doppel in der Ablage (JJJJMMTT_BUe_<Quelle>_n) zu einem Stand, der aus einer anderen Datei geladen ist – muss geoeff
        net werden
1862  Private Function DoppelZuGeladen(rel As String, geladenInfo As Object) As Boolean
1863      Dim tag As Date, quellen As Variant, teil As String, q As Variant
1864      If Not IstDoppel(rel) Then Exit Function
1865      If Not NameZerlegen(rel, tag, quellen, teil) Then Exit Function
1866      For Each q In quellen
1867          If NeuerAbzugZuGeladen(rel, NamensSchluessel(CStr(q), teil, tag), geladenInfo) Then DoppelZuGeladen = True
1868      Next q
1869      Exit Function
1870  Nein:
1871      DoppelZuGeladen = False
1872  End Function
1873
1874  Private Function IstDoppel(rel As String) As Boolean
1875      Dim nm As String
1876      nm = DateiName(rel)
1877      nm = Left$(nm, InStrRev(nm, ".") - 1)
1878      IstDoppel = (nm Like "########_BUe_*_#" Or nm Like "########_BUe_*_##")
1879  End Function
1880
1881  Private Function DateiName(pfad As String) As String
1882      DateiName = Mid$(pfad, InStrRev(Replace(pfad, "/", Application.PathSeparator), Application.PathSeparator) + 1)
1883  End Function
1884
1885  ' ersetzte oder nicht uebernommene Rohdateien und im Bereich MAKLER ignorierte HS-Abzuege (12.91) werden beim Laden ni
        cht mehr gelesen
1886  Private Function AusgemustertName(datei As String) As Boolean
1887      Dim nm As String
1888      nm = LCase$(Left$(datei, InStrRev(datei, ".") - 1))
1889      AusgemustertName = (nm Like "*_ersetzt" Or nm Like "*_ersetzt_#*" Or nm Like "*_verworfen" Or nm Like "*_verworfen
        _#*" Or _
1890                          nm Like "*" & LCase$(HS_ENDUNG) Or nm Like "*" & LCase$(HS_ENDUNG) & "_#*")
1891  End Function
1892
1893  ' Angaben fuer die Ersatz-Frage: 0 Quelle, 1 Stichtag, 2 geladen am (leer, wenn unbekannt), 3 Datensaetze geladen,
1894  ' 4 Rueckstand geladen, 5 Name der neuen Datei, 6 Zeilen der neuen Datei (rund: Blattende minus Kopfzeile), 7 Einheit
        (MAKLER)
1895  Private Function ErsatzDaten(st As Stand, info As Variant) As Variant
1896      Dim am As Variant
1897      If IsNumeric(info(0)) And Not IsEmpty(info(0)) Then am = CDate(info(0))
1898      ErsatzDaten = Array(st.Quelle, st.Stichtag, am, CDbl(info(2)), CDbl(info(3)), DateiName(st.Datei), CDbl(st.Zeilen)
        , st.Gruppe)
1899  End Function
1900
1901  ' ganze Zahl mit Punkt als Tausendertrenner, unabhaengig von der Spracheinstellung von Windows
1902  Public Function ZahlDE(n As Double) As String
1903      Dim s As String, i As Long, t As String, minus As Boolean
1904      minus = (n < 0)
1905      s = CStr(Abs(Round(n, 0)))
1906      For i = Len(s) To 1 Step -1
1907          t = Mid$(s, i, 1) & t
1908          If (Len(s) - i) Mod 3 = 2 And i > 1 Then t = "." & t
1909      Next i
1910      ZahlDE = IIf(minus, "-", "") & t
1911  End Function
1912
1913  ' ---------------------------------------------------------------- Celonis (Process Mining): Kopie jeder Datei, aus de
        r geladen wird
1914  ' Ziel: Konfiguration!B6 (z. B. H:\Process-Mining-Sync\Upload), leer = aus. Dateiname wie in der Ablage, ohne Doppel-E
        ndung _2,
1915  ' damit ein ersetzter Stand die alte Kopie ueberschreibt. Fehler halten das Laden nicht auf, sie stehen in Ladelog.txt
         und
1916  ' in der Abschlussmeldung.
1917  Private Sub CelonisEinmal(ordner As String, datei As String, gesendet As Object, ueberschreiben As Boolean, stichtag A
        s Date)
1918      Dim ab As Date, zn As String, al As String
1919      If gesendet.Exists(LCase$(datei)) Then Exit Sub        ' Sammeldatei mit mehreren Staenden: nur einmal
1920      ' Konfiguration!F8 „Celonis ab Stichtag“: aeltere Staende gingen schon frueher an Celonis (Neuaufbau, neue Master-
        Datei)
1921      ab = CelonisAb()
1922      If stichtag > 0 And ab > 0 And CLng(stichtag) < CLng(ab) Then
1923          If Len(Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B6").Value))) > 0 Then celonisAlt = celonisA
        lt + 1
1924          Exit Sub
1925      End If
1926      gesendet.Add LCase$(datei), True
1927      zn = ""
1928      If Not celonisNamen Is Nothing Then
1929          If celonisNamen.Exists(LCase$(datei)) Then zn = CStr(celonisNamen(LCase$(datei)))
```

```
    End If
    al = ""
    If Not celonisAlias Is Nothing Then
        If celonisAlias.Exists(LCase$(datei)) Then al = CStr(celonisAlias(LCase$(datei)))
    End If
    If CelonisSenden(PfadVon(ordner, datei), ueberschreiben, zn, al) Then
        CelonisOffenErledigt datei
    Else
        CelonisOffenMerken datei, zn, ueberschreiben          ' Ordner nicht erreichbar oder Kopie gescheitert: naechstes Laden versucht es erneut
    End If
End Sub

' Gescheiterte Uebergaben (Celonis-Ordner nicht erreichbar, Kopie fehlgeschlagen) im Blatt Celonis_Log, Spalten F:I (Rohdatei relativ,
' einheitlicher Name, seit, „ja“ = Ersatz: ueberschreibt die alte Kopie). Jedes Laden versucht sie vor der Verarbeitung erneut
' (CelonisOffenNachholen) – die Rohdatei wird ja nicht noch einmal geladen, sonst ginge sie nie an Celonis.
Private Sub CelonisOffenMerken(rel As String, zielName As String, ueberschreiben As Boolean)
    Dim ws As Worksheet, r As Long
    On Error Resume Next
    Set ws = CelonisBlatt()
    If ws Is Nothing Then Exit Sub
    If Len(CStr(ws.Range("I1").Value)) = 0 Then ws.Range("I1").Value = "Ersatz"     ' Blatt aus 12.65 ohne Spalte I
    r = CelonisOffenZeile(ws, rel)
    If r > 0 Then
        If ueberschreiben Then ws.Cells(r, 9).Value = "ja"
        Exit Sub
    End If
    r = ws.Cells(ws.Rows.Count, 6).End(xlUp).Row + 1
    If r < 2 Then r = 2
    ws.Cells(r, 6).Value = "'" & rel
    ws.Cells(r, 7).Value = "'" & zielName
    ws.Cells(r, 8).Value = Now
    ws.Cells(r, 9).Value = IIf(ueberschreiben, "ja", "")
    LogSchreiben "  Celonis: offen vermerkt: " & rel & IIf(ueberschreiben, " (Ersatz)", "")
End Sub

Private Sub CelonisOffenErledigt(rel As String)
    Dim ws As Worksheet, r As Long
    On Error Resume Next
    Set ws = ThisWorkbook.Worksheets("Celonis_Log")
    If ws Is Nothing Then Exit Sub
    r = CelonisOffenZeile(ws, rel)
    If r > 0 Then ws.Range(ws.Cells(r, 6), ws.Cells(r, 9)).Delete Shift:=xlUp
End Sub

' Rohdatei umbenannt und abgelegt (NeueDatenLaden, Schritt 1): eine offene Uebergabe – etwa eines Kollegen im SharePoint-Modus, der
' die Datei unter ihrem MIS-O-Namen vermerkt hat – auf den neuen Pfad umschreiben, damit CelonisOffenNachholen sie wiederfindet
Private Sub CelonisOffenUmbenennen(alt As String, neu As String)
    Dim ws As Worksheet, r As Long
    On Error Resume Next
    Set ws = ThisWorkbook.Worksheets("Celonis_Log")
    If ws Is Nothing Then Exit Sub
    r = CelonisOffenZeile(ws, alt)
    If r > 0 Then
        ws.Cells(r, 6).Value = "'" & neu
        LogSchreiben "  Celonis: offene Übergabe umgeschrieben: " & alt & " -> " & neu
    End If
End Sub

' Rohdatei einer offenen Uebergabe, die nicht mehr unter ihrem Pfad liegt (ein Kollege hat sie umbenannt und abgelegt): unter dem
' einheitlichen Namen im Monatsordner suchen (JJJJ\MM_Monat aus dem Datum am Namensanfang, wie ZielDateiname), auch die Doppel _2 … _9;
' gibt es mehrere, das zuletzt abgelegte (hoechstes _n). Ergebnis: relativer Pfad, leer = nicht gefunden.
Private Function CelonisOffenSuchen(ordner As String, rel As String, zielName As String) As String
    Dim nm As String, basis As String, endung As String, monat As String, kand As String, i As Long
    nm = zielName
    If Len(nm) = 0 Then nm = DateiName(rel)
    If InStrRev(nm, ".") = 0 Then Exit Function
    If Not (nm Like "20######*_BUe_*") Then Exit Function
    If Not (Mid$(nm, 5, 2) Like "[01]#") Then Exit Function
    If Not (Mid$(nm, 7, 2) Like "[0-3]#") Then Exit Function
    On Error GoTo Ende
    monat = MonatsOrdner(DateSerial(CInt(Left$(nm, 4)), CInt(Mid$(nm, 5, 2)), CInt(Mid$(nm, 7, 2))))
    basis = Left$(nm, InStrRev(nm, ".") - 1)
    endung = Mid$(nm, InStrRev(nm, "."))
    If IstDoppel(nm) Then basis = Left$(basis, InStrRev(basis, "_") - 1)
    For i = 1 To 9
        kand = monat & Application.PathSeparator & basis & IIf(i = 1, "", "_" & i) & endung
        If DateiVorhanden(PfadVon(ordner, kand)) Then CelonisOffenSuchen = kand
    Next i
Ende:
End Function

' Beim Ablegen (lokaler Lauf): Wurde die Datei unter ihrem bisherigen Namen schon an Celonis uebergeben (SharePoint-Modus bis 12.64),
' den einheitlichen Namen sofort dauerhaft als Alias ins Protokoll schreiben. Sonst kennt nur dieser Lauf die Zuordnung, und nach einem
' Neuaufbau oder mit einer neuen Master-Datei ginge die Datei erneut an Celonis. Nur bei gleicher Groesse (derselbe Abzug).
Private Sub CelonisAliasAblegen(ordner As String, altName As String, neuRel As String)
    Dim nm As String, basis As String, endung As String, g As Double
```

```
    On Error GoTo Ende
    If Len(Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B6").Value))) = 0 Then Exit Sub   ' Celonis aus
    If celonisLog Is Nothing Then Set celonisLog = CelonisLogLesen()
    If Not celonisLog.Exists(LCase$(altName)) Then Exit Sub
    nm = DateiName(neuRel)
    basis = Left$(nm, InStrRev(nm, ".") - 1)
    endung = Mid$(nm, InStrRev(nm, "."))
    If IstDoppel(nm) Then basis = Left$(basis, InStrRev(basis, "_") - 1)
    If StrComp(altName, basis & endung, vbTextCompare) = 0 Then Exit Sub
    If celonisLog.Exists(LCase$(basis & endung)) Then Exit Sub
    g = DateiGroesse(PfadVon(ordner, neuRel))
    If g <> celonisLog(LCase$(altName)) Then Exit Sub
    CelonisLogSchreiben basis & endung, g, "Alias von " & altName
Ende:
End Sub

Private Function CelonisOffenZeile(ws As Worksheet, rel As String) As Long
    Dim letzte As Long, a As Variant, r As Long
    letzte = ws.Cells(ws.Rows.Count, 6).End(xlUp).Row
    If letzte < 2 Then Exit Function
    a = ws.Range(ws.Cells(2, 6), ws.Cells(letzte + 1, 6)).Value2
    For r = 1 To UBound(a, 1) - 1
        If StrComp(CStr(Nz(a(r, 1))), rel, vbTextCompare) = 0 Then CelonisOffenZeile = r + 1: Exit Function
    Next r
End Function

' Offene Uebergaben erneut versuchen; Ergebnis: Anzahl nachgeholter Dateien (nur wirklich kopierte). Liegt die Rohdate
   i nicht mehr
' unter dem vermerkten Pfad, wird sie unter dem einheitlichen Namen gesucht (CelonisOffenSuchen); ist sie auch dort ni
   cht, steht das
' in der Abschlussmeldung und die Zeile entfaellt (sonst bliebe sie still fuer immer stehen).
Private Function CelonisOffenNachholen(ordner As String) As Long
    Dim ws As Worksheet, letzte As Long, a As Variant, r As Long, rel As String, gefunden As String, vorher As Long, e
   rsatz As Boolean
    If Len(Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B6").Value))) = 0 Then Exit Function
    On Error Resume Next
    Set ws = ThisWorkbook.Worksheets("Celonis_Log")
    On Error GoTo 0
    If ws Is Nothing Then Exit Function
    letzte = ws.Cells(ws.Rows.Count, 6).End(xlUp).Row
    If letzte < 2 Then Exit Function
    a = ws.Range(ws.Cells(2, 6), ws.Cells(letzte + 1, 9)).Value2
    For r = UBound(a, 1) - 1 To 1 Step -1
        rel = CStr(Nz(a(r, 1)))
        If Len(rel) > 0 Then
            If Not DateiVorhanden(PfadVon(ordner, rel)) Then
                gefunden = CelonisOffenSuchen(ordner, rel, CStr(Nz(a(r, 2))))
                If Len(gefunden) > 0 Then
                    LogSchreiben "Celonis: offene Übergabe " & rel & " liegt jetzt unter " & gefunden
                    ws.Cells(r + 1, 6).Value = "'" & gefunden
                Else
                    LogSchreiben "Celonis: offene Übergabe nicht mehr auffindbar, aus der Liste genommen: " & rel
                    CelonisFehlerMerken "offene Übergabe " & DateiName(rel) & " nicht mehr auffindbar (umbenannt oder
   gelöscht) und nicht übergeben – bitte prüfen"
                    ws.Range(ws.Cells(r + 1, 6), ws.Cells(r + 1, 9)).Delete Shift:=xlUp
                End If
                rel = gefunden
            End If
            If Len(rel) > 0 Then
                LogSchreiben "Celonis: offene Übergabe erneut: " & rel
                ersatz = (Len(CStr(Nz(a(r, 4)))) > 0)             ' Ersatz: ueberschreibt die alte Kopie trotz bekannt
   en Namens
                vorher = celonisAnzahl
                If CelonisSenden(PfadVon(ordner, rel), ersatz, CStr(Nz(a(r, 2)))) Then
                    ws.Range(ws.Cells(r + 1, 6), ws.Cells(r + 1, 9)).Delete Shift:=xlUp
                    If celonisAnzahl > vorher Then CelonisOffenNachholen = CelonisOffenNachholen + 1
                End If
            End If
        End If
    Next r
End Function

' Blatt Celonis_Log (ausgeblendet): A:D Protokoll der Uebergaben, F:I offene Uebergaben; wird bei Bedarf angelegt
Private Function CelonisBlatt() As Worksheet
    Dim ws As Worksheet, akt As Object
    On Error Resume Next
    Set ws = ThisWorkbook.Worksheets("Celonis_Log")
    On Error GoTo 0
    If ws Is Nothing Then
        Set akt = ActiveSheet
        Set ws = ThisWorkbook.Worksheets.Add(After:=ThisWorkbook.Worksheets(ThisWorkbook.Worksheets.Count))
        ws.Name = "Celonis_Log"
        ws.Range("A1:D1").Value = Array("Zeitpunkt", "Datei", "Bytes", "Herkunft")
        ws.Range("F1:I1").Value = Array("Offen: Rohdatei", "Name für Celonis", "seit", "Ersatz")
        ws.Visible = xlSheetVeryHidden
        If Not akt Is Nothing Then akt.Activate
    End If
    Set CelonisBlatt = ws
End Function

Private Sub CelonisFehlerMerken(t As String)
    If InStr(1, celonisFehler, t, vbTextCompare) > 0 Then Exit Sub
    celonisFehler = celonisFehler & IIf(Len(celonisFehler) > 0, "; ", "") & t
End Sub

Private Function CelonisSenden(quelle As String, ueberschreiben As Boolean, Optional zielName As String = "", Optional
```

```
        aliasName As String = "") As Boolean
2107    Dim ziel As String, nm As String, basis As String, endung As String, fehler As String, groesse As Double, bekannt
        As Double
2108    ziel = Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B6").Value))
2109    CelonisSenden = True
2110    If Len(ziel) = 0 Then Exit Function                    ' Celonis aus: nichts zu tun, kein Fehler
2111    If Right$(ziel, 1) <> Application.PathSeparator Then ziel = ziel & Application.PathSeparator
2112    If Not OrdnerVorhanden(ziel) Then
2113        CelonisFehlerMerken "Ordner " & ziel & " nicht erreichbar, Übergabe beim nächsten Laden"
2114        LogSchreiben "  Celonis: Ordner nicht erreichbar: " & ziel
2115        CelonisSenden = False
2116        Exit Function
2117    End If
2118    nm = DateiName(quelle)
2119    basis = Left$(nm, InStrRev(nm, ".") - 1)
2120    endung = Mid$(nm, InStrRev(nm, "."))
2121    If IstDoppel(nm) Then basis = Left$(basis, InStrRev(basis, "_") - 1)
2122    If Len(zielName) > 0 Then                              ' SharePoint: einheitlicher Name, obwohl die Rohdatei nic
       ht umbenannt ist
2123        basis = Left$(zielName, InStrRev(zielName, ".") - 1)
2124        endung = Mid$(zielName, InStrRev(zielName, "."))
2125    End If
2126    ' Schon uebergeben? Der Server holt die Dateien alle zwei Stunden ab und leert den Ordner – deshalb fuehrt das Too
       l ein
2127    ' eigenes Protokoll (Celonis_Upload_Log.txt neben der Master-Datei). Steht die Datei darin oder liegt sie noch im
       Ordner,
2128    ' wird sie nicht erneut kopiert. Ausnahme: ein aktuellerer Abzug (Ersatz oder Doppel …_2) mit anderer Groesse.
2129    groesse = DateiGroesse(quelle)
2130    If celonisLog Is Nothing Then Set celonisLog = CelonisLogLesen()
2131    If celonisLog.Exists(LCase$(basis & endung)) Then
2132        bekannt = celonisLog(LCase$(basis & endung))
2133    ElseIf Len(aliasName) > 0 And StrComp(aliasName, basis & endung, vbTextCompare) <> 0 And celonisLog.Exists(LCase$(
       aliasName)) Then
2134        ' unter dem frueheren Namen uebergeben (MIS-O-Name aus dem SharePoint-Modus bis 12.64, Originalname vor dem Um
       benennen):
2135        ' nur derselbe Abzug (gleiche Groesse) gilt als uebergeben und bekommt den Namen nachgetragen. Die Groessenpru
       efung steht im
2136        ' Rumpf, nicht in der Bedingung: VBA wertet And nicht verkuerzt aus.
2137        bekannt = celonisLog(LCase$(aliasName))
2138        If bekannt = groesse Then
2139            CelonisLogSchreiben basis & endung, bekannt, "Alias von " & aliasName
2140        Else
2141            bekannt = -1                                   ' anderer Abzug unter demselben Rohnamen: normal ueberge
       ben
2142        End If
2143    ElseIf DateiVorhanden(ziel & basis & endung) Then
2144        bekannt = DateiGroesse(ziel & basis & endung)
2145        CelonisLogSchreiben basis & endung, bekannt, "lag schon im Ordner"
2146    Else
2147        bekannt = -1
2148    End If
2149    If bekannt >= 0 Then
2150        If bekannt = groesse Or Not (ueberschreiben Or IstDoppel(nm)) Then
2151            celonisVorhanden = celonisVorhanden + 1
2152            Exit Function
2153        End If
2154    End If
2155    modFortschritt.FortschrittSetzen ladeAnteil, CStr(IIf(Len(ladeStatus) > 0, ladeStatus, "Übergabe an Celonis")), "Ü
       bergabe an Celonis: " & basis & endung   ' nur, wenn wirklich kopiert wird
2156    If IstUrl(quelle) And Len(CacheVon(quelle)) > 0 Then
2157        On Error Resume Next                               ' SharePoint: lokale Kopie aus diesem Laden
2158        FileCopy CacheVon(quelle), ziel & basis & endung
2159        If Err.Number <> 0 Then fehler = Err.Description
2160        On Error GoTo 0
2161    ElseIf IstUrl(quelle) Then
2162        fehler = UrlKopieren(quelle, ziel & basis & endung)        ' SharePoint: oeffnen und als Kopie speichern
2163    Else
2164        On Error Resume Next
2165        FileCopy quelle, ziel & basis & endung
2166        If Err.Number <> 0 Then fehler = Err.Description
2167        On Error GoTo 0
2168    End If
2169    If Len(fehler) = 0 Then
2170        celonisAnzahl = celonisAnzahl + 1
2171        CelonisLogSchreiben basis & endung, groesse, quelle
2172        LogSchreiben "  Celonis: " & nm & " -> " & ziel & basis & endung
2173    Else
2174        CelonisFehlerMerken basis & endung & " nicht kopiert (" & fehler & "), Übergabe beim nächsten Laden"
2175        LogSchreiben "  Celonis FEHLER (" & fehler & "): " & nm
2176        CelonisSenden = False
2177    End If
2178 End Function
2179
2180 Private Function DateiGroesse(pfad As String) As Double
2181    Dim e As Variant
2182    On Error Resume Next
2183    DateiGroesse = -2
2184    If IstUrl(pfad) Then                                   ' SharePoint: Groesse aus der Auflistung
2185        e = SpEintrag(pfad)
2186        If IsArray(e) Then
2187            If IsNumeric(e(2)) And Not IsEmpty(e(2)) Then DateiGroesse = CDbl(e(2))
2188        End If
2189    Else
2190        DateiGroesse = FileLen(pfad)
2191    End If
2192    On Error GoTo 0
```

```
End Function

' Datei von SharePoint als Kopie in einen lokalen Ordner sichern (FileCopy geht mit https nicht); Ergebnis leer = ok
Private Function UrlKopieren(url As String, ziel As String) As String
    Dim wb As Workbook
    On Error GoTo Fehler
    Set wb = QuelleOeffnen(url)
    Application.DisplayAlerts = False
    wb.SaveCopyAs ziel
    Application.DisplayAlerts = True
    wb.Close SaveChanges:=False
    Exit Function
Fehler:
    UrlKopieren = Err.Description
    On Error Resume Next
    Application.DisplayAlerts = True
    If Not wb Is Nothing Then wb.Close SaveChanges:=False
End Function

' Celonis-Protokoll an drei Stellen, damit es einen Wechsel der Master-Datei (neue Version, Neuaufbau, SharePoint) ueb
    ersteht:
' Celonis_Upload_Log.txt im Ordner der Master-Datei (lokal) bzw. in %LOCALAPPDATA%\BUE_Reporting, dort immer zusaetzli
    ch, und das
' ausgeblendete Blatt Celonis_Log in der Master-Datei. Gelesen wird die Vereinigung aller drei.
Private Function CelonisLogPfade() As Collection
    ' neben einer lokal erreichbaren Master-Datei (auch synchronisierter SharePoint-Ordner) und in %LOCALAPPDATA%\BUE_
    Reporting –
    ' dort je Bereich getrennt, damit HS- und Makler-Master auf demselben Rechner sich nicht gegenseitig Dateien als u
    ebergeben melden
    Dim c As New Collection, b As String
    On Error Resume Next
    b = BasisOrdner()
    If Len(b) > 0 And Not IstUrl(b) Then c.Add ProtokollPfad(b, CELONIS_LOG)
    c.Add AppDatenOrdner() & Application.PathSeparator & IIf(IstMakler(), "Celonis_Upload_Log_Makler.txt", CELONIS_LOG
    )
    Set CelonisLogPfade = c
End Function

' Konfiguration!F8: Staende vor diesem Stichtag gehen nicht an Celonis (0 = keine Grenze)
Private Function CelonisAb() As Date
    Dim v As Variant
    On Error Resume Next
    v = ThisWorkbook.Worksheets("Konfiguration").Range("F8").Value
    If IsDate(v) Then CelonisAb = CDate(v)
End Function

' Protokoll lesen: Dateiname (klein) -> Groesse der zuletzt uebergebenen Fassung. Zeilen: Zeitpunkt TAB Datei TAB Byte
    s TAB Herkunft
Private Function CelonisLogLesen() As Object
    Dim d As Object, f As Integer, zeile As String, t() As String, p As Variant, ws As Worksheet, a As Variant, r As L
    ong, letzte As Long
    Set d = CreateObject("Scripting.Dictionary")
    On Error Resume Next
    For Each p In CelonisLogPfade()
        If Len(Dir(CStr(p))) > 0 Then
            f = FreeFile
            Open CStr(p) For Input As #f
            Do While Not EOF(f)
                Line Input #f, zeile
                If Len(LogSchluessel(zeile)) > 0 Then d(LogSchluessel(zeile)) = LogGroesse(zeile)
            Loop
            Close #f
        End If
    Next p
    ' SharePoint ohne Dateipfad: das Protokoll neben der Master-Datei ueber Power Query lesen (dort schreiben auch Kol
    legen mit
    ' synchronisiertem Ordner hinein); nach dem Laden werden die neuen Zeilen dort angehaengt (CelonisSpZurueckschreib
    en)
    If IstUrl(BasisOrdner()) Then
        CelonisSpLesen
        If Not celonisSp Is Nothing Then
            For Each p In celonisSp
                If Len(LogSchluessel(CStr(p))) > 0 Then d(LogSchluessel(CStr(p))) = LogGroesse(CStr(p))
            Next p
        End If
    End If
    Set ws = Nothing
    Set ws = ThisWorkbook.Worksheets("Celonis_Log")
    If Not ws Is Nothing Then
        letzte = ws.Cells(ws.Rows.Count, 2).End(xlUp).Row
        If letzte >= 2 Then
            a = ws.Range(ws.Cells(2, 1), ws.Cells(letzte + 1, 3)).Value2
            For r = 1 To UBound(a, 1) - 1
                If Len(CStr(a(r, 2))) > 0 Then d(LCase$(Trim$(CStr(a(r, 2))))) = Val(a(r, 3))
            Next r
        End If
    End If
    Set CelonisLogLesen = d
End Function

' Feld des Protokolls: Tabulatoren und Anfuehrungszeichen ersetzen (Speichern als Text wuerde das Feld sonst in "…" se
    tzen)
Private Function LogText(t As String) As String
    LogText = Replace(Replace(Replace(t, vbTab, " "), """", "'"), vbLf, " ")
End Function

```

```
2279  ' Dateiname aus einer Protokollzeile (Feld 2), klein, ohne umschliessende Anfuehrungszeichen; leer, wenn die Zeile kei
         ns hat
2280  Private Function LogSchluessel(zeile As String) As String
2281      Dim t() As String
2282      t = Split(zeile, vbTab)
2283      If UBound(t) < 2 Then Exit Function
2284      LogSchluessel = LCase$(TextFeld(t(1)))
2285  End Function
2286
2287  ' Feld einer als Text gespeicherten Zeile: ohne umschliessende Anfuehrungszeichen, "" -> " (so speichert Excel Felder
         mit Komma)
2288  Private Function TextFeld(t As String) As String
2289      Dim k As String
2290      k = Trim$(t)
2291      If Len(k) >= 2 And Left$(k, 1) = """" And Right$(k, 1) = """" Then k = Replace(Mid$(k, 2, Len(k) - 2), """""", """
         ")
2292      TextFeld = k
2293  End Function
2294
2295  ' Groesse aus einer Protokollzeile (Feld 3)
2296  Private Function LogGroesse(zeile As String) As Double
2297      Dim t() As String
2298      t = Split(zeile, vbTab)
2299      If UBound(t) >= 2 Then LogGroesse = Val(Replace(t(2), """", ""))
2300  End Function
2301
2302  ' Celonis_Upload_Log.txt neben der Master-Datei auf SharePoint lesen (celonisSp bleibt Nothing, wenn das nicht gelingt
         – dann wird
2303  ' dort auch nichts zurueckgeschrieben, damit fremde Zeilen nicht verloren gehen)
2304  Private Sub CelonisSpLesen()
2305      Dim a As Variant, r As Long, m As String
2306      If Not celonisSp Is Nothing Then Exit Sub
2307      On Error GoTo Fehler
2308      m = SpTextFormel(BasisOrdner(), CELONIS_LOG, PROTOKOLL_ORDNER)
2309      If Len(m) = 0 Then Exit Sub
2310      a = SpTabelle(SP_TEXT_ABFRAGE, m, "L1", "Textdatei aus dem SharePoint-Ordner der Master-Datei (Celonis-Protokoll,
        wird vom Tool geschrieben)")
2311      Set celonisSp = New Collection
2312      If IsArray(a) Then
2313          For r = 1 To UBound(a, 1)
2314              If Len(CStr(Nz(a(r, 1)))) > 0 Then celonisSp.Add CStr(a(r, 1))
2315          Next r
2316      End If
2317      celonisSpFehler = ""
2318      LogSchreiben "  Celonis-Protokoll auf SharePoint gelesen: " & celonisSp.Count & " Zeilen"
2319      Exit Sub
2320  Fehler:
2321      Set celonisSp = Nothing
2322      celonisSpFehler = Err.Description
2323      LogSchreiben "  Celonis-Protokoll auf SharePoint nicht lesbar: " & Err.Description
2324  End Sub
2325
2326  ' Neue Zeilen an Celonis_Upload_Log.txt auf SharePoint anhaengen: Datei aus gelesenen und neuen Zeilen neu aufbauen un
         d ueber
2327  ' Excel als Text (Tabulator, Windows-1252 – wie Print #) unter derselben Adresse speichern. Ergebnis leer = ok, sonst
         Fehlertext.
2328  Private Function CelonisSpZurueckschreiben() As String
2329      Dim wb As Workbook, a() As Variant, n As Long, r As Long, c As Long, t() As String, z As Variant, ziel As String
2330      Dim bekannt As Object, p As Variant, f As Integer, zeile As String, k As String, neu As Collection
2331      If celonisSpNeu Is Nothing Then Exit Function
2332      If Not IstUrl(BasisOrdner()) Then Exit Function          ' lokaler/synchronisierter Ordner: Print # schreibt die D
        atei direkt
2333      If Len(Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B6").Value))) = 0 Then Exit Function   ' Celonis
         aus
2334      On Error GoTo Fehler
2335      ' unmittelbar vorher neu lesen: Zeilen, die Kollegen waehrend des Ladens angehaengt haben, bleiben erhalten
2336      Set celonisSp = Nothing
2337      CelonisSpLesen
2338      If celonisSp Is Nothing Then
2339          CelonisSpZurueckschreiben = "Protokoll " & CELONIS_LOG & " auf SharePoint nicht lesbar (" & celonisSpFehler &
        "), deshalb nicht fortgeschrieben"
2340          Exit Function
2341      End If
2342      Set bekannt = CreateObject("Scripting.Dictionary")
2343      For Each z In celonisSp
2344          k = LogSchluessel(CStr(z))
2345          If Len(k) > 0 Then bekannt(k) = True
2346      Next z
2347      ' neue Zeilen dieses Ladens und Uebergaben, die nur lokal vermerkt sind (12.62/12.63, frueher gescheitertes Schrei
        ben)
2348      Set neu = New Collection
2349      For Each z In celonisSpNeu
2350          k = LogSchluessel(CStr(z))
2351          If Len(k) > 0 And Not bekannt.Exists(k) Then neu.Add CStr(z): bekannt(k) = True
2352      Next z
2353      For Each p In CelonisLogPfade()
2354          If Len(Dir(CStr(p))) > 0 Then
2355              f = FreeFile
2356              Open CStr(p) For Input As #f
2357              Do While Not EOF(f)
2358                  Line Input #f, zeile
2359                  k = LogSchluessel(zeile)
2360                  If Len(k) > 0 And Not bekannt.Exists(k) Then neu.Add zeile: bekannt(k) = True
2361              Loop
2362              Close #f
2363              f = 0
```

```
2364          End If
2365      Next p
2366      If neu.Count = 0 Then Set celonisSpNeu = New Collection: Exit Function
2367      n = celonisSp.Count + neu.Count
2368      ReDim a(1 To n, 1 To 4)
2369      For Each z In celonisSp                                          ' vorhandene Zeilen normalisiert (sonst wuerden A
        nfuehrungszeichen,
2370          r = r + 1                                                    ' die Excel beim Speichern um Felder mit Komma se
        tzt, jedes Mal mehr)
2371          t = Split(CStr(z), vbTab)
2372          For c = 0 To UBound(t)
2373              If c <= 3 Then a(r, c + 1) = LogText(TextFeld(t(c))) Else a(r, 4) = a(r, 4) & " " & LogText(TextFeld(t(c))
      )
2374          Next c
2375      Next z
2376      For Each z In neu
2377          r = r + 1
2378          t = Split(CStr(z), vbTab)
2379          For c = 0 To UBound(t)
2380              If c <= 3 Then a(r, c + 1) = LogText(t(c)) Else a(r, 4) = a(r, 4) & " " & LogText(t(c))
2381          Next c
2382      Next z
2383      Set wb = Workbooks.Add(xlWBATWorksheet)
2384      wb.Worksheets(1).Range("A1:D" & n).NumberFormat = "@"          ' alles als Text, nichts umformatieren
2385      wb.Worksheets(1).Range("A1:D" & n).Value = a
2386      ziel = SpTextSpeichern(wb, BasisOrdner(), CELONIS_LOG)          ' xlTextWindows, in Protokolle (sonst neben der Ma
        ster-Datei)
2387      wb.Close SaveChanges:=False
2388      ThisWorkbook.Activate
2389      LogSchreiben "  Celonis-Protokoll auf SharePoint fortgeschrieben: " & neu.Count & " neue Zeilen"
2390      For Each z In neu
2391          celonisSp.Add z
2392      Next z
2393      Set celonisSpNeu = New Collection
2394      Exit Function
2395  Fehler:
2396      CelonisSpZurueckschreiben = Err.Description
2397      On Error Resume Next
2398      If f > 0 Then Close #f
2399      Application.DisplayAlerts = True
2400      If Not wb Is Nothing Then wb.Close SaveChanges:=False
2401      ThisWorkbook.Activate
2402      LogSchreiben "  Celonis-Protokoll auf SharePoint NICHT fortgeschrieben: " & CelonisSpZurueckschreiben
2403  End Function
2404
2405  Private Sub CelonisLogSchreiben(datei As String, groesse As Double, herkunft As String)
2406      Dim f As Integer, p As Variant, ws As Worksheet, r As Long, akt As Object, zeile As String
2407      On Error Resume Next
2408      If celonisLog Is Nothing Then Set celonisLog = CelonisLogLesen()
2409      celonisLog(LCase$(datei)) = groesse
2410      zeile = Format(Now, "yyyy-mm-dd hh:nn:ss") & vbTab & LogText(datei) & vbTab & CStr(groesse) & vbTab & LogText(herk
        unft)
2411      For Each p In CelonisLogPfade()
2412          f = FreeFile
2413          Open CStr(p) For Append As #f
2414          Print #f, zeile
2415          Close #f
2416      Next p
2417      If Not celonisSpNeu Is Nothing Then celonisSpNeu.Add zeile    ' fuer SharePoint vormerken (geschrieben am Ende des
        Ladens)
2418      Set ws = CelonisBlatt()
2419      r = ws.Cells(ws.Rows.Count, 2).End(xlUp).Row + 1
2420      If r < 2 Then r = 2
2421      ws.Range(ws.Cells(r, 1), ws.Cells(r, 4)).Value = Array(Now, datei, groesse, herkunft)
2422  End Sub
2423
2424  ' ------------------------------------------------------------------ Neue Themenpostkoerbe ausserhalb HS (Karte modStand
        ard nach dem Laden)
2425  Public Function NeuePostkoerbeAnzahl() As Long
2426      If Not neuePK Is Nothing Then NeuePostkoerbeAnzahl = neuePK.Count
2427  End Function
2428
2429  Public Function IstNeuerPostkorb(nm As String) As Boolean
2430      If Not neuePK Is Nothing Then IstNeuerPostkorb = neuePK.Exists(nm)
2431  End Function
2432
2433  Public Function NeuePostkoerbeText() As String
2434      Dim k As Variant, t As String
2435      If neuePK Is Nothing Then Exit Function
2436      For Each k In neuePK.Keys
2437          If Len(t) > 0 Then t = t & ", "
2438          t = t & k
2439      Next k
2440      NeuePostkoerbeText = t
2441  End Function
2442
2443  Public Sub NeuePostkoerbeErledigt()
2444      If Not neuePK Is Nothing Then neuePK.RemoveAll
2445  End Sub
2446
2447  ' ------------------------------------------------------------------ Ersatz-Frage (Karte in modTagesablauf)
2448  Public Function ErsatzFrageOffen() As Boolean
2449      If ersatzOffen Is Nothing Then Exit Function
2450      ErsatzFrageOffen = (ersatzOffen.Count > 0)
2451  End Function
2452
```

```
Public Function ErsatzAnzahl() As Long
    If Not ersatzOffen Is Nothing Then ErsatzAnzahl = ersatzOffen.Count
End Function

' Angabe k (siehe ErsatzDaten) des i-ten offenen Stands (1 = erster)
Public Function ErsatzWert(i As Long, k As Long) As Variant
    Dim w As Variant
    If ersatzOffen Is Nothing Then Exit Function
    If i < 1 Or i > ersatzOffen.Count Then Exit Function
    w = ersatzOffen.Items()(i - 1)
    ErsatzWert = w(k)
End Function

' Klartext fuer das Ersatzfenster ohne Formen (MsgBox)
Public Function ErsatzFrageText() As String
    Dim i As Long, t As String
    For i = 1 To ErsatzAnzahl()
        If Len(t) > 0 Then t = t & vbLf
        t = t & Strang(CStr(ErsatzWert(i, 0)), CStr(ErsatzWert(i, 7))) & " " & Format$(ErsatzWert(i, 1), "dd.mm.yyyy")
    & ": geladen " & ZahlDE(CDbl(ErsatzWert(i, 3))) & _
            " Datensätze, neue Datei " & ErsatzWert(i, 5) & " rund " & ZahlDE(CDbl(ErsatzWert(i, 6))) & " Zeilen"
    Next i
    ErsatzFrageText = t
End Function

Public Function ErsatzAusAblauf() As Boolean
    ErsatzAusAblauf = ersatzImAblauf
End Function

' Antwort auf die Frage: True = neue Datei ersetzt den Stand, False = alter Stand bleibt; danach laedt der Aufrufer er
   neut
Public Sub ErsatzEntscheiden(ersetzen As Boolean)
    Dim k As Variant
    If ersatzWahl Is Nothing Then Set ersatzWahl = CreateObject("Scripting.Dictionary")
    If ersatzOffen Is Nothing Then Exit Sub
    For Each k In ersatzOffen.Keys
        ersatzWahl(k) = ersetzen
    Next k
    ersatzOffen.RemoveAll
End Sub

Public Sub ErsatzAbbrechen()
    If Not ersatzOffen Is Nothing Then ersatzOffen.RemoveAll
    If Not ersatzWahl Is Nothing Then ersatzWahl.RemoveAll
End Sub

' ---------------------------------------------------------------- Hilfsfunktionen Verarbeitung
Private Function Nz(v As Variant) As Variant
    If IsError(v) Or IsEmpty(v) Or IsNull(v) Then Nz = "" Else Nz = v
End Function

Private Function Sauber(v As Variant) As String
    ' Text ohne Rand und ohne das Trennzeichen der Schluessel
    Sauber = Replace(Trim$(CStr(Nz(v))), TRENNER, "/")
End Function

Public Function Verdichten(t As String) As String
    Dim s As String
    s = Trim$(Replace(Replace(Replace(t, vbTab, " "), vbLf, " "), TRENNER, "/"))
    Do While InStr(s, "  ") > 0: s = Replace(s, "  ", " "): Loop
    Verdichten = s
End Function

Public Function PostkorbNorm(t As String) As String
    Dim s As String
    s = Verdichten(t)
    If Len(s) = 0 Then s = "(ohne Postkorb)"
    PostkorbNorm = s
End Function

Public Sub GevoTyp(gevo As String, ByRef typ As String, ByRef nr As Long)
    ' "SV 108 Bestandsübertragung" -> "108 Bestandsübertragung", 108 ; ". Ohne Zuordnung" -> "Ohne Zuordnung", 0
    Dim s As String, p As Long, erstes As String, zahl As String, i As Long
    s = Verdichten(gevo)
    nr = 0
    zahl = ""
    If Len(s) = 0 Then typ = "(ohne GeVo)": Exit Sub
    p = InStr(s, " ")
    If p = 3 Then
        erstes = Left$(s, 2)
        If erstes Like "[A-Z][A-Z]" Then s = Mid$(s, 4)
    End If
    Do While Len(s) > 0 And (Left$(s, 1) = "." Or Left$(s, 1) = " ")
        s = Mid$(s, 2)
    Loop
    typ = s
    For i = 1 To Len(s)
        If Mid$(s, i, 1) Like "#" Then zahl = zahl & Mid$(s, i, 1) Else Exit For
    Next i
    If Len(zahl) > 0 Then nr = CLng(zahl)
End Sub

Private Function AlterKlasse(v As Variant) As Long
    Dim t As Double
    If IsNumeric(v) Then t = CDbl(v) Else t = 0
    If t <= 7 Then
```

```
2547          AlterKlasse = 0
2548      ElseIf t <= 30 Then
2549          AlterKlasse = 1
2550      ElseIf t <= 90 Then
2551          AlterKlasse = 2
2552      Else
2553          AlterKlasse = 3
2554      End If
2555  End Function
2556
2557  Private Function SpartenZuordnung() As Object
2558      Dim d As Object, ws As Worksheet, r As Long, daten As Variant
2559      Set d = CreateObject("Scripting.Dictionary")
2560      Set ws = ThisWorkbook.Worksheets("Konfiguration")
2561      daten = ws.Range("A11:B40").Value2
2562      For r = 1 To UBound(daten, 1)
2563          If Len(Trim$(CStr(Nz(daten(r, 1))))) > 0 Then
2564              If Not d.Exists(Trim$(CStr(daten(r, 1)))) Then d.Add Trim$(CStr(daten(r, 1))), Trim$(CStr(Nz(daten(r, 2)))
      )
2565          End If
2566      Next r
2567      Set SpartenZuordnung = d
2568  End Function
2569
2570  ' ------------------------------------------------------------- Ordner
2571  Public Function RohdatenPfad() As String
2572      ' fuer den Tagesablauf: aufgeloester Rohdaten-Ordner (leer, wenn der Pfad nicht aufgeloest werden kann)
2573      On Error Resume Next
2574      RohdatenPfad = RohdatenOrdner()
2575      On Error GoTo 0
2576  End Function
2577
2578  ' Rohdaten-Ordner. Ist er nur als SharePoint-Adresse erreichbar (Master-Datei aus dem Browser in der App geoeffnet), w
       ird zuerst
2579  ' ein lokaler Ordner dazu gesucht (OneDrive-Verknuepfung oder Synchronisierung, auch wenn die Verknuepfung nur auf „Ro
       hdaten“
2580  ' oder auf die ganze Bibliothek „Dokumente“ gesetzt ist) – dann kann das Makro umbenennen und ablegen wie mit Dateipfa
       d.
2581  Private Function RohdatenOrdner() As String
2582      Dim p As String, l As String
2583      p = OrdnerAufloesen(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B3").Value), "Rohdaten")
2584      If IstUrl(p) Then
2585          If rohLokalUrl = p And Len(rohLokal) > 0 Then
2586              If OrdnerVorhanden(rohLokal) Then RohdatenOrdner = rohLokal: Exit Function
2587          End If
2588          l = LokalFuerOrdner(p)
2589          If Len(l) > 0 Then
2590              rohLokalUrl = p
2591              rohLokal = l & Application.PathSeparator
2592              LogSchreiben "  Rohdaten lokal gefunden (OneDrive-Verknuepfung/Sync): " & l
2593              RohdatenOrdner = rohLokal
2594              Exit Function
2595          End If
2596          LogSchreiben "  Rohdaten: kein lokaler Ordner zu " & p & " gefunden (OneDrive-Verknuepfung fehlt, OneDrive-Cli
      ent aus oder noch nicht synchronisiert)"
2597      End If
2598      RohdatenOrdner = p
2599  End Function
2600
2601  ' Lokaler Ordner zu einer SharePoint-Ordneradresse: Zuordnung des OneDrive-Clients (Registry), sonst Suche unter den O
       neDrive-
2602  ' Ordnern und ihren Unterordnern (eine Verknuepfung heisst wie der Ordner, auf den sie zeigt: „Rohdaten“, „BÜ - HS“,
2603  ' „Dokumente“ …) und unter den Bibliotheksordnern im Benutzerprofil. Jedes Ende der Adresse wird probiert. Ein Treffer
       zaehlt nur,
2604  ' wenn dort dieselben Dateien liegen wie in der SharePoint-Auflistung (Power Query) – ein fremder Ordner „Rohdaten“ wi
       rd so nicht genommen.
2605  Private Function LokalFuerOrdner(url As String) As String
2606      Dim ud As String, teile() As String, sep As String, kand As String, w As Variant, j As Long, m As Long
2607      On Error GoTo Ende
2608      sep = Application.PathSeparator
2609      ud = Replace(UrlDekodieren(url), "\", "/")
2610      Do While Right$(ud, 1) = "/": ud = Left$(ud, Len(ud) - 1): Loop
2611      kand = SyncPfad(ud)
2612      If Len(kand) > 0 Then
2613          If OrdnerVorhanden(kand & sep) Then
2614              If PasstZurListe(kand, url) Then LokalFuerOrdner = kand: Exit Function
2615          End If
2616      End If
2617      teile = Split(ud, "/")
2618      For Each w In SuchWurzeln()
2619          For j = 3 To UBound(teile)
2620              kand = CStr(w)
2621              For m = j To UBound(teile)
2622                  kand = kand & sep & teile(m)
2623              Next m
2624              If OrdnerVorhanden(kand & sep) Then
2625                  If PasstZurListe(kand, url) Then LokalFuerOrdner = kand: Exit Function
2626              End If
2627          Next j
2628      Next w
2629  Ende:
2630  End Function
2631
2632  ' Registry des OneDrive-Clients: laengste passende Bibliotheksadresse -> lokaler Ordner (ohne Pruefung)
2633  Private Function SyncPfad(ud As String) As String
2634      Const HKCU As Long = &H80000001
```

```
2635      Dim reg As Object, schluessel As Variant, k As Variant, ns As Variant, mp As Variant, basis As String, n As String
      , u As String, best As Long
2636      On Error GoTo Ende
2637      Set reg = GetObject("winmgmts:{impersonationLevel=impersonate}!\\.\root\default:StdRegProv")
2638      basis = "Software\SyncEngines\Providers\OneDrive"
2639      reg.EnumKey HKCU, basis, schluessel
2640      If Not IsArray(schluessel) Then Exit Function
2641      u = LCase$(ud) & "/"
2642      For Each k In schluessel
2643          ns = Empty: mp = Empty
2644          reg.GetStringValue HKCU, basis & "\" & CStr(k), "UrlNamespace", ns
2645          reg.GetStringValue HKCU, basis & "\" & CStr(k), "MountPoint", mp
2646          If Not IsNull(ns) And Not IsEmpty(ns) And Not IsNull(mp) And Not IsEmpty(mp) Then
2647              n = LCase$(Replace(UrlDekodieren(CStr(ns)), "\", "/"))
2648              If Right$(n, 1) <> "/" Then n = n & "/"
2649              If Len(n) > best And Left$(u, Len(n)) = n Then
2650                  SyncPfad = CStr(mp) & Application.PathSeparator & Replace(Mid$(ud & "/", Len(n) + 1), "/", Application
      .PathSeparator)
2651                  Do While Right$(SyncPfad, 1) = Application.PathSeparator: SyncPfad = Left$(SyncPfad, Len(SyncPfad) - 1
      ): Loop
2652                  best = Len(n)
2653              End If
2654          End If
2655      Next k
2656  Ende:
2657  End Function
2658
2659  ' Orte, unter denen eine OneDrive-Verknuepfung oder synchronisierte Bibliothek liegen kann
2660  Private Function SuchWurzeln() As Collection
2661      Dim c As New Collection, v As Variant, w As Variant, u As Variant
2662      On Error Resume Next
2663      For Each v In Array(Environ$("OneDriveCommercial"), Environ$("OneDrive"), Environ$("OneDriveConsumer"))
2664          If Len(CStr(v)) > 0 Then
2665              c.Add CStr(v)
2666              For Each w In UnterOrdner(CStr(v))
2667                  c.Add CStr(w)                                   ' Verknuepfung auf eine ganze Bibliothek („Dokumente“)
      oder einen Oberordner
2668              Next w
2669          End If
2670      Next v
2671      If Len(Environ$("USERPROFILE")) > 0 Then                 ' Bibliotheksordner (z. B. C:\Users\x\ERGO\PV Dashboard -
      Dokumente)
2672          For Each w In UnterOrdner(Environ$("USERPROFILE"))
2673              For Each u In UnterOrdner(CStr(w))
2674                  If InStr(1, CStr(u), " - ", vbTextCompare) > 0 Then c.Add CStr(u)
2675              Next u
2676          Next w
2677      End If
2678      Set SuchWurzeln = c
2679  End Function
2680
2681  ' Liegen im lokalen Ordner dieselben Dateien wie laut SharePoint-Auflistung? (mindestens drei bzw. alle, wenn es wenig
     er sind)
2682  Private Function PasstZurListe(lokal As String, url As String) As Boolean
2683      Dim d As Object, k As Variant, n As Long, treffer As Long
2684      On Error GoTo Ende
2685      Set d = SpListe(url)
2686      If d Is Nothing Then Exit Function
2687      If d.Count = 0 Then Exit Function
2688      For Each k In d.Keys
2689          n = n + 1
2690          If DateiVorhanden(lokal & Application.PathSeparator & CStr(k)) Then treffer = treffer + 1
2691          If treffer >= 3 Then PasstZurListe = True: Exit Function
2692          If n >= 60 Then Exit For
2693      Next k
2694      PasstZurListe = (treffer > 0 And treffer = d.Count)
2695  Ende:
2696  End Function
2697
2698  Private Function OrdnerAufloesen(eintrag As String, standard As String) As String
2699      ' Relativer Eintrag (Konfiguration) -> Ordner neben der Master-Datei; absoluter Pfad bleibt. Ergebnis endet mit Tr
      enner.
2700      ' Liegt die Master-Datei auf SharePoint und ist kein Dateipfad erreichbar, ist das Ergebnis eine https-Adresse (en
      det mit /).
2701      ' Ein Eintrag als SharePoint-Adresse (Adresszeile des Browsers) wird als Ordneradresse uebernommen.
2702      Dim p As String, b As String
2703      p = Trim$(eintrag)
2704      If Len(p) = 0 Then p = standard
2705      If IstUrl(p) Then
2706          p = SharePointOrdnerUrl(p)
2707      ElseIf InStr(p, ":") = 0 And Left$(p, 2) <> "\\" Then
2708          b = BasisOrdner()
2709          If IstUrl(b) Then
2710              p = b & "/" & Replace(p, "\", "/")
2711          Else
2712              p = b & Application.PathSeparator & p
2713          End If
2714      End If
2715      If IstUrl(p) Then
2716          If Right$(p, 1) <> "/" Then p = p & "/"
2717      ElseIf Right$(p, 1) <> Application.PathSeparator Then
2718          p = p & Application.PathSeparator
2719      End If
2720      OrdnerAufloesen = p
2721  End Function
2722
```

```
Public Function IstUrl(p As String) As Boolean
    IstUrl = (LCase$(Left$(p, 7)) = "http://" Or LCase$(Left$(p, 8)) = "https://")
End Function

' Ordner + relativer Pfad (Trenner \); bei einer https-Adresse mit /
Private Function PfadVon(ordner As String, rel As String) As String
    If IstUrl(ordner) Then
        PfadVon = ordner & Replace(rel, Application.PathSeparator, "/")
    Else
        PfadVon = ordner & rel
    End If
End Function

' Liegen die Rohdaten auf SharePoint ohne Dateipfad (Laden ueber https)? Fuer Karten und Knoepfe.
Public Function RohdatenAufSharePoint() As Boolean
    RohdatenAufSharePoint = IstUrl(RohdatenPfad())
End Function

' Textdatei des Tools im Ordner Protokolle neben der Master-Datei (lokal bzw. synchronisiert): Ordner anlegen, eine Da
   tei vom
' bisherigen Ort (neben der Master-Datei) beim ersten Zugriff dorthin umziehen. Geht das Anlegen nicht: bisheriger Ort
   .
Private Function ProtokollPfad(b As String, datei As String) As String
    Dim o As String, alt As String, sep As String
    sep = Application.PathSeparator
    o = b & sep & PROTOKOLL_ORDNER
    alt = b & sep & datei
    On Error Resume Next
    If Len(Dir(o, vbDirectory)) = 0 Then MkDir o
    If Len(Dir(o, vbDirectory)) = 0 Then ProtokollPfad = alt: Exit Function
    ProtokollPfad = o & sep & datei
    If Len(Dir(alt)) > 0 Then
        If Len(Dir(ProtokollPfad)) = 0 Then Name alt As ProtokollPfad
    End If
End Function

' Ladelog.txt: Protokolle neben der lokal erreichbaren Master-Datei, sonst %LOCALAPPDATA%\BUE_Reporting
Private Function LokalerProtokollPfad(datei As String) As String
    Dim b As String
    On Error Resume Next
    b = BasisOrdner()
    If Len(b) > 0 And Not IstUrl(b) Then LokalerProtokollPfad = ProtokollPfad(b, datei): Exit Function
    LokalerProtokollPfad = AppDatenOrdner() & Application.PathSeparator & datei
End Function

' SharePoint ohne Dateipfad: Mappe als Text (xlTextWindows) in <Master-Ordner>/Protokolle speichern; fehlt der Ordner
   (Excel
' kann ueber https keinen anlegen), neben die Master-Datei wie bisher. Ergebnis: Adresse; ein Fehler am bisherigen Ort
   geht weiter.
Private Function SpTextSpeichern(wb As Workbook, b As String, datei As String) As String
    Dim ziel As String, nr As Long, txt As String
    Application.DisplayAlerts = False
    ziel = b & "/" & PROTOKOLL_ORDNER & "/" & datei
    On Error Resume Next
    wb.SaveAs FileName:=ziel, FileFormat:=20
    nr = Err.Number
    On Error GoTo 0
    If nr <> 0 Then
        LogSchreiben "  Ordner " & PROTOKOLL_ORDNER & " auf SharePoint nicht beschreibbar (fehlt?) - " & datei & " neb
   en der Master-Datei"
        ziel = b & "/" & datei
        On Error Resume Next
        wb.SaveAs FileName:=ziel, FileFormat:=20
        nr = Err.Number: txt = Err.Description
        On Error GoTo 0
    End If
    Application.DisplayAlerts = True
    If nr <> 0 Then Err.Raise nr, , txt
    SpTextSpeichern = ziel
End Function

' Ordner fuer Logs, wenn die Master-Datei nur ueber https erreichbar ist: %LOCALAPPDATA%\BUE_Reporting (sonst %TEMP%)
Private Function LokalerOrdner() As String
    Dim b As String
    On Error Resume Next
    b = BasisOrdner()
    If Len(b) > 0 And Not IstUrl(b) Then LokalerOrdner = b: Exit Function
    LokalerOrdner = AppDatenOrdner()
End Function

' %LOCALAPPDATA%\BUE_Reporting (sonst %TEMP%\BUE_Reporting) – bleibt bei einem Wechsel der Master-Datei erhalten
Private Function AppDatenOrdner() As String
    Dim b As String
    On Error Resume Next
    b = Environ$("LOCALAPPDATA")
    If Len(b) = 0 Then b = Environ$("TEMP")
    b = b & Application.PathSeparator & "BUE_Reporting"
    If Len(Dir(b, vbDirectory)) = 0 Then MkDir b
    AppDatenOrdner = b
End Function

Public Function BasisOrdner() As String
    ' Ordner der Master-Datei. Liegt die Datei auf SharePoint/OneDrive, liefert ThisWorkbook.Path eine https-Adresse,
   mit der Dir(),
    ' Name und MkDir nicht arbeiten (Laufzeitfehler 52). Dann wird ein Dateipfad gesucht – ohne Warten, ohne Dialog:
    '   1. Konfiguration!B7 als Dateipfad (wenn die Master-Datei dort liegt)
```

```
2813      '   2. lokaler Pfad (Datei liegt auf C:, H:, \\server …)
2814      '   3. synchronisierte Bibliothek laut Windows-Registry (OneDrive-Client: UrlNamespace -> MountPoint)
2815      '   4. WebDAV-Pfad \\host@SSL\DavWWWRoot\… – nur wenn der Windows-Dienst WebClient bereits laeuft (ein Versuch)
2816      '   5. OneDrive-Ordner und Bibliotheksordner unter dem Benutzerprofil (Verknuepfung „Zu OneDrive hinzufuegen“, Syn
          c)
2817      '   6. Rohdaten-Ordner absolut eingetragen (Konfiguration!B3, z. B. H:\…\Rohdaten): dessen uebergeordneter Ordner
2818      '   7. sonst die https-Adresse selbst: Laden ueber SharePoint (Power Query listet, Workbooks.Open oeffnet ueber ht
          tps)
2819      ' Das Ergebnis wird je Sitzung gemerkt (auch die https-Adresse), damit die Suche nur einmal laeuft.
2820      Dim p As String, k As String, kand As String, roh As String
2821      k = Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B7").Value))
2822      If Len(k) > 0 And Not IstUrl(k) Then
2823          If Right$(k, 1) = Application.PathSeparator Then k = Left$(k, Len(k) - 1)
2824          If DateiVorhanden(k & Application.PathSeparator & ThisWorkbook.Name) Then
2825              BasisOrdner = k
2826              Exit Function
2827          End If
2828      End If
2829      p = ThisWorkbook.Path
2830      If Not IstUrl(p) Then
2831          BasisOrdner = p
2832          Exit Function
2833      End If
2834      If Len(basisCache) > 0 Then
2835          If basisCacheUrl = p Then BasisOrdner = basisCache: Exit Function
2836      End If
2837      kand = SyncZuordnung(p)
2838      If Len(kand) = 0 Then kand = WebDavPfad(p)
2839      If Len(kand) = 0 Then kand = ProfilSuche(p)
2840      If Len(kand) = 0 Then
2841          roh = Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B3").Value))
2842          If InStr(roh, ":") > 0 And Not IstUrl(roh) Or Left$(roh, 2) = "\\" Then
2843              If Right$(roh, 1) = Application.PathSeparator Then roh = Left$(roh, Len(roh) - 1)
2844              If OrdnerVorhanden(roh & Application.PathSeparator) And InStrRev(roh, Application.PathSeparator) > 3 Then
2845                  kand = Left$(roh, InStrRev(roh, Application.PathSeparator) - 1)      ' Logs, Präsentationen neben dem
          Rohdaten-Ordner
2846              End If
2847          End If
2848      End If
2849      If Len(kand) = 0 Then
2850          kand = p                                                                    ' SharePoint direkt ueber https
2851          Do While Right$(kand, 1) = "/": kand = Left$(kand, Len(kand) - 1): Loop
2852      End If
2853      basisCache = kand: basisCacheUrl = p
2854      BasisOrdner = kand
2855  End Function
2856
2857  ' Synchronisierte SharePoint-/OneDrive-Bibliothek: HKCU\Software\SyncEngines\Providers\OneDrive\<id> mit UrlNamespace
          (Adresse der
2858  ' Bibliothek) und MountPoint (lokaler Ordner). Die laengste passende Adresse gewinnt.
2859  Private Function SyncZuordnung(url As String) As String
2860      Const HKCU As Long = &H80000001
2861      Dim reg As Object, schluessel As Variant, k As Variant, ns As Variant, mp As Variant, basis As String
2862      Dim u As String, ud As String, n As String, best As Long, kand As String
2863      On Error GoTo Ende
2864      Set reg = GetObject("winmgmts:{impersonationLevel=impersonate}!\\.\root\default:StdRegProv")
2865      basis = "Software\SyncEngines\Providers\OneDrive"
2866      reg.EnumKey HKCU, basis, schluessel
2867      If Not IsArray(schluessel) Then Exit Function
2868      ud = Replace(UrlDekodieren(url), "\", "/")
2869      If Right$(ud, 1) <> "/" Then ud = ud & "/"
2870      u = LCase$(ud)
2871      For Each k In schluessel
2872          ns = Empty: mp = Empty
2873          reg.GetStringValue HKCU, basis & "\" & CStr(k), "UrlNamespace", ns
2874          reg.GetStringValue HKCU, basis & "\" & CStr(k), "MountPoint", mp
2875          If Not IsNull(ns) And Not IsEmpty(ns) And Not IsNull(mp) And Not IsEmpty(mp) Then
2876              n = LCase$(Replace(UrlDekodieren(CStr(ns)), "\", "/"))
2877              If Right$(n, 1) <> "/" Then n = n & "/"
2878              If Len(n) > best And Left$(u, Len(n)) = n Then
2879                  kand = CStr(mp) & Application.PathSeparator & Replace(Mid$(ud, Len(n) + 1), "/", Application.PathSepar
          ator)
2880                  Do While Right$(kand, 1) = Application.PathSeparator: kand = Left$(kand, Len(kand) - 1): Loop
2881                  If DateiVorhanden(kand & Application.PathSeparator & ThisWorkbook.Name) Then SyncZuordnung = kand: bes
          t = Len(n)
2882              End If
2883          End If
2884      Next k
2885  Ende:
2886  End Function
2887
2888  ' SharePoint als WebDAV-Pfad: https://host/sites/… -> \\host@SSL\DavWWWRoot\sites\… Nur wenn der Windows-Dienst WebCli
          ent laeuft –
2889  ' sonst blockiert schon die Pruefung des Pfads (Windows sucht ihn erst als Netzwerkfreigabe). Microsoft hat WebClient
          im
2890  ' November 2023 abgekuendigt, er startet nicht mehr von selbst; deshalb kein Startversuch und kein Warten.
2891  Private Function WebDavPfad(url As String) As String
2892      Dim kand As Variant
2893      On Error GoTo Ende
2894      If Len(WebDavVorschlag(url)) = 0 Then Exit Function
2895      If Not WebClientLaeuft() Then Exit Function
2896      For Each kand In Array(WebDavVorschlag(url), Replace(WebDavVorschlag(url), "@SSL\DavWWWRoot\", "@SSL\"))
2897          If DateiVorhanden(CStr(kand) & Application.PathSeparator & ThisWorkbook.Name) Then WebDavPfad = CStr(kand): Ex
          it Function
2898      Next kand
2899  Ende:
```

```
End Function

Private Function WebClientLaeuft() As Boolean
    Dim wmi As Object, dienste As Object, d As Object
    On Error GoTo Ende
    Set wmi = GetObject("winmgmts:{impersonationLevel=impersonate}!\\.\root\cimv2")
    Set dienste = wmi.ExecQuery("SELECT State FROM Win32_Service WHERE Name='WebClient'")
    For Each d In dienste
        If LCase$(CStr(d.State)) = "running" Then WebClientLaeuft = True
    Next d
Ende:
End Function

' Eintrag in der Konfiguration als SharePoint-Adresse (Adresszeile des Browsers) -> Ordneradresse https://host/sites/… /Ordner
' Erkennt …/Forms/AllItems.aspx?id=/sites/…&viewid=…, Freigabe-Adressen /:f:/r/… und die direkte Ordneradresse.
Private Function SharePointOrdnerUrl(eintrag As String) As String
    Dim u As String, host As String, rel As String, p As Long, q As Long
    u = Trim$(eintrag)
    p = InStr(9, u, "/")
    If p = 0 Then SharePointOrdnerUrl = u: Exit Function
    host = Left$(u, p - 1)
    p = InStr(1, u, "?id=", vbTextCompare)
    If p = 0 Then p = InStr(1, u, "&id=", vbTextCompare)
    If p > 0 Then
        rel = Mid$(u, p + 4)
        q = InStr(rel, "&")
        If q > 0 Then rel = Left$(rel, q - 1)
        u = host & UrlDekodieren(rel)
    Else
        q = InStr(u, "?")
        If q > 0 Then u = Left$(u, q - 1)
        u = Replace(Replace(u, "/:f:/r/", "/"), "/:x:/r/", "/")
        If InStr(1, u, "/Forms/", vbTextCompare) > 0 Then u = Left$(u, InStr(1, u, "/Forms/", vbTextCompare) - 1)
        u = UrlDekodieren(u)
    End If
    Do While Right$(u, 1) = "/": u = Left$(u, Len(u) - 1): Loop
    SharePointOrdnerUrl = u
End Function

Public Function WebDavVorschlag(url As String) As String
    Dim u As String, p As Long
    u = UrlDekodieren(url)
    If LCase$(Left$(u, 8)) <> "https://" Then Exit Function
    u = Mid$(u, 9)
    p = InStr(u, "/")
    If p = 0 Then Exit Function
    WebDavVorschlag = "\\" & Left$(u, p - 1) & "@SSL\DavWWWRoot\" & Replace(Mid$(u, p + 1), "/", "\")
    Do While Right$(WebDavVorschlag, 1) = "\": WebDavVorschlag = Left$(WebDavVorschlag, Len(WebDavVorschlag) - 1): Loop
End Function

' OneDrive-Ordner (Umgebungsvariablen) und Bibliotheksordner unter dem Benutzerprofil (z. B. C:\Users\x\ERGO\PV Dashboard - Dokumente):
' das Ende der Adresse (ab beliebigem Teil) darunter suchen, auch eine und zwei Ebenen tiefer
Private Function ProfilSuche(url As String) As String
    Dim teile() As String, wurzeln As Collection, w As Variant, unter As Collection, u2 As Collection, e As String, j As Long, m As Long, kand As String
    Dim sep As String, v As Variant
    On Error GoTo Ende
    sep = Application.PathSeparator
    teile = Split(Replace(UrlDekodieren(url), "\", "/"), "/")
    Set wurzeln = New Collection
    For Each v In Array(Environ$("OneDriveCommercial"), Environ$("OneDriveConsumer"), Environ$("OneDrive"), Environ$("USERPROFILE"))
        If Len(CStr(v)) > 0 Then wurzeln.Add CStr(v)
    Next v
    If Len(Environ$("USERPROFILE")) > 0 Then                  ' Bibliotheksordner: zwei Ebenen unter dem Profil
        Set unter = UnterOrdner(Environ$("USERPROFILE"))
        For Each w In unter
            wurzeln.Add CStr(w)
            Set u2 = UnterOrdner(CStr(w))
            For Each v In u2
                If InStr(1, CStr(v), " - ", vbTextCompare) > 0 Then wurzeln.Add CStr(v)
            Next v
        Next w
    End If
    For Each w In wurzeln
        For j = 3 To UBound(teile)
            kand = CStr(w)
            For m = j To UBound(teile)
                kand = kand & sep & teile(m)
            Next m
            If DateiVorhanden(kand & sep & ThisWorkbook.Name) Then ProfilSuche = kand: Exit Function
        Next j
    Next w
Ende:
End Function

Private Function UnterOrdner(ordner As String) As Collection
    Dim c As New Collection, e As String, sep As String
    sep = Application.PathSeparator
    On Error Resume Next
    e = Dir(ordner & sep & "*", vbDirectory)
    Do While Len(e) > 0
        If e <> "." And e <> ".." And Left$(e, 1) <> "." Then
```

```
2991              If (GetAttr(ordner & sep & e) And vbDirectory) = vbDirectory Then c.Add ordner & sep & e
2992          End If
2993          e = Dir()
2994      Loop
2995      Set UnterOrdner = c
2996  End Function
2997
2998  Private Function UrlDekodieren(u As String) As String
2999      ' %XX-Folgen (UTF-8) in Text zurueckwandeln, z. B. %C3%BC -> ü
3000      Dim i As Long, bytes() As Byte, n As Long, s As String, stream As Object
3001      On Error GoTo Roh
3002      ReDim bytes(0 To Len(u) * 3)
3003      n = 0
3004      i = 1
3005      Do While i <= Len(u)
3006          If Mid$(u, i, 1) = "%" And i + 2 <= Len(u) Then
3007              bytes(n) = CByte("&H" & Mid$(u, i + 1, 2))
3008              n = n + 1
3009              i = i + 3
3010          Else
3011              s = Mid$(u, i, 1)
3012              If AscW(s) < 128 Then
3013                  bytes(n) = AscW(s): n = n + 1
3014              Else
3015                  ' Zeichen ausserhalb ASCII unveraendert als UTF-8 uebernehmen
3016                  Dim t As Object
3017                  Set t = CreateObject("ADODB.Stream")
3018                  t.Type = 2: t.Charset = "utf-8": t.Open: t.WriteText s: t.Position = 0: t.Type = 1
3019                  Dim b() As Byte, q As Long
3020                  b = t.Read
3021                  t.Close
3022                  For q = 3 To UBound(b)          ' ohne BOM
3023                      bytes(n) = b(q): n = n + 1
3024                  Next q
3025              End If
3026              i = i + 1
3027          End If
3028      Loop
3029      If n = 0 Then UrlDekodieren = "": Exit Function
3030      ReDim Preserve bytes(0 To n - 1)
3031      Set stream = CreateObject("ADODB.Stream")
3032      stream.Type = 1: stream.Open: stream.Write bytes: stream.Position = 0
3033      stream.Type = 2: stream.Charset = "utf-8"
3034      UrlDekodieren = stream.ReadText
3035      stream.Close
3036      Exit Function
3037  Roh:
3038      UrlDekodieren = Replace(u, "%20", " ")
3039  End Function
3040
3041  Private Function OrdnerVorhanden(pfad As String) As Boolean
3042      If IstUrl(pfad) Then OrdnerVorhanden = Not (SpListe(pfad) Is Nothing): Exit Function
3043      On Error Resume Next
3044      OrdnerVorhanden = (Len(Dir(pfad, vbDirectory)) > 0)
3045      On Error GoTo 0
3046  End Function
3047
3048  Private Function DateiVorhanden(pfad As String) As Boolean
3049      If IstUrl(pfad) Then DateiVorhanden = IsArray(SpEintrag(pfad)): Exit Function
3050      On Error Resume Next
3051      DateiVorhanden = (Len(Dir(pfad)) > 0)
3052      On Error GoTo 0
3053  End Function
3054
3055  ' ------------------------------------------------------------- SharePoint ohne Dateipfad (https)
3056  ' Excel kann einen SharePoint-Ordner nicht mit Dir() lesen, Power Query aber schon – mit der Anmeldung des Office-Kont
       os, ohne
3057  ' OneDrive und ohne WebDAV. SpListe schreibt die Abfrage BUE_SharePoint_Liste fuer den Ordner (SharePoint.Contents, Or
       dner fuer
3058  ' Ordner; findet sie den Ordner so nicht, SharePoint.Files ueber die ganze Seite), laedt sie in das ausgeblendete Blat
       t SP_Liste
3059  ' und merkt das Ergebnis je Ordner fuer diese Sitzung: Dateiname relativ zum Ordner (mit Ablage JJJJ\MM_Monat), Aender
       ungsdatum,
3060  ' Groesse. Beim ersten Mal fragt Excel nach der Anmeldung (Organisationskonto/Microsoft-Konto, „Anmelden“, „Verbinden“
       ).
3061  Private Function SpListe(ordnerUrl As String) As Object
3062      Dim key As String, m As String, d As Object, a As Variant, r As Long, rel As String
3063      key = SpSchluessel(ordnerUrl)
3064      If spListen Is Nothing Then Set spListen = CreateObject("Scripting.Dictionary")
3065      If spListen.Exists(key) Then Set SpListe = spListen(key): Exit Function
3066      spFehler = ""
3067      m = SpFormel(ordnerUrl)
3068      If Len(m) = 0 Then spFehler = "Die Adresse ist keine SharePoint-Ordneradresse.": Exit Function
3069      On Error GoTo Fehler
3070      a = SpTabelle(SP_ABFRAGE, m, "A1", "Dateien eines SharePoint-Ordners für das BÜ-Reporting (wird vom Tool geschrieb
         en)")
3071      Set d = CreateObject("Scripting.Dictionary")
3072      d.CompareMode = vbTextCompare
3073      If IsArray(a) Then
3074          For r = 1 To UBound(a, 1)
3075              rel = Replace(CStr(Nz(a(r, 1))), "/", Application.PathSeparator)
3076              If Len(rel) > 0 Then d(rel) = Array(rel, a(r, 2), a(r, 3))
3077          Next r
3078      End If
3079      spListen.Add key, d
3080      Set SpListe = d
```

```
3081      Exit Function
3082  Fehler:
3083      spFehler = Err.Description
3084      Set SpListe = Nothing
3085  End Function
3086
3087  ' Power-Query-Abfrage schreiben, in das Blatt SP_Liste ab zelle laden und als Werte (1-basiert, 2D) liefern; Empty = k
         eine Zeilen.
3088  ' Fehler (Anmeldung abgebrochen, Adresse falsch) werden an den Aufrufer weitergegeben.
3089  Private Function SpTabelle(abfrage As String, formel As String, zelle As String, beschreibung As String) As Variant
3090      Dim ws As Worksheet, lo As ListObject, q As Object, a As Variant, e(1 To 1, 1 To 1) As Variant
3091      Set ws = SpBlatt()
3092      Set q = Nothing
3093      On Error Resume Next
3094      Set q = ThisWorkbook.Queries(abfrage)
3095      Set lo = ws.Range(zelle).ListObject
3096      On Error GoTo 0
3097      If q Is Nothing Then
3098          ThisWorkbook.Queries.Add abfrage, formel, beschreibung
3099      ElseIf q.Formula <> formel Then
3100          q.Formula = formel
3101      End If
3102      If lo Is Nothing Then
3103          Set lo = ws.ListObjects.Add(SourceType:=0, Source:="OLEDB;Provider=Microsoft.Mashup.OleDb.1;Data Source=$Workb
         ook$;Location=" & _
3104                                      abfrage & ";Extended Properties=""""", Destination:=ws.Range(zelle))
3105          With lo.QueryTable
3106              .CommandType = xlCmdSql
3107              .CommandText = Array("SELECT * FROM [" & abfrage & "]")
3108              .RefreshStyle = xlInsertDeleteCells
3109              .AdjustColumnWidth = False
3110          End With
3111      End If
3112      lo.QueryTable.BackgroundQuery = False
3113      Application.DisplayAlerts = True                          ' Anmeldefenster von Power Query muss erscheinen duerfen
3114      lo.QueryTable.Refresh BackgroundQuery:=False
3115      If lo.ListRows.Count = 0 Then Exit Function
3116      a = lo.DataBodyRange.Value2
3117      If Not IsArray(a) Then e(1, 1) = a: a = e                 ' eine Zelle: trotzdem 2D
3118      SpTabelle = a
3119  End Function
3120
3121  Private Function SpSchluessel(ordnerUrl As String) As String
3122      Dim u As String
3123      u = LCase$(UrlDekodieren(ordnerUrl))
3124      If Right$(u, 1) <> "/" Then u = u & "/"
3125      SpSchluessel = u
3126  End Function
3127
3128  ' Eintrag einer Datei (https-Adresse) aus der Auflistung ihres Ordners: Array(rel, geaendert, groesse), sonst Empty
3129  Private Function SpEintrag(pfad As String) As Variant
3130      Dim u As String, k As Variant, rel As String
3131      If spListen Is Nothing Then Exit Function
3132      u = LCase$(UrlDekodieren(pfad))
3133      For Each k In spListen.Keys
3134          If Left$(u, Len(CStr(k))) = CStr(k) Then
3135              rel = Replace(Mid$(UrlDekodieren(pfad), Len(CStr(k)) + 1), "/", Application.PathSeparator)
3136              If spListen(k).Exists(rel) Then SpEintrag = spListen(k)(rel): Exit Function
3137          End If
3138      Next k
3139  End Function
3140
3141  Private Function SpBlatt() As Worksheet
3142      Dim ws As Worksheet, akt As Object
3143      On Error Resume Next
3144      Set ws = ThisWorkbook.Worksheets(SP_BLATT)
3145      On Error GoTo 0
3146      If ws Is Nothing Then
3147          Set akt = ActiveSheet
3148          Set ws = ThisWorkbook.Worksheets.Add(After:=ThisWorkbook.Worksheets(ThisWorkbook.Worksheets.Count))
3149          ws.Name = SP_BLATT
3150          ws.Range("J1").Value = "Im Tool ausgeblendete Rohdateien (SharePoint: gelöscht oder nicht übernommen)"
3151          ws.Visible = xlSheetVeryHidden
3152          If Not akt Is Nothing Then akt.Activate
3153      End If
3154      Set SpBlatt = ws
3155  End Function
3156
3157  ' Power-Query-Formel fuer einen Ordner https://host/sites/<Seite>/<Bibliothek>/<Ordner>…: Dateien des Ordners und der
         Ablage
3158  ' JJJJ\MM_Monat (Spalten Rel, Geaendert, Groesse)
3159  Private Function SpFormel(ordnerUrl As String) As String
3160      Dim kopf As String, f As String
3161      kopf = SpKopf(ordnerUrl)
3162      If Len(kopf) = 0 Then Exit Function
3163      f = "let" & vbLf & kopf & _
3164          "    Groesse = (a) => try a[Size] otherwise null," & vbLf & _
3165          "    IstOrdner = (x) => Value.Is(x, type table)," & vbLf
3166      f = f & _
3167          "    Ebene = (t as table, rel as text, tiefe as number) as table =>" & vbLf & _
3168          "        let" & vbLf & _
3169          "            Dateien = Table.SelectRows(t, each not IstOrdner([Content]))," & vbLf & _
3170          "            Eigene = Table.FromColumns({List.Transform(Dateien[Name], each rel & _), Dateien[Date modified],
         List.Transform(Dateien[Attributes], Groesse)}, {""Rel"", ""Geaendert"", ""Groesse""})," & vbLf & _
3171          "            Unter = Table.SelectRows(t, each IstOrdner([Content]) and (if tiefe = 2 then Text.Length([Name])
         = 4 and Text.Length(Text.Select([Name], {""0""..""9""})) = 4" & _
```

```
3172          " else Text.Length([Name]) > 3 and Text.Length(Text.Select(Text.Start([Name], 2), {""0""..""9""})) = 2 an
      d Text.Middle([Name], 2, 1) = ""_"")," & vbLf & _
3173          "            Tiefer = if tiefe <= 0 then {} else List.Transform(Table.ToRecords(Unter), each @Ebene([Content],
       rel & [Name] & ""\"", tiefe - 1))" & vbLf & _
3174          "        in" & vbLf & _
3175          "            Table.Combine({Eigene} & Tiefer)," & vbLf
3176      f = f & _
3177          "    Navigiert = try List.Accumulate(Teile, SharePoint.Contents(Seite, [ApiVersion = 15]), (s, n) => s{[Name =
       n]}[Content]) otherwise null," & vbLf & _
3178          "    Ersatz = () =>" & vbLf & _
3179          "        let" & vbLf & _
3180          "            Alle = SharePoint.Files(Seite, [ApiVersion = 15])," & vbLf & _
3181          "            Treffer = Table.SelectRows(Alle, each Text.StartsWith(Text.Lower([Folder Path]), Ordner))," & vbL
      f & _
3182          "            Rel = List.Transform(Table.ToRecords(Treffer), each Text.Replace(Text.Middle([Folder Path], Text.
      Length(Ordner)), ""/"", ""\"") & [Name])" & vbLf & _
3183          "        in" & vbLf & _
3184          "            Table.FromColumns({Rel, Treffer[Date modified], List.Transform(Treffer[Attributes], Groesse)}, {"
      "Rel"", ""Geaendert"", ""Groesse""})," & vbLf & _
3185          "    Ergebnis = if Navigiert = null then Ersatz() else Ebene(Navigiert, """", 2)" & vbLf & _
3186          "in" & vbLf & _
3187          "    Ergebnis"
3188      SpFormel = f
3189  End Function
3190
3191  ' Gemeinsamer Anfang der Abfragen: Seite (https://host/sites/<Seite>), Teile (Bibliothek und Ordner) und Ordner (Adres
      se klein, mit /)
3192  Private Function SpKopf(ordnerUrl As String) As String
3193      Dim u As String, host As String, teile() As String, i As Long, ab As Long, seite As String, liste As String, p As
      Long
3194      u = UrlDekodieren(ordnerUrl)
3195      Do While Right$(u, 1) = "/": u = Left$(u, Len(u) - 1): Loop
3196      If Not IstUrl(u) Then Exit Function
3197      p = InStr(9, u, "/")
3198      If p = 0 Then Exit Function
3199      host = Left$(u, p - 1)
3200      teile = Split(Mid$(u, p + 1), "/")
3201      ab = 0
3202      seite = host
3203      If UBound(teile) >= 1 Then
3204          If LCase$(teile(0)) = "sites" Or LCase$(teile(0)) = "teams" Then seite = host & "/" & teile(0) & "/" & teile(1
      ): ab = 2
3205      End If
3206      For i = ab To UBound(teile)
3207          liste = liste & IIf(Len(liste) > 0, ", ", "") & MText(teile(i))
3208      Next i
3209      SpKopf = "    Seite = " & MText(seite) & "," & vbLf & _
3210               "    Teile = {" & liste & "}," & vbLf & _
3211               "    Ordner = " & MText(LCase$(u) & "/") & "," & vbLf
3212  End Function
3213
3214  ' Abfrage: Zeilen einer Textdatei im SharePoint-Ordner (Windows-1252 wie Print #), keine Datei = keine Zeilen. Ein Abr
      uffehler
3215  ' (Netz, Drosselung, Anmeldung) wird NICHT als „Datei fehlt“ verschluckt, sondern laesst die Abfrage scheitern – sonst
       wuerde das
3216  ' Protokoll beim Zurueckschreiben nur mit den neuen Zeilen ueberschrieben.
3217  ' unter (12.90): zuerst im Unterordner (Protokolle) suchen, sonst im Ordner der Master-Datei (bisheriger Ort)
3218  Private Function SpTextFormel(ordnerUrl As String, datei As String, Optional unter As String = "") As String
3219      Dim kopf As String
3220      kopf = SpKopf(ordnerUrl)
3221      If Len(kopf) = 0 Then Exit Function
3222      ' Pruefen: Im gelesenen Ordner muss die Master-Datei selbst liegen – nur dann bedeutet „Protokoll nicht gefunden“
      wirklich,
3223      ' dass es fehlt (sonst Fehler, und es wird nichts ueberschrieben)
3224      SpTextFormel = "let" & vbLf & kopf & _
3225          "    Name = " & MText(datei) & "," & vbLf & _
3226          "    Unter = " & MText(unter) & "," & vbLf & _
3227          "    Master = " & MText(ThisWorkbook.Name) & "," & vbLf & _
3228          "    Pruefen = (t as table) as table => if Table.IsEmpty(Table.SelectRows(t, each Text.Lower([Name]) = Text.Lo
      wer(Master))) then error ""Ordner der Master-Datei nicht erkannt"" else t," & vbLf & _
3229          "    Navigiert = try List.Accumulate(Teile, SharePoint.Contents(Seite, [ApiVersion = 15]), (s, n) => s{[Name =
       n]}[Content]) otherwise null," & vbLf & _
3230          "    Geprueft = if Navigiert = null then null else Pruefen(Navigiert)," & vbLf & _
3231          "    UnterZeile = if Geprueft = null or Unter = """" then null else Table.SelectRows(Geprueft, each Text.Lower
      ([Name]) = Text.Lower(Unter))," & vbLf & _
3232          "    ImUnter = if UnterZeile = null or Table.IsEmpty(UnterZeile) then null else Table.SelectRows(UnterZeile{0}
      [Content], each Text.Lower([Name]) = Text.Lower(Name))," & vbLf & _
3233          "    Gefunden = if Geprueft = null then null else Table.SelectRows(Geprueft, each Text.Lower([Name]) = Text.Lo
      wer(Name))," & vbLf & _
3234          "    Direkt = if ImUnter <> null and not Table.IsEmpty(ImUnter) then ImUnter{0}[Content] else if Gefunden = nu
      ll or Table.IsEmpty(Gefunden) then null else Gefunden{0}[Content]," & vbLf & _
3235          "    Ersatz = () =>" & vbLf & _
3236          "        let" & vbLf & _
3237          "            Alle = SharePoint.Files(Seite, [ApiVersion = 15])," & vbLf & _
3238          "            ImOrdner = Pruefen(Table.SelectRows(Alle, each Text.Lower([Folder Path]) = Ordner))," & vbLf & _
3239          "            Treffer = Table.SelectRows(ImOrdner, each Text.Lower([Name]) = Text.Lower(Name))," & vbLf & _
3240          "            TrefferU = if Unter = """" then Treffer else Table.SelectRows(Alle, each Text.Lower([Folder Path]
      ) = Ordner & Text.Lower(Unter) & ""/"" and Text.Lower([Name]) = Text.Lower(Name))" & vbLf & _
3241          "        in" & vbLf & _
3242          "            if Table.RowCount(TrefferU) > 0 then TrefferU{0}[Content] else if Table.RowCount(Treffer) = 0 the
      n null else Treffer{0}[Content]," & vbLf & _
3243          "    Inhalt = if Direkt <> null then Direkt else if Navigiert <> null then null else Ersatz()," & vbLf & _
3244          "    Zeilen = if Inhalt = null then {} else List.Select(Lines.FromBinary(Inhalt, null, null, 1252), each Text.
      Length(_) > 0)," & vbLf & _
3245          "    Ergebnis = Table.FromColumns({Zeilen}, {""Zeile""})" & vbLf & _
3246          "in" & vbLf & _
```

```
3247          "    Ergebnis"
3248  End Function
3249
3250  Private Function MText(t As String) As String
3251      MText = """" & Replace(t, """", """""") & """"
3252  End Function
3253
3254  ' Meldung, wenn der SharePoint-Ordner nicht aufgelistet werden konnte
3255  Private Function SpHinweis(ordner As String) As String
3256      SpHinweis = "Der SharePoint-Ordner konnte nicht aufgelistet werden:" & vbCrLf & ordner & vbCrLf & vbCrLf & _
3257          IIf(Len(spFehler) > 0, "Meldung: " & spFehler & vbCrLf & vbCrLf, "") & _
3258          "Excel liest SharePoint-Ordner über Power Query mit deiner Office-Anmeldung. Beim ersten Mal erscheint ein Anm
      eldefenster: " & _
3259          "links „Organisationskonto“ (oder „Microsoft-Konto“) wählen, „Anmelden“, dann „Verbinden“. Wurde es abgebroche
      n: " & _
3260          "Daten > Daten abrufen > Datenquelleneinstellungen, die SharePoint-Adresse wählen, „Berechtigungen löschen“ un
      d erneut laden. " & _
3261          "Liegt der Ordner woanders: in Konfiguration B3 die Adresse des Ordners aus der Adresszeile des Browsers einfü
      gen."
3262  End Function
3263
3264  ' SharePoint: Datei im Tool ausblenden (geloescht oder nicht uebernommen) – wird beim Laden und in der Liste uebergang
      en
3265  ' Ausgeblendet wird eine bestimmte Datei: SP_Liste!J Name, K Groesse (seit 12.89). MIS-O vergibt jeden Tag dieselben N
      amen
3266  ' („Rueckstand_Verbund.xlsx“) – nur der Name traf sonst auch den neuen Abzug des naechsten Tages (wurde ungeladen zu …
      _verworfen).
3267  Private Sub SpAusblenden(rel As String, Optional ordner As String = "")
3268      Dim ws As Worksheet, r As Long, groesse As Double
3269      If Len(ordner) = 0 Then ordner = RohdatenOrdner()
3270      groesse = DateiGroesse(PfadVon(ordner, rel))
3271      If SpAusgeblendet(rel, groesse) Then Exit Sub
3272      Set ws = SpBlatt()
3273      r = ws.Cells(ws.Rows.Count, 10).End(xlUp).Row + 1
3274      If r < 2 Then r = 2
3275      ws.Cells(r, 10).Value = "'" & rel
3276      If groesse >= 0 Then ws.Cells(r, 11).Value = groesse
3277  End Sub
3278
3279  ' groesse < 0: unbekannt. Ein Eintrag mit Groesse gilt nur fuer eine Datei derselben Groesse; ein Eintrag ohne Groesse
      (vor 12.89)
3280  ' nur fuer Namen, die das Tool vergeben hat (JJJJMMTT_BUe_…, je Stichtag eindeutig), nicht fuer MIS-O-Namen.
3281  Private Function SpAusgeblendet(rel As String, Optional groesse As Double = -1) As Boolean
3282      Dim ws As Worksheet, a As Variant, r As Long, letzte As Long
3283      On Error Resume Next
3284      Set ws = ThisWorkbook.Worksheets(SP_BLATT)
3285      On Error GoTo 0
3286      If ws Is Nothing Then Exit Function
3287      letzte = ws.Cells(ws.Rows.Count, 10).End(xlUp).Row
3288      If letzte < 2 Then Exit Function
3289      a = ws.Range(ws.Cells(2, 10), ws.Cells(letzte + 1, 11)).Value2
3290      For r = 1 To UBound(a, 1)
3291          If StrComp(CStr(Nz(a(r, 1))), rel, vbTextCompare) = 0 Then
3292              If IsNumeric(a(r, 2)) And Not IsEmpty(a(r, 2)) Then
3293                  If groesse < 0 Or CDbl(a(r, 2)) = groesse Then SpAusgeblendet = True: Exit Function
3294              ElseIf DateiName(rel) Like "########_BUe_*" Or DateiName(rel) Like "########-########_BUe_*" Then
3295                  SpAusgeblendet = True: Exit Function
3296              End If
3297          End If
3298      Next r
3299  End Function
3300
3301  ' ---------------------------------------------------------------- SharePoint: Ablage ueber die SharePoint-Schnittstel
      le (12.80)
3302  ' Excel kann auf SharePoint ohne Dateipfad nichts umbenennen oder verschieben (Name, MkDir und Kill brauchen einen Pfa
      d). Power
3303  ' Query kann es: Web.Contents mit Inhalt schickt einen POST an die SharePoint-Schnittstelle (_api), mit dem Office-Kon
      to
3304  ' (Organisationskonto) wie die Auflistung. SpAblagePruefen probiert das einmal je Laden harmlos aus (den Ordner Rohdat
      en
3305  ' „anlegen“, den es schon gibt); nur wenn das gelingt, legt das Tool auch auf SharePoint ab wie mit Dateipfad (JJJJ\MM
      _Monat,
3306  ' einheitlicher Name, …_verworfen, …_ersetzt). Ob eine Verschiebung gelungen ist, entscheidet danach eine frische Aufl
      istung:
3307  ' Power Query kann eine Abfrage beim Aktualisieren mehr als einmal auswerten, ein zweiter Versuch meldet dann „nicht g
      efunden“,
3308  ' obwohl die Datei schon am Ziel liegt.
3309
3310  ' Seite (https://host/sites/X) und server-relativer Pfad (/sites/X/Bibliothek/…) einer https-Adresse, ohne / am Ende
3311  Private Function SpZerlegen(url As String, ByRef seite As String, ByRef pfad As String) As Boolean
3312      Dim u As String, p As Long, teile() As String, host As String
3313      u = UrlDekodieren(url)
3314      Do While Right$(u, 1) = "/": u = Left$(u, Len(u) - 1): Loop
3315      If Not IstUrl(u) Then Exit Function
3316      p = InStr(9, u, "/")
3317      If p = 0 Then Exit Function
3318      host = Left$(u, p - 1)
3319      pfad = Mid$(u, p)
3320      teile = Split(Mid$(u, p + 1), "/")
3321      seite = host
3322      If UBound(teile) >= 1 Then
3323          If LCase$(teile(0)) = "sites" Or LCase$(teile(0)) = "teams" Then seite = host & "/" & teile(0) & "/" & teile(1
      )
3324      End If
3325      SpZerlegen = True
3326  End Function
```

```
3327
3328 ' Abfrage fuer Schritte auf SharePoint, je Schritt Array(Art, A, B) mit server-relativen Pfaden: „ordner“ legt A an (g
       ibt es
3329 ' ihn schon, antwortet SharePoint trotzdem mit 200), „verschieben“ verschiebt die Datei A nach B (ohne Ueberschreiben;
       erst
3330 ' MoveToUsingPath, sonst das aeltere moveto). Jeder Schritt wartet auf das Ergebnis des vorigen. Ergebnis je Schritt:
       Status, Text.
3331 Private Function SpAblageFormel(seite As String, schritte As Collection) As String
3332     Dim f As String, s As Variant, liste As String
3333     For Each s In schritte
3334         liste = liste & IIf(Len(liste) > 0, ", ", "") & "[Art = " & MText(CStr(s(0))) & ", A = " & MText(CStr(s(1))) &
       ", B = " & MText(CStr(s(2))) & "]"
3335     Next s
3336     Randomize
3337     f = "let" & vbLf & _
3338         "    Seite = " & MText(seite) & "," & vbLf & _
3339         "    Schritte = {" & liste & "}," & vbLf & _
3340         "    Lauf = " & MText(Format$(Now, "yyyymmddhhnnss") & "-" & CStr(Int(Rnd() * 1000000))) & "," & vbLf & _
3341         "    Ausdruck = (t as text) as text => ""'"" & Text.Replace(t, ""'"", ""''"") & ""'""," & vbLf & _
3342         "    Senden = (rel as text, q as record) as record =>" & vbLf & _
3343         "        let" & vbLf & _
3344         "            Antwort = Web.Contents(Seite, [RelativePath = rel, Query = q, Headers = [Accept = ""application/j
       son;odata=nometadata"", #""X-BUE-Lauf"" = Lauf], Content = Text.ToBinary("""), ManualStatusHandling = {400, 401, 4
       03, 404, 405, 409, 412, 423, 429, 500, 503}])," & vbLf & _
3345         "            Inhalt = Binary.Buffer(Antwort)," & vbLf & _
3346         "            Status = try Value.Metadata(Antwort)[Response.Status] otherwise 0" & vbLf & _
3347         "        in" & vbLf & _
3348         "            [Status = Status, Text = Text.Start(Text.Clean(try Text.FromBinary(Inhalt) otherwise """), 240)]
       ," & vbLf & _
3349         "    Gut = (e as record) as logical => e[Status] >= 200 and e[Status] < 300," & vbLf
3350     f = f & _
3351         "    Ausfuehren = (s as record) as record =>" & vbLf & _
3352         "        if s[Art] = ""ordner"" then" & vbLf & _
3353         "            let" & vbLf & _
3354         "                Neu = Senden(""_api/web/folders/add(@a)"", [#""@a"" = Ausdruck(s[A])])" & vbLf & _
3355         "            in" & vbLf & _
3356         "                if Gut(Neu) then Neu else Senden(""_api/web/folders/AddUsingPath(DecodedUrl=@a)"", [#""@a"" =
       Ausdruck(s[A])])" & vbLf & _
3357         "        else" & vbLf & _
3358         "            let" & vbLf & _
3359         "                Neu = Senden(""_api/web/GetFileByServerRelativePath(DecodedUrl=@a)/MoveToUsingPath(DecodedUrl
       =@b,moveOperations=0)"", [#""@a"" = Ausdruck(s[A]), #""@b"" = Ausdruck(s[B])])" & vbLf & _
3360         "            in" & vbLf & _
3361         "                if Gut(Neu) then Neu else Senden(""_api/web/GetFileByServerRelativeUrl(@a)/moveto(newurl=@b,f
       lags=0)"", [#""@a"" = Ausdruck(s[A]), #""@b"" = Ausdruck(s[B])])," & vbLf & _
3362         "    Schritt = (bisher as list, s as record) as list =>" & vbLf & _
3363         "        let" & vbLf & _
3364         "            Vorher = if List.IsEmpty(bisher) then 0 else List.Last(bisher)[Status]," & vbLf & _
3365         "            Versuch = if Vorher >= 0 then try Ausfuehren(s) else try Ausfuehren(s)," & vbLf & _
3366         "            Erg = if Versuch[HasError] then [Status = -1, Text = Text.Start(Versuch[Error][Message], 240)] el
       se Versuch[Value]" & vbLf & _
3367         "        in" & vbLf & _
3368         "            bisher & {[Art = s[Art], A = s[A], B = s[B], Status = Erg[Status], Text = Erg[Text]]}," & vbLf &
       _
3369         "    Ergebnis = List.Accumulate(Schritte, {}, Schritt)," & vbLf & _
3370         "    Tabelle = Table.FromRecords(Ergebnis, type table [Art = text, A = text, B = text, Status = number, Text =
       text])" & vbLf & _
3371         "in" & vbLf & _
3372         "    Tabelle"
3373     SpAblageFormel = f
3374 End Function
3375
3376 ' Schritte ausfuehren (Blatt SP_Liste ab T1); Ergebnis leer = alle Schritte mit Status 2xx, sonst der erste Fehler
3377 Private Function SpAblageAusfuehren(ordner As String, schritte As Collection) As String
3378     Dim seite As String, pfad As String, a As Variant, i As Long, st As Double
3379     If Not SpZerlegen(ordner, seite, pfad) Then SpAblageAusfuehren = "keine SharePoint-Adresse": Exit Function
3380     On Error GoTo Fehler
3381     a = SpTabelle(SP_ABLAGE_ABFRAGE, SpAblageFormel(seite, schritte), "T1", _
3382                   "Rohdateien auf SharePoint ablegen: Ordner anlegen, Datei verschieben (wird vom Tool geschrieben)")
3383     On Error GoTo 0
3384     If Not IsArray(a) Then SpAblageAusfuehren = "keine Antwort der Abfrage": Exit Function
3385     For i = 1 To UBound(a, 1)
3386         st = Val(CStr(Nz(a(i, 4))))
3387         If st < 200 Or st >= 300 Then
3388             SpAblageAusfuehren = CStr(Nz(a(i, 1))) & " " & CStr(Nz(a(i, 2))) & ": Status " & CStr(Nz(a(i, 4))) & " " &
       Left$(CStr(Nz(a(i, 5))), 160)
3389             Exit Function
3390         End If
3391     Next i
3392     Exit Function
3393 Fehler:
3394     SpAblageAusfuehren = "Power Query: " & Err.Description
3395 End Function
3396
3397 ' Einmal je Laden: geht die Ablage auf SharePoint? (Ordner Rohdaten „anlegen“ – gibt es ihn, aendert sich nichts)
3398 Private Sub SpAblagePruefen(ordner As String)
3399     Dim seite As String, pfad As String, schritte As Collection
3400     spAblage = -1
3401     spAblageFehler = ""
3402     If Not SpZerlegen(ordner, seite, pfad) Then spAblageFehler = "keine SharePoint-Adresse": Exit Sub
3403     Set schritte = New Collection
3404     schritte.Add Array("ordner", pfad, "")
3405     Application.StatusBar = "SharePoint: Ablage der Rohdateien wird geprüft …"
3406     spAblageFehler = SpAblageAusfuehren(ordner, schritte)
3407     If Len(spAblageFehler) = 0 Then spAblage = 1
3408     LogSchreiben "  SharePoint-Ablage: " & IIf(spAblage = 1, "moeglich (Schnittstelle antwortet)", "nicht moeglich - "
```

```
        & spAblageFehler)
3409  End Sub
3410
3411  ' Datei rel (relativ zum Ordner, Trenner \) nach neu verschieben, Ordner davor anlegen. Ergebnis leer = verschoben (la
        ut frischer
3412  ' Auflistung: neu da, rel weg)
3413  Private Function SpVerschieben(ordner As String, rel As String, neu As String) As String
3414      Dim seite As String, basis As String, schritte As Collection, teile() As String, i As Long, zwischen As String
3415      Dim fehler As String, liste As Object, sep As String
3416      sep = Application.PathSeparator
3417      If Not SpZerlegen(ordner, seite, basis) Then SpVerschieben = "keine SharePoint-Adresse": Exit Function
3418      Set schritte = New Collection
3419      teile = Split(neu, sep)
3420      For i = 0 To UBound(teile) - 1
3421          zwischen = zwischen & "/" & teile(i)
3422          schritte.Add Array("ordner", basis & zwischen, "")
3423      Next i
3424      schritte.Add Array("verschieben", basis & "/" & Replace(rel, sep, "/"), basis & "/" & Replace(neu, sep, "/"))
3425      fehler = SpAblageAusfuehren(ordner, schritte)
3426      Dim alt As Object
3427      If Not spListen Is Nothing Then
3428          If spListen.Exists(SpSchluessel(ordner)) Then
3429              Set alt = spListen(SpSchluessel(ordner))
3430              spListen.Remove SpSchluessel(ordner)
3431          End If
3432      End If
3433      Set liste = SpListe(ordner)
3434      If liste Is Nothing Then
3435          If Not alt Is Nothing Then spListen.Add SpSchluessel(ordner), alt     ' alte Auflistung behalten (Celonis, fre
        ie Namen)
3436          SpVerschieben = IIf(Len(fehler) > 0, fehler, "Auflistung danach nicht moeglich: " & spFehler)
3437      ElseIf liste.Exists(neu) And Not liste.Exists(rel) Then
3438          SpVerschieben = ""
3439          CacheUmziehen PfadVon(ordner, rel), PfadVon(ordner, neu)
3440      ElseIf Len(fehler) > 0 Then
3441          SpVerschieben = fehler
3442      Else
3443          SpVerschieben = "nach dem Verschieben nicht am Ziel"
3444      End If
3445  End Function
3446
3447  ' Ladeprotokoll Spalte B: alle Zeilen mit der Datei alt auf neu umschreiben (nach dem Ablegen einer schon geladenen Da
        tei, damit
3448  ' Nachrechnen, „Rohdatei öffnen“ und die Ersatz-Logik die Datei am neuen Ort finden). Liefert die Zahl der Zeilen.
3449  Private Function ProtokollDateiUmschreiben(alt As String, neu As String) As Long
3450      Dim ws As Worksheet, letzte As Long, a As Variant, r As Long
3451      InhaltUmbenennen alt, neu
3452      On Error GoTo Ende
3453      Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
3454      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
3455      If letzte < 5 Then Exit Function
3456      a = ws.Range(ws.Cells(5, 2), ws.Cells(letzte + 1, 2)).Value2
3457      For r = 1 To UBound(a, 1) - 1
3458          If StrComp(CStr(Nz(a(r, 1))), alt, vbTextCompare) = 0 Then
3459              ws.Cells(r + 4, 2).Value = "'" & neu
3460              ProtokollDateiUmschreiben = ProtokollDateiUmschreiben + 1
3461          End If
3462      Next r
3463      If ProtokollDateiUmschreiben > 0 Then LogSchreiben "  Ladeprotokoll: " & ProtokollDateiUmschreiben & " Zeile(n) "
        & alt & " -> " & neu
3464  Ende:
3465  End Function
3466
3467  ' Lokale Kopie (Cache) einer verschobenen Datei auch unter der neuen Adresse finden
3468  Private Sub CacheUmziehen(alt As String, neu As String)
3469      On Error Resume Next
3470      If cacheDateien Is Nothing Then Exit Sub
3471      If cacheDateien.Exists(LCase$(alt)) Then cacheDateien(LCase$(neu)) = cacheDateien(LCase$(alt))
3472  End Sub
3473
3474  ' Rohdatei umbenennen bzw. in die Ablage verschieben (rel und neu relativ zum Ordner); Ergebnis leer = gelungen
3475  Private Function DateiUmbenennen(ordner As String, rel As String, neu As String) As String
3476      If IstUrl(ordner) Then
3477          DateiUmbenennen = SpVerschieben(ordner, rel, neu)
3478          Exit Function
3479      End If
3480      On Error Resume Next
3481      Name ordner & rel As ordner & neu
3482      If Err.Number <> 0 Then DateiUmbenennen = Err.Description
3483      Err.Clear
3484  End Function
3485
3486  ' Rohdateien, die noch direkt im Ordner Rohdaten liegen (nicht in JJJJ\MM_Monat, nicht ausgeblendet)
3487  Private Function SpUnsortiert(ordner As String) As Long
3488      Dim liste As Object, k As Variant, nm As String
3489      On Error Resume Next
3490      Set liste = SpListe(ordner)
3491      If liste Is Nothing Then Exit Function
3492      For Each k In liste.Keys
3493          nm = CStr(k)
3494          If InStr(nm, Application.PathSeparator) = 0 And Left$(nm, 2) <> "~$" And InStr(nm, ".") > 0 Then
3495              If IstExcelDatei(nm) Then
3496                  If Not AusgemustertName(nm) And Not SpAusgeblendet(nm, ZahlOder(liste(k)(2), -1)) Then SpUnsortiert =
        SpUnsortiert + 1
3497              End If
3498          End If
```

```
    Next k
End Function

' Einmal je Sitzung anbieten, den Ordner mit OneDrive zu synchronisieren (nicht im geführten Tagesablauf: dort steht es in der Meldung)
Private Sub SyncAnbieten(k As Long)
    If spSyncGefragt Then Exit Sub
    If modTagesablauf.WartetAufLaden() Then Exit Sub          ' im Tagesablauf steht die Abhilfe in der Meldung
    spSyncGefragt = True
    If MsgBox(IIf(k = 1, "1 Rohdatei liegt", k & " Rohdateien liegen") & " unsortiert im SharePoint-Ordner Rohdaten, weil SharePoint das " & _
              "Ablegen über Excel nicht zugelassen hat (Grund im Ladelog unter %LOCALAPPDATA%\BUE_Reporting)." & vbCrLf & vbCrLf & "Ordner der " & _
              "Master-Datei jetzt im Browser öffnen? Dort oben einmal „Verknüpfung zu OneDrive hinzufügen“ klicken – danach legt das Tool " & _
              "die Dateien beim nächsten Laden selbst ab.", vbQuestion + vbYesNo, "Rohdateien ablegen") = vbYes Then OrdnerSynchronisieren
End Sub

' ---------------------------------------------------------------- Ordner einbinden (SharePoint-Modus)
' Power Query darf auf SharePoint nur lesen (POST mit Anmeldung lehnt es ab, Test am 09.10.2026). Verlaesslich ist die Verknuepfung
' „Verknüpfung zu OneDrive hinzufügen“ beim Ordner der Master-Datei: BasisOrdner findet den Ordner danach unter dem OneDrive-Ordner
' (ProfilSuche), auch wenn die Datei ueber „In App öffnen“ geoeffnet wird, und das Tool arbeitet mit Dateipfad (Name, MkDir).
' Diese Prozedur oeffnet den Ordner im Browser und sagt, wo geklickt wird.
Public Sub OrdnerSynchronisieren()
    Dim ordner As String
    On Error Resume Next
    ordner = ThisWorkbook.Path
    If Not IstUrl(ordner) Then
        MsgBox "Die Master-Datei liegt schon in einem Ordner mit Dateipfad – das Tool legt die Rohdateien dort selbst ab:" & vbCrLf & ordner, _
               vbInformation, "Ordner einbinden"
        Exit Sub
    End If
    LogSchreiben "Ordner der Master-Datei im Browser geoeffnet (Verknuepfung zu OneDrive): " & ordner
    ThisWorkbook.FollowHyperlink ordner
    MsgBox "Der Ordner der Master-Datei öffnet sich im Browser. Dort oben einmal „Verknüpfung zu OneDrive hinzufügen“ klicken." & vbCrLf & vbCrLf & _
           "Danach Excel schließen und die Master-Datei wie gewohnt öffnen (auch über „In App öffnen“). Beim nächsten Laden benennt das " & _
           "Tool die Rohdateien wieder selbst um und legt sie in JJJJ\MM_Monat ab.", vbInformation, "Ordner einbinden"
End Sub


' SharePoint: Wurde aus dieser Datei schon ein Stand geladen, ersetzt oder uebersprungen? (jede Zeile im Ladeprotokoll)
' Seit 12.89 bei MIS-O-Namen (nicht JJJJMMTT_BUe_…) nur, wenn auch die Groesse stimmt (Blatt Dateiinhalt): MIS-O vergibt jeden Tag
' dieselben Namen, ein neuer Abzug unter altem Namen (auf SharePoint ersetzt) wurde sonst nie geladen. Unbekannt -> oeffnen
' (ein schon geladener Stand wird dann in Schritt 4 uebersprungen, die Datei steht danach im Blatt Dateiinhalt).
Private Function DateiSchonGeladen(rel As String, ordner As String) As Boolean
    Dim ws As Worksheet, letzte As Long, a As Variant, r As Long, gefunden As Boolean, nm As String
    Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    If letzte < 5 Then Exit Function
    a = ws.Range(ws.Cells(5, 2), ws.Cells(letzte + 1, 2)).Value2
    For r = 1 To UBound(a, 1) - 1
        If StrComp(CStr(Nz(a(r, 1))), rel, vbTextCompare) = 0 Then gefunden = True: Exit For
    Next r
    If Not gefunden Then Exit Function
    nm = DateiName(rel)
    If nm Like "########_BUe_*" Or nm Like "########-########_BUe_*" Then DateiSchonGeladen = True: Exit Function
    If inhaltIndex Is Nothing Then InhaltLaden
    DateiSchonGeladen = inhaltIndex.Exists(LCase$(rel) & "|" & DateiKennung(ordner, rel))
End Function

Private Function ZahlOder(v As Variant, ersatz As Double) As Double
    ZahlOder = ersatz
    On Error Resume Next
    If IsNumeric(v) And Not IsEmpty(v) Then
        If Len(CStr(v)) > 0 Then ZahlOder = CDbl(v)
    End If
End Function

' Zeitwert aus der Auflistung (Datum als Zahl oder Text) fuer den Vergleich
Private Function ZeitWert(v As Variant) As Double
    On Error Resume Next
    If IsNumeric(v) And Not IsEmpty(v) Then
        ZeitWert = CDbl(v)
    ElseIf Len(CStr(v)) > 0 Then
        ZeitWert = CDbl(CDate(Left$(CStr(v), 19)))
    End If
End Function

' ---------------------------------------------------------------- Snapshot (Schluessel des letzten Stands je Quelle und Einheit)
' Blatt Snapshot_<Quelle>: A..E Schluessel und Merkmale, F "Gruppe|Einheit" (HS: "|"). Stichtag und Anzahl: H2/H3 fuer die
' leere Gruppe, J2:L41 (Gruppe, Stichtag, Schluessel) fuer jede Gruppe im Bereich MAKLER. Parameter einheit = Gruppe.
Private Function SnapshotBlatt(quelle As String) As Worksheet
    Set SnapshotBlatt = ThisWorkbook.Worksheets("Snapshot_" & quelle)
End Function
```

```
3579
3580  Private Function SnapshotLesen(quelle As String, einheit As String) As Object
3581      Dim ws As Worksheet, letzte As Long, daten As Variant, r As Long, d As Object
3582      Set ws = SnapshotBlatt(quelle)
3583      Set SnapshotLesen = Nothing
3584      If LetzterStand(quelle, einheit) = 0 Then Exit Function
3585      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
3586      Set d = CreateObject("Scripting.Dictionary")
3587      If letzte >= 2 Then
3588          daten = ws.Range(ws.Cells(2, 1), ws.Cells(letzte + 1, 6)).Value2      ' +1: immer zweidimensional
3589          For r = 1 To UBound(daten, 1) - 1
3590              If SnapGruppe(daten(r, 6)) = einheit Then
3591                  If Not d.Exists(CStr(daten(r, 1))) Then d.Add CStr(daten(r, 1)), Array(CStr(daten(r, 2)), CStr(daten(r
        , 3)), CStr(daten(r, 4)), CStr(daten(r, 5)), _
3592                                                                                  SnapEinheit(daten(r, 6)))
3593              End If
3594          Next r
3595      End If
3596      Set SnapshotLesen = d
3597  End Function
3598
3599  ' Schluessel einer Einheit ersetzen (die anderen Einheiten bleiben stehen); stichtag 0 = Einheit ohne Stand (leeren)
3600  Private Sub SnapshotSchreiben(quelle As String, einheit As String, stichtag As Date, neu As Object)
3601      Dim je As Object
3602      Set je = CreateObject("Scripting.Dictionary")
3603      je.Add einheit, neu
3604      SnapshotErsetzen quelle, stichtag, je
3605  End Sub
3606
3607  ' Schluessel mehrerer Einheiten in einem Schreibvorgang ersetzen: je = Einheit -> Dictionary der heutigen Schluessel
3608  Private Sub SnapshotErsetzen(quelle As String, stichtag As Date, je As Object)
3609      Dim ws As Worksheet, aus() As Variant, k As Variant, e As Variant, i As Long, w As Variant, letzte As Long, alt As
      Variant, r As Long, c As Long, n As Long
3610      Dim neu As Object
3611      Set ws = SnapshotBlatt(quelle)
3612      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
3613      For Each e In je.Keys
3614          n = n + je.Item(e).Count
3615      Next e
3616      If letzte >= 2 Then
3617          alt = ws.Range(ws.Cells(2, 1), ws.Cells(letzte + 1, 6)).Value2
3618          For r = 1 To UBound(alt, 1) - 1
3619              If Not je.Exists(SnapGruppe(alt(r, 6))) And Len(CStr(Nz(alt(r, 1)))) > 0 Then n = n + 1
3620          Next r
3621      End If
3622      ws.Range(ws.Cells(2, 1), ws.Cells(ws.Rows.Count, 6)).ClearContents
3623      If n > 0 Then
3624          ReDim aus(1 To n, 1 To 6)
3625          If letzte >= 2 Then
3626              For r = 1 To UBound(alt, 1) - 1
3627                  If Not je.Exists(SnapGruppe(alt(r, 6))) And Len(CStr(Nz(alt(r, 1)))) > 0 Then
3628                      i = i + 1
3629                      For c = 1 To 6
3630                          aus(i, c) = alt(r, c)
3631                      Next c
3632                  End If
3633              Next r
3634          End If
3635          For Each e In je.Keys
3636              Set neu = je.Item(e)
3637              For Each k In neu.Keys
3638                  i = i + 1
3639                  w = neu(k)
3640                  aus(i, 1) = CStr(k): aus(i, 2) = w(0): aus(i, 3) = w(1): aus(i, 4) = w(2): aus(i, 5) = w(3)
3641                  aus(i, 6) = CStr(e) & TRENNER & CStr(w(4))
3642              Next k
3643          Next e
3644          ws.Range(ws.Cells(2, 1), ws.Cells(n + 1, 6)).Value = aus
3645      End If
3646      For Each e In je.Keys
3647          StandVermerken ws, CStr(e), stichtag, je.Item(e).Count
3648      Next e
3649  End Sub
3650
3651  ' Spalte F des Snapshots: "Gruppe|Einheit" (aeltere Zeilen ohne "|": nur Gruppe)
3652  Private Function SnapGruppe(v As Variant) As String
3653      Dim t As String
3654      t = CStr(Nz(v))
3655      If InStr(t, TRENNER) > 0 Then SnapGruppe = Left$(t, InStr(t, TRENNER) - 1) Else SnapGruppe = t
3656  End Function
3657
3658  Private Function SnapEinheit(v As Variant) As String
3659      Dim t As String
3660      t = CStr(Nz(v))
3661      If InStr(t, TRENNER) > 0 Then SnapEinheit = Mid$(t, InStr(t, TRENNER) + 1)
3662  End Function
3663
3664  Private Sub StandVermerken(ws As Worksheet, einheit As String, stichtag As Date, anzahl As Long)
3665      Dim r As Long, frei As Long
3666      If Len(einheit) = 0 Then
3667          If stichtag = 0 Then
3668              ws.Range("H2:H3").ClearContents
3669          Else
3670              ws.Range("H2").Value = stichtag
3671              ws.Range("H2").NumberFormat = "dd.mm.yyyy"
3672              ws.Range("H3").Value = anzahl
```

```
3673          End If
3674          Exit Sub
3675      End If
3676      For r = 2 To SNAP_EINHEITEN + 1
3677          If CStr(ws.Cells(r, 10).Value) = einheit Then frei = r: Exit For
3678          If frei = 0 And Len(CStr(ws.Cells(r, 10).Value)) = 0 Then frei = -r
3679      Next r
3680      If frei < 0 Then frei = -frei
3681      If frei = 0 Then Err.Raise vbObjectError + 13, , "Zu viele Einheiten (mehr als " & SNAP_EINHEITEN & ") im Blatt "
         & ws.Name & "."
3682      ws.Cells(frei, 10).Value = einheit
3683      If stichtag = 0 Then
3684          ws.Cells(frei, 11).ClearContents
3685          ws.Cells(frei, 12).ClearContents
3686      Else
3687          ws.Cells(frei, 11).Value = stichtag
3688          ws.Cells(frei, 11).NumberFormat = "dd.mm.yyyy"
3689          ws.Cells(frei, 12).Value = anzahl
3690      End If
3691  End Sub
3692
3693  ' Verlauf!AL/AM (erste/letzte Position der Fakten-Zeilen je Stichtag) aus den Fakten neu bestimmen – nach dem Ersetzen
       ,
3694  ' wenn Zeilen aufgerueckt sind und ein Stichtag nicht mehr am Stueck steht
3695  Public Sub VerlaufGrenzenNeu()
3696      Dim wsF As Worksheet, wsV As Worksheet, letzte As Long, lv As Long, a As Variant, i As Long, d As Object, k As Lon
         g, w As Variant, r As Long
3697      Set wsF = ThisWorkbook.Worksheets("Fakten")
3698      Set wsV = ThisWorkbook.Worksheets("Verlauf")
3699      Set d = CreateObject("Scripting.Dictionary")
3700      letzte = wsF.Cells(wsF.Rows.Count, 1).End(xlUp).Row
3701      If letzte >= 2 Then
3702          a = wsF.Range(wsF.Cells(2, 1), wsF.Cells(letzte + 1, 1)).Value2       ' +1: immer zweidimensional
3703          For i = 1 To UBound(a, 1) - 1
3704              If IsNumeric(a(i, 1)) And Not IsEmpty(a(i, 1)) Then
3705                  k = CLng(a(i, 1))
3706                  If d.Exists(k) Then
3707                      w = d(k)
3708                      w(1) = i
3709                      d(k) = w
3710                  Else
3711                      d.Add k, Array(i, i)
3712                  End If
3713              End If
3714          Next i
3715      End If
3716      lv = wsV.Cells(wsV.Rows.Count, 1).End(xlUp).Row
3717      For r = 6 To lv
3718          w = wsV.Cells(r, 1).Value2
3719          If IsNumeric(w) And Not IsEmpty(w) Then
3720              If d.Exists(CLng(w)) Then
3721                  w = d(CLng(w))
3722                  wsV.Cells(r, 38).Value = w(0)
3723                  wsV.Cells(r, 39).Value = w(1)
3724              Else
3725                  wsV.Cells(r, 38).Value = 1
3726                  wsV.Cells(r, 39).Value = 1
3727              End If
3728          End If
3729      Next r
3730  End Sub
3731
3732  ' Stichtag des zuletzt geladenen Stands einer Quelle und Einheit (0 = noch keiner)
3733  Private Function LetzterStand(quelle As String, einheit As String) As Date
3734      Dim v As Variant, ws As Worksheet, r As Long
3735      Set ws = SnapshotBlatt(quelle)
3736      If Len(einheit) = 0 Then
3737          v = ws.Range("H2").Value
3738      Else
3739          For r = 2 To SNAP_EINHEITEN + 1
3740              If CStr(ws.Cells(r, 10).Value) = einheit Then v = ws.Cells(r, 11).Value: Exit For
3741          Next r
3742      End If
3743      If IsDate(v) Then LetzterStand = CDate(v) Else LetzterStand = 0
3744  End Function
3745
3746  ' juengster Stand einer Quelle ueber alle Einheiten
3747  Private Function LetzterStandQuelle(quelle As String) As Date
3748      Dim ws As Worksheet, r As Long, v As Variant
3749      Set ws = SnapshotBlatt(quelle)
3750      LetzterStandQuelle = LetzterStand(quelle, "")
3751      For r = 2 To SNAP_EINHEITEN + 1
3752          v = ws.Cells(r, 11).Value
3753          If IsDate(v) Then
3754              If CDate(v) > LetzterStandQuelle Then LetzterStandQuelle = CDate(v)
3755          End If
3756      Next r
3757  End Function
3758
3759  ' ------------------------------------------------------------- Verlauf, Filterliste, Protokoll
3760  Private Sub VerlaufErgaenzen(stichtag As Date, pos1 As Long, pos2 As Long)
3761      ' Zeile je Stichtag anlegen (Formeln der Vorzeile kopieren) und die Blockgrenzen der Fakten-Zeilen dieses Stichtag
         s
3762      ' in AL/AM als Werte fuehren (Position im Bereich Fakten!X2:X300000): erste = kleinste, letzte = groesste Position
3763      ' Ein nachgereichter Stichtag (12.88) wird an seiner Stelle eingefuegt: letzte Zeile kopieren, dann die Werte A, A
         L, AM ab der
```

```
3764      ' Einfuegestelle eine Zeile nach unten (die uebrigen Spalten sind Formeln der eigenen Zeile)
3765      Dim ws As Worksheet, letzte As Long, r As Long, zeile As Long, c As Variant, v As Variant
3766      Set ws = ThisWorkbook.Worksheets("Verlauf")
3767      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
3768      zeile = 0
3769      For r = 6 To letzte
3770          If ws.Cells(r, 1).Value = stichtag Then zeile = r: Exit For
3771      Next r
3772      If zeile = 0 Then
3773          If letzte < 6 Then letzte = 6
3774          If IsEmpty(ws.Cells(6, 1).Value) Then
3775              zeile = 6
3776          Else
3777              If letzte + 1 > VERLAUF_MAX Then Err.Raise vbObjectError + 3, , "Das Blatt Verlauf ist voll (mehr als " &
      VERLAUF_MAX & " Stichtage)."
3778              ws.Rows(letzte).Copy ws.Rows(letzte + 1)
3779              Application.CutCopyMode = False
3780              zeile = letzte + 1
3781              For r = 6 To letzte
3782                  v = ws.Cells(r, 1).Value2
3783                  If IsNumeric(v) And Not IsEmpty(v) Then
3784                      If CLng(v) > CLng(stichtag) Then zeile = r: Exit For
3785                  End If
3786              Next r
3787              If zeile <= letzte Then
3788                  For Each c In Array(1, 38, 39)
3789                      ws.Range(ws.Cells(zeile + 1, c), ws.Cells(letzte + 1, c)).Value = ws.Range(ws.Cells(zeile, c), ws.
      Cells(letzte, c)).Value
3790                  Next c
3791                  LogSchreiben "  Verlauf: Stichtag " & Format(stichtag, "dd.mm.yyyy") & " in Zeile " & zeile & " eingef
      uegt"
3792              End If
3793          End If
3794          ws.Cells(zeile, 1).Value = stichtag
3795          ws.Cells(zeile, 38).Value = pos1
3796          ws.Cells(zeile, 39).Value = pos2
3797      Else
3798          If Val(ws.Cells(zeile, 38).Value) = 0 Or pos1 < Val(ws.Cells(zeile, 38).Value) Then ws.Cells(zeile, 38).Value
      = pos1
3799          If pos2 > Val(ws.Cells(zeile, 39).Value) Then ws.Cells(zeile, 39).Value = pos2
3800      End If
3801  End Sub
3802
3803  Private Sub PostkoerbeRegistrieren(neue As Object)
3804      ' Unbekannte Themenpostkoerbe kommen mit Kennzeichen WAHR in die Filterliste (Filter!B, ohne Chip)
3805      Dim ws As Worksheet, r As Long, k As Variant, vorhanden As Object, frei As Long, liste As Variant
3806      Set ws = ThisWorkbook.Worksheets("Filter")
3807      Set vorhanden = CreateObject("Scripting.Dictionary")
3808      liste = ws.Range(ws.Cells(PK_VON, 2), ws.Cells(PK_BIS, 2)).Value2
3809      frei = 0
3810      For r = 1 To UBound(liste, 1)
3811          If Len(Trim$(CStr(Nz(liste(r, 1))))) > 0 Then
3812              If Not vorhanden.Exists(CStr(liste(r, 1))) Then vorhanden.Add CStr(liste(r, 1)), True
3813          ElseIf frei = 0 Then
3814              frei = PK_VON + r - 1
3815          End If
3816      Next r
3817      For Each k In neue.Keys
3818          If Not vorhanden.Exists(CStr(k)) Then
3819              If frei = 0 Or frei > PK_BIS Then Err.Raise vbObjectError + 4, , "Die Filterliste der Themenpostkörbe ist
      voll (Filter!B" & PK_VON & ":B" & PK_BIS & "); Postkorb '" & k & "' kann nicht aufgenommen werden."
3820              ws.Cells(frei, 2).Value = CStr(k)
3821              ws.Cells(frei, 3).Value = True
3822              ' Standard-Auswahl (Konfiguration!I, zeilengleich): HS-Postkoerbe (… 23724) nach Vorschlag (KO/MV "ja", so
      nst "nein",
3823              ' z. B. GR, AZ, 72, BÜ); der Chip im Cockpit (Filter!C) folgt dem Vorschlag, ein neuer HS-Postkorb startet
       also ausgeblendet;
3824              ' Postkoerbe anderer Bereiche (z. B. ODPB) erst einmal alle "ja" – die Karte nach dem Laden fragt, welche
      heraus sollen.
3825              ' Bereich MAKLER: alle Postkoerbe der Abzuege gehoeren dazu ("ja"), keine Frage.
3826              If IstMakler() Then
3827                  ThisWorkbook.Worksheets("Konfiguration").Cells(frei + 6, 9).Value = "ja"
3828              ElseIf CStr(k) Like "* 23724" Or CStr(k) = "(ohne Postkorb)" Then
3829                  ThisWorkbook.Worksheets("Konfiguration").Cells(frei + 6, 9).Value = IIf(InStr(STANDARD_VORSCHLAG, "|"
      & CStr(k) & "|") > 0, "ja", "nein")
3830                  ws.Cells(frei, 3).Value = (InStr(STANDARD_VORSCHLAG, "|" & CStr(k) & "|") > 0)
3831              Else
3832                  ThisWorkbook.Worksheets("Konfiguration").Cells(frei + 6, 9).Value = "ja"
3833                  If neuePK Is Nothing Then Set neuePK = CreateObject("Scripting.Dictionary")
3834                  If Not neuePK.Exists(CStr(k)) Then neuePK.Add CStr(k), True
3835              End If
3836              vorhanden.Add CStr(k), True
3837              frei = frei + 1
3838          End If
3839      Next k
3840  End Sub
3841
3842  ' Hinweis einer Ladeprotokoll-Zeile: gilt der Stand als geladen? Ja bei leerem Hinweis (bis 12.68), „erster Stand …“ u
       nd der
3843  ' Verkettungs-Angabe, die seit 12.69 jeder normal geladene Stand traegt („Verkettungskennzeichen: …“, „ohne Spalte
3844  ' Verkettungskennzeichen: …“, ggf. mit „· … Zeilen der HS-Einheit … ignoriert“). Nein bei „übersprungen …“, „ersetzt …
       “,
3845  ' „neu gerechnet …“, „doppelt im Ladeprotokoll …“ und „keine Datensätze …“. Bis 12.91 zaehlten nur leer und „erster“ –
        siehe
3846  ' DoppelteStaendeBereinigen.
3847  Public Function StandGueltig(h As String) As Boolean
```

```
    StandGueltig = (Len(h) = 0 Or h Like "erster*" Or h Like "Verkettungskennzeichen*" Or h Like "ohne Spalte Verkettungskennzeichen*")
End Function

' ---------------------------------------------------------------- Doppelt geladene Staende (12.92)
' Bis 12.91 galt ein Stand nur mit leerem Hinweis oder „erster Stand …“ als geladen, seit 12.69 traegt aber jeder normal geladene
' Stand die Verkettungs-Angabe. Mit Dateipfad (OneDrive-Verknuepfung, Dateien in der Ablage) las „Neue Daten laden“ deshalb den
' zuletzt geladenen Stand jeder Quelle beim naechsten Laden noch einmal ein: gegen den eigenen Snapshot, also Ruecksta nd doppelt,
' Eingaenge und Abgaenge 0. Bereinigt wird einmal, wenn das Ladeprotokoll einen Stand mehrfach gueltig fuehrt:
' - Ladeprotokoll: alle Zeilen mit derselben Datei -> die erste bleibt, die weiteren heissen „doppelt im Ladeprotokoll …“ (auch
'   im Bereich MAKLER, wo jedes Laden die aelteren Staende als Nachzuegler neu rechnete und je Stand eine Zeile anhaengte); verschiedene
'   Dateien (Ersetzen, dessen alte Zeile bis 12.91 nicht markiert wurde) -> die letzte bleibt, die frueheren heissen „ersetzt …“.
' - Fakten: je Stichtag, Quelle, Sparte, VSS, Postkorb, GeVo, Einheit und Strang bleibt die erste Zeile (ein Durchlauf schreibt
'   jeden dieser Schluessel genau einmal, der zweite Durchlauf steht dahinter). Danach Verlauf!AL:AM neu.
' Ergebnis: die doppelt geladenen Staende als Text („OPAG 18.10., 19.10. · Verbund 22.10.“), leer = nichts doppelt.
Private Function DoppelteStaendeBereinigen() As String
    Dim wsL As Worksheet, wsF As Worksheet, letzte As Long, p As Variant, r As Long, i As Long, c As Long, k As String
    Dim gruppen As Object, g As Variant, zeilen As Collection, gleich As Boolean, behalten As Long, zeit As String, markiert As Long
    Dim doppelt As Object, sn As String, blk As Variant, ein As Variant, aus() As Variant, ausE() As Variant, m As Long, weg As Long
    Dim schl As Object, fk As String, tage As Variant, t As Long, j As Long, txt As String, teil As String
    On Error GoTo Fehler                                ' Bereinigung darf das Laden nie blockieren
    Set wsL = ThisWorkbook.Worksheets("Ladeprotokoll")
    letzte = wsL.Cells(wsL.Rows.Count, 1).End(xlUp).Row
    If letzte < 6 Then Exit Function
    p = wsL.Range(wsL.Cells(5, 1), wsL.Cells(letzte + 1, SP_GRUPPE_L)).Value2          ' +1: immer zweidimensional
    Set gruppen = CreateObject("Scripting.Dictionary")
    For r = 1 To UBound(p, 1) - 1
        If IsNumeric(p(r, 5)) And Not IsEmpty(p(r, 5)) And Len(CStr(Nz(p(r, 4)))) > 0 Then
            If StandGueltig(CStr(Nz(p(r, 10)))) Then
                k = StandSchluessel(CStr(p(r, 4)), CStr(Nz(p(r, SP_GRUPPE_L))), CDate(p(r, 5)))
                If Not gruppen.Exists(k) Then gruppen.Add k, New Collection
                gruppen.Item(k).Add r
            End If
        End If
    Next r
    zeit = Format(Now, "dd.mm.yyyy hh:nn")
    For Each g In gruppen.Keys
        Set zeilen = gruppen.Item(g)
        If zeilen.Count > 1 Then
            gleich = True
            For i = 2 To zeilen.Count
                If StrComp(DateiName(CStr(Nz(p(zeilen(i), 2)))), DateiName(CStr(Nz(p(zeilen(1), 2)))), vbTextCompare) <> 0 Then gleich = False
            Next i
            If gleich Then behalten = 1 Else behalten = zeilen.Count
            For i = 1 To zeilen.Count
                If i <> behalten Then
                    r = zeilen(i)
                    If gleich Then
                        wsL.Cells(r + 4, 10).Value = "doppelt im Ladeprotokoll – am " & zeit & " bereinigt (12.92)"
                    Else
                        wsL.Cells(r + 4, 10).Value = "ersetzt (Ladeprotokoll am " & zeit & " bereinigt, 12.92)"
                    End If
                    markiert = markiert + 1
                End If
            Next i
        End If
    Next g
    If markiert = 0 Then Exit Function
    LogSchreiben "Ladeprotokoll: " & markiert & " mehrfach gueltige Zeile(n) markiert"

    ' Fakten: gleiche Schluessel je Stichtag und Quelle nur einmal (die erste Zeile bleibt), der Rest rueckt auf
    Set wsF = ThisWorkbook.Worksheets("Fakten")
    letzte = wsF.Cells(wsF.Rows.Count, 1).End(xlUp).Row
    If letzte >= 3 Then
        blk = wsF.Range(wsF.Cells(2, 1), wsF.Cells(letzte + 1, 15)).Value2
        ein = wsF.Range(wsF.Cells(2, SP_EINHEIT_F), wsF.Cells(letzte + 1, SP_ALTSUM_F)).Value2   ' R Einheit, S Strang, T, U, V
        ReDim aus(1 To UBound(blk, 1), 1 To 15)
        ReDim ausE(1 To UBound(blk, 1), 1 To 5)
        Set schl = CreateObject("Scripting.Dictionary")
        Set doppelt = CreateObject("Scripting.Dictionary")  ' Strang -> Dictionary der Stichtage mit entfernten Zeilen
        For r = 1 To UBound(blk, 1) - 1
            fk = CStr(CLng(Val(blk(r, 1)))) & TRENNER & CStr(Nz(blk(r, 2))) & TRENNER & CStr(Nz(blk(r, 3))) & TRENNER & CStr(Nz(blk(r, 4))) & _
                 TRENNER & CStr(Nz(blk(r, 5))) & TRENNER & CStr(Nz(blk(r, 6))) & TRENNER & CStr(Nz(ein(r, 1))) & TRENNER & CStr(Nz(ein(r, 2)))
            If schl.Exists(fk) Then
                weg = weg + 1
                sn = Strang(CStr(Nz(blk(r, 2))), CStr(Nz(ein(r, 2))))
                If Not doppelt.Exists(sn) Then doppelt.Add sn, CreateObject("Scripting.Dictionary")
                doppelt.Item(sn).Item(CLng(Val(blk(r, 1)))) = True
            Else
                schl.Add fk, True
                m = m + 1
                For c = 1 To 15
                    aus(m, c) = blk(r, c)
```

```
                Next c
                For c = 1 To 5
                    ausE(m, c) = ein(r, c)
                Next c
            End If
        Next r
        If weg > 0 Then
            wsF.Range(wsF.Cells(2, 1), wsF.Cells(m + 1, 15)).Value2 = aus
            wsF.Range(wsF.Cells(2, SP_EINHEIT_F), wsF.Cells(m + 1, SP_ALTSUM_F)).Value2 = ausE
            wsF.Range(wsF.Cells(m + 2, 1), wsF.Cells(letzte, SP_ALTSUM_F)).ClearContents
        End If
        Erase aus
        Erase ausE
    End If
    LogSchreiben "Fakten: " & weg & " doppelte Zeile(n) entfernt"
    VerlaufGrenzenNeu

    ' Meldungstext: je Strang die Stichtage aufsteigend
    If weg = 0 Then Exit Function
    For Each g In doppelt.Keys
        tage = doppelt.Item(g).Keys
        For i = 0 To UBound(tage) - 1
            For j = i + 1 To UBound(tage)
                If CLng(tage(j)) < CLng(tage(i)) Then t = tage(i): tage(i) = tage(j): tage(j) = t
            Next j
        Next i
        teil = ""
        For i = 0 To UBound(tage)
            teil = teil & IIf(i > 0, ", ", "") & Format$(CDate(CLng(tage(i))), "dd.mm.")
        Next i
        txt = txt & IIf(Len(txt) > 0, " · ", "") & CStr(g) & " " & teil
    Next g
    LogSchreiben "Doppelt geladene Staende bereinigt: " & txt
    DoppelteStaendeBereinigen = txt
    Exit Function
Fehler:
    LogSchreiben "Bereinigung doppelter Staende abgebrochen (Fehler " & Err.Number & ": " & Err.Description & ") – Lad
    en laeuft weiter"
    Err.Clear
End Function

' Zusatz fuer die Abschlussmeldung (einmal, danach geleert)
Private Function DoppeltMeldung() As String
    If Len(doppeltText) = 0 Then Exit Function
    DoppeltMeldung = "." & vbCrLf & "Bereinigt: Diese Stände waren bis 12.91 doppelt geladen, ihr Rückstand zählte zwe
    ifach – " & doppeltText & _
                     ". Fakten und Ladeprotokoll sind korrigiert"
    doppeltText = ""
End Function

Private Function ProtokollLesen() As Object
    ' Alle protokollierten Staende (Quelle|Stichtag) – geladene und uebersprungene – als Dictionary
    Dim ws As Worksheet, letzte As Long, daten As Variant, r As Long, d As Object, hinweis As String, k As String
    Set d = CreateObject("Scripting.Dictionary")
    Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    If letzte >= 5 Then
        daten = ws.Range(ws.Cells(5, 1), ws.Cells(letzte + 1, SP_GRUPPE_L)).Value2     ' +1: immer zweidimensional
        For r = 1 To UBound(daten, 1) - 1
            If IsNumeric(daten(r, 5)) And Not IsEmpty(daten(r, 5)) And Len(CStr(Nz(daten(r, 4)))) > 0 Then
                hinweis = CStr(Nz(daten(r, 10)))
                If StandGueltig(hinweis) Or Left$(hinweis, 12) = "übersprungen" Then
                    ' Stand und Namensschluessel des abgelegten Dateinamens (fuer AbgelegtUndGeladen)
                    k = StandSchluessel(CStr(daten(r, 4)), CStr(Nz(daten(r, SP_GRUPPE_L))), CDate(daten(r, 5)))
                    If Not d.Exists(k) Then d.Add k, True
                    k = NamensSchluessel(CStr(daten(r, 4)), EinheitDateiteil(CStr(Nz(daten(r, SP_GRUPPE_L)))), CDate(d
    aten(r, 5)))
                    If Not d.Exists(k) Then d.Add k, True
                    k = NamensSchluesselDatei(CStr(Nz(daten(r, 2))), CStr(daten(r, 4)), CDate(daten(r, 5)))
                    If Len(k) > 0 Then
                        If Not d.Exists(k) Then d.Add k, True
                    End If
                End If
            End If
        Next r
    End If
    Set ProtokollLesen = d
End Function

Private Sub Protokoll(st As Stand, zeilen As Long, rueckstand As Long, eingang As Long, abgang As Long, hinweis As Str
    ing)
    Dim ws As Worksheet, r As Long
    Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
    r = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row + 1
    If r < 5 Then r = 5
    ws.Range(ws.Cells(r, 1), ws.Cells(r, SP_GRUPPE_L)).Value = Array(Now, st.Datei, st.Blatt, st.Quelle, st.Stichtag,
    zeilen, rueckstand, eingang, abgang, hinweis, st.Gruppe)
    ws.Cells(r, 1).NumberFormat = "dd.mm.yyyy hh:mm"
    ws.Cells(r, 5).NumberFormat = "dd.mm.yyyy"
End Sub

' ---------------------------------------------------------------- Dateiinhalt (12.88)
' Blatt Dateiinhalt: je geprueftem Blatt einer Rohdatei und je Einheit eine Zeile – A Datei, B Kennung (Groesse in Byt
    e, seit 12.89; SharePoint und OneDrive liefern dieselbe),
' C Blatt, D Quelle, E Stichtag, F Einheit (Strang), G Zeilen, H Einheiten der Datei, I geprueft am. Zum Nachschauen,
    was in welcher
```

```
4019  ' Datei stand, und damit eine schon gepruefte Datei in Schritt 1 nicht erneut geoeffnet wird. J2 haelt die Einheiten-Z
       uordnung
4020  ' (Konfiguration F7, Q11:R40), mit der geprueft wurde; aendert sie sich, wird das Blatt geleert und neu aufgebaut.
4021  Private Function InhaltBlatt() As Worksheet
4022      On Error Resume Next
4023      Set InhaltBlatt = ThisWorkbook.Worksheets("Dateiinhalt")
4024  End Function
4025
4026  Private Function InhaltFingerabdruck() As String
4027      Dim a As Variant, r As Long, t As String
4028      On Error Resume Next
4029      t = UCase$(Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("F7").Value)))
4030      If t = "MAKLER" Then t = t & ";HS" & HS_EINHEITEN        ' seit 12.91 ohne HS-Zeilen geprueft: aeltere Eintraege n
         eu pruefen
4031      a = ThisWorkbook.Worksheets("Konfiguration").Range("Q11:R40").Value2
4032      If IsArray(a) Then
4033          For r = 1 To UBound(a, 1)
4034              If Len(CStr(Nz(a(r, 1)))) + Len(CStr(Nz(a(r, 2)))) > 0 Then t = t & ";" & UCase$(Trim$(CStr(Nz(a(r, 1)))))
         & "=" & UCase$(Trim$(CStr(Nz(a(r, 2)))))
4035          Next r
4036      End If
4037      InhaltFingerabdruck = t
4038  End Function
4039
4040  Private Function DateiKennung(ordner As String, rel As String) As String
4041      Dim e As Variant
4042      On Error GoTo Ende
4043      If IstUrl(ordner) Then
4044          e = SpEintrag(PfadVon(ordner, rel))
4045          If IsArray(e) Then
4046              If IsNumeric(e(2)) And Not IsEmpty(e(2)) Then
4047                  If Len(CStr(e(2))) > 0 Then DateiKennung = CStr(CDbl(e(2)))
4048              End If
4049          End If
4050      Else
4051          DateiKennung = CStr(CDbl(FileLen(ordner & rel)))
4052      End If
4053  Ende:
4054  End Function
4055
4056  ' Index aus dem Blatt: "datei|kennung" (klein) -> Collection von Array(Blatt, Quelle, Stichtag, Einheit, Zeilen, Einhe
     iten der Datei)
4057  Private Sub InhaltLaden()
4058      Dim ws As Worksheet, letzte As Long, a As Variant, r As Long, k As String
4059      Set inhaltIndex = CreateObject("Scripting.Dictionary")
4060      Set ws = InhaltBlatt()
4061      If ws Is Nothing Then Exit Sub
4062      If CStr(ws.Range("J2").Value) <> InhaltFingerabdruck() Then
4063          letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
4064          If letzte >= 5 Then ws.Range(ws.Cells(5, 1), ws.Cells(letzte, 9)).ClearContents
4065          ws.Range("J2").Value = "'" & InhaltFingerabdruck()
4066          If letzte >= 5 Then LogSchreiben "Dateiinhalt geleert (Einheiten-Zuordnung geaendert)"
4067          Exit Sub
4068      End If
4069      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
4070      If letzte < 5 Then Exit Sub
4071      a = ws.Range(ws.Cells(5, 1), ws.Cells(letzte + 1, 9)).Value2
4072      For r = 1 To UBound(a, 1) - 1
4073          If Len(CStr(Nz(a(r, 1)))) > 0 And IsNumeric(a(r, 5)) And Not IsEmpty(a(r, 5)) Then
4074              k = LCase$(CStr(a(r, 1))) & "|" & CStr(Nz(a(r, 2)))
4075              If Not inhaltIndex.Exists(k) Then inhaltIndex.Add k, New Collection
4076              inhaltIndex.Item(k).Add Array(CStr(Nz(a(r, 3))), CStr(Nz(a(r, 4))), CDate(a(r, 5)), CStr(Nz(a(r, 6))), Val
        (a(r, 7)), CStr(Nz(a(r, 8))))
4077          End If
4078      Next r
4079  End Sub
4080
4081  ' Blaetter und Einheiten einer schon gepruefften Datei (gleiche Kennung) aus dem Index; False = unbekannt, Datei oeffne
     n
4082  Private Function InhaltBekannt(ordner As String, datei As String, bl() As Stand, blS() As Object, nBl As Long) As Bool
     ean
4083      Dim kenn As String, k As String, z As Variant, b As Long, gef As Long, st As Stand
4084      If inhaltIndex Is Nothing Then InhaltLaden
4085      kenn = DateiKennung(ordner, datei)
4086      If Len(kenn) = 0 Then Exit Function
4087      k = LCase$(datei) & "|" & kenn
4088      If Not inhaltIndex.Exists(k) Then Exit Function
4089      nBl = 0
4090      For Each z In inhaltIndex.Item(k)
4091          gef = -1
4092          For b = 0 To nBl - 1
4093              If bl(b).Blatt = z(0) And bl(b).Quelle = z(1) And CLng(bl(b).Stichtag) = CLng(z(2)) Then gef = b: Exit For
4094          Next b
4095          If gef < 0 Then
4096              st.Datei = datei: st.Blatt = z(0): st.Quelle = z(1): st.Stichtag = z(2): st.DateiGruppe = z(5): st.Gruppe
        = z(5)
4097              st.Zeilen = 0: st.Ersetzen = False
4098              ReDim Preserve bl(0 To nBl)
4099              ReDim Preserve blS(0 To nBl)
4100              bl(nBl) = st
4101              Set blS(nBl) = CreateObject("Scripting.Dictionary")
4102              gef = nBl
4103              nBl = nBl + 1
4104          End If
4105          blS(gef).Item(CStr(z(3))) = CLng(z(4))
4106          bl(gef).Zeilen = bl(gef).Zeilen + CLng(z(4))
```

```
    Next z
    InhaltBekannt = (nBl > 0)
End Function

' Ergebnis der Pruefung einer geoeffneten Datei vermerken (je Blatt und Einheit eine Zeile)
Private Sub InhaltMerken(ordner As String, datei As String, bl() As Stand, blS() As Object, nBl As Long)
    Dim ws As Worksheet, kenn As String, b As Long, u As Variant, r As Long, k As String, n As Long, aus() As Variant
    If nBl = 0 Then Exit Sub
    Set ws = InhaltBlatt()
    If ws Is Nothing Then Exit Sub
    If inhaltIndex Is Nothing Then InhaltLaden
    kenn = DateiKennung(ordner, datei)
    If Len(kenn) = 0 Then Exit Sub
    k = LCase$(datei) & "|" & kenn
    If inhaltIndex.Exists(k) Then Exit Sub
    inhaltIndex.Add k, New Collection
    For b = 0 To nBl - 1
        n = n + blS(b).Count
    Next b
    ReDim aus(1 To n, 1 To 9)
    n = 0
    For b = 0 To nBl - 1
        For Each u In blS(b).Keys
            n = n + 1
            aus(n, 1) = "'" & datei: aus(n, 2) = "'" & kenn: aus(n, 3) = "'" & bl(b).Blatt: aus(n, 4) = bl(b).Quelle
            aus(n, 5) = bl(b).Stichtag: aus(n, 6) = "'" & CStr(u): aus(n, 7) = CLng(blS(b).Item(u)): aus(n, 8) = "'" &
    bl(b).DateiGruppe: aus(n, 9) = Now
            inhaltIndex.Item(k).Add Array(bl(b).Blatt, bl(b).Quelle, bl(b).Stichtag, CStr(u), CLng(blS(b).Item(u)), bl
   (b).DateiGruppe)
        Next u
    Next b
    r = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row + 1
    If r < 5 Then r = 5
    ws.Range(ws.Cells(r, 1), ws.Cells(r + n - 1, 9)).Value = aus
    ws.Range(ws.Cells(r, 5), ws.Cells(r + n - 1, 5)).NumberFormat = "dd.mm.yyyy"
    ws.Range(ws.Cells(r, 9), ws.Cells(r + n - 1, 9)).NumberFormat = "dd.mm.yyyy hh:mm"
End Sub

' Datei wurde umbenannt bzw. abgelegt: Eintraege folgen ihr (die Kennung bleibt, die Groesse aendert sich nicht)
Private Sub InhaltUmbenennen(alt As String, neu As String)
    Dim ws As Worksheet, letzte As Long, a As Variant, r As Long, n As Long
    On Error GoTo Ende
    Set ws = InhaltBlatt()
    If ws Is Nothing Then Exit Sub
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    If letzte < 5 Then Exit Sub
    a = ws.Range(ws.Cells(5, 1), ws.Cells(letzte + 1, 1)).Value2
    For r = 1 To UBound(a, 1) - 1
        If StrComp(CStr(Nz(a(r, 1))), alt, vbTextCompare) = 0 Then ws.Cells(r + 4, 1).Value = "'" & neu: n = n + 1
    Next r
    If n > 0 Then Set inhaltIndex = Nothing                  ' beim naechsten Zugriff neu lesen
Ende:
End Sub

' Bereich MAKLER: stammen die geladenen Staende noch aus der Zeit vor 12.88 (Strang = Gruppe mehrerer Einheiten, „MSUE
   +ODPB“)?
Private Function AlteGruppenStraenge() As Boolean
    Dim q As Variant, a As Variant, r As Long
    If Not IstMakler() Then Exit Function
    On Error Resume Next
    For Each q In Array("OPAG", "Verbund")
        a = Empty
        a = SnapshotBlatt(CStr(q)).Range(SnapshotBlatt(CStr(q)).Cells(2, 10), SnapshotBlatt(CStr(q)).Cells(SNAP_EINHEI
   TEN + 1, 10)).Value2
        If IsArray(a) Then
            For r = 1 To UBound(a, 1)
                If InStr(CStr(Nz(a(r, 1))), "+") > 0 Then AlteGruppenStraenge = True: Exit Function
            Next r
        End If
    Next q
End Function

' ---------------------------------------------------------------- Bereich und Einheiten
' Konfiguration!F7: HS (ein Strang je Quelle) oder MAKLER (je Quelle ein Strang je Einheit, siehe Kopf des Moduls)
Public Function IstMakler() As Boolean
    On Error Resume Next
    IstMakler = (UCase$(Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("F7").Value))) = "MAKLER")
    On Error GoTo 0
End Function

' Bereich MAKLER (12.91): gehoert eine Einheit (EinheitAusOE) zum HS-Reporting (OE HSB4 K -> HSB)? Deren Zeilen ignori
   ert der
' Makler-Master. Nur in Wegen des Bereichs MAKLER gefragt (je Datenzeile, deshalb ohne IstMakler).
Public Function IstHsEinheit(e As String) As Boolean
    If Len(e) = 0 Then Exit Function
    IstHsEinheit = (InStr(1, HS_EINHEITEN, "|" & UCase$(Trim$(e)) & "|", vbBinaryCompare) > 0)
End Function

' Bereich MAKLER: HS-Einheit, deren Zeilen schon in den Fakten stehen (vor 12.91 mitgeladene HS-Abzuege), sonst leer
Private Function HsDatenGeladen() As String
    Dim liste As Variant, i As Long
    If Not IstMakler() Then Exit Function
    On Error GoTo Ende
    liste = EinheitenListe()
    For i = LBound(liste) To UBound(liste)
        If IstHsEinheit(CStr(liste(i))) Then HsDatenGeladen = CStr(liste(i)): Exit Function
```

```
4198      Next i
4199  Ende:
4200  End Function
4201
4202  ' Bereich MAKLER: reiner HS-Abzug im Ordner Rohdaten oder in der Ablage – nicht laden, nicht an Celonis; die Datei ble
         ibt, wo sie
4203  ' liegt, und heisst danach …_HS-ignoriert (wird nicht mehr gelesen). SharePoint ohne Dateipfad: im Tool ausgeblendet.
4204  Private Sub HsAbzugAusmustern(ordner As String, datei As String)
4205      Dim neuName As String, fehler As String, p As Long
4206      hsDateien.Add DateiName(datei)
4207      If IstUrl(ordner) And spAblage <> 1 Then
4208          SpAusblenden datei, ordner
4209          hsAusgeblendet = hsAusgeblendet + 1
4210          LogSchreiben "  HS-Abzug (OE HSB4 K) im Makler-Reporting ignoriert, ausgeblendet: " & datei
4211          Exit Sub
4212      End If
4213      p = InStrRev(datei, ".")
4214      On Error Resume Next
4215      neuName = FreierDateiname(ordner, Left$(datei, p - 1) & HS_ENDUNG & Mid$(datei, p))
4216      Err.Clear
4217      On Error GoTo 0
4218      fehler = DateiUmbenennen(ordner, datei, neuName)
4219      If Len(fehler) = 0 Then
4220          LogSchreiben "  HS-Abzug (OE HSB4 K) im Makler-Reporting ignoriert: " & datei & " -> " & neuName
4221      Else
4222          LogSchreiben "  HS-Abzug (OE HSB4 K) im Makler-Reporting ignoriert, nicht umbenannt (" & fehler & "): " & date
         i
4223      End If
4224  End Sub
4225
4226  ' Hinweis fuer die Abschlussmeldung: ignorierte HS-Abzuege und HS-Zeilen dieses Ladens (leer, wenn keine)
4227  Private Function HsMeldung() As String
4228      Dim t As String, d As Variant, n As Long
4229      If Not hsDateien Is Nothing Then
4230          If hsDateien.Count > 0 Then
4231              For Each d In hsDateien
4232                  n = n + 1
4233                  If n <= 3 Then t = t & IIf(n > 1, ", ", "") & CStr(d)
4234              Next d
4235              If hsDateien.Count > 3 Then t = t & " …"
4236              t = IIf(hsDateien.Count = 1, "1 HS-Abzug", hsDateien.Count & " HS-Abzüge") & " ignoriert (OE HSB4 K, gehör
         t ins HS-Reporting): " & t
4237              If hsAusgeblendet = hsDateien.Count Then
4238                  t = t & " – im Tool ausgeblendet"
4239              Else
4240                  t = t & " – umbenannt in …" & HS_ENDUNG
4241              End If
4242          End If
4243      End If
4244      If hsZeilen > 0 Then t = t & IIf(Len(t) > 0, "; ", "") & ZahlDE(CDbl(hsZeilen)) & IIf(hsZeilen = 1, " Zeile", " Ze
         ilen") & _
4245           " der HS-Einheit (OE HSB4 K) in Makler-Abzügen ignoriert"
4246      HsMeldung = t
4247  End Function
4248
4249  ' Zusatz fuer das Ladeprotokoll eines Strangs
4250  Private Function HsText(n As Long) As String
4251      If n > 0 Then HsText = " · " & ZahlDE(CDbl(n)) & IIf(n = 1, " Zeile", " Zeilen") & " der HS-Einheit (OE HSB4 K) ig
         noriert"
4252  End Function
4253
4254  ' Anzeige- und Sortiername eines Strangs: "Verbund" bzw. "Verbund · ODPB D"
4255  Public Function Strang(quelle As String, einheit As String) As String
4256      If Len(einheit) > 0 Then Strang = quelle & " · " & einheit Else Strang = quelle
4257  End Function
4258
4259  ' Schluessel eines Stands: Quelle[|Einheit]|Stichtag (ohne Einheit wie bisher Quelle|Stichtag)
4260  Private Function StandSchluessel(quelle As String, einheit As String, tag As Date) As String
4261      If Len(einheit) > 0 Then
4262          StandSchluessel = quelle & TRENNER & einheit & TRENNER & CLng(tag)
4263      Else
4264          StandSchluessel = quelle & TRENNER & CLng(tag)
4265      End If
4266  End Function
4267
4268  ' Schluessel aus dem abgelegten Dateinamen: "@Quelle[_Einheitteil]|Stichtag"
4269  Private Function NamensSchluessel(quelle As String, teil As String, tag As Date) As String
4270      NamensSchluessel = "@" & quelle & IIf(Len(teil) > 0, "_" & teil, "") & TRENNER & CLng(tag)
4271  End Function
4272
4273  ' Namensschluessel aus dem Dateinamen einer Protokollzeile (seit 12.88 traegt der Name die Einheiten der Datei, die Ze
         ile nur
4274  ' ihre Einheit): passt der Stichtag im Namen zur Zeile, gilt die abgelegte Datei damit als geladen. Sonst leer.
4275  Private Function NamensSchluesselDatei(rel As String, quelle As String, tag As Date) As String
4276      Dim t As Date, quellen As Variant, teil As String
4277      If Len(rel) = 0 Then Exit Function
4278      If Not NameZerlegen(rel, t, quellen, teil) Then Exit Function
4279      If CLng(t) = CLng(tag) Then NamensSchluesselDatei = NamensSchluessel(quelle, teil, tag)
4280  End Function
4281
4282  ' Einheit als Teil eines Dateinamens: Leerzeichen und Unterstriche werden zu "-", unzulaessige Zeichen fallen weg
4283  Private Function EinheitDateiteil(einheit As String) As String
4284      Dim i As Long, ch As String, t As String
4285      For i = 1 To Len(einheit)
4286          ch = Mid$(einheit, i, 1)
4287          Select Case ch
```

```
            Case " ", "_": If Right$(t, 1) <> "-" Then t = t & "-"
            Case "\", "/", ":", "*", "?", """", "<", ">", "|", "'", ".", ","
            Case Else: t = t & ch
        End Select
    Next i
    EinheitDateiteil = t
End Function

' Abgelegter Name JJJJMMTT_BUe_<OPAG|Verbund|OPAG_Verbund>[_Einheit][_n]: Stichtag, Quellen und Einheit-Teil.
' Sammeldateien (JJJJMMTT-JJJJMMTT_…) nennen nicht jeden Stichtag: False (werden geoeffnet).
Private Function NameZerlegen(rel As String, ByRef tag As Date, ByRef quellen As Variant, ByRef teil As String) As Boo
   lean
    Dim nm As String, t() As String, i As Long
    nm = DateiName(rel)
    nm = Left$(nm, InStrRev(nm, ".") - 1)
    If Not nm Like "########_BUe_*" Then Exit Function
    On Error GoTo Nein
    tag = DateSerial(CInt(Left$(nm, 4)), CInt(Mid$(nm, 5, 2)), CInt(Mid$(nm, 7, 2)))
    t = Split(Mid$(nm, 14), "_")
    i = 0
    If t(0) = "OPAG" And UBound(t) >= 1 Then
        If t(1) = "Verbund" Then quellen = Array("OPAG", "Verbund"): i = 2 Else quellen = Array("OPAG"): i = 1
    ElseIf t(0) = "OPAG" Then
        quellen = Array("OPAG"): i = 1
    ElseIf t(0) = "Verbund" Then
        quellen = Array("Verbund"): i = 1
    Else
        Exit Function
    End If
    teil = ""
    If UBound(t) >= i Then
        If Not t(i) Like String(Len(t(i)), "#") Then teil = t(i)          ' _2, _3 … ist das Doppel, keine Einheit
    End If
    NameZerlegen = True
    Exit Function
Nein:
    NameZerlegen = False
End Function

' Gruppe eines Abzugs (Bereich MAKLER): die Einheiten aller Zeilen (Spalte OE_NR, siehe EinheitAusOE), sortiert und mi
   t "+"
' verbunden, z. B. "MSUE+ODPB+ODPP". Ohne Spalte OE_NR: die Kopfzeile "oe_nr = '…'" bzw. "oe_nr in (…)", sonst leer.
' zaehl: Zeilen je Einheit der Gruppe, rest: Zeilen anderer OEs, mitOE: Spalte OE_NR vorhanden (12.88)
Private Function GruppeErkennen(ws As Worksheet, kopfdaten As Variant, kopf As Long, letzteSpalte As Long, zaehl As Ob
   ject, _
                                ByRef rest As Long, ByRef mitOE As Boolean, ByRef hs As Long) As String
    Dim r As Long, c As Long, t As String, p As Long, werte As Object, liste As Variant, teile() As String, i As Long,
    e As String
    Set werte = CreateObject("Scripting.Dictionary")
    rest = 0: mitOE = False: hs = 0
    For c = 1 To letzteSpalte
        If UCase$(Trim$(CStr(Nz(kopfdaten(kopf, c))))) = "OE_NR" Then
            mitOE = True
            liste = ws.Range(ws.Cells(kopf + 1, c), ws.Cells(ws.UsedRange.Row + ws.UsedRange.Rows.Count, c)).Value2
            For r = 1 To UBound(liste, 1)
                t = Trim$(CStr(Nz(liste(r, 1))))
                If Len(t) > 0 And Not t Like "Anzahl*" Then
                    t = EinheitAusOE(t)
                    If IstHsEinheit(t) Then
                        hs = hs + 1                                  ' Zeile des HS-Reportings (12.91): zaehlt nirgen
   ds
                    ElseIf GruppenEinheit(t) Then
                        If Not werte.Exists(t) Then werte.Add t, True
                        zaehl(t) = zaehl(t) + 1
                    Else
                        rest = rest + 1
                    End If
                End If
            Next r
            GruppeErkennen = SortiertVerbunden(werte)
            Exit Function
        End If
    Next c
    For r = 1 To kopf - 1
        For c = 1 To letzteSpalte
            t = Trim$(CStr(Nz(kopfdaten(r, c))))
            If LCase$(Left$(t, 5)) = "oe_nr" Then
                p = InStr(t, "=")
                If p = 0 Then p = InStr(1, t, " in ", vbTextCompare) + 3
                If p > 3 Then
                    teile = Split(EinheitNorm(Mid$(t, p + 1)), "+")
                    For i = 0 To UBound(teile)
                        e = EinheitAusOE(teile(i))
                        If IstHsEinheit(e) Then
                            hs = hs + 1
                        ElseIf GruppenEinheit(e) Then
                            If Not werte.Exists(e) Then werte.Add e, True
                        Else
                            rest = rest + 1
                        End If
                    Next i
                    GruppeErkennen = SortiertVerbunden(werte)
                    Exit Function
                End If
            End If
        Next c
```

```
4379      Next r
4380  End Function
4381
4382  ' Zaehlt eine Einheit fuer die Gruppe? Nur die in Konfiguration!R11:R40 genannten Einheiten (ODPB, ODPP, MSUE, KBI …), damit
4383  ' einzelne Zeilen anderer OEs (z. B. „ODP D“ selbst) die Gruppe nicht von Tag zu Tag aendern. Leere Tabelle: jede Einheit.
4384  ' Die Liste wird je Laden einmal gelesen (gruppenListe; wird fuer jede Datenzeile gefragt).
4385  Private Function GruppenEinheit(e As String) As Boolean
4386      Dim liste As Variant, r As Long
4387      If gruppenListe Is Nothing Then
4388          Set gruppenListe = CreateObject("Scripting.Dictionary")
4389          gruppenListe.CompareMode = vbTextCompare
4390          On Error Resume Next
4391          liste = ThisWorkbook.Worksheets("Konfiguration").Range("R11:R40").Value2
4392          On Error GoTo 0
4393          If IsArray(liste) Then
4394              For r = 1 To UBound(liste, 1)
4395                  If Len(Trim$(CStr(Nz(liste(r, 1))))) > 0 Then gruppenListe(Trim$(CStr(liste(r, 1)))) = True
4396              Next r
4397          End If
4398      End If
4399      If IstHsEinheit(e) Then Exit Function                   ' HS-Einheit (12.91) ist nie ein Makler-Strang
4400      GruppenEinheit = (gruppenListe.Count = 0 Or gruppenListe.Exists(e))
4401  End Function
4402
4403  ' Haupteinheit eines Blatts: die Einheit mit den meisten Zeilen (gleich viele: alphabetisch die erste); leer, wenn keine
4404  Private Function HauptAusZaehlung(zaehl As Object) As String
4405      Dim k As Variant, best As Long
4406      For Each k In zaehl.Keys
4407          If CLng(zaehl(k)) > best Or (CLng(zaehl(k)) = best And CStr(k) < HauptAusZaehlung) Then
4408              best = CLng(zaehl(k)): HauptAusZaehlung = CStr(k)
4409          End If
4410      Next k
4411  End Function
4412
4413  ' dieselbe Regel auf den gelesenen Datenzeilen (Spalte OE_NR, ohne Fusszeilen „Anzahl …“)
4414  Private Function HauptStrang(daten As Variant, sp() As Long) As String
4415      Dim r As Long, t As String, zaehl As Object
4416      If sp(9) = 0 Then Exit Function
4417      Set zaehl = CreateObject("Scripting.Dictionary")
4418      For r = 1 To UBound(daten, 1)
4419          t = Trim$(CStr(Nz(daten(r, sp(9)))))
4420          If Len(t) > 0 And Not t Like "Anzahl*" Then
4421              t = EinheitAusOE(t)
4422              If GruppenEinheit(t) Then zaehl(t) = zaehl(t) + 1
4423          End If
4424      Next r
4425      HauptStrang = HauptAusZaehlung(zaehl)
4426  End Function
4427
4428  ' Strang einer Datenzeile (Bereich MAKLER, Spalte OE_NR): ihre Einheit, wenn sie in Konfiguration!R11:R40 steht, sonst die
4429  ' Haupteinheit des Blatts. Ohne Spalte OE_NR entscheidet der Aufrufer (ein Strang je Blatt).
4430  Private Function ZeileStrang(daten As Variant, r As Long, sp() As Long, haupt As String) As String
4431      Dim e As String
4432      e = ZeilenEinheit(daten, r, sp)
4433      If e <> "(ohne OE)" And Len(e) > 0 Then
4434          If GruppenEinheit(e) Then ZeileStrang = e: Exit Function
4435      End If
4436      ZeileStrang = haupt
4437  End Function
4438
4439  ' Verkettungskennzeichen „verkettet, nicht führend“ -> Zeile zaehlt nicht (wie im MIS-O-Bericht). Tolerant geschrieben:
4440  ' Gross/Klein, Bindestrich/Unterstrich, ohne Umlaut („nicht fuehrend“, „nicht-führend“, „Verkettet_nicht_fuehrend“)
4441  Private Function NichtFuehrend(daten As Variant, r As Long, sp() As Long) As Boolean
4442      Dim t As String
4443      If sp(10) = 0 Then Exit Function
4444      t = LCase$(CStr(Nz(daten(r, sp(10)))))
4445      t = Replace(Replace(Replace(t, "-", " "), "_", " "), "ue", "ü")
4446      Do While InStr(t, "  ") > 0
4447          t = Replace(t, "  ", " ")
4448      Loop
4449      NichtFuehrend = (InStr(t, "nicht führ") > 0 Or InStr(t, "nichtführ") > 0)
4450  End Function
4451
4452  ' Hinweis fuer das Ladeprotokoll: ob der Abzug die Spalte Verkettungskennzeichen hat und wie viele verkettete, nicht fuehrende
4453  ' Vorgaenge ausgelassen wurden. Ohne Spalte zaehlen sie mit — MIS-O-Berichte mit Verkettungsfilter zeigen dann weniger.
4454  Private Function VerkettungHinweis(sp() As Long, n As Long, mitTrenner As Boolean) As String
4455      VerkettungHinweis = VerkettungText(sp(10) > 0, n, mitTrenner)
4456  End Function
4457
4458  Private Function VerkettungText(mitSpalte As Boolean, n As Long, mitTrenner As Boolean) As String
4459      Dim t As String
4460      If Not mitSpalte Then
4461          t = "ohne Spalte Verkettungskennzeichen: verkettete, nicht führende Vorgänge zählen mit (MIS-O-Bericht mit Verkettungsfilter zeigt weniger)"
4462      Else
4463          t = "Verkettungskennzeichen: " & Format(n, "#,##0") & IIf(n = 1, " nicht führender Vorgang", " nicht führende Vorgänge") & " ausgelassen"
4464      End If
4465      VerkettungText = IIf(mitTrenner, " · ", "") & t
```

```
End Function

Private Function SortiertVerbunden(d As Object) As String
    Dim w As Variant, i As Long, j As Long, tmp As Variant
    If d.Count = 0 Then Exit Function
    w = d.Keys
    For i = 0 To UBound(w) - 1
        For j = i + 1 To UBound(w)
            If w(j) < w(i) Then tmp = w(i): w(i) = w(j): w(j) = tmp
        Next j
    Next i
    SortiertVerbunden = Join(w, "+")
End Function

' Einheit einer Datenzeile (Bereich MAKLER, Spalte OE_NR), sonst leer
Private Function ZeilenEinheit(daten As Variant, r As Long, sp() As Long) As String
    If sp(9) = 0 Then Exit Function                     ' StandDatenLesen setzt sp(9) nur im Bereich MAKLER
    ZeilenEinheit = EinheitAusOE(Trim$(CStr(Nz(daten(r, sp(9))))))
End Function

' OE_NR -> Einheit (OE-Ebene 4): laengster passender Anfang aus Konfiguration!Q11:Q40 -> R; sonst die fuehrenden Buchstaben
' ohne angehaengtes "HH" (ODPB1 D -> ODPB, ODPBHH -> ODPB, MSUE2 M -> MSUE). Ergebnisse werden je Laden zwischengespeichert.
Public Function EinheitAusOE(oe As String) As String
    Dim t As String, i As Long, ch As String, zuordnung As Variant, r As Long, best As Long, anf As String
    t = UCase$(Trim$(oe))
    If Len(t) = 0 Then EinheitAusOE = "(ohne OE)": Exit Function
    If oeEinheit Is Nothing Then Set oeEinheit = CreateObject("Scripting.Dictionary")
    If oeEinheit.Exists(t) Then EinheitAusOE = oeEinheit(t): Exit Function
    On Error Resume Next
    zuordnung = ThisWorkbook.Worksheets("Konfiguration").Range("Q11:R40").Value2
    On Error GoTo 0
    If IsArray(zuordnung) Then
        For r = 1 To UBound(zuordnung, 1)
            anf = UCase$(Trim$(CStr(Nz(zuordnung(r, 1)))))
            If Len(anf) > best And Len(Trim$(CStr(Nz(zuordnung(r, 2))))) > 0 Then
                If Left$(t, Len(anf)) = anf Then best = Len(anf): EinheitAusOE = Trim$(CStr(zuordnung(r, 2)))
            End If
        Next r
    End If
    If best = 0 Then
        For i = 1 To Len(t)
            ch = Mid$(t, i, 1)
            If ch Like "[A-Z]" Then EinheitAusOE = EinheitAusOE & ch Else Exit For
        Next i
        If Len(EinheitAusOE) > 4 And Right$(EinheitAusOE, 2) = "HH" Then EinheitAusOE = Left$(EinheitAusOE, Len(EinheitAusOE) - 2)
        If Len(EinheitAusOE) = 0 Then EinheitAusOE = t
    End If
    oeEinheit.Add t, EinheitAusOE
End Function

' Namen der Einheiten fuer die Sicht "Einheit" der Rueckstandsdiagramme (Filter!L22:L25): bis zu vier Einheiten alphabetisch,
' bei mehr als vier die ersten drei und "Übrige"
Public Sub EinheitenNamenSetzen()
    Dim liste As Variant, n As Long, i As Long, aus(1 To 4, 1 To 1) As Variant
    liste = EinheitenListe()
    n = UBound(liste) - LBound(liste) + 1
    If n < 0 Then n = 0
    For i = 1 To 4
        If i <= n Then aus(i, 1) = liste(LBound(liste) + i - 1)
    Next i
    If n > 4 Then aus(4, 1) = "Übrige"
    ThisWorkbook.Worksheets("Filter").Range("L22:L25").Value = aus
    EinheitChipsSetzen liste
End Sub

' Einheiten-Chips im Cockpit (Bereich Makler, 12.86): Filter!X5:X10 die ersten sechs Einheiten der Fakten (sortiert), Y = im Filter.
' Bekannte Einheiten behalten ihren Zustand, neue kommen mit WAHR; leere Zeilen stehen auf WAHR (ohne Chip, ohne Wirkung).
' Ab der siebten Einheit gibt es keinen Chip — solche Einheiten zaehlen immer (nicht in EH_Liste).
Private Sub EinheitChipsSetzen(liste As Variant)
    Dim ws As Worksheet, alt As Object, i As Long, n As Long, namen(1 To 6, 1 To 1) As Variant, zust(1 To 6, 1 To 1) As Variant, e As String
    Set ws = ThisWorkbook.Worksheets("Filter")
    Set alt = CreateObject("Scripting.Dictionary")
    For i = 5 To 10
        e = CStr(Nz(ws.Cells(i, 24).Value))
        If Len(e) > 0 And Not alt.Exists(e) Then alt.Add e, (ws.Cells(i, 25).Value <> False)
    Next i
    n = UBound(liste) - LBound(liste) + 1
    If n < 0 Then n = 0
    For i = 1 To 6
        zust(i, 1) = True
        If i <= n Then
            e = CStr(liste(LBound(liste) + i - 1))
            namen(i, 1) = e
            If alt.Exists(e) Then zust(i, 1) = alt(e)
        Else
            namen(i, 1) = Empty
        End If
    Next i
    ws.Range("X5:X10").Value = namen
```

```
4555      ws.Range("Y5:Y10").Value = zust
4556      If n > 6 Then LogSchreiben "Einheiten-Chips: " & n & " Einheiten, Chips nur für die ersten sechs"
4557  End Sub
4558
4559  ' "'ODPB D'" / "('ODPB D', 'ODPP D')" -> "ODPB D" / "ODPB D+ODPP D"
4560  Private Function EinheitNorm(t As String) As String
4561      Dim s As String, teile() As String, i As Long, e As String
4562      s = Replace(Replace(Replace(Replace(t, "'", ""), Chr(34), ""), "(", ""), ")", "")
4563      teile = Split(s, ",")
4564      For i = 0 To UBound(teile)
4565          e = Trim$(teile(i))
4566          Do While InStr(e, "  ") > 0: e = Replace(e, "  ", " "): Loop
4567          If Len(e) > 0 Then EinheitNorm = EinheitNorm & IIf(Len(EinheitNorm) > 0, "+", "") & e
4568      Next i
4569  End Function
4570
4571  ' Einheiten in den geladenen Fakten (Spalte R), sortiert – fuer die Einheiten-Auswahl (modWerkzeuge.EinheitZeigen, Dat
        eien vor 12.70)
4572  Public Function EinheitenListe() As Variant
4573      Dim ws As Worksheet, letzte As Long, a As Variant, d As Object, i As Long, j As Long, w As Variant, tmp As Variant
        , t As String
4574      Set ws = ThisWorkbook.Worksheets("Fakten")
4575      Set d = CreateObject("Scripting.Dictionary")
4576      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
4577      If letzte >= 2 Then
4578          a = ws.Range(ws.Cells(2, SP_EINHEIT_F), ws.Cells(letzte + 1, SP_EINHEIT_F)).Value2
4579          For i = 1 To UBound(a, 1) - 1
4580              t = CStr(Nz(a(i, 1)))
4581              If Len(t) > 0 Then
4582                  If Not d.Exists(t) Then d.Add t, True
4583              End If
4584          Next i
4585      End If
4586      If d.Count = 0 Then EinheitenListe = Array(): Exit Function
4587      w = d.Keys
4588      For i = 0 To UBound(w) - 1
4589          For j = i + 1 To UBound(w)
4590              If w(j) < w(i) Then tmp = w(i): w(i) = w(j): w(j) = tmp
4591          Next j
4592      Next i
4593      EinheitenListe = w
4594  End Function
4595
4596  ' ---------------------------------------------------------------- Tool leeren (Knopf "Zurücksetzen" im Cockpit)
4597  ' Leert alle geladenen Staende: Fakten, Umbuchungen, Verlauf (bis auf die erste Formelzeile), Ladeprotokoll, Snapshots
        , die
4598  ' Postkorb-Liste im Filter und die Standard-Auswahl der Postkoerbe. Mit anwesenheit zusaetzlich Mitarbeiter und Kalend
        er.
4599  ' Die Rohdateien bleiben im Ordner; "Neue Daten laden" liest sie danach wieder ein (auch aus der Monatsablage).
4600  Public Sub ToolLeeren(anwesenheit As Boolean)
4601      Dim ws As Worksheet, letzte As Long, q As Variant, calc As XlCalculation
4602      If lademodusAktiv Then Exit Sub
4603      calc = Application.Calculation
4604      Application.ScreenUpdating = False
4605      Application.EnableEvents = False
4606      Application.Calculation = xlCalculationManual
4607      On Error GoTo Ende
4608      LogSchreiben "---- Tool leeren" & IIf(anwesenheit, " (mit Anwesenheit)", "")
4609      Set ws = ThisWorkbook.Worksheets("Fakten")
4610      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
4611      If letzte >= 2 Then ws.Range(ws.Cells(2, 1), ws.Cells(letzte, 15)).ClearContents
4612      If letzte >= 2 Then ws.Range(ws.Cells(2, SP_EINHEIT_F), ws.Cells(letzte, SP_ALTSUM_F)).ClearContents
4613      If letzte >= 3 Then ws.Range(ws.Cells(3, FAKTEN_SPALTEN), ws.Cells(letzte, FAKTEN_SPALTEN + 1)).ClearContents   '
        Formeln ab Zeile 3 (Zeile 2 bleibt Vorlage)
4614      Set ws = ThisWorkbook.Worksheets("Umbuchungen")
4615      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
4616      If letzte >= 2 Then ws.Range(ws.Cells(2, 1), ws.Cells(letzte, 10)).ClearContents
4617      If letzte >= 2 Then ws.Range(ws.Cells(2, SP_EINHEIT_U), ws.Cells(letzte, SP_EINHEIT_VOR_U)).ClearContents
4618      Set ws = ThisWorkbook.Worksheets("Verlauf")
4619      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
4620      If letzte >= 7 Then ws.Rows("7:" & letzte).ClearContents
4621      ws.Range("A6").ClearContents
4622      ws.Range("AL6").Value = 1: ws.Range("AM6").Value = 1
4623      Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
4624      letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
4625      If letzte >= 5 Then ws.Range(ws.Cells(5, 1), ws.Cells(letzte, SP_GRUPPE_L)).ClearContents
4626      For Each q In Array("OPAG", "Verbund")
4627          Set ws = SnapshotBlatt(CStr(q))
4628          ws.Range(ws.Cells(2, 1), ws.Cells(ws.Rows.Count, 6)).ClearContents
4629          ws.Range("H2:H3").ClearContents
4630          ws.Range(ws.Cells(2, 10), ws.Cells(SNAP_EINHEITEN + 1, 12)).ClearContents
4631      Next q
4632      ' Postkoerbe: Liste und Kennzeichen im Filter, Standard-Auswahl in der Konfiguration (zeilengleich I11:I50)
4633      ThisWorkbook.Worksheets("Filter").Range(ThisWorkbook.Worksheets("Filter").Cells(PK_VON, 2), ThisWorkbook.Worksheet
        s("Filter").Cells(PK_BIS, 3)).ClearContents
4634      ThisWorkbook.Worksheets("Konfiguration").Range("I11:I50").ClearContents
4635      ThisWorkbook.Worksheets("Filter").Range("L16").Value = "Gesamt"
4636      EinheitenNamenSetzen
4637      If anwesenheit Then AnwesenheitLeeren
4638      modCockpit.BezugFormelSetzen
4639  Ende:
4640      Dim fehlertext As String
4641      If Err.Number <> 0 Then fehlertext = Err.Description
4642      On Error Resume Next
4643      Application.Calculation = xlCalculationAutomatic
4644      Application.Calculate
```

```
    modCockpit.DiagrammeAnpassen True
    Application.Calculation = calc
    Application.EnableEvents = True
    Application.ScreenUpdating = True
    If Len(fehlertext) > 0 Then
        LogSchreiben "FEHLER beim Leeren: " & fehlertext
        modFortschritt.HinweisZeigen "Fehler", "Beim Zurücksetzen ist ein Fehler aufgetreten: " & fehlertext, True
    End If
End Sub

' Mitarbeiter (Stammdaten A:N ab MA_START) und Kalender (Eintraege ab Zeile 7) leeren, Formeln bleiben
Private Sub AnwesenheitLeeren()
    Dim ws As Worksheet, rg As Range
    Set ws = ThisWorkbook.Worksheets("Mitarbeiter")
    On Error Resume Next
    Set rg = ws.Range(ws.Cells(MA_START, 1), ws.Cells(MA_START + MA_ZEILEN - 1, 14)).SpecialCells(xlCellTypeConstants)
    If Not rg Is Nothing Then rg.ClearContents
    ws.Range("W5:W7").ClearContents                         ' letzte Datei, Zeitpunkt, Format
    Set rg = Nothing
    Set ws = ThisWorkbook.Worksheets("Kalender")
    Set rg = ws.Range(ws.Cells(MA_START + PL_VERSATZ, 4), ws.Cells(MA_START + PL_VERSATZ + MA_ZEILEN - 1, ws.Columns.C
   ount)).SpecialCells(xlCellTypeConstants)
    If Not rg Is Nothing Then rg.ClearContents
    On Error GoTo 0
End Sub

' ------------------------------------------------------------- Beladung läuft (orange Box oben rechts im Cockpit)
' Filter!L28 = Name, L29 = Beginn. Gesetzt beim Start von „Neue Daten laden“, geleert am Ende (auch nach Fehler, Ersat
   z-Frage,
' leerem Ordner). Die Datei wird jeweils gespeichert, damit Kollegen auf SharePoint die Box sehen bzw. danach die neue
   n Werte.
' Wer die Datei schon vorher offen hatte, sieht L28/L29 aber nicht (seine Kopie wird nicht nachgefuehrt). Deshalb zusa
   etzlich eine
' Sperrdatei neben der Master-Datei (<Master>_Beladung.txt, erste Zeile „Name<TAB>JJJJ-MM-TT hh:mm:ss“, nach dem Ende
' „frei<TAB>Name<TAB>Ende“), die jeder Klick frisch liest: lokal bzw. im synchronisierten Ordner direkt, im SharePoint
   -Modus ueber
' Power Query (Abfrage BUE_SharePoint_Sperre, Blatt SP_Liste ab N1) – geschrieben wie das Celonis-Protokoll.
Public Function NutzerName() As String
    Dim t As String
    t = Trim$(Application.UserName)
    If InStr(t, "(") > 1 Then t = Trim$(Left$(t, InStr(t, "(") - 1))   ' "Brenneisen, Eike (HSB4K-EXTERN)" -> "Brennei
   sen, Eike"
    If Len(t) = 0 Then t = Environ$("USERNAME")
    NutzerName = t
End Function

' Laeuft gerade die Beladung eines anderen? Ergebnis: Hinweistext, sonst leer. Quellen: L28/L29 der eigenen Kopie und
   die Sperrdatei
' (siehe oben). Meldet die Sperrdatei „frei“ mit einem Ende nach dem Beginn in L28/L29, ist jene Beladung vorbei (die
   Datei wurde
' danach nicht mehr gespeichert, etwa nach einem Abbruch beim Ersetzen).
Public Function BeladungAndererText() As String
    Dim ws As Worksheet, wer As String, seit As Variant, beginn As Double, ende As Double, z As String, t() As String
    On Error Resume Next
    Set ws = ThisWorkbook.Worksheets("Filter")
    wer = Trim$(CStr(ws.Range("L28").Value))
    seit = ws.Range("L29").Value2                           ' Zahl (Datum als Seriennummer), unabhaengig vom Zellform
   at
    If IsNumeric(seit) And Not IsEmpty(seit) Then beginn = CDbl(seit)
    z = BeladungDateiLesen()
    t = Split(z, vbTab)
    If UBound(t) >= 1 Then
        If StrComp(TextFeld(t(0)), "frei", vbTextCompare) = 0 Then
            If UBound(t) >= 2 Then ende = ZeitAusText(TextFeld(t(2)))
            If ende > 0 And ende >= beginn Then wer = ""
        Else
            BeladungAndererText = BeladungHinweis(TextFeld(t(0)), ZeitAusText(TextFeld(t(1))))
            If Len(BeladungAndererText) > 0 Then Exit Function
        End If
    End If
    BeladungAndererText = BeladungHinweis(wer, beginn)
End Function

' Hinweistext zu einer Beladung (Name, Beginn als Zahl); leer, wenn keine, die eigene (etwa nach einem Absturz von Exc
   el) oder
' aelter als 3 Stunden (gilt dann als abgebrochen)
Private Function BeladungHinweis(wer As String, seit As Double) As String
    If Len(wer) = 0 Or seit <= 0 Then Exit Function
    If CDbl(Now) - seit > 0.125 Then Exit Function
    If StrComp(wer, NutzerName(), vbTextCompare) = 0 Then Exit Function
    BeladungHinweis = wer & " lädt gerade neue Daten (seit " & Format$(CDate(seit), "hh:nn") & " Uhr). Bitte warten, b
   is die Beladung " & _
        "fertig ist. Danach die Datei schließen, ohne zu speichern, und neu öffnen, damit die neuen Werte erscheinen."
End Function

' Name der Sperrdatei: je Master-Datei (HS und Makler koennen im selben Ordner liegen)
Private Function BeladungDatei() As String
    Dim nm As String
    nm = ThisWorkbook.Name
    If InStrRev(nm, ".") > 1 Then nm = Left$(nm, InStrRev(nm, ".") - 1)
    BeladungDatei = nm & "_Beladung.txt"
End Function

' erste Zeile der Sperrdatei; leer = keine Datei, leer oder nicht lesbar (dann gilt nur L28/L29)
Private Function BeladungDateiLesen() As String
    Dim b As String, m As String, a As Variant, f As Integer, z As String
```

```
4730      On Error GoTo Fehler
4731      b = BasisOrdner()
4732      If Len(b) = 0 Then Exit Function
4733      If IstUrl(b) Then
4734          m = SpTextFormel(b, BeladungDatei(), PROTOKOLL_ORDNER)
4735          If Len(m) = 0 Then Exit Function
4736          a = SpTabelle(SP_SPERRE_ABFRAGE, m, "N1", "Sperrdatei „Beladung läuft“ aus dem SharePoint-Ordner der Master-Da
          tei (wird vom Tool geschrieben)")
4737          If IsArray(a) Then BeladungDateiLesen = CStr(Nz(a(1, 1)))
4738      Else
4739          If Len(Dir(ProtokollPfad(b, BeladungDatei()))) = 0 Then Exit Function
4740          f = FreeFile
4741          Open ProtokollPfad(b, BeladungDatei()) For Input As #f
4742          If Not EOF(f) Then Line Input #f, z
4743          Close #f
4744          f = 0
4745          BeladungDateiLesen = z
4746      End If
4747      Exit Function
4748  Fehler:
4749      z = Err.Description
4750      On Error Resume Next
4751      If f > 0 Then Close #f
4752      LogSchreiben "  Sperrdatei " & BeladungDatei() & " nicht lesbar: " & z
4753  End Function
4754
4755  ' Sperrdatei schreiben (an: Beginn, aus: „frei“ mit Ende). Fehler halten das Laden nicht auf, sie stehen im Ladelog.
4756  Private Sub BeladungDateiSchreiben(an As Boolean)
4757      Dim b As String, zeile As Variant, f As Integer, wb As Workbook, bild As Boolean, fehler As String
4758      bild = Application.ScreenUpdating
4759      On Error GoTo Fehler
4760      b = BasisOrdner()
4761      If Len(b) = 0 Then Exit Sub
4762      If an Then
4763          zeile = Array(NutzerName(), Format$(Now, "yyyy-mm-dd hh:nn:ss"))
4764      Else
4765          zeile = Array("frei", NutzerName(), Format$(Now, "yyyy-mm-dd hh:nn:ss"))
4766      End If
4767      If IstUrl(b) Then                                       ' SharePoint ohne Dateipfad: ueber Excel als Text speiche
         rn
4768          Application.ScreenUpdating = False
4769          Set wb = Workbooks.Add(xlWBATWorksheet)
4770          wb.Worksheets(1).Range("A1").Resize(1, UBound(zeile) + 1).NumberFormat = "@"
4771          wb.Worksheets(1).Range("A1").Resize(1, UBound(zeile) + 1).Value = zeile
4772          SpTextSpeichern wb, b, BeladungDatei()                  ' xlTextWindows, in Protokolle (sonst neben der Master
         -Datei)
4773          wb.Close SaveChanges:=False
4774          Set wb = Nothing
4775          ThisWorkbook.Activate
4776          Application.ScreenUpdating = bild
4777      Else
4778          f = FreeFile
4779          Open ProtokollPfad(b, BeladungDatei()) For Output As #f
4780          Print #f, Join(zeile, vbTab)
4781          Close #f
4782          f = 0
4783      End If
4784      Exit Sub
4785  Fehler:
4786      fehler = Err.Description
4787      On Error Resume Next
4788      If f > 0 Then Close #f
4789      Application.DisplayAlerts = True
4790      If Not wb Is Nothing Then wb.Close SaveChanges:=False: ThisWorkbook.Activate
4791      Application.ScreenUpdating = bild
4792      LogSchreiben "  Sperrdatei " & BeladungDatei() & " nicht geschrieben: " & fehler
4793  End Sub
4794
4795  ' "2026-10-08 09:15:00" -> Zeitpunkt als Zahl (unabhaengig von der Spracheinstellung), sonst 0
4796  Private Function ZeitAusText(t As String) As Double
4797      Dim s As String
4798      s = Trim$(t)
4799      If Not (s Like "####-##-## ##:##:##") Then Exit Function
4800      On Error Resume Next
4801      ZeitAusText = CDbl(DateSerial(CInt(Left$(s, 4)), CInt(Mid$(s, 6, 2)), CInt(Mid$(s, 9, 2)))) + _
4802                    CDbl(TimeSerial(CInt(Mid$(s, 12, 2)), CInt(Mid$(s, 15, 2)), CInt(Mid$(s, 18, 2))))
4803  End Function
4804
4805  ' Sperre setzen/aufheben: L28/L29, Sperrdatei, dann die Master-Datei speichern (speichern = False: nur Sperre aufheben
         , etwa nach
4806  ' einem Abbruch beim Ersetzen). Vor dem Speichern wird das Ladefenster (LADE_*) entfernt, sonst laege es in der gespei
        cherten Datei
4807  ' ueber dem Cockpit und fienge die Klicks ab.
4808  Private Sub BeladungMarkieren(an As Boolean, Optional speichern As Boolean = True)
4809      Dim ws As Worksheet
4810      On Error Resume Next
4811      Set ws = ThisWorkbook.Worksheets("Filter")
4812      If an Then
4813          ws.Range("L28").Value = NutzerName()
4814          ws.Range("L29").Value = Now
4815      Else
4816          ws.Range("L28:L29").ClearContents
4817          modFortschritt.FortschrittSchliessen
4818      End If
4819      BeladungDateiSchreiben an
4820      If ThisWorkbook.ReadOnly Or Not speichern Then Exit Sub
```

```
    Application.DisplayAlerts = False
    ThisWorkbook.Save
    Application.DisplayAlerts = True
    If Err.Number <> 0 Then LogSchreiben "  Speichern fuer die Beladungs-Box nicht moeglich: " & Err.Description
    Err.Clear
End Sub

' ---------------------------------------------------------------- Ältester Vorgang (Knopf an der Kachel im Cockpit)
' Faktenzeile mit dem hoechsten Rueckstandsalter am letzten Stichtag der Quelle nach aktivem Filter – dieselbe Rechnung wie die
' Kachel (Filter!O31/O32 Stichtag, O47/O48 Alter). Ergebnis Array: 0 Quelle, 1 Stichtag, 2 Alter, 3 Sparte, 4 VSS, 5 Postkorb,
' 6 GeVo, 7 Einheit, 8 Gruppe, 9 Schluessel (leer bei Staenden, die vor 12.64 geladen wurden), 10 Datei, 11 Blatt; Empty = keiner.
Public Function AeltesterVorgang(quelle As String) As Variant
    Dim wsF As Worksheet, wsI As Worksheet, tag As Variant, alter As Variant, letzte As Long, a As Variant, e As Variant, r As Long
    Dim w As Variant, datei As String, blatt As String, wsL As Worksheet, p As Variant
    Set wsI = ThisWorkbook.Worksheets("Filter")
    tag = wsI.Range(IIf(quelle = "OPAG", "O31", "O32")).Value2
    alter = wsI.Range(IIf(quelle = "OPAG", "O48", "O47")).Value2
    If Not IsNumeric(tag) Or IsEmpty(tag) Then Exit Function
    If Not IsNumeric(alter) Or IsEmpty(alter) Then Exit Function
    If CDbl(tag) <= 0 Then Exit Function
    Set wsF = ThisWorkbook.Worksheets("Fakten")
    letzte = wsF.Cells(wsF.Rows.Count, 1).End(xlUp).Row
    If letzte < 2 Then Exit Function
    a = wsF.Range(wsF.Cells(2, 1), wsF.Cells(letzte + 1, FAKTEN_SPALTEN)).Value2
    e = wsF.Range(wsF.Cells(2, SP_EINHEIT_F), wsF.Cells(letzte + 1, SP_VORGANG_F)).Value2
    For r = 1 To UBound(a, 1) - 1
        If Val(a(r, 1)) = CLng(tag) And CStr(Nz(a(r, 2))) = quelle And Val(a(r, FAKTEN_SPALTEN)) = 1 Then
            If Val(e(r, 3)) = CDbl(alter) Then
                w = Array(quelle, CDate(tag), CDbl(alter), CStr(Nz(a(r, 3))), CStr(Nz(a(r, 4))), CStr(Nz(a(r, 5))), CStr(Nz(a(r, 6))), _
                          CStr(Nz(e(r, 1))), CStr(Nz(e(r, 2))), CStr(Nz(e(r, 4))), "", "")
                Exit For
            End If
        End If
    Next r
    If Not IsArray(w) Then Exit Function
    ' Rohdatei des Stands aus dem Ladeprotokoll (gueltige Zeile zu Quelle, Stichtag, Gruppe)
    Set wsL = ThisWorkbook.Worksheets("Ladeprotokoll")
    letzte = wsL.Cells(wsL.Rows.Count, 1).End(xlUp).Row
    If letzte >= 5 Then
        p = wsL.Range(wsL.Cells(5, 1), wsL.Cells(letzte + 1, SP_GRUPPE_L)).Value2
        For r = 1 To UBound(p, 1) - 1
            If CStr(Nz(p(r, 4))) = quelle And Val(p(r, 5)) = CLng(tag) And CStr(Nz(p(r, SP_GRUPPE_L))) = CStr(w(8)) Then
                If StandGueltig(CStr(Nz(p(r, 10)))) Then datei = CStr(Nz(p(r, 2))): blatt = CStr(Nz(p(r, 3)))
            End If
        Next r
    End If
    w(10) = datei: w(11) = blatt
    AeltesterVorgang = w
End Function

' Rohdatei des aeltesten Vorgangs sichtbar oeffnen (schreibgeschuetzt) und die Zeile gelb markieren. Gesucht wird ueber den
' Schluessel (VSS, Ordnungsbegriff, LNR …); fehlt er: die Zeile mit VSS, Postkorb und GeVo und dem hoechsten Alter.
' Ergebnis leer = markiert, sonst Hinweistext.
Public Function VorgangMarkieren(quelle As String) As String
    Dim v As Variant, ordner As String, pfad As String, wb As Workbook, ws As Worksheet, w As Worksheet, pruef As Stand
    Dim kopfdaten As Variant, kopf As Long, letzteSpalte As Long, r As Long, c As Long, sp() As Long, daten As Variant
    Dim letzte As Range, letzteZeile As Long, ziel As Long, best As Double, alt As Double, nm As String, st As Stand, mitSchluessel As Boolean
    v = AeltesterVorgang(quelle)
    If Not IsArray(v) Then VorgangMarkieren = "Kein Rückstand " & quelle & " am letzten Stichtag.": Exit Function
    If Len(v(10)) = 0 Then VorgangMarkieren = "Die Rohdatei dieses Stands steht nicht im Ladeprotokoll.": Exit Function
    On Error GoTo Fehler
    ReDim sp(1 To 10)
    Set spListen = Nothing
    ordner = RohdatenOrdner()
    st.Datei = CStr(v(10)): st.Quelle = quelle: st.Stichtag = v(1): st.Gruppe = CStr(v(8))
    If Not IstUrl(ordner) Then
        If Not DateiVorhanden(ordner & st.Datei) Then st.Datei = StandDatei(ordner, st)
    End If
    If Len(st.Datei) = 0 Then VorgangMarkieren = "Die Rohdatei " & v(10) & " liegt nicht mehr im Ordner Rohdaten.": Exit Function
    pfad = PfadVon(ordner, st.Datei)
    nm = DateiName(pfad)
    On Error Resume Next
    Set wb = Workbooks(nm)
    On Error GoTo Fehler
    If wb Is Nothing Then
        Application.StatusBar = "Öffne " & nm & " …"
        Set wb = Workbooks.Open(FileName:=pfad, UpdateLinks:=0, ReadOnly:=True, IgnoreReadOnlyRecommended:=True)
        Application.StatusBar = False
    End If
    On Error Resume Next
    Set ws = wb.Worksheets(CStr(v(11)))
    On Error GoTo Fehler
    If ws Is Nothing Then
        For Each w In wb.Worksheets
            If BlattErkennen(w, pruef) Then
```

```
                If pruef.Quelle = quelle And CLng(pruef.Stichtag) = CLng(v(1)) Then Set ws = w: Exit For
            End If
        Next w
    End If
    If ws Is Nothing Then VorgangMarkieren = "Blatt des Stands in " & nm & " nicht gefunden.": Exit Function
    letzteSpalte = ws.UsedRange.Column + ws.UsedRange.Columns.Count - 1
    kopfdaten = ws.Range(ws.Cells(1, 1), ws.Cells(MAX_KOPFZEILEN, letzteSpalte)).Value2
    For r = 1 To MAX_KOPFZEILEN
        For c = 1 To letzteSpalte
            If Trim$(CStr(Nz(kopfdaten(r, c)))) = "Themenpostkorb" Then kopf = r
        Next c
        If kopf > 0 Then Exit For
    Next r
    If kopf = 0 Then VorgangMarkieren = "Kopfzeile in " & nm & " nicht gefunden.": Exit Function
    For c = 1 To letzteSpalte
        Select Case UCase$(Trim$(CStr(Nz(kopfdaten(kopf, c)))))
            Case "VERWALTUNGSSYSTEMSCHLÜSSEL": sp(1) = c
            Case "ORDNUNGSBEGRIFF": sp(2) = c
            Case "VORG_VERBU_LNR": sp(3) = c
            Case "THEMENPOSTKORB": sp(4) = c
            Case "RÜCKSTANDSALTER_TAGE": sp(5) = c
            Case "GEVO_SL": sp(6) = c
            Case "OPS_AUFTR_ID": sp(7) = c
            Case "OPS_ARBGUT_LNR": sp(8) = c
        End Select
    Next c
    If sp(1) = 0 Or sp(2) = 0 Or sp(3) = 0 Or sp(4) = 0 Or sp(5) = 0 Or sp(6) = 0 Then VorgangMarkieren = "Spalten in
    " & nm & " nicht erkannt.": Exit Function
    Set letzte = ws.Columns(sp(1)).Find(What:="*", LookIn:=xlValues, SearchOrder:=xlByRows, SearchDirection:=xlPreviou
    s)
    If letzte Is Nothing Then letzteZeile = kopf Else letzteZeile = letzte.Row
    If letzteZeile <= kopf Then VorgangMarkieren = "Keine Datensätze in " & nm & ".": Exit Function
    daten = ws.Range(ws.Cells(kopf + 1, 1), ws.Cells(letzteZeile + 1, letzteSpalte)).Value2      ' +1: immer zweidimen
    sional
    mitSchluessel = (Len(CStr(v(9))) > 0)
    best = -1
    For r = 1 To UBound(daten, 1) - 1
        If mitSchluessel Then
            If DatensatzSchluessel(daten, r, sp, quelle) = CStr(v(9)) Then ziel = r: Exit For
        ElseIf Sauber(daten(r, sp(1))) = CStr(v(4)) Then
            If PostkorbNorm(CStr(Nz(daten(r, sp(4))))) = CStr(v(5)) And Verdichten(CStr(Nz(daten(r, sp(6))))) = CStr(v
    (6)) Then
                If IsNumeric(daten(r, sp(5))) And Not IsEmpty(daten(r, sp(5))) Then
                    alt = CDbl(daten(r, sp(5)))
                    If alt > best Then best = alt: ziel = r
                End If
            End If
        End If
    Next r
    If ziel = 0 Then VorgangMarkieren = "Der Vorgang wurde in " & nm & " nicht gefunden.": Exit Function
    wb.Activate
    ws.Activate
    ws.Range(ws.Cells(kopf + ziel, 1), ws.Cells(kopf + ziel, letzteSpalte)).Interior.Color = RGB(255, 230, 120)
    Application.Goto ws.Cells(kopf + ziel, 1), True
    ws.Range(ws.Cells(kopf + ziel, 1), ws.Cells(kopf + ziel, letzteSpalte)).Select
    Exit Function
Fehler:
    Application.StatusBar = False
    VorgangMarkieren = "Rohdatei konnte nicht geöffnet werden: " & Err.Description
End Function

' ------------------------------------------------------------- Rohdateien (Knopf "Rohdaten löschen" im Cockpit)
' Alle Excel-Dateien im Rohdaten-Ordner und in der Ablage (relative Pfade, neueste zuerst nach Dateiname/Datum)
Public Function RohdateienListe() As Variant
    Dim ordner As String, dateien() As String, n As Long, i As Long, j As Long, t As String, aus() As String
    On Error GoTo Leer
    Set spListen = Nothing
    ordner = RohdatenOrdner()
    If Not OrdnerVorhanden(ordner) Then GoTo Leer
    n = DateienSammeln(ordner, dateien)
    If n = 0 Then GoTo Leer
    For i = 0 To n - 2
        For j = i + 1 To n - 1
            If StrComp(DateiName(dateien(j)), DateiName(dateien(i)), vbTextCompare) > 0 Then t = dateien(i): dateien(i
    ) = dateien(j): dateien(j) = t
        Next j
    Next i
    ReDim aus(0 To n - 1)
    For i = 0 To n - 1
        aus(i) = dateien(i)
    Next i
    RohdateienListe = aus
    Exit Function
Leer:
    RohdateienListe = Array()
End Function

' Wurde aus dieser Datei (relativer Pfad) ein Stand geladen? (Ladeprotokoll, gueltige Zeilen)
Public Function RohdateiGeladen(rel As String) As Boolean
    Dim ws As Worksheet, letzte As Long, a As Variant, r As Long, h As String
    Set ws = ThisWorkbook.Worksheets("Ladeprotokoll")
    letzte = ws.Cells(ws.Rows.Count, 1).End(xlUp).Row
    If letzte < 5 Then Exit Function
    a = ws.Range(ws.Cells(5, 1), ws.Cells(letzte + 1, 10)).Value2
    For r = 1 To UBound(a, 1) - 1
        h = CStr(Nz(a(r, 10)))
```

```
4997          If StandGueltig(h) Then
4998              If StrComp(DateiName(CStr(Nz(a(r, 2)))), DateiName(rel), vbTextCompare) = 0 Then RohdateiGeladen = True: E
        xit Function
4999          End If
5000      Next r
5001  End Function
5002
5003  Public Function RohdateiGroesse(rel As String) As Double
5004      RohdateiGroesse = DateiGroesse(PfadVon(RohdatenOrdner(), rel))
5005  End Function
5006
5007  ' Datei endgueltig loeschen; Ergebnis leer = geloescht, sonst Fehlertext. Auf SharePoint kann Excel nicht loeschen: di
     e Datei wird
5008  ' im Tool ausgeblendet (beim Laden und in dieser Liste uebergangen), endgueltig loescht man sie im Browser.
5009  Public Function RohdateiLoeschen(rel As String) As String
5010      If RohdatenAufSharePoint() Then
5011          SpAusblenden rel
5012          LogSchreiben "Rohdatei ausgeblendet (SharePoint): " & rel
5013          Exit Function
5014      End If
5015      On Error Resume Next
5016      Kill RohdatenOrdner() & rel
5017      If Err.Number <> 0 Then RohdateiLoeschen = Err.Description Else LogSchreiben "Rohdatei gelöscht: " & rel
5018      On Error GoTo 0
5019  End Function
5020
5021  ' ============================================================================
5022  ' Anwesenheitsliste (MAK je Mitarbeiter und Kalenderwoche) importieren
5023  ' Zwei Formate: Enthaelt die Datei ein Blatt "Verwaltung" (Telefonie- und Abwesenheitsplan, eine Zeile je Tag, eine Sp
     alte je
5024  ' Person), uebernimmt PlanUebernehmen die Tage direkt (siehe dort). Sonst gilt die MAK-Liste je KW:
5025  ' Erwartet im Ordner "Anwesenheit" (Konfiguration!B4) die neueste Excel-Datei mit einer Kopfzeile "Mitarbeiter",
5026  ' den Spalten "Makler BÜ Gesundheit" / "Makler BÜ Komposit" (Sparte) und Wochenspalten "NN KW" / "NN. KW".
5027  ' Ergebnis: Stammdaten im Blatt "Mitarbeiter" (BUe-Kapazitaet = groesster Wochenwert), Abweichungen je Tag im Blatt "K
     alender"
5028  ' (Urlaub = "U", sonstiger Text = "A", abweichende Zahl = Zahl, Kapazitaet oder leer = Zelle leer).
5029  ' Faellt die KW-Nummer gegenueber der Vorspalte, beginnt ein neues Jahr (Liste ueber den Jahreswechsel).
5030  ' Mitarbeiter!W5:W7 = Datei, Zeitpunkt und Format ("Plan" / "MAK-Liste") des letzten Imports.
5031  ' imLaden = True: Aufruf aus NeueDatenLaden (AnwesenheitImLaden) – laeuft in dessen Lademodus und Fortschritt, ohne ei
     gene
5032  ' Neuberechnung (die folgt dort); ein Fehler steht dann nur in der Meldung (importMeldung), nie als Karte.
5033  ' ============================================================================
5034  Public Sub AnwesenheitLaden(Optional stumm As Boolean = False, Optional imLaden As Boolean = False)
5035      AnwesenheitStatusLeeren
5036      Dim ordner As String, datei As String, neueste As String, neuesteZeit As Date, pfad As String
5037      Dim wbQ As Workbook, ws As Worksheet, daten As Variant
5038      Dim r As Long, c As Long, kopf As Long, cMa As Long, cEinheit As Long, cGes As Long, cKom As Long
5039      Dim kwSpalten() As Long, kwNummern() As Long, kwJahre() As Long, nKw As Long, t As String, jahr As Long
5040      Dim wsM As Worksheet, wsP As Worksheet, wsW As Worksheet, ma As String, sparte As String, w As Variant
5041      Dim zeile As Long, kap As Double, spalte As Variant, anzahl As Long
5042      Dim kwSpalteZiel() As Long, cMin As Long, cMax As Long, zeilenWerte As Variant, zi As Long, schluesselZeile As Var
        iant, tg As Long, eintrag As Variant
5043      Dim eigenerLademodus As Boolean, offenVorher As Boolean
5044
5045      If lademodusAktiv And Not imLaden Then Exit Sub
5046      If imLaden Then stumm = True
5047      anwImLaden = imLaden
5048      If Not imLaden Then Set spListen = Nothing             ' im Laden ist die Auflistung schon frisch
5049      On Error Resume Next
5050      ordner = OrdnerAufloesen(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B4").Value), "Anwesenheit")
5051      If Err.Number <> 0 Then
5052          t = Err.Description
5053          On Error GoTo 0
5054          AnwNichtUebernommen t
5055          If Not stumm Then modFortschritt.HinweisZeigen "Fehler", t, True
5056          Exit Sub
5057      End If
5058      ' ab hier jeder Fehler (auch beim Auflisten auf SharePoint) in die eigene Behandlung – im Laden darf er das Laden
        nicht abbrechen
5059      On Error GoTo Fehler
5060      jahr = CLng(Val(ThisWorkbook.Worksheets("Konfiguration").Range("B5").Value))
5061      If jahr = 0 Then jahr = Year(Date)
5062
5063      If Not OrdnerVorhanden(ordner) Then
5064          AnwNichtUebernommen "Ordner nicht gefunden (" & ordner & ")"
5065          If Not stumm Then
5066              If IstUrl(ordner) Then
5067                  modFortschritt.HinweisZeigen "Fehler", SpHinweis(ordner), True
5068              Else
5069                  modFortschritt.HinweisZeigen "Fehler", "Der Ordner für die Anwesenheitsliste wurde nicht gefunden:" &
        vbCrLf & ordner, True
5070              End If
5071          End If
5072          Exit Sub
5073      End If
5074      If IstUrl(ordner) Then                                 ' SharePoint: neueste Datei laut Auflistung (nur der Ordn
        er selbst)
5075          Dim spL As Object, spK As Variant, spZeit As Double, spBest As Double
5076          Set spL = SpListe(ordner)
5077          If spL Is Nothing Then Err.Raise vbObjectError + 10, , "Der SharePoint-Ordner der Anwesenheit ist nicht lesbar
       (" & spFehler & ")."
5078          spBest = -1
5079          For Each spK In spL.Keys
5080              datei = CStr(spL(spK)(0))
5081              If InStr(datei, Application.PathSeparator) = 0 And Left$(datei, 2) <> "~$" And InStr(datei, ".") > 0 Then
5082                  If IstExcelDatei(datei) Then
```

```
5083                    spZeit = ZeitWert(spL(spK)(1))
5084                    If spZeit > spBest Then spBest = spZeit: neueste = datei
5085                End If
5086            End If
5087        Next spK
5088    Else
5089        datei = Dir(ordner & "*.xls*")
5090        Do While Len(datei) > 0
5091            If Left$(datei, 2) <> "~$" And IstExcelDatei(datei) Then
5092                If FileDateTime(ordner & datei) > neuesteZeit Then
5093                    neuesteZeit = FileDateTime(ordner & datei)
5094                    neueste = datei
5095                End If
5096            End If
5097            datei = Dir()
5098        Loop
5099    End If
5100    If Len(neueste) = 0 Then
5101        AnwNichtUebernommen "keine Excel-Datei in " & ordner
5102        If Not stumm Then modFortschritt.HinweisZeigen "Hinweis", "Im Ordner " & ordner & " liegt keine Excel-Datei.",
        False
5103        Exit Sub
5104    End If
5105    pfad = PfadVon(ordner, neueste)
5106
5107    LogSchreiben "---- AnwesenheitLaden " & neueste & IIf(imLaden, " (beim Laden)", "")
5108    If Not imLaden Then
5109        LadeModusEin "Anwesenheitsliste wird importiert"
5110        eigenerLademodus = True
5111    End If
5112    AnwSetzen 0.1, "Anwesenheitsliste lesen", neueste
5113    ' Der Plan wird laufend gepflegt und ist oft gerade offen: dann die offene Mappe lesen (auch noch nicht Gespeicher
       tes)
5114    ' und sie offen lassen – QuelleOeffnen lehnt offene Dateien ab (richtig fuer Rohdaten)
5115    On Error Resume Next
5116    Set wbQ = Workbooks(DateiName(neueste))
5117    On Error GoTo Fehler
5118    If wbQ Is Nothing Then
5119        Set wbQ = QuelleOeffnen(pfad)
5120    Else
5121        offenVorher = True
5122        LogSchreiben "  Datei ist in Excel geöffnet, gelesen wird die offene Mappe"
5123    End If
5124    Set ws = Nothing
5125    On Error Resume Next
5126    Set ws = wbQ.Worksheets("Verwaltung")
5127    On Error GoTo Fehler
5128    If Not ws Is Nothing Then                               ' Telefonie- und Abwesenheitsplan (eine Zeile je Tag)
5129        daten = ws.Range("A1:CZ900").Value2
5130        If Not offenVorher Then wbQ.Close SaveChanges:=False
5131        Set wbQ = Nothing
5132        Dim planInfo As String
5133        AnwSetzen 0.3, "Anwesenheitsplan je Tag übernehmen"
5134        anzahl = PlanUebernehmen(daten, planInfo)
5135        wsM_Stand neueste, "Plan"
5136        modCockpit.AlphabetischSortieren
5137        modCockpit.KalenderVorbelegen
5138        LogSchreiben "  Anwesenheitsplan: " & anzahl & " Mitarbeitende, " & planInfo & "; " & anwHeute
5139        If Not imLaden Then
5140            AnwSetzen 0.95, "MAK und Produktivität werden neu berechnet"
5141            Application.Calculation = xlCalculationAutomatic
5142            Application.Calculate
5143            modCockpit.DiagrammeAnpassen True
5144            AnwSetzen 1, "Fertig"
5145            LadeModusAus
5146        End If
5147        anwPlan = True
5148        importMeldung = "Anwesenheitsplan übernommen: " & neueste & ", " & anzahl & " Mitarbeitende, " & planInfo
5149        If Not stumm Then modFortschritt.HinweisZeigen "Hinweis", importMeldung & "." & vbCrLf & anwHeute & "." & vbCr
       Lf & vbCrLf & _
5150            "„x“ im Plan = abwesend. Neue Personen zählen voll (BÜ-Kapazität 1,0); eine im Blatt 'Mitarbeiter' geänder
       te Kapazität und die Sparte bleiben. " & _
5151            "„Neue Daten laden“ übernimmt den Plan jedes Mal neu.", False
5152        Exit Sub
5153    End If
5154    Set ws = wbQ.Worksheets(1)
5155    daten = ws.UsedRange.Value2
5156    If Not offenVorher Then wbQ.Close SaveChanges:=False
5157    Set wbQ = Nothing
5158    If Not IsArray(daten) Then Err.Raise vbObjectError + 2, , "Die Datei enthält keine Tabelle."
5159    AnwSetzen 0.3, "Kopfzeile und Wochenspalten erkennen"
5160
5161    For r = 1 To UBound(daten, 1)
5162        For c = 1 To UBound(daten, 2)
5163            If InStr(1, CStr(Nz(daten(r, c))), "Mitarbeiter", vbTextCompare) > 0 Then kopf = r: cMa = c
5164        Next c
5165        If kopf > 0 Then Exit For
5166    Next r
5167    If kopf = 0 Then Err.Raise vbObjectError + 2, , "Keine Kopfzeile mit 'Mitarbeiter' gefunden."
5168    nKw = 0
5169    For c = 1 To UBound(daten, 2)
5170        t = Replace(Replace(CStr(Nz(daten(kopf, c))), vbLf, " "), ".", "")
5171        t = Trim$(t)
5172        If InStr(1, t, "Gesundheit", vbTextCompare) > 0 Then cGes = c
5173        If InStr(1, t, "Komposit", vbTextCompare) > 0 Then cKom = c
5174        If InStr(1, t, "Einheit", vbTextCompare) > 0 Then cEinheit = c
```

```
5175          If Len(t) > 2 Then                                    ' And wertet nicht verkuerzt aus: Left$(t, -2) waere Fehl
      er 5
5176              If UCase$(Right$(t, 2)) = "KW" Then
5177                  If IsNumeric(Trim$(Left$(t, Len(t) - 2))) Then
5178                      ReDim Preserve kwSpalten(0 To nKw): ReDim Preserve kwNummern(0 To nKw): ReDim Preserve kwJahre(0 T
      o nKw)
5179                      kwSpalten(nKw) = c: kwNummern(nKw) = CLng(Trim$(Left$(t, Len(t) - 2)))
5180                      If nKw > 0 Then
5181                          If kwNummern(nKw) < kwNummern(nKw - 1) Then jahr = jahr + 1     ' Jahreswechsel in der Liste
5182                      End If
5183                      kwJahre(nKw) = jahr
5184                      nKw = nKw + 1
5185                  End If
5186              End If
5187          End If
5188      Next c
5189      If nKw = 0 Then Err.Raise vbObjectError + 3, , "Keine Wochenspalten ('NN KW') gefunden."
5190      If UBound(daten, 1) <= kopf Then Err.Raise vbObjectError + 4, , "Keine Mitarbeiterzeilen unter der Kopfzeile."
5191
5192      Set wsM = ThisWorkbook.Worksheets("Mitarbeiter")
5193      Set wsP = ThisWorkbook.Worksheets("Kalender")
5194      Set wsW = ThisWorkbook.Worksheets("Wirksam")
5195      ' Erste Kalenderspalte je KW einmal bestimmen (Wirksam!6 = KW-Schluessel je Tag; eine KW hat bis zu 5 Tagesspalten
      )
5196      ReDim kwSpalteZiel(0 To nKw - 1)
5197      cMin = 0: cMax = 0
5198      For c = 0 To nKw - 1
5199          spalte = Application.Match(kwJahre(c) * 100 + kwNummern(c), wsW.Rows(6), 0)
5200          If IsError(spalte) Then
5201              kwSpalteZiel(c) = 0
5202          Else
5203              kwSpalteZiel(c) = CLng(spalte)
5204              If cMin = 0 Or kwSpalteZiel(c) < cMin Then cMin = kwSpalteZiel(c)
5205              If kwSpalteZiel(c) > cMax Then cMax = kwSpalteZiel(c)
5206          End If
5207      Next c
5208      If cMin = 0 Then Err.Raise vbObjectError + 5, , "Keine der Wochenspalten passt zum Jahr " & kwJahre(0) & " (Konfig
      uration!B5) im Kalender."
5209      schluesselZeile = wsW.Range(wsW.Cells(6, cMin), wsW.Cells(6, cMax + 5)).Value2
5210
5211      For r = kopf + 1 To UBound(daten, 1)
5212          ma = Trim$(CStr(Nz(daten(r, cMa))))
5213          AnwSetzen 0.3 + 0.6 * (r - kopf) / (UBound(daten, 1) - kopf), "Übernehme " & ma
5214          If Len(ma) > 0 And UCase$(ma) <> "MAK" Then
5215              sparte = "Komposit"
5216              If cGes > 0 Then
5217                  ' IsNumeric(Empty) ist True, CDbl("") aber Fehler 13 – deshalb Leerzelle zuerst ausschliessen
5218                  If Not IsEmpty(daten(r, cGes)) Then
5219                      If IsNumeric(daten(r, cGes)) Then
5220                          If CDbl(daten(r, cGes)) > 0 Then sparte = "Gesundheit"
5221                      End If
5222                  End If
5223              End If
5224              kap = 0
5225              For c = 0 To nKw - 1
5226                  w = daten(r, kwSpalten(c))
5227                  If IsNumeric(w) And Not IsEmpty(w) Then
5228                      If CDbl(w) > kap Then kap = CDbl(w)
5229                  End If
5230              Next c
5231              zeile = MitarbeiterZeile(wsM, ma)
5232              If zeile = 0 Then Err.Raise vbObjectError + 6, , "Kein Platz mehr im Blatt Mitarbeiter (maximal " & MA_ZEI
      LEN & " Zeilen)."
5233              wsM.Cells(zeile, 1).Value = ma
5234              If cEinheit > 0 Then wsM.Cells(zeile, 2).Value = Trim$(CStr(Nz(daten(r, cEinheit))))
5235              wsM.Cells(zeile, 3).Value = sparte
5236              wsM.Cells(zeile, 4).Value = kap
5237              If IsEmpty(wsM.Cells(zeile, 7).Value) Then wsM.Cells(zeile, 7).Value = True
5238              anzahl = anzahl + 1
5239              ' Kalenderzeile als ein Block: vorhandene Werte lesen, Wochenwerte der Liste auf die Tage der KW, einmal s
      chreiben.
5240              ' Nur Abweichungen werden eingetragen ("Urlaub" = U, andere Abwesenheit = A, abweichende Zahl = anwesender
       Anteil
5241              ' Zahl / BUe-Kapazitaet); leere Wochen und volle Kapazitaet lassen vorhandene Tageseintraege stehen.
5242              zeilenWerte = wsP.Range(wsP.Cells(zeile + PL_VERSATZ, cMin), wsP.Cells(zeile + PL_VERSATZ, cMax + 5)).Valu
      e2
5243              For c = 0 To nKw - 1
5244                  If kwSpalteZiel(c) > 0 Then
5245                      w = daten(r, kwSpalten(c))
5246                      eintrag = Empty
5247                      If IsEmpty(w) Or Len(Trim$(CStr(Nz(w)))) = 0 Then
5248                          eintrag = Empty
5249                      ElseIf IsNumeric(w) Then
5250                          If Abs(CDbl(w) - kap) >= 0.0001 Then
5251                              If kap > 0 Then eintrag = Round(Application.Max(0, Application.Min(1, CDbl(w) / kap)), 2)
      Else eintrag = 0
5252                          End If
5253                      ElseIf InStr(1, CStr(w), "urlaub", vbTextCompare) > 0 Then
5254                          eintrag = "U"
5255                      Else
5256                          eintrag = "A"
5257                      End If
5258                      If Not IsEmpty(eintrag) Then
5259                          For tg = 0 To 4
5260                              zi = kwSpalteZiel(c) - cMin + 1 + tg
5261                              If zi <= UBound(zeilenWerte, 2) Then
```

```
5262                                If schluesselZeile(1, zi) = kwJahre(c) * 100 + kwNummern(c) Then zeilenWerte(1, zi) =
    eintrag
5263                            End If
5264                        Next tg
5265                    End If
5266                End If
5267            Next c
5268            wsP.Range(wsP.Cells(zeile + PL_VERSATZ, cMin), wsP.Cells(zeile + PL_VERSATZ, cMax + 5)).Value2 = zeilenWer
    te
5269        End If
5270    Next r
5271    wsM_Stand neueste, "MAK-Liste"
5272    modCockpit.AlphabetischSortieren                    ' neue Mitarbeitende einsortieren, Plan wandert zeilengleich mi
    t
5273    modCockpit.KalenderVorbelegen                       ' frei laut Arbeitszeitmodell und Feiertage eintragen
5274    LogSchreiben "  " & anzahl & " Mitarbeitende"
5275    If Not imLaden Then
5276        AnwSetzen 0.95, "MAK und Produktivität werden neu berechnet"
5277        Application.Calculation = xlCalculationAutomatic
5278        Application.Calculate
5279        LogSchreiben "  Diagramme anpassen"
5280        modCockpit.DiagrammeAnpassen True
5281        AnwSetzen 1, "Fertig"
5282        LadeModusAus
5283    End If
5284    LogSchreiben "  Fertig"
5285    importMeldung = "MAK-Liste importiert: " & neueste & ", " & anzahl & " Mitarbeitende, KW " & kwNummern(0) & "/" &
    kwJahre(0) & " bis KW " & kwNummern(nKw - 1) & "/" & kwJahre(nKw - 1)
5286    If Not stumm Then modFortschritt.HinweisZeigen "Hinweis", importMeldung & "." & vbCrLf & vbCrLf & "Arbeitszeitmode
    ll und neue Mitarbeitende pflegst du im Blatt 'Mitarbeiter', Abwesenheiten je Tag im 'Kalender'.", False
5287    Exit Sub
5288
5289 Fehler:
5290    Dim fehlertext As String, nummer As Long
5291    nummer = Err.Number
5292    fehlertext = Err.Description                        ' eigene Meldungen (Err.Raise vbObjectError + n) ohne Nummer
5293    If nummer < vbObjectError Or nummer >= vbObjectError + 100 Then fehlertext = "Fehler " & nummer & ": " & fehlertex
    t
5294    On Error Resume Next
5295    If Not wbQ Is Nothing And Not offenVorher Then wbQ.Close SaveChanges:=False
5296    If eigenerLademodus Then LadeModusAus
5297    LogSchreiben "  Anwesenheit FEHLER " & nummer & ": " & Err.Description & " – " & fehlertext
5298    AnwNichtUebernommen fehlertext
5299    If Not stumm Then modFortschritt.HinweisZeigen "Fehler", "Fehler beim Import der Anwesenheitsliste: " & fehlertext
    , True
5300 End Sub
5301
5302 ' Import versucht, aber nicht gelungen: Meldung fuer Laden und Tagesablauf (der Kalender bleibt, wie er ist)
5303 Private Sub AnwNichtUebernommen(grund As String)
5304    anwPlan = False
5305    anwFehlschlag = True
5306    importMeldung = "Anwesenheit nicht übernommen: " & grund & " – MAK und Produktivität rechnen mit dem bisherigen St
    and im Kalender"
5307 End Sub
5308
5309 ' Anwesenheit innerhalb von „Neue Daten laden“ (vor Neuberechnung und Speichern, siehe AnwesenheitAutomatisch).
5310 ' Rueckgabe: Meldung fuer die Abschlusskarte (leer = nicht versucht).
5311 Private Function AnwesenheitImLaden() As String
5312    Dim vorText As String
5313    AnwesenheitStatusLeeren
5314    If IstMakler() Then
5315        ladeOrt = "Bearbeiter MSUED ergänzen"
5316        If ZusatzMitarbeiterSetzen() > 0 Then vorText = vorText & IIf(Len(vorText) > 0, "." & vbCrLf, "") & "Bearbeite
    r der Einheit " & ZUSATZ_EINHEIT & " im Blatt Mitarbeiter ergänzt"
5317    End If
5318    If Not AnwesenheitAutomatisch() Then AnwesenheitImLaden = vorText: Exit Function
5319    ladeOrt = "Anwesenheit übernehmen"
5320    LogSchreiben "Anwesenheit übernehmen"
5321    AnwesenheitLaden True, True
5322    AnwesenheitImLaden = vorText
5323    If Len(importMeldung) > 0 Then AnwesenheitImLaden = vorText & IIf(Len(vorText) > 0, "." & vbCrLf, "") & importMeld
    ung
5324 End Function
5325
5326 ' Uebernimmt „Neue Daten laden“ die Anwesenheit mit? Ja, solange noch niemand im Blatt Mitarbeiter steht (erster Impor
    t, beide
5327 ' Formate), und immer, wenn sie aus dem Telefonie- und Abwesenheitsplan kommt (Mitarbeiter!W7 = "Plan"; im Bereich Mak
    ler auch
5328 ' bei leerem W7 – Dateien vor 12.70): der Plan wird laufend gepflegt, Abwesenheiten sollen ohne eigenen Klick in MAK u
    nd
5329 ' Produktivitaet. Die MAK-Liste je KW (W7 = "MAK-Liste") bleibt ein einmaliger Import (danach „Anwesenheit laden“), da
    mit
5330 ' gepflegte Sparte, Kapazitaet und Eintraege im Kalender stehen bleiben.
5331 Public Function AnwesenheitAutomatisch() As Boolean
5332    Dim wsM As Worksheet
5333    On Error Resume Next
5334    Set wsM = ThisWorkbook.Worksheets("Mitarbeiter")
5335    If Len(Trim$(CStr(Nz(wsM.Range("A" & MA_START).Value)))) = 0 Then
5336        AnwesenheitAutomatisch = True
5337    ElseIf StrComp(Trim$(CStr(Nz(wsM.Range("W7").Value))), "Plan", vbTextCompare) = 0 Then
5338        AnwesenheitAutomatisch = True
5339    Else
5340        AnwesenheitAutomatisch = IstMakler() And StrComp(Trim$(CStr(Nz(wsM.Range("W7").Value))), "MAK-Liste", vbTextCo
    mpare) <> 0
5341    End If
5342    On Error GoTo 0
```

```
End Function

' Ergebnis des letzten Imports fuer den Tagesablauf: aus dem Plan? heutige Anwesenheit laut Plan, Fehlertext (leer = kein Fehler)
Public Function AnwesenheitAusPlan() As Boolean
    AnwesenheitAusPlan = anwPlan
End Function

Public Function AnwesenheitHeute() As String
    AnwesenheitHeute = anwHeute
End Function

Public Function AnwesenheitFehltHeute() As Long
    AnwesenheitFehltHeute = anwFehltHeute
End Function

Public Function AnwesenheitMeldung() As String
    AnwesenheitMeldung = importMeldung
End Function

Public Function AnwesenheitFehler() As String
    If anwFehlschlag Then AnwesenheitFehler = importMeldung
End Function

Public Function AnwesenheitDatei() As String
    On Error Resume Next
    AnwesenheitDatei = CStr(Nz(ThisWorkbook.Worksheets("Mitarbeiter").Range("W5").Value))
End Function

' Beginn des Tagesablaufs: Ergebnis eines frueheren Imports dieser Sitzung vergessen
Public Sub AnwesenheitStatusLeeren()
    importMeldung = ""
    anwPlan = False: anwFehlschlag = False: anwHeute = "": anwFehltHeute = -1
End Sub

' Fortschritt des Anwesenheits-Imports; innerhalb von „Neue Daten laden“ auf dem Stueck zwischen Laden (0,9) und Neuberechnung (0,92)
Private Sub AnwSetzen(ByVal anteil As Double, ByVal status As String, Optional ByVal detail As String = "")
    If anwImLaden Then anteil = 0.9 + 0.02 * anteil
    modFortschritt.FortschrittSetzen anteil, status, detail
End Sub

Private Sub wsM_Stand(datei As String, art As String)
    With ThisWorkbook.Worksheets("Mitarbeiter")
        .Range("W5").Value = datei
        .Range("W6").Value = Now
        .Range("W6").NumberFormat = "dd.mm.yyyy hh:mm"
        .Range("W7").Value = art
    End With
End Sub

' Telefonie- und Abwesenheitsplan (Blatt "Verwaltung", z. B. ODPB3D + KBIG2D) uebernehmen:
' - Kopf: Zeile mit "Datum" (Spalte der Tage), Zeile mit "Mitarbeiter" (1 = zaehlt mit), Zeile mit "Team ..." (Beginn je Team),
'   Namen ueber diesen Zeilen; Spalte "Arbeitsfrei" (1 = Feiertag).
' - Personen: jede Spalte mit 1 in der Mitarbeiter-Zeile innerhalb eines Teamblocks (Block endet an der ersten Spalte ohne
'   Namen und ohne 1). Personen ausserhalb der Teams (z. B. Leitung) bleiben aussen vor.
' - Keine Schaetzung des BUe-Anteils: neue Personen zaehlen voll (BUe-Kapazitaet 1,0); eine im Blatt Mitarbeiter gepflegte
'   Kapazitaet und die Sparte bleiben (neu: Komposit).
' - Je Arbeitstag Mo-Fr: "x" (gross oder klein) = abwesend -> Kalender "A" (vorhandenes U/K bleibt); jeder andere Eintrag oder leer
'   = anwesend -> ein vorhandenes A wird geloescht (es stammt aus dem Plan). Von Hand eingetragene U/K, Anteile und Feiertage
'   bleiben unberuehrt – auch an vergangenen Tagen, damit fruehere Wochen beim erneuten Uebernehmen nicht kippen.
' - Heute laut Plan (anwHeute, anwFehltHeute) fuer den Tagesablauf; aktive Personen im Blatt Mitarbeiter, die im Plan fehlen,
'   nennt die Meldung (sie zaehlen weiter, bis dort ein Austritt steht).
Private Function PlanUebernehmen(daten As Variant, ByRef info As String) As Long
    Dim zDatum As Long, cDatum As Long, zMa As Long, zTeam As Long, cFrei As Long, r As Long, c As Long, t As String
    Dim namen() As String, teams() As String, spalten() As Long, n As Long, team As String, k As Long, nm As String
    Dim wsM As Worksheet, wsP As Worksheet, wsW As Worksheet, spalteVon As Object, letzte As Long, datum As Double
    Dim zeile As Long, werte As Variant, cMin As Long, cMax As Long, v As String, alt As String, abw As Long, tage As Long
    Dim erster As Double, letzter As Double, i As Long
    Dim rHeute As Long, heuteFrei As Boolean, nFehlt As Long, fehlt As String, imPlan As Object, ohne As String, nmZ As Variant

    For r = 1 To 20
        For c = 1 To UBound(daten, 2)
            t = Trim$(CStr(Nz(daten(r, c))))
            If zDatum = 0 And StrComp(t, "Datum", vbTextCompare) = 0 Then zDatum = r: cDatum = c
            If zMa = 0 And StrComp(t, "Mitarbeiter", vbTextCompare) = 0 Then zMa = r
            If zTeam = 0 And LCase$(Left$(t, 5)) = "team " Then zTeam = r
            If cFrei = 0 And InStr(1, t, "Arbeitsfrei", vbTextCompare) > 0 Then cFrei = c
        Next c
    Next r
    If zDatum = 0 Or zMa = 0 Or zTeam = 0 Then Err.Raise vbObjectError + 7, , "Im Blatt Verwaltung fehlen die Zeilen Datum, Mitarbeiter oder Team."

    ' Personen je Teamblock
    ReDim namen(1 To UBound(daten, 2)): ReDim teams(1 To UBound(daten, 2)): ReDim spalten(1 To UBound(daten, 2))
    For c = 1 To UBound(daten, 2)
        t = Trim$(CStr(Nz(daten(zTeam, c))))
        If LCase$(Left$(t, 5)) = "team " Then team = t
```

```
5428          nm = ""
5429          For k = zMa - 1 To 1 Step -1                          ' Name: erster Text ueber der Mitarbeiter-Zeile, der kei
      n Teamkopf ist
5430              t = Trim$(Replace(CStr(Nz(daten(k, c))), vbLf, " "))
5431              If Len(t) > 0 And LCase$(Left$(t, 5)) <> "team " Then nm = t: Exit For
5432          Next k
5433          If Len(nm) = 0 And CStr(Nz(daten(zMa, c))) <> "1" Then team = ""     ' Luecke: Teamblock zu Ende
5434          If Len(team) > 0 And Len(nm) > 0 And CStr(Nz(daten(zMa, c))) = "1" Then
5435              n = n + 1: namen(n) = nm: teams(n) = team: spalten(n) = c
5436          End If
5437      Next c
5438      If n = 0 Then Err.Raise vbObjectError + 8, , "Im Anwesenheitsplan wurden keine Personen in einem Team gefunden."
5439
5440      Set wsM = ThisWorkbook.Worksheets("Mitarbeiter")
5441      Set wsP = ThisWorkbook.Worksheets("Kalender")
5442      Set wsW = ThisWorkbook.Worksheets("Wirksam")
5443      Set spalteVon = CreateObject("Scripting.Dictionary")         ' Datum -> Spalte im Kalender (Wirksam!5)
5444      letzte = wsW.Cells(5, wsW.Columns.Count).End(xlToLeft).Column
5445      werte = wsW.Range(wsW.Cells(5, 1), wsW.Cells(5, letzte)).Value2
5446      For c = 1 To letzte
5447          If IsNumeric(werte(1, c)) And Not IsEmpty(werte(1, c)) Then
5448              If CDbl(werte(1, c)) > 40000 Then spalteVon(CLng(werte(1, c))) = c
5449          End If
5450      Next c
5451      For r = zDatum + 1 To UBound(daten, 1)
5452          If IsNumeric(daten(r, cDatum)) And Not IsEmpty(daten(r, cDatum)) Then
5453              If spalteVon.Exists(CLng(daten(r, cDatum))) Then
5454                  c = spalteVon(CLng(daten(r, cDatum)))
5455                  If cMin = 0 Or c < cMin Then cMin = c
5456                  If c > cMax Then cMax = c
5457              End If
5458              ' Zeile von heute (nur Arbeitstag ohne "Arbeitsfrei" = 1) fuer die heutige Anwesenheit
5459              If CLng(daten(r, cDatum)) = CLng(Date) Then
5460                  t = ""
5461                  If cFrei > 0 Then t = CStr(Nz(daten(r, cFrei)))
5462                  If Weekday(Date, vbMonday) > 5 Or t = "1" Then heuteFrei = True Else rHeute = r
5463              End If
5464          End If
5465      Next r
5466      If cMin = 0 Then Err.Raise vbObjectError + 9, , "Kein Tag des Anwesenheitsplans liegt im Zeitraum des Kalenders."
5467
5468      For i = 1 To n
5469          AnwSetzen 0.3 + 0.6 * i / n, "Übernehme " & namen(i)
5470          zeile = MitarbeiterZeile(wsM, namen(i))
5471          If zeile = 0 Then Err.Raise vbObjectError + 6, , "Kein Platz mehr im Blatt Mitarbeiter (maximal " & MA_ZEILEN
      & " Zeilen)."
5472          wsM.Cells(zeile, 1).Value = namen(i)
5473          wsM.Cells(zeile, 2).Value = teams(i)
5474          If Len(Trim$(CStr(Nz(wsM.Cells(zeile, 3).Value)))) = 0 Then wsM.Cells(zeile, 3).Value = "Komposit"
5475          ' neue Person: voll (keine Schaetzung des BUe-Anteils); eine gepflegte Kapazitaet bleibt beim erneuten Ueberne
      hmen
5476          If Len(Trim$(CStr(Nz(wsM.Cells(zeile, 4).Value)))) = 0 Then wsM.Cells(zeile, 4).Value = 1
5477          If IsEmpty(wsM.Cells(zeile, 7).Value) Then wsM.Cells(zeile, 7).Value = True
5478          werte = wsP.Range(wsP.Cells(zeile + PL_VERSATZ, cMin), wsP.Cells(zeile + PL_VERSATZ, cMax)).Value2
5479          For r = zDatum + 1 To UBound(daten, 1)
5480              If IsNumeric(daten(r, cDatum)) And Not IsEmpty(daten(r, cDatum)) Then
5481                  datum = CDbl(daten(r, cDatum))
5482                  If spalteVon.Exists(CLng(datum)) And Weekday(datum, vbMonday) <= 5 Then
5483                      If cFrei = 0 Then
5484                          t = ""
5485                      Else
5486                          t = CStr(Nz(daten(r, cFrei)))
5487                      End If
5488                      If t <> "1" Then
5489                          c = spalteVon(CLng(datum)) - cMin + 1
5490                          v = LCase$(Trim$(CStr(Nz(daten(r, spalten(i))))))
5491                          alt = UCase$(Trim$(CStr(Nz(werte(1, c)))))
5492                          If v = "x" Then
5493                              If alt <> "U" And alt <> "K" Then werte(1, c) = "A"
5494                              abw = abw + 1
5495                          ElseIf alt = "A" Then
5496                              werte(1, c) = Empty
5497                          End If
5498                          If i = 1 Then
5499                              tage = tage + 1
5500                              If erster = 0 Or datum < erster Then erster = datum
5501                              If datum > letzter Then letzter = datum
5502                          End If
5503                      End If
5504                  End If
5505              End If
5506          Next r
5507          wsP.Range(wsP.Cells(zeile + PL_VERSATZ, cMin), wsP.Cells(zeile + PL_VERSATZ, cMax)).Value2 = werte
5508          If rHeute > 0 Then
5509              If LCase$(Trim$(CStr(Nz(daten(rHeute, spalten(i)))))) = "x" Then
5510                  nFehlt = nFehlt + 1
5511                  fehlt = fehlt & IIf(Len(fehlt) > 0, ", ", "") & namen(i)
5512              End If
5513          End If
5514      Next i
5515      info = tage & " Arbeitstage vom " & Format(erster, "dd.mm.yyyy") & " bis " & Format(letzter, "dd.mm.yyyy") & ", "
      & abw & " Abwesenheitstage"
5516
5517      ' heute laut Plan
5518      If rHeute > 0 Then
5519          anwFehltHeute = nFehlt
```

```
5520          If nFehlt = 0 Then
5521              anwHeute = "Laut Plan sind heute alle " & n & " da"
5522          Else
5523              anwHeute = "Laut Plan sind heute " & (n - nFehlt) & " von " & n & " da, " & IIf(nFehlt = 1, "es fehlt ", "
        es fehlen ") & fehlt
5524          End If
5525      ElseIf heuteFrei Or Weekday(Date, vbMonday) > 5 Then
5526          anwFehltHeute = -1
5527          anwHeute = "Laut Plan ist heute arbeitsfrei"
5528      Else
5529          anwFehltHeute = -2
5530          anwHeute = "Der heutige Tag steht nicht im Plan" & IIf(letzter > 0, " (er reicht vom " & Format(erster, "dd.mm
        .yyyy") & " bis " & _
5531                      Format(letzter, "dd.mm.yyyy") & ")", "") & " – heute zählen alle als da"
5532      End If
5533
5534      ' aktive Personen im Blatt Mitarbeiter, die der Plan nicht (mehr) fuehrt
5535      Set imPlan = CreateObject("Scripting.Dictionary")
5536      imPlan.CompareMode = vbTextCompare
5537      For i = 1 To n
5538          imPlan(namen(i)) = True
5539      Next i
5540      If IstMakler() Then                                         ' Bearbeiter MSUED stehen nicht im Plan und zaehlen imme
        r mit
5541          ZusatzMitarbeiterSetzen
5542          For Each nmZ In Split(ZUSATZ_MA, "|")
5543              imPlan(CStr(nmZ)) = True
5544          Next nmZ
5545      End If
5546      For r = MA_START To MA_START + MA_ZEILEN - 1
5547          nm = Trim$(CStr(Nz(wsM.Cells(r, 1).Value)))
5548          If Len(nm) > 0 Then
5549              If Not imPlan.Exists(nm) Then
5550                  If MitarbeiterAktiv(wsM, r) Then ohne = ohne & IIf(Len(ohne) > 0, ", ", "") & nm
5551              End If
5552          End If
5553      Next r
5554      If Len(ohne) > 0 Then info = info & "; nicht im Plan, zählen weiter (Austritt im Blatt Mitarbeiter setzen): " & oh
        ne
5555      PlanUebernehmen = n
5556  End Function
5557
5558  ' Bereich MAKLER: die Bearbeiter aus ZUSATZ_MA ins Blatt Mitarbeiter, falls sie fehlen (Einheit = ZUSATZ_EINHEIT, dami
        t die
5559  ' Einheiten-Chips sie zuordnen; Sparte Komposit, Kapazitaet 1, aktiv). Vorhandene Zeilen bleiben, wie sie gepflegt sin
        d; ein
5560  ' Austritt im Blatt Mitarbeiter nimmt eine Person heraus. Kalender: ohne Eintrag = anwesend. Ergebnis: Anzahl neu ange
        legt.
5561  Private Function ZusatzMitarbeiterSetzen() As Long
5562      Dim wsM As Worksheet, nm As Variant, zeile As Long
5563      Set wsM = ThisWorkbook.Worksheets("Mitarbeiter")
5564      For Each nm In Split(ZUSATZ_MA, "|")
5565          zeile = MitarbeiterZeile(wsM, CStr(nm))
5566          If zeile = 0 Then Exit For
5567          If Len(Trim$(CStr(Nz(wsM.Cells(zeile, 1).Value)))) = 0 Then
5568              wsM.Cells(zeile, 1).Value = CStr(nm)
5569              wsM.Cells(zeile, 2).Value = ZUSATZ_EINHEIT
5570              wsM.Cells(zeile, 3).Value = "Komposit"
5571              wsM.Cells(zeile, 4).Value = 1
5572              wsM.Cells(zeile, 7).Value = True
5573              ZusatzMitarbeiterSetzen = ZusatzMitarbeiterSetzen + 1
5574              LogSchreiben "  Mitarbeiter ergänzt: " & nm & " (" & ZUSATZ_EINHEIT & ")"
5575          End If
5576      Next nm
5577  End Function
5578
5579  ' Zaehlt die Person heute mit? Aktiv (G) nicht FALSCH und kein Austritt (F) vor heute
5580  Private Function MitarbeiterAktiv(wsM As Worksheet, r As Long) As Boolean
5581      Dim a As Variant, aus As Variant
5582      MitarbeiterAktiv = True
5583      a = wsM.Cells(r, 7).Value
5584      aus = wsM.Cells(r, 6).Value
5585      If VarType(a) = vbBoolean Then
5586          If a = False Then MitarbeiterAktiv = False
5587      End If
5588      If Not IsError(aus) Then
5589          If IsDate(aus) Then
5590              If CDate(aus) < Date Then MitarbeiterAktiv = False
5591          End If
5592      End If
5593  End Function
5594
5595  Private Function MitarbeiterZeile(wsM As Worksheet, ma As String) As Long
5596      ' Zeile des Mitarbeiters im Blatt Mitarbeiter (Name in Spalte A ab Zeile MA_START), sonst erste freie Zeile; 0 = k
        ein Platz
5597      Dim r As Long
5598      For r = MA_START To MA_START + MA_ZEILEN - 1
5599          If StrComp(Trim$(CStr(wsM.Cells(r, 1).Value)), ma, vbTextCompare) = 0 Then MitarbeiterZeile = r: Exit Function
5600      Next r
5601      For r = MA_START To MA_START + MA_ZEILEN - 1
5602          If Len(Trim$(CStr(wsM.Cells(r, 1).Value))) = 0 Then MitarbeiterZeile = r: Exit Function
5603      Next r
5604      MitarbeiterZeile = 0
5605  End Function
```

modCockpit.bas

```
Option Explicit
' ================================================================================
' modCockpit – Schaltflaechen des Cockpits: Filter setzen, Ansicht aktualisieren
' Die Chips im Cockpit sind Zellen mit HYPERLINK auf ChipHover (Rollover: Filter!Q1 = Chip unter der Maus);
' der Klick landet ueber Worksheet_SelectionChange (Blattmodul Cockpit) in ChipSchalten und schaltet
' Filter!C (Themenpostkoerbe, Zeilen PK_VON..PK_BIS), Filter!H (GeVo-Typen, Zeilen 5..18) bzw. Filter!T (Sparten, Zeil
  en 5..8) um.
' Alle Kennzahlen und Diagramme rechnen ueber Formeln; DiagrammeAnpassen setzt nur veraenderte Bereiche.
' Diagramme: 1/2 Rueckstand OPAG/Verbund, 5 MAK, 6 Altersstruktur, 7/8 Produktivitaet Verbund/OPAG (feste Zeilen
' Diagrammdaten!50:56, Achse J bzw. I, Tag/KW/Woche nach Ansicht),
' 9 Rueckstand je Themenpostkorb; Kachel Eingaenge/Abgaenge: BW_Gesamt, BW_Komposit, BW_Leben, BW_KV
' (Quelle Verbund/OPAG nach Filter!L13, umgeschaltet ueber die Bookmark-Chips BWQ_Verbund/BWQ_OPAG).
' PPT: 10/11 Rueckstand, 12 MAK, 13/14 Bewegungen, 15 Altersstruktur.
' Ansicht Filter!L9: "Tag" (letzte 7 Stichtage), "KW" (Montag bis Sonntag der KW des Bezugstags, Knopf "Aktuelle KW"),
' "Woche" (letzte 7 KW). Achse je Quelle (Diagrammdaten!BR OPAG, BS Verbund): in der Wochensicht steht unter der KW da
  s
' Datum des verwendeten Stands (Sonntag der KW, sonst letzter Stand; * = abgeschlossene KW ohne Sonntagsstand);
' Bewegungen nutzen dieselbe Achse ohne Stern (BT OPAG, BU Verbund).
' ================================================================================

Private aktualisierungGeplant As Boolean   ' Aktualisierung per OnTime bereits geplant
Private reihenFest As Boolean              ' Diagrammdaten-Reihen (feste Zeilen) sind gesetzt
Private letztesK1 As Long                  ' zuletzt gesetztes KW-Fenster
Private letztesP As Long                   ' zuletzt gesetzte letzte Postkorbzeile
' Bezugstag Cockpit!L12: Formel = neuester Stand, Datum als Wert = frueherer Stichtag
Public Const BEZUG_FORMEL As String = "=IF(COUNT(Verlauf!$A$6:$A$1000)=0,"""",MAX(Verlauf!$A$6:$A$1000))"

Public Sub FilterAllePostkoerbe()
    If modLaden.LadenLaeuft() Then Exit Sub
    SetzePostkoerbe True
End Sub

Public Sub FilterAktiveBearbeitung()
    ' Knopf "Standard-Postkörbe": nur die Themenpostkoerbe der Standard-Auswahl (Konfiguration!I11:I50 = "ja")
    If modLaden.LadenLaeuft() Then Exit Sub
    StandardPostkoerbeSetzen
    Aktualisieren
End Sub

Private Sub StandardPostkoerbeSetzen()
    Dim wsF As Worksheet, wsK As Worksheet, r As Long, namen As Variant, std As Variant, aus() As Variant
    Set wsF = ThisWorkbook.Worksheets("Filter")
    Set wsK = ThisWorkbook.Worksheets("Konfiguration")
    namen = wsF.Range(wsF.Cells(modLaden.PK_VON, 2), wsF.Cells(modLaden.PK_BIS, 2)).Value2
    std = wsK.Range("I11:I50").Value2                                  ' zeilengleich mit Filter!B5:B44
    ReDim aus(1 To UBound(namen, 1), 1 To 1)
    For r = 1 To UBound(namen, 1)
        If Len(CStr(namen(r, 1))) > 0 Then aus(r, 1) = (LCase$(Trim$(CStr(std(r, 1)))) = "ja") Else aus(r, 1) = Empty
    Next r
    wsF.Range(wsF.Cells(modLaden.PK_VON, 3), wsF.Cells(modLaden.PK_BIS, 3)).Value = aus
End Sub

' Knopf "Auf Standard zurücksetzen": Themenpostkoerbe und GeVo-Typen wie in der Standard-Auswahl (Konfiguration H:L),
' alle Sparten. Sind alle GeVo-Typen "ja", gilt der Modus "Alle GeVo", sonst "Manuelle Auswahl" mit den Kaestchen.
Public Sub StandardAnwenden()
    Dim wsF As Worksheet, wsK As Worksheet, r As Long, alle As Boolean, ja As Boolean
    If modLaden.LadenLaeuft() Then Exit Sub
    Set wsF = ThisWorkbook.Worksheets("Filter")
    Set wsK = ThisWorkbook.Worksheets("Konfiguration")
    StandardPostkoerbeSetzen
    alle = True
    For r = 0 To 13
        ja = (LCase$(Trim$(CStr(wsK.Cells(11 + r, 12).Value))) <> "nein")
        wsF.Cells(5 + r, 8).Value = ja
        If Not ja And (r = 13 Or Len(CStr(wsF.Cells(5 + r, 7).Value)) > 0) Then alle = False
    Next r
    wsF.Range("L5").Value = IIf(alle, "Alle GeVo", "Manuelle Auswahl")
    wsF.Range("T5:T8").Value = True
    wsF.Range("Y5:Y10").Value = True                    ' Einheiten-Chips (Bereich Makler): alle
    Aktualisieren
End Sub

Public Sub FilterAlleGeVo()
    If modLaden.LadenLaeuft() Then Exit Sub
    ThisWorkbook.Worksheets("Filter").Range("L5").Value = "Alle GeVo"
    Aktualisieren
End Sub

Public Sub FilterNurBUE()
    If modLaden.LadenLaeuft() Then Exit Sub
    ThisWorkbook.Worksheets("Filter").Range("L5").Value = "Nur BÜ-Schlüssel"
    Aktualisieren
End Sub

Public Sub FilterZuruecksetzen()
    ' frueher: alles an; jetzt: zurueck auf die Standard-Auswahl
    StandardAnwenden
End Sub

Public Sub FilterAlleSparten()
    If modLaden.LadenLaeuft() Then Exit Sub
    ThisWorkbook.Worksheets("Filter").Range("T5:T8").Value = True
    Aktualisieren
End Sub
```

```
 93
 94  Public Sub FilterAlleEinheiten()
 95      If modLaden.LadenLaeuft() Then Exit Sub
 96      ThisWorkbook.Worksheets("Filter").Range("Y5:Y10").Value = True
 97      Aktualisieren
 98  End Sub
 99
100  ' Knopf „nur“ neben einem Einheiten-Chip: nur diese Einheit im Filter, alle anderen aus (kein Umschalten)
101  Private Sub EinheitNur(i As Long)
102      Dim wsF As Worksheet, r As Long
103      If modLaden.LadenLaeuft() Then Exit Sub
104      Set wsF = ThisWorkbook.Worksheets("Filter")
105      If Len(CStr(wsF.Cells(4 + i, 24).Value)) = 0 Then Exit Sub
106      For r = 5 To 10
107          wsF.Cells(r, 25).Value = (r = 4 + i Or Len(CStr(wsF.Cells(r, 24).Value)) = 0)
108      Next r
109      Aktualisieren
110  End Sub
111
112  Public Sub EinheitNur1()
113      EinheitNur 1
114  End Sub
115
116  Public Sub EinheitNur2()
117      EinheitNur 2
118  End Sub
119
120  Public Sub EinheitNur3()
121      EinheitNur 3
122  End Sub
123
124  Public Sub EinheitNur4()
125      EinheitNur 4
126  End Sub
127
128  Public Sub EinheitNur5()
129      EinheitNur 5
130  End Sub
131
132  Public Sub EinheitNur6()
133      EinheitNur 6
134  End Sub
135
136  Public Sub Aktualisieren(Optional mitPPT As Boolean = False)
137      ' mitPPT: auch die Diagramme des Blatts PPT nachziehen (vor dem PowerPoint-Export)
138      aktualisierungGeplant = False
139      Application.Calculate
140      FilterAnzeigen
141      DiagrammeAnpassen False, mitPPT
142  End Sub
143
144  Public Sub Hilfe()
145      If modLaden.LadenLaeuft() Then Exit Sub
146      Dim titel As String
147      titel = CStr(IIf(modLaden.IstMakler(), "BÜ-Reporting Makler", "BÜ-Reporting HSB4 K"))
148      modFortschritt.HinweisZeigen titel, titel & vbCrLf & vbCrLf & _
149              "Neue Daten laden: liest alle Rückstandsabzüge (OPAG und Verbund) aus dem Ordner 'Rohdaten' neben dieser Da
         tei, " & _
150              "erkennt Stichtag und Quelle selbst und lädt nur, was noch fehlt. Ein neuer Abzug zu einem schon geladenen
         Tag ersetzt den alten nach Rückfrage." & vbCrLf & vbCrLf & _
151              "Anwesenheit laden: importiert die MAK-Liste oder den Telefonie- und Abwesenheitsplan in die Blätter Mitarb
         eiter und Kalender; den Plan übernimmt auch jedes 'Neue Daten laden'. Arbeitszeit und Verteilung auf Mo–Fr stehen i
         m Blatt Mitarbeiter; im Kalender (Kalender öffnen) je Tag: leer = da, U Urlaub, K Krankheit, A abwesend, Zahl = anw
         esender Anteil (0,5 = halber Tag; Samstag 1 = kommt). Feste freie Tage (kursives A) und Feiertage (FT) trägt das To
         ol selbst ein." & vbCrLf & vbCrLf & _
152              "PowerPoint-Folie erzeugen: kopiert die Kacheln des Blatts PPT auf eine Folie der ERGO-Vorlage." & vbCrLf &
          vbCrLf & _
153              "Filter: Die Chips für Sparten, Themenpostkörbe und GeVo-Typen schalten per Klick um (gefüllt = im Filter)
         und wirken sofort auf alle Kennzahlen und Diagramme; über den Rückstandssäulen zeigt ein Badge die Veränderung zum
         vorherigen Stand (grün = weniger, gelb = mehr). " & _
154              "'Standard-Postkörbe' wählt die Postkörbe der Standard-Auswahl, 'Auf Standard zurücksetzen' stellt Postkörb
         e und GeVo-Typen auf den Standard. " & _
155              "Was zum Standard gehört, legt 'Standard festlegen' fest (oder Konfiguration, Spalten H bis L)." & vbCrLf &
          vbCrLf & _
156              "Werkzeuge: 'SLA anpassen' setzt die SLA-Linien (0 = keine Linie), 'Rohdaten löschen' entfernt einzelne Roh
         dateien (auf Wunsch mit Neuaufbau), 'Zurücksetzen' leert das Tool (die Rohdateien bleiben). " & _
157              "Bereich Makler: Mehrere Einheiten (z. B. ODPB, ODPP, KBI) liefern je Tag eigene Abzüge; das Tool zeigt sie
          zusammen; die Chips 'Sparte | Einheit' über dem Rückstand stapeln nach Einheit." & vbCrLf & vbCrLf & _
158              "Eingänge = Schlüssel, die im neuen Stand erstmals vorkommen; Abgänge = Schlüssel des Vortags, die fehlen "
          & _
159              "(abgearbeitet oder weitergeleitet). Produktivität = Abgänge je MAK (neueste KW hochger.): Wochensicht je K
         W, " & _
160              "Tagessicht und Aktuelle KW Ø je Arbeitstag; Basis in der Kachel." & vbCrLf & vbCrLf & _
161              "Diagramme zeigen immer 7 Plätze: Tagessicht = letzte 7 Stichtage und Wochensicht = letzte 7 KW, neueste Pe
         riode jeweils rechts; 'Aktuelle KW' = Montag (links) bis Sonntag der Kalenderwoche des Bezugstags (Tage ohne Stand
         und nach dem Bezugstag bleiben leer). " & _
162              "In der Wochensicht steht unter jeder KW das Datum des Stands (Sonntag, sonst letzter Stand; * = ohne Sonnt
         ag). Werte, die nicht in ihr Segment passen, stehen mit Führungslinie neben der Säule." & vbCrLf & vbCrLf & _
163              "Eingänge und Abgänge: je Sparte eine Zeile, darüber Gesamt; Eingänge oben, Abgänge unten, Delta in der Box
          (grün = Abbau, gelb = Aufbau). Die Chips Verbund/OPAG in der Kopfleiste schalten die Quelle um; rechts die Summen
         (Tag: letzte 7 Stichtage, Aktuelle KW: die Tage der KW, Woche: letzte KW).", False
164  End Sub
165
166  Private Sub SetzePostkoerbe(wert As Boolean)
167      Dim ws As Worksheet, r As Long, namen As Variant, aus() As Variant
168      Set ws = ThisWorkbook.Worksheets("Filter")
```

```
    namen = ws.Range(ws.Cells(modLaden.PK_VON, 2), ws.Cells(modLaden.PK_BIS, 2)).Value2
    ReDim aus(1 To UBound(namen, 1), 1 To 1)
    For r = 1 To UBound(namen, 1)
        If Len(CStr(namen(r, 1))) > 0 Then aus(r, 1) = wert Else aus(r, 1) = Empty
    Next r
    ws.Range(ws.Cells(modLaden.PK_VON, 3), ws.Cells(modLaden.PK_BIS, 3)).Value = aus
End Sub

' ---------------------------------------------------------------- Chips (Filterschaltflaechen)
Public Function ChipHover(nr As Long) As String
    ' Wird von HYPERLINK() in den Chipzellen beim Ueberfahren mit der Maus ausgewertet (Rollover): merkt den Chip in Filter!Q1,
    ' die bedingten Formate heben ihn hervor. Der Link zeigt auf die Zelle selbst; der Klick loest Worksheet_SelectionChange aus.
    ' Geschrieben wird NUR beim Ueberfahren (CalculationState = xlDone), nie waehrend einer Neuberechnung oder des Ladens:
    ' Zellschreiben aus einer UDF mitten in der Berechnung bringt neuere Excel-Versionen zum Absturz.
    ' Konfiguration!B8 = NEIN schaltet den Hover ganz ab (Klick bleibt).
    ' Der Link zeigt auf Spalte A der Chipzeile (dort steht die Chip-Nummer); ist diese Zelle schon markiert, auf Spalte H
    ' derselben Zeile – so aendert jeder Klick die Auswahl und Worksheet_SelectionChange feuert auch beim zweiten Klick.
    Dim z As Range, ziel As Range
    On Error Resume Next
    Set z = Application.Caller
    Set ziel = z.Parent.Cells(z.Row, 1)
    If Not ActiveCell Is Nothing Then
        If ActiveCell.Parent.Name = z.Parent.Name And ActiveCell.Address = ziel.Address Then Set ziel = z.Parent.Cells(z.Row, 8)
    End If
    ChipHover = "#'" & z.Parent.Name & "'!" & ziel.Address
    If modLaden.LadenLaeuft() Then Exit Function
    If Application.CalculationState <> xlDone Then Exit Function
    If UCase$(Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B8").Value))) = "NEIN" Then Exit Function
    If ThisWorkbook.Worksheets("Filter").Range("Q1").Value <> nr Then ThisWorkbook.Worksheets("Filter").Range("Q1").Value = nr
    On Error GoTo 0
End Function

' Themenpostkoerbe (Filter!B5:B44) nach Rueckstand am letzten Stand absteigend sortieren (Verbund + OPAG, Filter!D:E; bei
' Gleichstand nach Saetzen in Fakten, Filter!U). Die zehn Chips im Cockpit zeigen damit die groessten Postkoerbe der geladenen
' Daten (Postkoerbe ohne Daten erscheinen nicht). Filterkennzeichen (C) und Standard (Konfiguration!I) wandern zeilengleich mit;
' alle Formeln suchen den Postkorb ueber den Namen. Aufruf nach jedem Laden.
Public Sub PostkoerbeSortieren()
    Dim wsF As Worksheet, wsK As Worksheet, n As Long, i As Long, j As Long, t As Long
    Dim namen As Variant, aktiv As Variant, std As Variant, d As Variant, e As Variant, u As Variant
    Dim wert() As Double, folge() As Long, neuN() As Variant, neuA() As Variant, neuS() As Variant, geaendert As Boolean
    Set wsF = ThisWorkbook.Worksheets("Filter")
    Set wsK = ThisWorkbook.Worksheets("Konfiguration")
    n = modLaden.PK_BIS - modLaden.PK_VON + 1
    namen = wsF.Range(wsF.Cells(modLaden.PK_VON, 2), wsF.Cells(modLaden.PK_BIS, 2)).Value2
    aktiv = wsF.Range(wsF.Cells(modLaden.PK_VON, 3), wsF.Cells(modLaden.PK_BIS, 3)).Value2
    d = wsF.Range(wsF.Cells(modLaden.PK_VON, 4), wsF.Cells(modLaden.PK_BIS, 4)).Value2
    e = wsF.Range(wsF.Cells(modLaden.PK_VON, 5), wsF.Cells(modLaden.PK_BIS, 5)).Value2
    u = wsF.Range(wsF.Cells(modLaden.PK_VON, 21), wsF.Cells(modLaden.PK_BIS, 21)).Value2
    std = wsK.Range(wsK.Cells(modLaden.PK_VON + 6, 9), wsK.Cells(modLaden.PK_BIS + 6, 9)).Value2
    ReDim wert(1 To n): ReDim folge(1 To n)
    For i = 1 To n
        folge(i) = i
        If Len(Trim$(CStr(namen(i, 1)))) = 0 Then
            wert(i) = -2
        Else
            wert(i) = Zahl(d(i, 1)) + Zahl(e(i, 1)) + Zahl(u(i, 1)) / 1000000#      ' Rueckstand, bei Gleichstand Saetze
            If Zahl(u(i, 1)) = 0 Then wert(i) = -1                                  ' ohne Daten ans Ende (vor die leeren Plaetze)
        End If
    Next i
    For i = 2 To n                                                  ' stabile Einfuegesortierung, absteigend
        t = folge(i)
        j = i - 1
        Do While j >= 1
            If wert(folge(j)) >= wert(t) Then Exit Do
            folge(j + 1) = folge(j)
            j = j - 1
        Loop
        folge(j + 1) = t
    Next i
    ReDim neuN(1 To n, 1 To 1): ReDim neuA(1 To n, 1 To 1): ReDim neuS(1 To n, 1 To 1)
    For i = 1 To n
        neuN(i, 1) = namen(folge(i), 1): neuA(i, 1) = aktiv(folge(i), 1): neuS(i, 1) = std(folge(i), 1)
        If folge(i) <> i Then geaendert = True
    Next i
    If Not geaendert Then Exit Sub
    wsF.Range(wsF.Cells(modLaden.PK_VON, 2), wsF.Cells(modLaden.PK_BIS, 2)).Value2 = neuN
    wsF.Range(wsF.Cells(modLaden.PK_VON, 3), wsF.Cells(modLaden.PK_BIS, 3)).Value2 = neuA
    wsK.Range(wsK.Cells(modLaden.PK_VON + 6, 9), wsK.Cells(modLaden.PK_BIS + 6, 9)).Value2 = neuS
    modLaden.LogSchreiben "Themenpostkörbe nach Rückstand sortiert"
End Sub

Private Function Zahl(v As Variant) As Double
    If IsError(v) Or IsEmpty(v) Then Exit Function
```

```
    If IsNumeric(v) Then Zahl = CDbl(v)
End Function

Public Sub ChipSchalten(nr As Long)
    If modLaden.LadenLaeuft() Then Exit Sub
    ' Schaltet den Chip nr um: 1..99 Themenpostkorb (Filter!C), 101..114 GeVo-Typ (Filter!H, wechselt in die manuelle Auswahl),
    ' 201..204 Sparte (Filter!T), 301..306 Einheit (Filter!Y, Bereich Makler)
    Dim wsF As Worksheet, r As Long, i As Long
    Set wsF = ThisWorkbook.Worksheets("Filter")
    If nr >= 300 Then
        r = 4 + (nr - 300)
        If Len(CStr(wsF.Cells(r, 24).Value)) = 0 Then Exit Sub           ' kein Chip (keine Einheit in dieser Zeile)
        wsF.Cells(r, 25).Value = Not CBool(wsF.Cells(r, 25).Value)
    ElseIf nr < 100 Then
        r = 4 + nr
        If Val(wsF.Cells(r, 21).Value) = 0 Then Exit Sub        ' ausgeblendeter Chip (Postkorb ohne Daten)
        wsF.Cells(r, 3).Value = Not CBool(wsF.Cells(r, 3).Value)
    ElseIf nr >= 200 Then
        r = 4 + (nr - 200)
        If Val(wsF.Cells(r, 22).Value) = 0 And Application.WorksheetFunction.Count(ThisWorkbook.Worksheets("Fakten").Range("A2:A3")) > 0 Then Exit Sub
        wsF.Cells(r, 20).Value = Not CBool(wsF.Cells(r, 20).Value)
    Else
        r = 4 + (nr - 100)
        If wsF.Range("L5").Value <> "Manuelle Auswahl" Then
            For i = 5 To 18
                wsF.Cells(i, 8).Value = CBool(wsF.Cells(i, 9).Value)
            Next i
            wsF.Range("L5").Value = "Manuelle Auswahl"
        End If
        wsF.Cells(r, 8).Value = Not CBool(wsF.Cells(r, 8).Value)
    End If
    ' Kennzahlen rechnen sofort ueber Formeln; Diagramme und Etiketten gleich danach nachziehen, damit der Klick
    ' nicht blockiert und der naechste Klick auf denselben Chip sofort wirkt
    AktualisierungPlanen
End Sub

Public Sub AuswahlWegPlanen()
    ' Nach dem Chip-Klick die Auswahl auf die neutrale Zelle H1 setzen – per OnTime, also nachdem Excel den Hyperlink
    ' fertig verfolgt hat (ein Select im Ereignis selbst wird von der Hyperlink-Navigation wieder ueberschrieben)
    On Error Resume Next
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modCockpit.AuswahlWeg"
    On Error GoTo 0
End Sub

Public Sub AuswahlWeg()
    On Error Resume Next
    If ActiveSheet.Name = "Cockpit" Then
        Application.EnableEvents = False
        ActiveSheet.Range("H1").Select
        Application.EnableEvents = True
    End If
    On Error GoTo 0
End Sub

Public Sub AktualisierungPlanen()
    If aktualisierungGeplant Then Exit Sub
    On Error GoTo Direkt
    aktualisierungGeplant = True
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modCockpit.Aktualisieren"
    Exit Sub
Direkt:
    aktualisierungGeplant = False
    Aktualisieren
End Sub

Public Sub FilterAnzeigen()
    ' Faerbt die Ansichts-Chips (Formen ANS_Tag/ANS_KW/ANS_Woche) nach Filter!L9 = "Tag", "KW" oder "Woche" (alles andere rechnet
    ' wie "Tag"); die Filter-Chips sind Zellen mit bedingtem Format
    Dim ws As Worksheet, wsF As Worksheet, ansicht As String
    Set ws = ThisWorkbook.Worksheets("Cockpit")
    Set wsF = ThisWorkbook.Worksheets("Filter")
    On Error Resume Next                                        ' direkt per Name statt Schleife ueber alle Formen
    ansicht = CStr(wsF.Range("L9").Value)
    ChipFaerben ws.Shapes("ANS_Tag"), (ansicht <> "Woche" And ansicht <> "KW")
    ChipFaerben ws.Shapes("ANS_KW"), (ansicht = "KW")
    ChipFaerben ws.Shapes("ANS_Woche"), (ansicht = "Woche")
    BookmarkFaerben ws.Shapes("BWQ_Verbund"), (wsF.Range("L13").Value <> "OPAG"), (wsF.Range("L13").Value = "OPAG")
    BookmarkFaerben ws.Shapes("BWQ_OPAG"), (wsF.Range("L13").Value = "OPAG"), (wsF.Range("L13").Value = "OPAG")
    Dim i As Long                                               ' „nur“-Knoepfe der Einheiten-Chips (Makler) nur mit Einheit zeigen
    For i = 1 To 6
        ws.Shapes("BTN_EHNur" & i).Visible = (Len(CStr(wsF.Cells(4 + i, 24).Value)) > 0)
    Next i
    On Error GoTo 0
End Sub

Public Sub AnsichtTag()
    If modLaden.LadenLaeuft() Then Exit Sub
    ThisWorkbook.Worksheets("Filter").Range("L9").Value = "Tag"
    Aktualisieren
End Sub

Public Sub AnsichtWoche()
```

```
344      If modLaden.LadenLaeuft() Then Exit Sub
345      ThisWorkbook.Worksheets("Filter").Range("L9").Value = "Woche"
346      Aktualisieren
347  End Sub
348
349  ' Knopf "Aktuelle KW" unter der Tagessicht: Tagessicht von Montag bis Sonntag der KW des Bezugstags (Filter!O40, ohne
       frueheren
350  ' Bezugstag der neueste Stand, also die laufende Woche). Tage ohne Stand und nach dem Bezugstag bleiben leer, ihre Bes
       chriftung
351  ' steht trotzdem an der Achse; zurueck/vor/Neuester Stand verschieben den Bezugstag und damit die KW (alles ueber Form
       eln).
352  Public Sub AnsichtKW()
353      If modLaden.LadenLaeuft() Then Exit Sub
354      ThisWorkbook.Worksheets("Filter").Range("L9").Value = "KW"
355      Aktualisieren
356  End Sub
357
358  ' Beim Oeffnen (Workbook_Open) immer Wochensicht – nur schreiben, wenn noetig (sonst fragt Excel beim Schliessen ohne
       Grund)
359  Public Sub WochensichtBeimOeffnen()
360      On Error Resume Next
361      With ThisWorkbook.Worksheets("Filter").Range("L9")
362          If CStr(.Value) <> "Woche" Then .Value = "Woche"
363      End With
364      On Error GoTo 0
365  End Sub
366
367  ' ---------------------------------------------------------------- Bezugstag (Cockpit!L12, Konstante BEZUG_FORMEL oben
       im Modul)
368
369  Public Sub BezugFormelSetzen()
370      Dim war As Boolean
371      war = Application.EnableEvents
372      Application.EnableEvents = False
373      ThisWorkbook.Worksheets("Cockpit").Range("L12").Formula = BEZUG_FORMEL
374      Application.EnableEvents = war
375  End Sub
376  Public Sub BezugZurueck()
377      BezugSetzen -1
378  End Sub
379
380  Public Sub BezugVor()
381      BezugSetzen 1
382  End Sub
383
384  Public Sub BezugNeuester()
385      If modLaden.LadenLaeuft() Then Exit Sub
386      BezugFormelSetzen
387      Aktualisieren
388  End Sub
389
390  Private Sub BezugSetzen(richtung As Long)
391      ' Blaettert vom wirksamen Bezugstag (Filter!O40) zum vorherigen bzw. naechsten geladenen Stichtag; der neueste Sti
       chtag
392      ' wird als Formel gespeichert, damit neue Staende nach dem Laden automatisch erscheinen.
393      Dim wsV As Worksheet, akt As Double, r As Long, letzte As Long, d As Double, ziel As Double, maxD As Double
394      If modLaden.LadenLaeuft() Then Exit Sub
395      Set wsV = ThisWorkbook.Worksheets("Verlauf")
396      akt = Val(ThisWorkbook.Worksheets("Filter").Range("O40").Value2)
397      If akt = 0 Then Exit Sub
398      letzte = wsV.Cells(wsV.Rows.Count, 1).End(xlUp).Row
399      ziel = 0: maxD = 0
400      For r = 6 To letzte
401          d = Val(wsV.Cells(r, 1).Value2)
402          If d > maxD Then maxD = d
403          If richtung < 0 Then
404              If d < akt And d > ziel Then ziel = d
405          Else
406              If d > akt And (ziel = 0 Or d < ziel) Then ziel = d
407          End If
408      Next r
409      If ziel = 0 Then Exit Sub                                    ' schon am Anfang bzw. Ende
410      If ziel >= maxD Then
411          BezugFormelSetzen
412      Else
413          Application.EnableEvents = False
414          ThisWorkbook.Worksheets("Cockpit").Range("L12").Value = CDate(ziel)
415          Application.EnableEvents = True
416      End If
417      Aktualisieren
418  End Sub
419
420  Public Sub QuelleVerbund()
421      If modLaden.LadenLaeuft() Then Exit Sub
422      ThisWorkbook.Worksheets("Filter").Range("L13").Value = "Verbund"
423      Aktualisieren
424  End Sub
425
426  Public Sub QuelleOPAG()
427      If modLaden.LadenLaeuft() Then Exit Sub
428      ThisWorkbook.Worksheets("Filter").Range("L13").Value = "OPAG"
429      Aktualisieren
430  End Sub
431
432  Private Sub BookmarkFaerben(shp As Shape, aktiv As Boolean, opag As Boolean)
433      ' Bookmark-Chips in der Kopfleiste der Kachel Eingaenge/Abgaenge: aktiv = weiss mit Schrift in der Kopffarbe,
```

```
434      ' inaktiv = Kopffarbe mit weisser Schrift und weisser Kontur (Kopf OPAG rot, Verbund petrol)
435      Dim kopf As Long
436      If opag Then kopf = RGB(142, 15, 30) Else kopf = RGB(11, 111, 125)
437      With shp
438          .Fill.Visible = msoTrue
439          .Fill.Solid
440          .Line.Visible = msoTrue
441          If aktiv Then
442              .Fill.ForeColor.RGB = RGB(255, 255, 255)
443              .Line.ForeColor.RGB = RGB(255, 255, 255)
444              .TextFrame2.TextRange.Font.Fill.ForeColor.RGB = kopf
445              .TextFrame2.TextRange.Font.Bold = msoTrue
446          Else
447              .Fill.ForeColor.RGB = kopf
448              .Line.ForeColor.RGB = RGB(255, 255, 255)
449              .Line.Weight = 1
450              .TextFrame2.TextRange.Font.Fill.ForeColor.RGB = RGB(255, 255, 255)
451              .TextFrame2.TextRange.Font.Bold = msoFalse
452          End If
453      End With
454  End Sub
455
456  Private Sub DeltaBoxenRunden(ch As Chart)
457      ' Delta-Etiketten der Kachel Eingaenge/Abgaenge als abgerundetes Rechteck, ohne Umbruch und mit der Form an den Te
         xt angepasst
458      ' (die Datei traegt beides bereits; Excel-Versionen, die es beim Oeffnen verwerfen, bekommen es hier)
459      Dim s As Series
460      On Error Resume Next
461      For Each s In ch.SeriesCollection
462          If Left$(s.Name, 5) = "Delta" Then
463              If s.HasDataLabels Then
464                  With s.DataLabels.Format
465                      .AutoShapeType = msoShapeRoundedRectangle
466                      .TextFrame2.WordWrap = msoFalse
467                      .TextFrame2.AutoSize = msoAutoSizeShapeToFitText
468                  End With
469              End If
470          End If
471      Next s
472      On Error GoTo 0
473  End Sub
474
475  Private Sub ChipFaerben(shp As Shape, aktiv As Boolean)
476      With shp
477          If aktiv Then
478              .Fill.Visible = msoTrue
479              .Fill.Solid
480              .Fill.ForeColor.RGB = RGB(15, 148, 167)
481              .Line.Visible = msoFalse
482              .TextFrame2.TextRange.Font.Fill.ForeColor.RGB = RGB(255, 255, 255)
483              .TextFrame2.TextRange.Font.Bold = msoTrue
484          Else
485              .Fill.Visible = msoTrue
486              .Fill.Solid
487              .Fill.ForeColor.RGB = RGB(255, 255, 255)
488              .Line.Visible = msoTrue
489              .Line.ForeColor.RGB = RGB(208, 213, 218)
490              .Line.Weight = 0.75
491              .TextFrame2.TextRange.Font.Fill.ForeColor.RGB = RGB(107, 107, 107)
492              .TextFrame2.TextRange.Font.Bold = msoFalse
493          End If
494      End With
495  End Sub
496
497  ' ---------------------------------------------------------------- Diagrammbereiche nachziehen
498  Public Sub DiagrammeAnpassen(Optional erzwingen As Boolean = False, Optional mitPPT As Boolean = False)
499      ' Setzt die Datenbereiche der Diagramme nur, wenn sie sich geaendert haben: Diagrammdaten (immer 7 Perioden) einma
         l,
500      ' das KW-Fenster (7 Kalenderwochen bis zur KW des letzten Stichtags) bei neuem k1, die Postkorbliste bei neuem p.
501      ' Danach Etiketten pruefen (Werte neben schmalen Segmenten). erzwingen = True nach dem Laden.
502      Dim ws As Worksheet, wsK As Worksheet, wsF As Worksheet, wsP As Worksheet
503      Dim k1 As Long, k2 As Long, p As Long, nm As Variant
504      Set ws = ThisWorkbook.Worksheets("Cockpit")
505      Set wsK = ThisWorkbook.Worksheets("KW")
506      Set wsF = ThisWorkbook.Worksheets("Filter")
507      Set wsP = ThisWorkbook.Worksheets("PPT")
508      k2 = 5 + CLng(Val(wsK.Range("AQ6").Value))
509      k1 = k2 - 6
510      If k1 < 6 Then k1 = 6
511      k2 = k1 + 6
512      p = wsF.Cells(wsF.Rows.Count, 2).End(xlUp).Row
513      If p < modLaden.PK_VON Then p = modLaden.PK_VON
514      If p > modLaden.PK_BIS Then p = modLaden.PK_BIS
515      On Error Resume Next
516      If erzwingen Then reihenFest = False: letztesK1 = 0: letztesP = 0
517      If Not reihenFest Then
518          Err.Clear
519          ' Bereich MAKLER: vierte Stapelreihe (Sonstige bzw. vierte Einheit, Diagrammdaten!BI/BJ) vor der Summenreihe
520          ' Achse je Quelle: BR (OPAG), BS (Verbund) – in der Wochensicht mit dem Datum des Stands (Sonntag, sonst letzt
         er) unter der KW
521          If modLaden.IstMakler() Then
522              SetzeReihen ws.ChartObjects("Chart 1").Chart, "Diagrammdaten", "BR", 4, 10, Array("C", "D", "E", "BI", "F"
         , "Q", "R", "S", "T")
523              SetzeReihen ws.ChartObjects("Chart 2").Chart, "Diagrammdaten", "BS", 4, 10, Array("G", "H", "I", "BJ", "J"
         , "V", "W", "X", "Y")
524              SetzeReihen wsP.ChartObjects("Chart 10").Chart, "Diagrammdaten", "BR", 4, 10, Array("C", "D", "E", "BI", "
```

```
F")
525             SetzeReihen wsP.ChartObjects("Chart 11").Chart, "Diagrammdaten", "BS", 4, 10, Array("G", "H", "I", "BJ", "
J")
526         Else
527             SetzeReihen ws.ChartObjects("Chart 1").Chart, "Diagrammdaten", "BR", 4, 10, Array("C", "D", "E", "F", "Q",
 "R", "S", "T")
528             SetzeReihen ws.ChartObjects("Chart 2").Chart, "Diagrammdaten", "BS", 4, 10, Array("G", "H", "I", "J", "V",
 "W", "X", "Y")
529             ' PPT-Folie (Blatt PPT): 10/11 Rueckstand OPAG/Verbund, 12 MAK, 13/14 Bewegungen OPAG/Verbund (Summen), 15
 Altersstruktur
530             SetzeReihen wsP.ChartObjects("Chart 10").Chart, "Diagrammdaten", "BR", 4, 10, Array("C", "D", "E", "F")
531             SetzeReihen wsP.ChartObjects("Chart 11").Chart, "Diagrammdaten", "BS", 4, 10, Array("G", "H", "I", "J")
532         End If
533         ' 13/14: Eingaenge, Abgaenge, Delta-Box Abbau, Delta-Box Aufbau (Text der Boxen haengt fest an Diagrammdaten!A
X/BA);
534         ' Achse BT/BU ohne Stern (der Stern zum fehlenden Sonntagsstand gehoert nur zum Rueckstand)
535         SetzeReihen wsP.ChartObjects("Chart 13").Chart, "Diagrammdaten", "BT", 4, 10, Array("K", "L", "AV", "AW")
536         SetzeReihen wsP.ChartObjects("Chart 14").Chart, "Diagrammdaten", "BU", 4, 10, Array("N", "O", "AY", "AZ")
537         ' Beim Bau sind die Wertachsen fest gesetzt (fuer die Etikettengeometrie); in Excel rechnet die Automatik
538         For Each nm In Array("Chart 1", "Chart 2")
539             With ws.ChartObjects(nm).Chart.Axes(xlValue)
540                 .MinimumScaleIsAuto = True
541                 .MaximumScaleIsAuto = True
542             End With
543         Next nm
544         For Each nm In Array("Chart 10", "Chart 11")
545             With wsP.ChartObjects(nm).Chart.Axes(xlValue)
546                 .MinimumScaleIsAuto = True
547                 .MaximumScaleIsAuto = True
548             End With
549         Next nm
550         reihenFest = (Err.Number = 0)
551         Err.Clear
552     End If
553     If k1 <> letztesK1 Then
554         SetzeReihen ws.ChartObjects("Chart 5").Chart, "KW", "C", k1, k2, Array("M", "L", "N")      ' Komposit/Leben un
ten, KV darueber
555         SetzeReihen wsP.ChartObjects("Chart 12").Chart, "KW", "C", k1, k2, Array("M", "L", "N")
556         letztesK1 = k1
557     End If
558     If p <> letztesP Then
559         SetzeReihen ws.ChartObjects("Chart 9").Chart, "Filter", "B", modLaden.PK_VON, p, Array("D", "E")
560         letztesP = p
561     End If
562     For Each nm In Array("Chart 1", "Chart 2")
563         EtikettenAuslagern ws.ChartObjects(nm).Chart, 0
564     Next nm
565     ' Produktivitaet (Chart 7 Verbund, Chart 8 OPAG): Wert einer zu niedrigen Saeule ueber die Saeule statt weiss hine
in (12.89)
566     For Each nm In Array("Chart 7", "Chart 8")
567         EtikettenUeberSaeule ws.ChartObjects(nm).Chart
568     Next nm
569     If erzwingen Then                                           ' Delta-Boxen behalten ihre Form; nur nach dem Laden neu
runden
570         For Each nm In Array("BW_Gesamt", "BW_Komposit", "BW_Leben", "BW_KV")
571             DeltaBoxenRunden ws.ChartObjects(nm).Chart
572         Next nm
573     End If
574     ' Altersstruktur: zu schmale Segmente ohne Etikett (die Werte ueber 90 Tage stehen im Untertitel). PPT-Rueckstand:
 wie im
575     ' Cockpit – passt ein Wert nicht ins Segment (meist Leben), steht er mit Fuehrungslinie neben der Saeule (Schrift
7 pt)
576     EtikettenAuslagern ws.ChartObjects("Chart 6").Chart, 2, True
577     ' Blatt PPT: nur nach dem Laden und vor dem Export (sieht sonst niemand – spart bei jedem Chip-Klick die Haelfte)
578     If erzwingen Or mitPPT Then
579         For Each nm In Array("Chart 10", "Chart 11")
580             EtikettenAuslagern wsP.ChartObjects(nm).Chart, 0, False, 7
581         Next nm
582         EtikettenAuslagern wsP.ChartObjects("Chart 15").Chart, 2, True
583     End If
584     On Error GoTo 0
585 End Sub
586
587 ' ---------------------------------------------------------------- Navigation
588 Public Sub PlanungOeffnen()
589     ' Knopf "Kalender öffnen"
590     If modLaden.LadenLaeuft() Then Exit Sub
591     AlphabetischSortieren
592     ZurAktuellenKW
593 End Sub
594
595 ' Mitarbeiter und ihre Kalenderzeilen alphabetisch sortieren. Mitarbeiter!A5:N64 (Eingaben inkl. Arbeitszeitmodell) un
d
596 ' Kalender!D7:... gehoeren zeilengleich zusammen (Kalender-Zeile = Mitarbeiter-Zeile + 2) und werden mit derselben
597 ' Reihenfolge umgestellt; Kalender!A:C, Mitarbeiter!O:T und Wirksam folgen ueber Formeln. Leere Zeilen ans Ende.
598 Public Sub AlphabetischSortieren()
599     Const MA_START As Long = 5, MA_ZEILEN As Long = 60, PL_START As Long = 7, PL_SPALTE0 As Long = 4, MA_SPALTEN As Lo
ng = 14
600     Dim wsM As Worksheet, wsP As Worksheet, stamm As Variant, plan As Variant, neuS As Variant, neuP As Variant
601     Dim folge() As Long, i As Long, j As Long, t As Long, c As Long, letzte As Long, geaendert As Boolean, warE As Boo
lean
602     On Error GoTo Ende
603     warE = Application.EnableEvents
604     Application.EnableEvents = False
605     Set wsM = ThisWorkbook.Worksheets("Mitarbeiter")
606     Set wsP = ThisWorkbook.Worksheets("Kalender")
```

```
    letzte = wsP.Cells(6, wsP.Columns.Count).End(xlToLeft).Column
    stamm = wsM.Range(wsM.Cells(MA_START, 1), wsM.Cells(MA_START + MA_ZEILEN - 1, MA_SPALTEN)).Value2
    plan = wsP.Range(wsP.Cells(PL_START, PL_SPALTE0), wsP.Cells(PL_START + MA_ZEILEN - 1, letzte)).Value2
    ReDim folge(1 To MA_ZEILEN)
    For i = 1 To MA_ZEILEN
        folge(i) = i
    Next i
    ' Einfuegesortierung (hoechstens 60 Zeilen), Vergleich ohne Gross/Klein, Umlaute nach Systemsprache
    For i = 2 To MA_ZEILEN
        t = folge(i)
        j = i - 1
        Do While j >= 1
            If Not Vorher(CStr(stamm(t, 1)), CStr(stamm(folge(j), 1))) Then Exit Do
            folge(j + 1) = folge(j)
            j = j - 1
        Loop
        folge(j + 1) = t
    Next i
    For i = 1 To MA_ZEILEN
        If folge(i) <> i Then geaendert = True
    Next i
    If geaendert Then
        neuS = stamm: neuP = plan
        For i = 1 To MA_ZEILEN
            For c = 1 To UBound(stamm, 2)
                neuS(i, c) = stamm(folge(i), c)
            Next c
            For c = 1 To UBound(plan, 2)
                neuP(i, c) = plan(folge(i), c)
            Next c
        Next i
        wsM.Range(wsM.Cells(MA_START, 1), wsM.Cells(MA_START + MA_ZEILEN - 1, MA_SPALTEN)).Value2 = neuS
        wsP.Range(wsP.Cells(PL_START, PL_SPALTE0), wsP.Cells(PL_START + MA_ZEILEN - 1, letzte)).Value2 = neuP
        modLaden.LogSchreiben "Mitarbeiter und Kalender alphabetisch sortiert"
    End If
Ende:
    Application.EnableEvents = warE
End Sub

' a vor b? Leere Namen immer ans Ende
Private Function Vorher(a As String, b As String) As Boolean
    a = Trim$(a): b = Trim$(b)
    If Len(a) = 0 Then Vorher = False: Exit Function
    If Len(b) = 0 Then Vorher = True: Exit Function
    Vorher = (StrComp(a, b, vbTextCompare) < 0)
End Function

Public Sub CockpitOeffnen()
    If modLaden.LadenLaeuft() Then Exit Sub
    ThisWorkbook.Worksheets("Cockpit").Activate
    ThisWorkbook.Worksheets("Cockpit").Range("A1").Select
End Sub

Public Sub ZurAktuellenKW()
    ' Knopf "Zu heute" bzw. "Kalender öffnen": Kalender aktivieren und den heutigen Tag neben die fixierten Spalten ho
   len
    ThisWorkbook.Worksheets("Kalender").Activate
    KalenderZuHeute
End Sub

' Spalte des heutigen Tags im Kalender (am Wochenende der naechste Montag); 0 = ausserhalb des Kalenders
Public Function KalenderSpalteHeute() As Long
    Dim ws As Worksheet, c As Long, letzte As Long
    On Error GoTo Ende
    Set ws = ThisWorkbook.Worksheets("Kalender")
    letzte = ws.Cells(6, ws.Columns.Count).End(xlToLeft).Column
    For c = 4 To letzte
        If IsDate(ws.Cells(6, c).Value) Then
            If CDate(ws.Cells(6, c).Value) >= Date Then KalenderSpalteHeute = c: Exit Function
        End If
    Next c
Ende:
End Function

' Kalender (muss aktiv sein): "frei"/"FT" nachtragen, vergangene Wochen einklappen und so blaettern, dass heute sichtb
   ar ist
Public Sub KalenderZuHeute()
    Dim c As Long, cm As Long
    On Error Resume Next
    If ActiveSheet.Name <> "Kalender" Then Exit Sub
    KalenderVorbelegen
    cm = KalenderEinklappen()
    c = KalenderSpalteHeute()
    If c = 0 Then Exit Sub
    ActiveWindow.ScrollColumn = IIf(c - 2 < cm, cm, c - 2)
    ActiveSheet.Cells(7, c).Select
End Sub

' Traegt im Kalender ein, was ohnehin feststeht: "FT" an Feiertagen (Konfiguration!N11:N60) und ein kursives "A" (abwe
    send)
' an Wochentagen, an denen die Person laut Arbeitszeitmodell nicht arbeitet (Mitarbeiter!J:N = 0). Angepasst werden nu
    r leere
' Zellen, "FT" und die kursiven "A" des Tools — eigene Eintraege (U, K, A, Anteil) bleiben stehen. Aendert sich das Mo
    dell,
' verschwinden alte automatische "A" wieder.
Public Sub KalenderVorbelegen()
```

```
    Const PL_START As Long = 7, PL_SPALTE0 As Long = 4, MA_ZEILEN As Long = 60, MA_START As Long = 5
    Dim wsP As Worksheet, wsM As Worksheet, wsK As Worksheet, ft As Object, letzte As Long
    Dim plan As Variant, namen As Variant, modell As Variant, daten As Variant, liste As Variant
    Dim r As Long, c As Long, wt As Long, v As Variant, ziel As Variant, alt As String, auto As Boolean, warE As Boolean, n As Long
    Dim zelle As Range, warS As Boolean
    On Error GoTo Ende
    warE = Application.EnableEvents
    warS = Application.ScreenUpdating                ' im Laden aus: nicht mitten im Ladefenster einschalten
    Application.EnableEvents = False
    Set wsP = ThisWorkbook.Worksheets("Kalender")
    Set wsM = ThisWorkbook.Worksheets("Mitarbeiter")
    Set wsK = ThisWorkbook.Worksheets("Konfiguration")
    letzte = wsP.Cells(6, wsP.Columns.Count).End(xlToLeft).Column
    If letzte < PL_SPALTE0 Then GoTo Ende
    Set ft = CreateObject("Scripting.Dictionary")
    liste = wsK.Range("N11:N60").Value2
    For r = 1 To UBound(liste, 1)
        If IsNumeric(liste(r, 1)) And Not IsEmpty(liste(r, 1)) Then ft(CLng(liste(r, 1))) = True
    Next r
    daten = wsP.Range(wsP.Cells(6, PL_SPALTE0), wsP.Cells(6, letzte)).Value2
    namen = wsP.Range(wsP.Cells(PL_START, 1), wsP.Cells(PL_START + MA_ZEILEN - 1, 1)).Value2
    modell = wsM.Range(wsM.Cells(MA_START, 10), wsM.Cells(MA_START + MA_ZEILEN - 1, 14)).Value2   ' J:N = Mo..Fr
    plan = wsP.Range(wsP.Cells(PL_START, PL_SPALTE0), wsP.Cells(PL_START + MA_ZEILEN - 1, letzte)).Value2
    Application.ScreenUpdating = False
    For r = 1 To MA_ZEILEN
        For c = 1 To UBound(plan, 2)
            v = plan(r, c)
            alt = UCase$(Trim$(CStr(v)))
            auto = False
            If alt = "A" Then auto = wsP.Cells(PL_START + r - 1, PL_SPALTE0 + c - 1).Font.Italic    ' "A" des Tools ist kursiv
            If IsEmpty(v) Or alt = "" Or alt = "FT" Or auto Then
                ziel = Empty
                If Len(Trim$(CStr(namen(r, 1)))) > 0 And IsNumeric(daten(1, c)) Then
                    If ft.Exists(CLng(daten(1, c))) Then
                        ziel = "FT"
                    Else
                        wt = Weekday(CDate(daten(1, c)), vbMonday)
                        If wt <= 5 Then
                            If Not IsEmpty(modell(r, wt)) And IsNumeric(modell(r, wt)) Then
                                If CDbl(modell(r, wt)) = 0 Then ziel = "A"
                            End If
                        End If
                    End If
                End If
                If CStr(ziel) <> CStr(v) Or (ziel = "A" And Not auto) Then
                    Set zelle = wsP.Cells(PL_START + r - 1, PL_SPALTE0 + c - 1)
                    zelle.Value = ziel
                    zelle.Font.Italic = (CStr(ziel) = "A")
                    n = n + 1
                End If
            End If
        Next c
    Next r
    If n > 0 Then modLaden.LogSchreiben "Kalender: " & n & " Einträge (A laut Arbeitszeitmodell, FT) angepasst"
Ende:
    Application.ScreenUpdating = warS
    Application.EnableEvents = warE
End Sub

' Alle Tage vor dem Montag der aktuellen Woche als eine Gliederungsgruppe zusammenfassen und zuklappen ("+" oben klappt auf).
' Rueckgabe: Spalte des Montags der aktuellen Woche (erste sichtbare Tagesspalte).
Public Function KalenderEinklappen() As Long
    Dim ws As Worksheet, c As Long, letzte As Long, cm As Long, montag As Date
    On Error Resume Next
    Set ws = ThisWorkbook.Worksheets("Kalender")
    letzte = ws.Cells(6, ws.Columns.Count).End(xlToLeft).Column
    montag = Date - Weekday(Date, vbMonday) + 1
    cm = 4
    For c = 4 To letzte
        If IsDate(ws.Cells(6, c).Value) Then
            If CDate(ws.Cells(6, c).Value) >= montag Then cm = c: Exit For
        End If
    Next c
    KalenderEinklappen = cm
    ws.Range(ws.Columns(4), ws.Columns(letzte)).Ungroup
    ws.Range(ws.Columns(4), ws.Columns(letzte)).Ungroup
    ws.Range(ws.Columns(4), ws.Columns(letzte)).EntireColumn.Hidden = False
    If cm > 4 Then
        ws.Outline.SummaryColumn = xlRight
        ws.Range(ws.Columns(4), ws.Columns(cm - 1)).Group
        ws.Range(ws.Columns(4), ws.Columns(cm - 1)).EntireColumn.Hidden = True
    End If
End Function

Private Sub SetzeReihen(ch As Chart, blatt As String, katSpalte As String, z1 As Long, z2 As Long, spalten As Variant)
    Dim i As Long
    For i = 1 To ch.SeriesCollection.Count
        If i - 1 <= UBound(spalten) Then
            ch.SeriesCollection(i).XValues = "=" & blatt & "!$" & katSpalte & "$" & z1 & ":$" & katSpalte & "$" & z2
            ch.SeriesCollection(i).Values = "=" & blatt & "!$" & spalten(i - 1) & "$" & z1 & ":$" & spalten(i - 1) & "$" & z2
        End If
    Next i
```

```
End Sub

' ------------------------------------------------------------- Gruppierte Saeulen: passt der Wert nicht in die Saeule (Saeule
' niedriger als das Etikett), steht er dunkel ueber der Saeule, sonst weiss innen am Ende
Public Sub EtikettenUeberSaeule(ch As Chart)
    Dim s As Series, p As Point, lbl As DataLabel, j As Long, i As Long, w As Variant, v As Double
    On Error Resume Next
    For j = 1 To ch.SeriesCollection.Count
        Set s = ch.SeriesCollection(j)
        If s.ChartType = xlColumnClustered Then
            w = s.Values
            For i = 1 To s.Points.Count
                v = 0
                If IsNumeric(w(i)) Then v = CDbl(w(i))
                Set p = s.Points(i)
                If v <> 0 And p.HasDataLabel Then
                    Set lbl = p.DataLabel
                    If p.Height < lbl.Height + 2 Then
                        lbl.Position = xlLabelPositionOutsideEnd
                        lbl.Font.Color = RGB(74, 74, 74)
                    Else
                        lbl.Position = xlLabelPositionInsideEnd
                        lbl.Font.Color = RGB(255, 255, 255)
                    End If
                End If
            Next i
        End If
    Next j
    On Error GoTo 0
End Sub

' ------------------------------------------------------------- Etiketten wie in think-cell: passt der Wert nicht ins Segment,
' steht er mit Fuehrungslinie neben der Saeule (abwechselnd links und rechts, ohne Ueberlappung) bzw. ueber dem Balken.
Public Sub EtikettenAuslagern(ch As Chart, dunkleReihen As Long, Optional ausblenden As Boolean = False, Optional groesse As Single = 8)
    Dim s As Series, p As Point, lbl As DataLabel, werte() As Variant, reihenTyp() As Long, leader() As Boolean
    Dim i As Long, j As Long, n As Long, nReihen As Long, y As Single, x As Single, v As Double
    Dim links As Boolean, letzteL As Single, letzteR As Single, balken As Boolean
    Dim lb As Single, lh As Single, luecke As Single, kat As Single, letzteBalken As Long
    Dim ausL() As Object, ausY() As Single, ausH() As Single, nAus As Long, k As Long, m As Long, tmpO As Object, tmpS As Single
    Dim oben As Single, unten As Single
    On Error Resume Next
    n = ch.SeriesCollection(1).Points.Count
    nReihen = ch.SeriesCollection.Count
    If n = 0 Or nReihen = 0 Then Exit Sub
    ' Reihenwerte und Typen einmal lesen (Values je Punkt zu lesen ist teuer)
    ReDim werte(1 To nReihen): ReDim reihenTyp(1 To nReihen): ReDim leader(1 To nReihen)
    For j = 1 To nReihen
        Set s = ch.SeriesCollection(j)
        reihenTyp(j) = 0
        If s.ChartType = xlColumnStacked Then reihenTyp(j) = 1
        If s.ChartType = xlBarStacked Or s.ChartType = xlBarStacked100 Then reihenTyp(j) = 2
        If Left$(s.Name, 5) = "Delta" Or s.Name = "Achsenluft" Then reihenTyp(j) = 0   ' unsichtbare Badge-Segmente: Etiketten bleiben, wie sie sind
        If reihenTyp(j) > 0 Then werte(j) = s.Values
        If reihenTyp(j) = 2 Then letzteBalken = j
    Next j
    kat = ch.PlotArea.InsideWidth / n
    ReDim ausL(1 To nReihen): ReDim ausY(1 To nReihen): ReDim ausH(1 To nReihen)
    oben = 0: unten = ch.ChartArea.Height
    For i = 1 To n
        links = False: letzteL = -1E+9: letzteR = -1E+9      ' immer rechts neben der Saeule: links schneidet die Vorgaenger-Saeule ab
        nAus = 0                                              ' ausgelagerte Saeulen-Etiketten dieses Punkts (werden unten verteilt)
        For j = 1 To nReihen
            If reihenTyp(j) > 0 Then
                Set s = ch.SeriesCollection(j)
                balken = (reihenTyp(j) = 2)
                v = 0
                If IsNumeric(werte(j)(i)) Then v = CDbl(werte(j)(i))
                Set p = s.Points(i)
                ' im Modus ausblenden wurde das Etikett vielleicht beim letzten Mal entfernt: fuer die Messung wieder anlegen
                If ausblenden And v <> 0 And Not p.HasDataLabel Then p.HasDataLabel = True
                If v <> 0 And p.HasDataLabel Then
                    Set lbl = p.DataLabel
                    lb = lbl.Width: lh = lbl.Height
                    If ausblenden Then
                        ' Altersstruktur: das letzte Segment (über 90 Tage) bekommt seinen Wert rechts neben das Balkenende, dort mit
                        ' „> 90 Tage:“ davor – eine nackte Zahl neben dem Balken war nicht zuzuordnen
                        If balken And j = letzteBalken Then
                            lbl.NumberFormat = "#,##0": lb = lbl.Width
                        End If
                        If balken And j = letzteBalken And p.Width < lb + 2 Then
                            lbl.NumberFormat = """> 90 Tage: ""#,##0"
                            lbl.Position = xlLabelPositionCenter
                            lbl.Left = p.Left + p.Width + 4
                            lbl.Top = p.Top + p.Height / 2 - lh / 2
                            lbl.Font.Color = RGB(26, 26, 26)
                            GoTo NaechsterPunkt
```

```
876                         End If
877                         If (balken And p.Width < lb + 2) Or (Not balken And (p.Height < lh + 1 Or p.Width < lb + 2)) T
       hen
878                             p.HasDataLabel = False
879                             GoTo NaechsterPunkt
880                         End If
881                     End If
882                     If balken Then
883                         If p.Width < lb + 2 Then
884                             lbl.Position = xlLabelPositionCenter
885                             x = p.Left + p.Width / 2 - lb / 2
886                             If x < letzteR Then x = letzteR
887                             lbl.Left = x
888                             lbl.Top = p.Top - lh - 3
889                             letzteR = x + lb + 2
890                             lbl.Font.Color = RGB(74, 74, 74)
891                             If Not leader(j) Then s.HasLeaderLines = True: leader(j) = True
892                         Else
893                             lbl.Position = xlLabelPositionCenter
894                             If j <= dunkleReihen Then lbl.Font.Color = RGB(26, 26, 26) Else lbl.Font.Color = RGB(255,
       255, 255)
895                         End If
896                     Else
897                         If p.Height < lh + 1 Or p.Width < lb + 2 Then
898                             lbl.Position = xlLabelPositionCenter
899                             y = p.Top + p.Height / 2 - lh / 2
900                             lbl.Left = p.Left + p.Width + IIf(groesse <= 7, 2, 5)      ' PPT (7 pt): dicht an der Saeu
       le
901                             ' Hoehe erst nach allen Reihen festlegen: mehrere kleine Segmente uebereinander wuerden si
       ch sonst
902                             ' ueberlagern (bisher nur gegen das zuletzt gesetzte Etikett geprueft)
903                             nAus = nAus + 1: Set ausL(nAus) = lbl: ausY(nAus) = y: ausH(nAus) = lh
904                             lbl.Font.Color = RGB(74, 74, 74)
905                             If Not leader(j) Then s.HasLeaderLines = True: leader(j) = True
906                             ' Passt das Etikett nicht in die Luecke zwischen den Saeulen, Schrift verkleinern
907                             luecke = kat - p.Width
908                             If luecke < 0 Then luecke = 0
909                             If lb + 6 > luecke And i > 1 And i < n Then lbl.Font.Size = groesse - 1 Else lbl.Font.Size
        = groesse
910                             ' PPT (7 pt): weisser Grund, damit der Wert auch ueber dem Rand der Nachbarsaeule lesbar b
       leibt
911                             If groesse <= 7 Then lbl.Format.Fill.Visible = True: lbl.Format.Fill.ForeColor.RGB = RGB(2
       55, 255, 255): lbl.Format.Fill.Solid
912                         Else
913                             lbl.Position = xlLabelPositionCenter
914                             lbl.Font.Color = RGB(255, 255, 255)
915                             If groesse <= 7 Then lbl.Format.Fill.Visible = False
916                         End If
917                     End If
918                 End If
919             End If
920 NaechsterPunkt:
921         Next j
922         If nAus > 0 Then
923             ' nach Wunschhoehe sortieren (oben zuerst), dann von oben nach unten ohne Ueberlappung stapeln; ragt der S
       tapel
924             ' unten heraus, von unten wieder nach oben schieben
925             For k = 2 To nAus
926                 For m = k To 2 Step -1
927                     If ausY(m) < ausY(m - 1) Then
928                         Set tmpO = ausL(m): Set ausL(m) = ausL(m - 1): Set ausL(m - 1) = tmpO
929                         tmpS = ausY(m): ausY(m) = ausY(m - 1): ausY(m - 1) = tmpS
930                         tmpS = ausH(m): ausH(m) = ausH(m - 1): ausH(m - 1) = tmpS
931                     End If
932                 Next m
933             Next k
934             If ausY(1) < oben Then ausY(1) = oben
935             For k = 2 To nAus
936                 If ausY(k) < ausY(k - 1) + ausH(k - 1) + 1 Then ausY(k) = ausY(k - 1) + ausH(k - 1) + 1
937             Next k
938             If ausY(nAus) + ausH(nAus) > unten Then
939                 ausY(nAus) = unten - ausH(nAus)
940                 For k = nAus - 1 To 1 Step -1
941                     If ausY(k) + ausH(k) + 1 > ausY(k + 1) Then ausY(k) = ausY(k + 1) - ausH(k) - 1
942                 Next k
943             End If
944             For k = 1 To nAus
945                 ausL(k).Top = ausY(k)
946                 Set ausL(k) = Nothing
947             Next k
948         End If
949     Next i
950     On Error GoTo 0
951 End Sub
952
953 ' Beim Oeffnen: steht Excel auf manuell, auf automatisch stellen und alles neu berechnen
954 Public Sub RechnenPruefen()
955     If Application.Calculation <> xlCalculationAutomatic Then
956         Application.Calculation = xlCalculationAutomatic
957         Application.CalculateFull
958     End If
959 End Sub
```

modTagesablauf.bas

```
Option Explicit
' ================================================================================
' modTagesablauf – gefuehrter Tagesablauf in drei Schritten (Karten als Formen, ohne UserForm)
'
'   Start (Knopf "Neue Daten laden" im Cockpit)
'   1. Rohdaten: "Liegen die neuen Abzuege im Ordner Rohdaten?"  Ja -> modLaden.NeueDatenLaden
'      (dessen Abschlussmeldung faengt modFortschritt.HinweisZeigen ab und reicht sie hierher weiter)
'   2. Anwesenheit: "Ist heute jemand nicht da?"  Ja -> Blatt Kalender beim heutigen Tag, die Spalte von heute blinkt
'      kurz auf; eine Karte am rechten Rand bleibt offen, waehrend eingetragen wird (U, K, A oder Anteil),
'      "Fertig" schliesst den Schritt ab.  Nein -> direkt zu 3.
'      Kommt die Anwesenheit aus dem Telefonie- und Abwesenheitsplan (das Laden hat ihn eben uebernommen), fragt der Schritt
'      nicht, sondern zeigt, wer heute laut Plan fehlt (PlanZeigen); "Im Kalender ansehen" oeffnet den Kalender wie oben.
'   3. Erfolg: "Alles fuer heute erledigt" – optional "PowerPoint-Folie erzeugen".
'   Filter!O42 haelt den Zeitpunkt des letzten abgeschlossenen Tagesablaufs (Anzeige im Cockpit-Kopf).
'
' Klicks auf eine Karte loeschen die Karte nie im eigenen Klick, sondern per OnTime gleich danach
' (wie modFortschritt.HinweisSchliessen) – das Loeschen der geklickten Form im eigenen Makro kann Excel abstuerzen lassen.
' ================================================================================

Private Const PRAEFIX As String = "TAGW_"
Private Const KAL_START As Long = 7             ' erste Mitarbeiterzeile im Kalender (Kopf: 4 Monat, 5 KW, 6 Tag)
Private Const KAL_ZEILEN As Long = 60
Private Const SCHRIFT As String = "Liberation Sans"

Private phase As String                         ' "", "frage", "laden", "anwesenheit", "fertig"
Private ladeText As String
Private ladeFehler As Boolean
Private nachLadenGeplant As Boolean
Private abwZeile As Long                        ' Spalte von heute im Kalender (0 = nicht gefunden)
Private abwVorbelegt As Boolean                 ' (nicht mehr genutzt)
Private eingetragen As Long                     ' Abwesenheiten heute (fuer die Erfolgsmeldung)
Private geladenText As String
Private planText As String                      ' Anwesenheit kam aus dem Plan: Zeile fuer die Erfolgsmeldung (sonst leer)

' ---------------------------------------------------------------- Farben
Private Const C_TEAL As Long = 8220427            ' RGB(11, 111, 125)
Private Const C_TEAL_HELL As Long = 10982415            ' RGB(15, 148, 167)
Private Const C_ROT As Long = 2627007           ' RGB(191, 21, 40)
Private Const C_SW As Long = 1710618            ' RGB(26, 26, 26)
Private Const C_GRAU As Long = 7039851            ' RGB(107, 107, 107)
Private Const C_GRAUL As Long = 14341584            ' RGB(208, 213, 218)
Private Const C_GRAUXL As Long = 16184563           ' RGB(243, 244, 246)
Private Const C_GRUEN As Long = 5209886           ' RGB(30, 127, 79)
Private Const C_GRUENH As Long = 15004637           ' RGB(221, 243, 228)
Private Const C_GELBD As Long = 23178           ' RGB(138, 90, 0)
Private Const C_GELBH As Long = 13103099            ' RGB(251, 239, 199)

Public Function TagesablaufVersion() As String
    TagesablaufVersion = "1"
End Function

' Wird von modFortschritt.HinweisZeigen gefragt: laeuft gerade das Laden aus dem Tagesablauf?
Public Function WartetAufLaden() As Boolean
    WartetAufLaden = (phase = "laden")
End Function

' Abschlussmeldung von NeueDatenLaden (oder eines Fehlers darin) uebernehmen und Schritt 2 planen
Public Sub LadeErgebnis(titel As String, text As String, fehler As Boolean)
    If Len(ladeText) > 0 Then ladeText = ladeText & vbCrLf
    ladeText = ladeText & text
    If fehler Then ladeFehler = True
    If Not nachLadenGeplant Then
        nachLadenGeplant = True
        On Error Resume Next
        Application.OnTime Now + TimeSerial(0, 0, 1), "'" & ThisWorkbook.Name & "'!modTagesablauf.NachDemLaden"
        On Error GoTo 0
    End If
End Sub

' ---------------------------------------------------------------- Einstieg
Public Sub Start()
    Dim sperre As String
    If modLaden.LadenLaeuft() Then Exit Sub
    sperre = modLaden.BeladungAndererText()                 ' einmal lesen (im SharePoint-Modus ein Abruf der Sperrdatei)
    If Len(sperre) > 0 Then                                 ' jemand anderes laedt gerade: Hinweis statt zweiter Beladung
        modFortschritt.HinweisZeigen "Beladung läuft", sperre, False
        Exit Sub
    End If
    On Error Resume Next
    KartenEntfernen
    modFortschritt.HinweisEntfernen
    phase = "frage"
    ladeText = "": ladeFehler = False: nachLadenGeplant = False
    abwZeile = 0: abwVorbelegt = False: eingetragen = 0: geladenText = "": planText = ""
    modLaden.AnwesenheitStatusLeeren
    ThisWorkbook.Worksheets("Cockpit").Activate
    On Error GoTo 0
    Schritt1Zeigen
```

```
End Sub

Private Sub Schritt1Zeigen()
    Dim ws As Worksheet, kx As Single, ky As Single, kb As Single, kh As Single, ordner As String
    On Error GoTo Ersatz
    Set ws = ActiveSheet
    ordner = modLaden.RohdatenPfad()
    kb = 580: kh = 250
    KarteModal ws, kb, kh, kx, ky
    Kopf ws, kx, ky, kb, 1, "Rohdaten ablegen"
    Beschriftung ws, "Frage", kx + 28, ky + 76, kb - 56, 26, "Liegen die neuen Abzüge im Ordner Rohdaten?", 15, True, C_SW
    Beschriftung ws, "Text", kx + 28, ky + 108, kb - 56, 34, _
         "Die heutigen MIS-O-Abzüge für OPAG und Verbund gehören in diesen Ordner. Beim Laden werden sie erkannt, " & _
         CStr(IIf(Left$(LCase$(ordner), 4) = "http", "direkt von SharePoint gelesen und bleiben dort liegen.", "umbenannt und in die Monatsablage verschoben.")), 9.5, False, C_GRAU, True
    Feld ws, "Pfad", kx + 28, ky + 148, kb - 56, 26, ordner
    Knopf ws, "Ja", kx + kb - 28 - 150, ky + kh - 50, 150, 30, "Ja, jetzt laden", 1, "modTagesablauf.KlickLaden"
    Knopf ws, "Ordner", kx + kb - 28 - 150 - 12 - 130, ky + kh - 50, 130, 30, "Ordner öffnen", 2, "modTagesablauf.KlickOrdner"
    Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 80, 30, "Abbrechen", 3, "modTagesablauf.KlickAbbrechen"
    Knopf ws, "Weiter", kx + 104, ky + kh - 50, 120, 30, "Ohne Laden weiter", 3, "modTagesablauf.KlickOhneLaden"
    Exit Sub
Ersatz:
    ErsatzFrage1
End Sub

' ---------------------------------------------------------------- Klicks Schritt 1
Public Sub KlickLaden()
    Planen "LadenStarten"
End Sub

Public Sub KlickOhneLaden()
    geladenText = "Rohdaten: heute ohne neues Laden."
    Planen "OhneLadenWeiter"
End Sub

' ohne neue Rohdaten: einen laufend gepflegten Anwesenheitsplan trotzdem frisch uebernehmen (MAK des Tages), dann Schritt 2
Public Sub OhneLadenWeiter()
    KartenEntfernen
    modLaden.AnwesenheitStatusLeeren
    If modLaden.AnwesenheitAutomatisch() And Not modLaden.LadenLaeuft() Then modLaden.AnwesenheitLaden True
    Schritt2Zeigen
End Sub

Public Sub KlickOrdner()
    Dim ordner As String
    On Error Resume Next
    ordner = modLaden.RohdatenPfad()
    If Len(ordner) = 0 Then Exit Sub
    Shell "explorer.exe """ & ordner & """", vbNormalFocus
    If Err.Number <> 0 Then
        Err.Clear
        ThisWorkbook.FollowHyperlink ordner
    End If
    On Error GoTo 0
End Sub

Public Sub KlickAbbrechen()
    phase = ""
    If abwVorbelegt Then VorbelegungZuruecknehmen
    Planen "KartenEntfernenUndZurueck"
End Sub

Public Sub Nichts()
    ' Klick auf den abgedunkelten Hintergrund: nichts tun (die Karte ist modal)
End Sub

Public Sub LadenStarten()
    KartenEntfernen
    If modLaden.LadenLaeuft() Then Exit Sub
    phase = "laden"
    ladeText = "": ladeFehler = False: nachLadenGeplant = False
    modLaden.NeueDatenLaden
    ' Ein schon geladener Stand kam neu: NeueDatenLaden hat nichts verarbeitet und zeigt die Ersatz-Frage (ErsatzFrageZeigen)
    If modLaden.ErsatzFrageOffen() Then
        phase = "ersatz"
        Exit Sub
    End If
    ' NeueDatenLaden laeuft synchron; seine Meldungen sind ueber HinweisZeigen -> LadeErgebnis gesammelt.
    ' Ab hier nichts mehr abfangen, Schritt 2 (oder die Fehlerkarte) folgt per OnTime.
    phase = "nachladen"
    If Not nachLadenGeplant Then
        nachLadenGeplant = True
        Planen "NachDemLaden"
    End If
End Sub

Public Sub NachDemLaden()
    nachLadenGeplant = False
    If phase <> "nachladen" Then Exit Sub
    If ladeFehler Then
        FehlerNachLadenZeigen
```

```
    Else
        geladenText = KurzText(IIf(Len(ladeText) > 0, ladeText, "geladen."))
        If modLaden.NeuePostkoerbeAnzahl() > 0 Then
            phase = "standard"                          ' neue Themenpostkoerbe (nicht HS): erst fragen, welche heraus sollen
            modStandard.Zeigen True
        Else
            Schritt2Zeigen
        End If
    End If
End Sub

' nach der Karte „Neue Themenpostkörbe“ im Tagesablauf mit Schritt 2 weiter
Public Sub NachStandardWeiter()
    If phase = "standard" Then Schritt2Zeigen
End Sub

Private Sub FehlerNachLadenZeigen()
    Dim ws As Worksheet, kx As Single, ky As Single, kb As Single, kh As Single
    On Error GoTo Ersatz
    phase = "frage"
    ThisWorkbook.Worksheets("Cockpit").Activate
    Set ws = ActiveSheet
    kb = 540
    kh = 184 + PilleHoehe(ladeText, kb - 56)
    KarteModal ws, kb, kh, kx, ky
    Kopf ws, kx, ky, kb, 1, "Rohdaten ablegen"
    Beschriftung ws, "Frage", kx + 28, ky + 76, kb - 56, 26, "Das Laden hat nicht geklappt", 15, True, C_SW
    Pille ws, "Ergebnis", kx + 28, ky + 110, kb - 56, PilleHoehe(ladeText, kb - 56), ladeText, C_GELBH, C_GELBD
    Knopf ws, "Ja", kx + kb - 28 - 150, ky + kh - 50, 150, 30, "Nochmal laden", 1, "modTagesablauf.KlickLaden"
    Knopf ws, "Weiter", kx + kb - 28 - 150 - 12 - 150, ky + kh - 50, 150, 30, "Ohne Laden weiter", 2, "modTagesablauf.KlickOhneLaden"
    Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 80, 30, "Abbrechen", 3, "modTagesablauf.KlickAbbrechen"
    Exit Sub
Ersatz:
    MsgBox ladeText, vbExclamation, "Tagesablauf"
    phase = ""
End Sub

' ------------------------------------------------------------- Ersatz-Frage: neuer Abzug fuer einen schon geladenen Stand
Public Sub ErsatzFrageZeigen()
    Dim ws As Worksheet, kx As Single, ky As Single, kb As Single, kh As Single, n As Long, i As Long
    Dim py As Single, ph As Single, ry As Single, zh As Single, c1 As Single, c2 As Single, c3 As Single
    Dim q As String, tag As Date, am As Variant, alt As Double, neu As Double, unter As String
    If Not modLaden.ErsatzFrageOffen() Then Exit Sub
    On Error GoTo Ersatz
    KartenEntfernen
    modFortschritt.HinweisEntfernen
    ThisWorkbook.Worksheets("Cockpit").Activate
    Set ws = ActiveSheet
    n = modLaden.ErsatzAnzahl()
    kb = 640: zh = 50
    ph = 30 + n * zh
    kh = 116 + ph + 108
    KarteModal ws, kb, kh, kx, ky
    If modLaden.ErsatzAusAblauf() Then
        Kopf ws, kx, ky, kb, 1, "Rohdaten ablegen"
    Else
        Beschriftung ws, "Schritt", kx + 28, ky + 20, kb - 56, 14, "ROHDATEN LADEN", 8, True, C_TEAL
    End If
    Beschriftung ws, "Frage", kx + 28, ky + 74, kb - 56, 24, IIf(n = 1, "Diesen Stand gibt es schon", "Diese Stände gibt es schon"), 15, True, C_SW
    ' Tabelle: je Stand eine Zeile – Quelle und Stichtag | bisher geladen | neuer Abzug
    py = ky + 108
    c1 = kx + 44: c2 = kx + 260: c3 = kx + 430
    Flaeche ws, "Tab", kx + 28, py, kb - 56, ph, C_GRAUXL
    Beschriftung ws, "H1", c1, py + 9, 200, 12, "STAND", 7.5, True, C_GRAU
    Beschriftung ws, "H2", c2, py + 9, 160, 12, "BISHER GELADEN", 7.5, True, C_GRAU
    Beschriftung ws, "H3", c3, py + 9, 170, 12, "NEUER ABZUG", 7.5, True, C_TEAL
    For i = 1 To n
        q = CStr(modLaden.ErsatzWert(i, 0))
        tag = CDate(modLaden.ErsatzWert(i, 1))
        am = modLaden.ErsatzWert(i, 2)
        alt = CDbl(modLaden.ErsatzWert(i, 3))
        neu = CDbl(modLaden.ErsatzWert(i, 6))
        ry = py + 28 + (i - 1) * zh
        If i > 1 Then Linie ws, "L" & i, kx + 40, ry, kb - 80
        Marke ws, "Q" & i, c1, ry + 9, 62, 20, q, IIf(q = "OPAG", C_ROT, C_TEAL)
        Beschriftung ws, "D" & i, c1 + 72, ry + 7, 130, 16, Choose(Weekday(tag, vbMonday), "Mo", "Di", "Mi", "Do", "Fr", "Sa", "So") & " " & _
                  Format$(tag, "dd.mm.yyyy"), 10.5, True, C_SW
        If Len(CStr(modLaden.ErsatzWert(i, 7))) > 0 Then Beschriftung ws, "DU" & i, c1 + 72, ry + 25, 130, 13, "Einheiten " & CStr(modLaden.ErsatzWert(i, 7)), 8, False, C_GRAU
        Beschriftung ws, "A" & i, c2, ry + 7, 160, 16, modLaden.ZahlDE(alt) & " Datensätze", 10.5, True, C_SW
        If IsDate(am) Then unter = "geladen " & Format$(am, "dd.mm.") & " um " & Format$(am, "hh:nn") & " Uhr" Else unter = "geladen"
        Beschriftung ws, "AU" & i, c2, ry + 25, 160, 13, unter, 8, False, C_GRAU
        Beschriftung ws, "N" & i, c3, ry + 7, 170, 16, "rund " & modLaden.ZahlDE(neu) & " Zeilen", 10.5, True, IIf(neu > alt * 1.05, C_GRUEN, C_SW)
        Beschriftung ws, "NU" & i, c3, ry + 25, 170, 13, KurzName(CStr(modLaden.ErsatzWert(i, 5))), 8, False, C_GRAU
    Next i
    Beschriftung ws, "Hinweis", kx + 28, py + ph + 12, kb - 56, 30, _
          "Übernehmen rechnet Eingänge und Abgänge dieses Tages gegen den Stand davor neu, danach alle späteren Stände dieser Quelle (und Einheiten). " & _
          "Die alte Datei bleibt als …_ersetzt in der Ablage.", 8.5, False, C_GRAU, True
```

```
    Knopf ws, "Ersetzen", kx + kb - 28 - 190, ky + kh - 50, 190, 30, "Neuen Abzug übernehmen", 1, "modTagesablauf.KlickErsetzen"
    Knopf ws, "Behalten", kx + kb - 28 - 190 - 12 - 160, ky + kh - 50, 160, 30, "Alten Stand behalten", 2, "modTagesablauf.KlickBehalten"
    Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 90, 30, "Abbrechen", 3, "modTagesablauf.KlickErsatzAbbrechen"
    Exit Sub
Ersatz:
    On Error Resume Next
    KartenEntfernen
    modLaden.ErsatzEntscheiden (MsgBox("Diesen Stand gibt es schon:" & vbCrLf & vbCrLf & modLaden.ErsatzFrageText() & vbCrLf & vbCrLf & _
                                "Soll der neue Abzug den alten Stand ersetzen?", vbYesNo + vbQuestion, "Rohdaten laden") = vbYes)
    ErsatzWeiter
End Sub

' Flaeche (abgerundet, ohne Rand) als Hintergrund einer Tabelle
Private Sub Flaeche(ws As Worksheet, nm As String, x As Single, y As Single, b As Single, h As Single, farbe As Long)
    With ws.Shapes.AddShape(5, x, y, b, h)
        .Name = PRAEFIX & nm
        .Adjustments(1) = 0.06
        .Fill.ForeColor.RGB = farbe
        .Line.Visible = msoFalse
        .OnAction = "modTagesablauf.Nichts"
    End With
End Sub

' duenne Trennlinie zwischen Tabellenzeilen
Private Sub Linie(ws As Worksheet, nm As String, x As Single, y As Single, b As Single)
    With ws.Shapes.AddLine(x, y, x + b, y)
        .Name = PRAEFIX & nm
        .Line.ForeColor.RGB = C_GRAUL
        .Line.Weight = 0.75
    End With
End Sub

' Marke der Quelle (OPAG rot, Verbund petrol) – Pille mit weisser, fetter Schrift
Private Sub Marke(ws As Worksheet, nm As String, x As Single, y As Single, b As Single, h As Single, inhalt As String, farbe As Long)
    With ws.Shapes.AddShape(5, x, y, b, h)
        .Name = PRAEFIX & nm
        .Adjustments(1) = 0.5
        .Fill.ForeColor.RGB = farbe
        .Line.Visible = msoFalse
        .OnAction = "modTagesablauf.Nichts"
        With .TextFrame2
            .MarginLeft = 2: .MarginRight = 2: .MarginTop = 0: .MarginBottom = 0
            .WordWrap = msoFalse
            .VerticalAnchor = msoAnchorMiddle
            .TextRange.Text = inhalt
            .TextRange.Font.Name = SCHRIFT
            .TextRange.Font.Size = 8.5
            .TextRange.Font.Bold = msoTrue
            .TextRange.Font.Fill.ForeColor.RGB = RGB(255, 255, 255)
            .TextRange.ParagraphFormat.Alignment = msoAlignCenter
        End With
    End With
End Sub

' Dateiname kurz (ohne Ordner, hoechstens 34 Zeichen)
Private Function KurzName(t As String) As String
    KurzName = t
    If Len(t) > 34 Then KurzName = Left$(t, 15) & "…" & Right$(t, 18)
End Function

Public Sub KlickErsetzen()
    modLaden.ErsatzEntscheiden True
    Planen "ErsatzWeiter"
End Sub

Public Sub KlickBehalten()
    modLaden.ErsatzEntscheiden False
    Planen "ErsatzWeiter"
End Sub

Public Sub KlickErsatzAbbrechen()
    modLaden.ErsatzAbbrechen
    If modLaden.ErsatzAusAblauf() Then
        KlickAbbrechen
    Else
        Planen "KartenEntfernenUndZurueck"
    End If
End Sub

' nach der Antwort erneut laden – im Tagesablauf mit dessen Fortsetzung (Schritt 2), sonst direkt
Public Sub ErsatzWeiter()
    KartenEntfernen
    If modLaden.ErsatzAusAblauf() Then
        LadenStarten
    Else
        modLaden.NeueDatenLaden
    End If
End Sub

' ------------------------------------------------------------- Schritt 2: Anwesenheit heute
Public Sub Schritt2Zeigen()
```

```
357      Dim ws As Worksheet, kx As Single, ky As Single, kb As Single, kh As Single, y As Single, warn As String, info As
       String
358      planText = ""
359      If modLaden.AnwesenheitAusPlan() And modLaden.AnwesenheitFehltHeute() <> -2 Then
360          PlanZeigen
361          Exit Sub
362      End If
363      On Error GoTo Ersatz
364      KartenEntfernen
365      phase = "anwesenheit"
366      ThisWorkbook.Worksheets("Cockpit").Activate
367      Set ws = ActiveSheet
368      ' Plan erwartet, aber nicht gelesen – oder gelesen, ohne den heutigen Tag: zeigen, dann wie bisher fragen.
369      ' Eine gelungene Uebernahme (erste MAK-Liste) steht gruen darueber.
370      warn = modLaden.AnwesenheitFehler()
371      If modLaden.AnwesenheitAusPlan() Then warn = modLaden.AnwesenheitHeute() & "."
372      If Len(warn) = 0 Then info = modLaden.AnwesenheitMeldung()
373      If Len(info) > 0 Then info = info & "."
374      kb = 540: kh = 250
375      If Len(geladenText) > 0 Then kh = kh + PilleHoehe(geladenText, kb - 56) + 12
376      If Len(info) > 0 Then kh = kh + PilleHoehe(info, kb - 56) + 12
377      If Len(warn) > 0 Then kh = kh + PilleHoehe(warn, kb - 56) + 12
378      KarteModal ws, kb, kh, kx, ky
379      Kopf ws, kx, ky, kb, 2, "Anwesenheit heute"
380      y = ky + 76
381      If Len(geladenText) > 0 Then
382          Pille ws, "Ergebnis", kx + 28, y, kb - 56, PilleHoehe(geladenText, kb - 56), geladenText, C_GRUENH, C_GRUEN
383          y = y + PilleHoehe(geladenText, kb - 56) + 12
384      End If
385      If Len(info) > 0 Then
386          Pille ws, "Anwesenheit", kx + 28, y, kb - 56, PilleHoehe(info, kb - 56), info, C_GRUENH, C_GRUEN
387          y = y + PilleHoehe(info, kb - 56) + 12
388      End If
389      If Len(warn) > 0 Then
390          Pille ws, "Warnung", kx + 28, y, kb - 56, PilleHoehe(warn, kb - 56), warn, C_GELBH, C_GELBD
391          y = y + PilleHoehe(warn, kb - 56) + 12
392      End If
393      Beschriftung ws, "Frage", kx + 28, y, kb - 56, 26, "Ist heute jemand nicht da?", 15, True, C_SW
394      Beschriftung ws, "Text", kx + 28, y + 32, kb - 56, 48, _
395           "Urlaub, krank, Fortbildung oder ein halber Tag: Wer heute fehlt, zählt nicht in die MAK des Tages. " & _
396           "Die Produktivität rechnet sonst mit zu vielen Köpfen. Heute ist " & WochentagLang(Date) & ", " & Format(Date
       , "dd.mm.yyyy") & ".", _
397           9.5, False, C_GRAU, True
398      Knopf ws, "Ja", kx + kb - 28 - 196, ky + kh - 50, 196, 30, "Ja, Abwesenheit eintragen", 1, "modTagesablauf.KlickAb
       wesend"
399      Knopf ws, "Nein", kx + kb - 28 - 196 - 12 - 130, ky + kh - 50, 130, 30, "Nein, alle da", 2, "modTagesablauf.KlickA
       lleDa"
400      Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 80, 30, "Abbrechen", 3, "modTagesablauf.KlickAbbrechen"
401      Exit Sub
402  Ersatz:
403      If MsgBox("Ist heute jemand nicht da (Urlaub, krank, halber Tag)?", vbQuestion + vbYesNo, "Tagesablauf – Anwesenhe
       it") = vbYes Then
404          AbwesenheitOeffnen
405      Else
406          Schritt3Zeigen
407      End If
408  End Sub
409
410  ' Schritt 2, wenn die Anwesenheit aus dem Telefonie- und Abwesenheitsplan kommt: nichts eintragen, nur zeigen, wer heu
       te fehlt
411  Private Sub PlanZeigen()
412      Dim ws As Worksheet, kx As Single, ky As Single, kb As Single, kh As Single, y As Single, ph As Single
413      Dim heute As String, fuellung As Long, schrift As Long
414      heute = modLaden.AnwesenheitHeute() & "."
415      planText = "Anwesenheit aus dem Plan übernommen. " & heute
416      On Error GoTo Ersatz
417      KartenEntfernen
418      phase = "anwesenheit"
419      ThisWorkbook.Worksheets("Cockpit").Activate
420      Set ws = ActiveSheet
421      kb = 540
422      ph = PilleHoehe(heute, kb - 56)
423      kh = 270 + ph
424      If Len(geladenText) > 0 Then kh = kh + PilleHoehe(geladenText, kb - 56) + 12
425      KarteModal ws, kb, kh, kx, ky
426      Kopf ws, kx, ky, kb, 2, "Anwesenheit heute"
427      y = ky + 76
428      If Len(geladenText) > 0 Then
429          Pille ws, "Ergebnis", kx + 28, y, kb - 56, PilleHoehe(geladenText, kb - 56), geladenText, C_GRUENH, C_GRUEN
430          y = y + PilleHoehe(geladenText, kb - 56) + 12
431      End If
432      Beschriftung ws, "Frage", kx + 28, y, kb - 56, 26, "Die Anwesenheit kommt aus dem Plan", 15, True, C_SW
433      Beschriftung ws, "Text", kx + 28, y + 32, kb - 56, 34, _
434           "Eben übernommen aus " & modLaden.AnwesenheitDatei() & ". Heute ist " & WochentagLang(Date) & ", " & Format(D
       ate, "dd.mm.yyyy") & ".", _
435           9.5, False, C_GRAU, True
436      If modLaden.AnwesenheitFehltHeute() > 0 Then
437          fuellung = C_GELBH: schrift = C_GELBD
438      ElseIf modLaden.AnwesenheitFehltHeute() = 0 Then
439          fuellung = C_GRUENH: schrift = C_GRUEN
440      Else
441          fuellung = C_GRAUXL: schrift = C_SW
442      End If
443      Pille ws, "Heute", kx + 28, y + 72, kb - 56, ph, heute, fuellung, schrift
444      Beschriftung ws, "Hinweis", kx + 28, y + 82 + ph, kb - 56, 30, _
445           "Abwesenheiten trägst du im Plan ein; „Neue Daten laden“ übernimmt ihn jedes Mal neu. Ein A im Kalender ohne
```

```
       „x“ im Plan entfernt das nächste Laden; U, K und Anteile bleiben.", _
446          8.5, False, C_GRAU, True
447      Knopf ws, "Weiter", kx + kb - 28 - 150, ky + kh - 50, 150, 30, "Weiter", 1, "modTagesablauf.KlickAlleDa"
448      Knopf ws, "Kalender", kx + kb - 28 - 150 - 12 - 170, ky + kh - 50, 170, 30, "Im Kalender ansehen", 2, "modTagesabl
       auf.KlickAbwesend"
449      Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 80, 30, "Abbrechen", 3, "modTagesablauf.KlickAbbrechen"
450      Exit Sub
451  Ersatz:
452      On Error Resume Next
453      KartenEntfernen
454      MsgBox heute, vbInformation, "Tagesablauf – Anwesenheit"
455      Schritt3Zeigen
456  End Sub
457
458  Public Sub KlickAlleDa()
459      Planen "Schritt3Zeigen"
460  End Sub
461
462  Public Sub KlickAbwesend()
463      Planen "AbwesenheitOeffnen"
464  End Sub
465
466  Public Sub AbwesenheitOeffnen()
467      ' Kalender beim heutigen Tag oeffnen; die Spalte von heute blinkt kurz, eine Karte am rechten Rand bleibt offen
468      Dim ws As Worksheet, kx As Single, ky As Single, kb As Single, kh As Single, frei As String, letzte As Long
469      On Error GoTo Ersatz
470      KartenEntfernen
471      Set ws = ThisWorkbook.Worksheets("Kalender")
472      ws.Activate
473      modCockpit.KalenderZuHeute
474      abwZeile = modCockpit.KalenderSpalteHeute()
475      If abwZeile = 0 Then GoTo Ersatz
476      ws.Cells(KAL_START, abwZeile).Select
477      frei = HeuteFrei()
478      kb = 300: kh = IIf(Len(frei) > 0, 412, 356)
479      With ActiveWindow.VisibleRange
480          kx = .Left + .Width - kb - 12
481          ky = .Top + 6
482      End With
483      If kx < ws.Cells(1, abwZeile).Left + ws.Cells(1, abwZeile).Width + 12 Then kx = ws.Cells(1, abwZeile).Left + ws.Ce
       lls(1, abwZeile).Width + 12
484      Karte ws, kx, ky, kb, kh
485      Kopf ws, kx, ky, kb, 2, "Anwesenheit heute"
486      If Len(planText) > 0 Then                          ' Anwesenheit aus dem Plan: ansehen, kurzfristig ergaenzen
487          Beschriftung ws, "Frage", kx + 24, ky + 72, kb - 48, 24, "Heute laut Plan", 13, True, C_SW
488          Beschriftung ws, "Text", kx + 24, ky + 104, kb - 48, 150, _
489               "Spalte von heute (rot): A = „x“ im Plan, kursives A = fester freier Tag, FT = Feiertag, leer = da." & vb
       Lf & vbLf & _
490               "Ergänzen: U Urlaub, K Krankheit, 0,5 = halber Tag – das bleibt stehen. Ein A ohne „x“ im Plan entfernt d
       as nächste Laden; " & _
491               "Abwesenheiten deshalb im Plan eintragen.", 9.5, False, C_GRAU, True
492      Else
493          Beschriftung ws, "Frage", kx + 24, ky + 72, kb - 48, 24, "Wer fehlt heute?", 13, True, C_SW
494          Beschriftung ws, "Text", kx + 24, ky + 104, kb - 48, 150, _
495               "In der Spalte von heute (rot) je Person eintragen:" & vbLf & _
496               "U  Urlaub" & vbLf & _
497               "K  Krankheit" & vbLf & _
498               "A  abwesend" & vbLf & _
499               "0,5  nur halber Tag da (Zahl von 0 bis 1)" & vbLf & vbLf & _
500               "Leer = da laut Arbeitszeitmodell; kursives A = fester freier Tag. Mehrere Tage: nach rechts weiter eintr
       agen.", 9.5, False, C_GRAU, True
501      End If
502      If Len(frei) > 0 Then
503          Pille ws, "Frei", kx + 24, ky + 278, kb - 48, 34, "Heute laut Modell frei: " & frei, C_GRAUXL, C_GRAU
504      End If
505      Knopf ws, "Zeigen", kx + 24, ky + kh - 88, kb - 48, 28, "Nochmal zeigen, wo", 2, "modTagesablauf.KlickZeigen"
506      Knopf ws, "Fertig", kx + 24, ky + kh - 50, kb - 48 - 92, 30, "Fertig", 1, "modTagesablauf.KlickFertig"
507      Knopf ws, "Abbrechen", kx + kb - 24 - 84, ky + kh - 50, 84, 30, "Abbrechen", 3, "modTagesablauf.KlickAbbrechen"
508      Hervorheben
509      Exit Sub
510  Ersatz:
511      On Error Resume Next
512      MsgBox "Bitte im Blatt 'Kalender' in der Spalte von heute je abwesender Person U, K, A oder einen Anteil eintragen
       " & _
513             "und danach im Cockpit erneut 'Neue Daten laden' klicken.", vbInformation, "Anwesenheit heute"
514      phase = ""
515  End Sub
516
517  ' Namen, die heute laut Arbeitszeitmodell (Mitarbeiter!J:N = 0) nicht arbeiten
518  Private Function HeuteFrei() As String
519      Dim wsM As Worksheet, r As Long, wt As Long, v As Variant, t As String
520      On Error Resume Next
521      wt = Weekday(Date, vbMonday)
522      If wt > 5 Then Exit Function
523      Set wsM = ThisWorkbook.Worksheets("Mitarbeiter")
524      For r = 5 To 64
525          If Len(Trim$(CStr(wsM.Cells(r, 1).Value))) > 0 Then
526              v = wsM.Cells(r, 9 + wt).Value
527              If Not IsEmpty(v) Then
528                  If IsNumeric(v) Then
529                      If CDbl(v) = 0 Then t = t & IIf(Len(t) > 0, ", ", "") & CStr(wsM.Cells(r, 1).Value)
530                  End If
531              End If
532          End If
533      Next r
534      HeuteFrei = t
```

```
End Function

Public Sub KlickZeigen()
    Planen "Hervorheben"
End Sub

' Die Spalte von heute (alle Mitarbeiterzeilen) und ihr Kopf blinken dreimal auf und verschwinden wieder
Public Sub Hervorheben()
    Dim ws As Worksheet, zeile As Range, kopf As Range, i As Long, t As Double, sichtbar As Boolean, letzte As Long
    On Error Resume Next
    If abwZeile = 0 Then Exit Sub
    Set ws = ThisWorkbook.Worksheets("Kalender")
    If Not ActiveSheet Is ws Then Exit Sub
    letzte = KAL_START
    For i = KAL_START To KAL_START + KAL_ZEILEN - 1
        If Len(CStr(ws.Cells(i, 1).Value)) > 0 Then letzte = i
    Next i
    Set zeile = ws.Range(ws.Cells(KAL_START, abwZeile), ws.Cells(letzte, abwZeile))
    Set kopf = ws.Range(ws.Cells(4, abwZeile), ws.Cells(6, abwZeile))
    Rahmen ws, "HL_Zeile", zeile
    Rahmen ws, "HL_Kopf", kopf
    For i = 1 To 6
        sichtbar = (i Mod 2 = 1)
        ws.Shapes(PRAEFIX & "HL_Zeile").Visible = IIf(sichtbar, msoTrue, msoFalse)
        ws.Shapes(PRAEFIX & "HL_Kopf").Visible = IIf(sichtbar, msoTrue, msoFalse)
        t = Timer
        Do While Timer - t < 0.35 And Timer >= t
            DoEvents
        Loop
    Next i
    ws.Shapes(PRAEFIX & "HL_Zeile").Visible = msoTrue
    ws.Shapes(PRAEFIX & "HL_Kopf").Visible = msoTrue
    t = Timer
    Do While Timer - t < 1.2 And Timer >= t
        DoEvents
    Loop
    ws.Shapes(PRAEFIX & "HL_Zeile").Delete
    ws.Shapes(PRAEFIX & "HL_Kopf").Delete
    ws.Cells(KAL_START, abwZeile).Select
    On Error GoTo 0
End Sub

Private Sub Rahmen(ws As Worksheet, nm As String, rg As Range)
    On Error Resume Next
    ws.Shapes(PRAEFIX & nm).Delete
    On Error GoTo 0
    With ws.Shapes.AddShape(5, rg.Left - 3, rg.Top - 3, rg.Width + 6, rg.Height + 6)
        .Name = PRAEFIX & nm
        .Adjustments(1) = 0.18
        .Fill.ForeColor.RGB = C_TEAL_HELL
        .Fill.Transparency = 0.8
        .Line.ForeColor.RGB = C_TEAL_HELL
        .Line.Weight = 2.5
        .OnAction = "modTagesablauf.Nichts"
    End With
End Sub

Public Sub KlickFertig()
    Planen "AbwesenheitFertig"
End Sub

Public Sub AbwesenheitFertig()
    ' Eintraege von heute zaehlen (jede nicht leere Zelle in der Spalte von heute)
    Dim ws As Worksheet, r As Long
    On Error Resume Next
    Set ws = ThisWorkbook.Worksheets("Kalender")
    eingetragen = 0
    If abwZeile > 0 Then
        For r = KAL_START To KAL_START + KAL_ZEILEN - 1
            If Len(CStr(ws.Cells(r, 1).Value)) > 0 And Len(Trim$(CStr(ws.Cells(r, abwZeile).Value))) > 0 Then
                ' Feiertag und das kursive A des Arbeitszeitmodells sind keine Abwesenheitsmeldung
                If UCase$(Trim$(CStr(ws.Cells(r, abwZeile).Value))) <> "FT" And Not ws.Cells(r, abwZeile).Font.Italic
   Then eingetragen = eingetragen + 1
            End If
        Next r
    End If
    modCockpit.AlphabetischSortieren
    If Len(planText) > 0 And eingetragen <> modLaden.AnwesenheitFehltHeute() And modLaden.AnwesenheitFehltHeute() >= 0
    Then
        planText = planText & " Im Kalender stehen heute " & eingetragen & IIf(eingetragen = 1, " Abwesenheit.", " Abw
   esenheiten.")
    End If
    On Error GoTo 0
    Schritt3Zeigen
End Sub

' (frueher: vorbelegtes Datum im Blatt Abwesenheit zuruecknehmen — im Kalender gibt es keine Vorbelegung)
Private Sub VorbelegungZuruecknehmen()
    abwVorbelegt = False
End Sub

' ---------------------------------------------------------------- Schritt 3: Erfolg
Public Sub Schritt3Zeigen()
    Dim ws As Worksheet, kx As Single, ky As Single, kb As Single, kh As Single, anw As String, zeilen As String
    On Error Resume Next
    KartenEntfernen
```

```
    phase = "fertig"
    ThisWorkbook.Worksheets("Cockpit").Activate
    Application.Calculate
    modCockpit.DiagrammeAnpassen True
    ThisWorkbook.Worksheets("Filter").Range("O42").Value = Now
    modLaden.LogSchreiben "Tagesablauf abgeschlossen"
    On Error GoTo Ersatz
    Set ws = ActiveSheet
    If Len(planText) > 0 Then
        anw = planText
    ElseIf eingetragen = 0 Then
        anw = "Anwesenheit geprüft: heute alle da."
    ElseIf eingetragen = 1 Then
        anw = "Anwesenheit geprüft: 1 Abwesenheit für heute eingetragen."
    Else
        anw = "Anwesenheit geprüft: " & eingetragen & " Abwesenheiten für heute eingetragen."
    End If
    zeilen = IIf(Len(geladenText) > 0, geladenText & vbLf, "") & anw
    kb = 540: kh = 240 + PilleHoehe(zeilen, kb - 56)
    KarteModal ws, kb, kh, kx, ky
    Kopf ws, kx, ky, kb, 3, "Fertig"
    Haken ws, kx + 28, ky + 76
    Beschriftung ws, "Frage", kx + 80, ky + 78, kb - 108, 26, "Du hast alles für heute gemacht", 15, True, C_SW
    Beschriftung ws, "Sub", kx + 80, ky + 104, kb - 108, 18, "Kennzahlen und Diagramme sind auf dem neuesten Stand.",
    9.5, False, C_GRAU
    Pille ws, "Ergebnis", kx + 28, ky + 138, kb - 56, PilleHoehe(zeilen, kb - 56), zeilen, C_GRAUXL, C_SW
    Beschriftung ws, "Text", kx + 28, ky + 146 + PilleHoehe(zeilen, kb - 56), kb - 56, 18, "Wenn du magst, kannst du j
    etzt die PowerPoint-Folie erstellen.", 9.5, False, C_GRAU
    Knopf ws, "PPT", kx + kb - 28 - 200, ky + kh - 50, 200, 30, "PowerPoint-Folie erzeugen", 4, "modTagesablauf.KlickP
    PT"
    Knopf ws, "Schliessen", kx + kb - 28 - 200 - 12 - 110, ky + kh - 50, 110, 30, "Schließen", 2, "modTagesablauf.Klic
    kSchliessen"
    Exit Sub
Ersatz:
    On Error Resume Next
    If MsgBox("Du hast alles für heute gemacht." & vbCrLf & anw & vbCrLf & vbCrLf & "PowerPoint-Folie jetzt erstellen?
    ", vbQuestion + vbYesNo, "Tagesablauf") = vbYes Then
        phase = ""
        modFolienauswahl.Zeigen
    End If
    phase = ""
End Sub

Public Sub KlickPPT()
    Planen "PPTStarten"
End Sub

Public Sub PPTStarten()
    KartenEntfernen
    phase = ""
    modFolienauswahl.Zeigen
End Sub

Public Sub KlickSchliessen()
    phase = ""
    Planen "KartenEntfernen"
End Sub

Public Sub KartenEntfernenUndZurueck()
    KartenEntfernen
    On Error Resume Next
    ThisWorkbook.Worksheets("Cockpit").Activate
    On Error GoTo 0
End Sub

' ---------------------------------------------------------------- Bausteine
Private Sub Planen(prozedur As String)
    On Error Resume Next
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modTagesablauf." & prozedur
    On Error GoTo 0
End Sub

Public Sub KartenEntfernen()
    Dim i As Long, ws As Worksheet
    On Error Resume Next
    For Each ws In ThisWorkbook.Worksheets
        For i = ws.Shapes.Count To 1 Step -1
            If Left$(ws.Shapes(i).Name, Len(PRAEFIX)) = PRAEFIX Then ws.Shapes(i).Delete
        Next i
    Next ws
    On Error GoTo 0
End Sub

' abgedunkelter Hintergrund ueber dem sichtbaren Bereich und Karte in der Mitte
Public Sub KarteModal(ws As Worksheet, kb As Single, kh As Single, ByRef kx As Single, ByRef ky As Single)
    Dim sichtbar As Range
    Set sichtbar = ActiveWindow.VisibleRange
    With ws.Shapes.AddShape(1, sichtbar.Left, sichtbar.Top, sichtbar.Width, sichtbar.Height)
        .Name = PRAEFIX & "Schleier"
        .Fill.ForeColor.RGB = C_SW
        .Fill.Transparency = 0.5
        .Line.Visible = msoFalse
        .OnAction = "modTagesablauf.Nichts"
    End With
    kx = sichtbar.Left + (sichtbar.Width - kb) / 2
    ky = sichtbar.Top + (sichtbar.Height - kh) / 2 - 30
```

```
    If ky < sichtbar.Top + 10 Then ky = sichtbar.Top + 10
    Karte ws, kx, ky, kb, kh
End Sub

Public Sub Karte(ws As Worksheet, kx As Single, ky As Single, kb As Single, kh As Single)
    With ws.Shapes.AddShape(5, kx, ky, kb, kh)
        .Name = PRAEFIX & "Karte"
        .Adjustments(1) = 0.04
        .Fill.ForeColor.RGB = RGB(255, 255, 255)
        .Line.ForeColor.RGB = C_GRAUL
        .Line.Weight = 0.75
        .Shadow.Visible = msoTrue
        .Shadow.Blur = 22
        .Shadow.OffsetX = 0
        .Shadow.OffsetY = 8
        .Shadow.Transparency = 0.7
        .Shadow.ForeColor.RGB = RGB(0, 0, 0)
        .OnAction = "modTagesablauf.Nichts"
    End With
End Sub

' Kopf der Karte: "TAGESABLAUF · SCHRITT n VON 3", Fortschritt in drei Segmenten mit Beschriftung
Private Sub Kopf(ws As Worksheet, kx As Single, ky As Single, kb As Single, schritt As Long, titel As String)
    Dim i As Long, sb As Single, sx As Single, namen As Variant, farbe As Long, rand As Single
    namen = Array("Rohdaten", "Anwesenheit", "Fertig")
    rand = IIf(kb < 400, 24, 28)
    If kb < 400 Then                            ' schmale Karte (neben der Tabelle): ohne Schrittnamen
        Beschriftung ws, "Schritt", kx + 24, ky + 20, kb - 48, 14, "TAGESABLAUF  ·  SCHRITT " & schritt & " VON 3", 8,
    True, C_TEAL
    Else
        Beschriftung ws, "Schritt", kx + 28, ky + 20, kb - 56, 14, "TAGESABLAUF  ·  SCHRITT " & schritt & " VON 3  ·
   " & UCase$(titel), 8, True, C_TEAL
    End If
    sb = (kb - 2 * rand - 2 * 8) / 3
    For i = 0 To 2
        sx = kx + rand + i * (sb + 8)
        If i + 1 < schritt Then
            farbe = C_TEAL
        ElseIf i + 1 = schritt Then
            farbe = C_TEAL_HELL
        Else
            farbe = C_GRAUXL
        End If
        With ws.Shapes.AddShape(5, sx, ky + 40, sb, 5)
            .Name = PRAEFIX & "Seg" & i
            .Adjustments(1) = 0.5
            .Fill.ForeColor.RGB = farbe
            .Line.Visible = msoFalse
            .OnAction = "modTagesablauf.Nichts"
        End With
        Beschriftung ws, "SegT" & i, sx, ky + 49, sb, 12, CStr(namen(i)), 7.5, (i + 1 = schritt), IIf(i + 1 <= schritt
   , C_SW, C_GRAU)
    Next i
End Sub

Public Sub Beschriftung(ws As Worksheet, nm As String, x As Single, y As Single, b As Single, h As Single, inhalt As S
   tring, _
                groesse As Single, fett As Boolean, farbe As Long, Optional umbruch As Boolean = False)
    With ws.Shapes.AddTextbox(msoTextOrientationHorizontal, x, y, b, h)
        .Name = PRAEFIX & nm
        .Fill.Visible = msoFalse
        .Line.Visible = msoFalse
        .OnAction = "modTagesablauf.Nichts"
        .TextFrame2.MarginLeft = 0: .TextFrame2.MarginRight = 0: .TextFrame2.MarginTop = 0: .TextFrame2.MarginBottom =
    0
        .TextFrame2.WordWrap = IIf(umbruch, msoTrue, msoFalse)
        .TextFrame2.VerticalAnchor = IIf(umbruch, msoAnchorTop, msoAnchorMiddle)
        With .TextFrame2.TextRange
            .Text = inhalt
            .Font.Name = SCHRIFT
            .Font.Size = groesse
            .Font.Bold = fett
            .Font.Fill.ForeColor.RGB = farbe
            .ParagraphFormat.SpaceAfter = 0
        End With
    End With
End Sub

' Feld mit Pfad (grau hinterlegt, eine Zeile, links gekuerzt, wenn zu lang)
Private Sub Feld(ws As Worksheet, nm As String, x As Single, y As Single, b As Single, h As Single, inhalt As String)
    Dim t As String
    t = inhalt
    If Len(t) > 78 Then t = "…" & Right$(t, 77)
    With ws.Shapes.AddShape(5, x, y, b, h)
        .Name = PRAEFIX & nm
        .Adjustments(1) = 0.2
        .Fill.ForeColor.RGB = C_GRAUXL
        .Line.ForeColor.RGB = C_GRAUL
        .Line.Weight = 0.75
        .OnAction = "modTagesablauf.KlickOrdner"
        With .TextFrame2
            .MarginLeft = 10: .MarginRight = 10: .MarginTop = 0: .MarginBottom = 0
            .WordWrap = msoFalse
            .VerticalAnchor = msoAnchorMiddle
            .TextRange.Text = t
            .TextRange.Font.Name = "Consolas"
```

```
            .TextRange.Font.Size = 9
            .TextRange.Font.Fill.ForeColor.RGB = C_SW
            .TextRange.ParagraphFormat.Alignment = msoAlignLeft
        End With
    End With
End Sub

' hinterlegter Hinweisblock (Ergebnis des Ladens, Zusammenfassung)
Public Sub Pille(ws As Worksheet, nm As String, x As Single, y As Single, b As Single, h As Single, inhalt As String,
   fuellung As Long, schrift As Long)
    With ws.Shapes.AddShape(5, x, y, b, h)
        .Name = PRAEFIX & nm
        .Adjustments(1) = 0.12
        .Fill.ForeColor.RGB = fuellung
        .Line.Visible = msoFalse
        .OnAction = "modTagesablauf.Nichts"
        With .TextFrame2
            .MarginLeft = 12: .MarginRight = 12: .MarginTop = 6: .MarginBottom = 6
            .WordWrap = msoTrue
            .VerticalAnchor = msoAnchorMiddle
            .TextRange.Text = inhalt
            .TextRange.Font.Name = SCHRIFT
            .TextRange.Font.Size = 9
            .TextRange.Font.Fill.ForeColor.RGB = schrift
            .TextRange.ParagraphFormat.Alignment = msoAlignLeft
        End With
    End With
End Sub

' art: 1 = primaer (petrol), 2 = sekundaer (weiss mit Rand), 3 = Textknopf (grau), 4 = PowerPoint (rot)
Public Sub Knopf(ws As Worksheet, nm As String, x As Single, y As Single, b As Single, h As Single, inhalt As String,
   art As Long, makro As String)
    With ws.Shapes.AddShape(5, x, y, b, h)
        .Name = PRAEFIX & "K_" & nm
        .Adjustments(1) = 0.25
        Select Case art
            Case 1, 4
                .Fill.ForeColor.RGB = IIf(art = 1, C_TEAL, C_ROT)
                .Line.Visible = msoFalse
            Case 2
                .Fill.ForeColor.RGB = RGB(255, 255, 255)
                .Line.ForeColor.RGB = C_GRAUL
                .Line.Weight = 1
            Case Else
                .Fill.Visible = msoFalse
                .Line.Visible = msoFalse
        End Select
        .OnAction = makro
        With .TextFrame2
            .MarginLeft = 4: .MarginRight = 4: .MarginTop = 0: .MarginBottom = 0
            .WordWrap = msoFalse
            .VerticalAnchor = msoAnchorMiddle
            .TextRange.Text = inhalt
            .TextRange.Font.Name = SCHRIFT
            .TextRange.Font.Size = 9.5
            .TextRange.Font.Bold = IIf(art = 3, msoFalse, msoTrue)
            .TextRange.Font.Fill.ForeColor.RGB = IIf(art = 1 Or art = 4, RGB(255, 255, 255), IIf(art = 2, C_SW, C_GRAU
   ))
            .TextRange.ParagraphFormat.Alignment = msoAlignCenter
        End With
    End With
End Sub

' gruener Kreis mit weissem Haken (Linienzug, keine Schriftzeichen)
Private Sub Haken(ws As Worksheet, x As Single, y As Single)
    Dim ff As Object
    With ws.Shapes.AddShape(9, x, y, 40, 40)
        .Name = PRAEFIX & "HakenKreis"
        .Fill.ForeColor.RGB = C_GRUEN
        .Line.Visible = msoFalse
        .OnAction = "modTagesablauf.Nichts"
    End With
    On Error Resume Next
    Set ff = ws.Shapes.BuildFreeform(msoEditingAuto, x + 11, y + 21)
    ff.AddNodes msoSegmentLine, msoEditingAuto, x + 17.5, y + 27.5
    ff.AddNodes msoSegmentLine, msoEditingAuto, x + 29.5, y + 14
    With ff.ConvertToShape
        .Name = PRAEFIX & "Haken"
        .Fill.Visible = msoFalse
        .Line.ForeColor.RGB = RGB(255, 255, 255)
        .Line.Weight = 3
        .OnAction = "modTagesablauf.Nichts"
    End With
    On Error GoTo 0
End Sub

' Hoehe eines Hinweisblocks: je Absatz und je ca. 85 Zeichen (9 pt auf 480 pt Breite) eine Zeile zu 14 pt, plus Rand
Private Function PilleHoehe(t As String, breite As Single) As Single
    Dim teile() As String, i As Long, n As Long, jeZeile As Long
    jeZeile = Int(breite / 5.6)
    teile = Split(Replace(t, vbCrLf, vbLf), vbLf)
    For i = 0 To UBound(teile)
        n = n + 1 + Int(Len(teile(i)) / jeZeile)
    Next i
    If n < 1 Then n = 1
    PilleHoehe = 16 + 14 * n
```

```
903  End Function
904
905  ' Lademeldung einzeilig mit "Rohdaten:" davor (mehrere Meldungen werden mit Leerzeichen verbunden)
906  Private Function KurzText(t As String) As String
907      Dim s As String
908      s = Replace(t, vbCrLf, " ")
909      s = Replace(s, vbLf, " ")
910      If Left$(s, 1) <> "R" Then s = "Rohdaten: " & s
911      KurzText = s
912  End Function
913
914  Private Function WochentagLang(d As Date) As String
915      WochentagLang = Choose(Weekday(d, vbMonday), "Montag", "Dienstag", "Mittwoch", "Donnerstag", "Freitag", "Samstag",
         "Sonntag")
916  End Function
917
918  ' Fallback ohne Formen (z. B. geschuetztes Blatt): klassische Abfrage
919  Private Sub ErsatzFrage1()
920      On Error Resume Next
921      KartenEntfernen
922      If MsgBox("Liegen die neuen Abzüge im Ordner Rohdaten?" & vbCrLf & modLaden.RohdatenPfad(), vbQuestion + vbYesNo,
        "Tagesablauf – Rohdaten") = vbYes Then
923          LadenStarten
924      Else
925          phase = ""
926      End If
927  End Sub
```

modWerkzeuge.bas

```
Option Explicit
' ============================================================================
' modWerkzeuge – Karten (Formen, ohne UserForm) fuer die Werkzeug-Knoepfe im Cockpit
'   SLA anpassen …      Konfiguration!F3 (SLA Verbund, 0 = keine Linie) und F4 (SLA OPAG, leer = Vorschlag) eingeben
'   Einheit …           nur Bereich MAKLER, nur Dateien vor 12.70 (Knopf BTN_Einheit, seither entfernt): Gesamt (Standard) oder
'                       eine Einheit (Filter!L16); Fakten!P und Umbuchungen!K/L zaehlen dann nur Zeilen dieser Einheit
'   Rohdaten löschen …  Dateien im Ordner Rohdaten (und der Monatsablage) auswaehlen und endgueltig loeschen; auf Wunsch
'                       danach das Tool aus den uebrigen Dateien neu aufbauen
'   Zurücksetzen …      alle geladenen Staende leeren (modLaden.ToolLeeren), optional auch die Anwesenheit
'   Sparte | Einheit    nur Bereich MAKLER: Chips in der Kopfleiste „Rückstand OPAG“; Filter!L19 schaltet die Stapel beider
'                       Rueckstandsdiagramme (Cockpit und Folie) zwischen Sparten und Einheiten (Namen Filter!L22:L25) um
' Bausteine (Karte, Beschriftung, Knopf, Pille) kommen aus modTagesablauf; alle Formen tragen den Praefix TAGW_ und
' verschwinden mit modTagesablauf.KartenEntfernen. Klicks loeschen die geklickte Form nie im eigenen Makro (OnTime).
' ============================================================================

Private Const PRAEFIX As String = "TAGW_"
Private Const SCHRIFT As String = "Liberation Sans"
Private Const SEITE As Long = 10                    ' Rohdateien je Seite

Private Const C_TEAL As Long = 8220427              ' RGB(11, 111, 125)
Private Const C_ROT As Long = 2627007               ' RGB(191, 21, 40)
Private Const C_SW As Long = 1710618                ' RGB(26, 26, 26)
Private Const C_GRAU As Long = 7039851              ' RGB(107, 107, 107)
Private Const C_GRAUL As Long = 14341584            ' RGB(208, 213, 218)
Private Const C_GRAUXL As Long = 16184563           ' RGB(243, 244, 246)
Private Const C_GELBD As Long = 23178               ' RGB(138, 90, 0)
Private Const C_GELBH As Long = 13103099            ' RGB(251, 239, 199)
Private Const C_ROTH As Long = 15460859             ' RGB(251, 233, 235)

Private kx As Single, ky As Single, kb As Single, kh As Single
Private einheiten As Variant                        ' Auswahl der Einheiten-Karte (0 = Gesamt)
Private klickNr As Long
Private dateien As Variant                          ' Rohdateien (relative Pfade)
Private gewaehlt() As Boolean
Private geladen() As Boolean
Private seiteNr As Long
Private mitAnwesenheit As Boolean
Private updateName As String                        ' Aeltere Datei aktualisieren: Name der geoeffneten alten Master-Datei

Public Function WerkzeugeVersion() As String
    WerkzeugeVersion = "1"
End Function

Private Sub Planen(prozedur As String)
    On Error Resume Next
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modWerkzeuge." & prozedur
    On Error GoTo 0
End Sub

Private Function Bereit() As Boolean
    If modLaden.LadenLaeuft() Then Exit Function
    On Error Resume Next
    modTagesablauf.KartenEntfernen
    modFortschritt.HinweisEntfernen
    ThisWorkbook.Worksheets("Cockpit").Activate
    On Error GoTo 0
    Bereit = True
End Function

Public Sub Schliessen()
    modTagesablauf.KartenEntfernen
End Sub

Public Sub KlickSchliessen()
    Planen "Schliessen"
End Sub

Private Sub Ueberschrift(ws As Worksheet, marke As String, frage As String)
    modTagesablauf.Beschriftung ws, "Schritt", kx + 28, ky + 22, kb - 56, 14, marke, 8, True, C_TEAL
    modTagesablauf.Beschriftung ws, "Frage", kx + 28, ky + 42, kb - 56, 24, frage, 15, True, C_SW
End Sub

' Rechteck mit Text und Makro (Zeilen der Dateiliste, Umschalter)
Private Sub Zeile(ws As Worksheet, nm As String, x As Single, y As Single, b As Single, h As Single, inhalt As String, _
    an As Boolean, makro As String, _
                  Optional groesse As Single = 9)
    On Error Resume Next
    ws.Shapes(PRAEFIX & nm).Delete
    On Error GoTo 0
    With ws.Shapes.AddShape(5, x, y, b, h)
        .Name = PRAEFIX & nm
        .Adjustments(1) = 0.2
        If an Then
            .Fill.ForeColor.RGB = C_TEAL
            .Line.Visible = msoFalse
        Else
            .Fill.ForeColor.RGB = RGB(255, 255, 255)
            .Line.ForeColor.RGB = C_GRAUL
            .Line.Weight = 1
```

```
        End If
        .OnAction = makro
        With .TextFrame2
            .MarginLeft = 10: .MarginRight = 10: .MarginTop = 0: .MarginBottom = 0
            .WordWrap = msoFalse
            .VerticalAnchor = msoAnchorMiddle
            .TextRange.Text = inhalt
            .TextRange.Font.Name = SCHRIFT
            .TextRange.Font.Size = groesse
            .TextRange.Font.Bold = IIf(an, msoTrue, msoFalse)
            .TextRange.Font.Fill.ForeColor.RGB = IIf(an, RGB(255, 255, 255), C_SW)
            .TextRange.ParagraphFormat.Alignment = msoAlignLeft
        End With
    End With
End Sub

' Eingabefeld: Textfeld ohne Makro (Klick hinein, tippen)
Private Sub Eingabe(ws As Worksheet, nm As String, x As Single, y As Single, b As Single, h As Single, inhalt As Strin
   g)
    With ws.Shapes.AddTextbox(msoTextOrientationHorizontal, x, y, b, h)
        .Name = PRAEFIX & nm
        .Fill.ForeColor.RGB = RGB(255, 255, 255)
        .Line.ForeColor.RGB = C_TEAL
        .Line.Weight = 1.25
        With .TextFrame2
            .MarginLeft = 10: .MarginRight = 10: .MarginTop = 2: .MarginBottom = 2
            .WordWrap = msoFalse
            .VerticalAnchor = msoAnchorMiddle
            .TextRange.Text = inhalt
            .TextRange.Font.Name = SCHRIFT
            .TextRange.Font.Size = 12
            .TextRange.Font.Bold = msoTrue
            .TextRange.Font.Fill.ForeColor.RGB = C_SW
            .TextRange.ParagraphFormat.Alignment = msoAlignRight
        End With
    End With
End Sub

Private Function FeldText(ws As Worksheet, nm As String) As String
    On Error Resume Next
    FeldText = Trim$(ws.Shapes(PRAEFIX & nm).TextFrame2.TextRange.Text)
    On Error GoTo 0
End Function

Private Function Wert(v As Variant) As Double
    If IsNumeric(v) And Not IsEmpty(v) Then Wert = CDbl(v)
End Function

' "5.500" / "5500" / "5 500" -> 5500; leer -> -1; keine Zahl -> -2
Private Function ZahlLesen(t As String) As Double
    Dim s As String, i As Long
    s = Replace(Replace(Replace(Trim$(t), ".", ""), " ", ""), Chr(160), "")
    s = Replace(s, ",", ".")
    If Len(s) = 0 Then ZahlLesen = -1: Exit Function
    For i = 1 To Len(s)
        If InStr("0123456789.", Mid$(s, i, 1)) = 0 Then ZahlLesen = -2: Exit Function
    Next i
    ZahlLesen = Round(Val(s), 0)
End Function

' ============================================================================ SLA anpassen
Public Sub SlaZeigen()
    Dim ws As Worksheet, wsK As Worksheet, v As Variant, vorschlag As String
    If Not Bereit() Then Exit Sub
    Set ws = ActiveSheet
    Set wsK = ThisWorkbook.Worksheets("Konfiguration")
    kb = 520: kh = 330
    modTagesablauf.KarteModal ws, kb, kh, kx, ky
    Ueberschrift ws, "VORGABEN  ·  SLA", "SLA anpassen"
    modTagesablauf.Beschriftung ws, "Text", kx + 28, ky + 72, kb - 56, 30, _
        "Die SLA erscheint als gestrichelte Linie in den Rückstandsdiagrammen. 0 oder leer = keine Linie.", 9.5, False
    , C_GRAU, True
    modTagesablauf.Beschriftung ws, "L1", kx + 28, ky + 118, 260, 16, "Rückstand Verbund (Vorgänge)", 10.5, True, C_SW
    modTagesablauf.Beschriftung ws, "L1u", kx + 28, ky + 136, 260, 14, "Konfiguration F3", 8, False, C_GRAU
    v = wsK.Range("F3").Value
    Eingabe ws, "E_Verbund", kx + kb - 28 - 150, ky + 116, 150, 30, CStr(IIf(Wert(v) > 0, modLaden.ZahlDE(Wert(v)), ""
    ))
    modTagesablauf.Beschriftung ws, "L2", kx + 28, ky + 170, 260, 16, "Rückstand OPAG (Arbeitsgüter)", 10.5, True, C_S
    W
    vorschlag = CStr(wsK.Range("D5").Value)
    modTagesablauf.Beschriftung ws, "L2u", kx + 28, ky + 188, kb - 56 - 170, 26, "Konfiguration F4 · " & vorschlag, 8,
    False, C_GRAU, True
    v = wsK.Range("F4").Value
    Eingabe ws, "E_OPAG", kx + kb - 28 - 150, ky + 168, 150, 30, CStr(IIf(Wert(v) > 0, modLaden.ZahlDE(Wert(v)), ""))
    modTagesablauf.Beschriftung ws, "Fehler", kx + 28, ky + 226, kb - 56, 16, "", 9, True, C_ROT
    modTagesablauf.Beschriftung ws, "Tipp", kx + 28, ky + 244, kb - 56, 16, "In ein Feld klicken, Zahl eingeben, dann
    „Speichern“.", 8.5, False, C_GRAU
    modTagesablauf.Knopf ws, "Speichern", kx + kb - 28 - 150, ky + kh - 50, 150, 30, "Speichern", 1, "modWerkzeuge.Kli
    ckSlaSpeichern"
    modTagesablauf.Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 90, 30, "Abbrechen", 3, "modWerkzeuge.KlickSchliessen
    "
End Sub

Public Sub KlickSlaSpeichern()
    Planen "SlaSpeichern"
End Sub
```

```
177
178 Public Sub SlaSpeichern()
179     Dim ws As Worksheet, wsK As Worksheet, vb As Double, vo As Double
180     Set ws = ActiveSheet
181     Set wsK = ThisWorkbook.Worksheets("Konfiguration")
182     vb = ZahlLesen(FeldText(ws, "E_Verbund"))
183     vo = ZahlLesen(FeldText(ws, "E_OPAG"))
184     If vb = -2 Or vo = -2 Then
185         On Error Resume Next
186         ws.Shapes(PRAEFIX & "Fehler").TextFrame2.TextRange.Text = "Bitte nur ganze Zahlen eingeben (z. B. 5.500)."
187         On Error GoTo 0
188         Exit Sub
189     End If
190     If vb < 0 Then vb = 0
191     wsK.Range("F3").Value = vb
192     If vo <= 0 Then wsK.Range("F4").ClearContents Else wsK.Range("F4").Value = vo
193     modTagesablauf.KartenEntfernen
194     Application.Calculate
195     modCockpit.DiagrammeAnpassen True
196     modFortschritt.HinweisZeigen "SLA gespeichert", "SLA Verbund: " & IIf(vb > 0, modLaden.ZahlDE(vb), "keine Linie")
      & _
197         " · SLA OPAG: " & IIf(vo > 0, modLaden.ZahlDE(vo), "Vorschlag aus Verbund") & ".", False
198 End Sub
199
200 ' ============================================================================ Einheit (Bereich MAKLER)
201 Public Sub EinheitZeigen()
202     Dim ws As Worksheet, liste As Variant, i As Long, n As Long, aktuell As String, b As Single, x As Single, y As Sin
      gle, t As String
203     If Not Bereit() Then Exit Sub
204     Set ws = ActiveSheet
205     liste = modLaden.EinheitenListe()
206     n = UBound(liste) - LBound(liste) + 1
207     If n < 0 Then n = 0
208     ReDim einheiten(0 To n)
209     einheiten(0) = "Gesamt"
210     For i = 1 To n
211         einheiten(i) = liste(LBound(liste) + i - 1)
212     Next i
213     aktuell = CStr(ThisWorkbook.Worksheets("Filter").Range("L16").Value)
214     If Len(aktuell) = 0 Then aktuell = "Gesamt"
215     kb = 520: kh = 190 + 38 * Int((n + 2) / 2)
216     modTagesablauf.KarteModal ws, kb, kh, kx, ky
217     Ueberschrift ws, "ANSICHT  ·  EINHEIT", "Welche Einheit anzeigen?"
218     modTagesablauf.Beschriftung ws, "Text", kx + 28, ky + 72, kb - 56, 30, _
219         "Gesamt zeigt alle Einheiten zusammen (Standard). Eine Einheit filtert Rückstand, Eingänge und Abgänge; MAK un
      d Produktivität bleiben Gesamtwerte.", _
220         9, False, C_GRAU, True
221     b = (kb - 56 - 10) / 2
222     For i = 0 To n
223         x = kx + 28 + (i Mod 2) * (b + 10)
224         y = ky + 112 + (i \ 2) * 38
225         t = CStr(IIf(i = 0, "Gesamt (alle Einheiten)", einheiten(i)))
226         Zeile ws, "E" & Format(i, "00"), x, y, b, 30, t, (CStr(einheiten(i)) = aktuell), "modWerkzeuge.KlickEinheit",
      10
227     Next i
228     If n = 0 Then modTagesablauf.Beschriftung ws, "Leer", kx + 28, ky + 152, kb - 56, 16, "Noch keine Einheiten gelade
      n – erst „Neue Daten laden“.", 9, False, C_GRAU
229     modTagesablauf.Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 90, 30, "Schließen", 3, "modWerkzeuge.KlickSchliessen
      "
230 End Sub
231
232 Public Sub KlickEinheit()
233     Dim nm As String
234     On Error Resume Next
235     nm = CStr(Application.Caller)
236     On Error GoTo 0
237     If InStr(nm, PRAEFIX & "E") <> 1 Then Exit Sub
238     klickNr = CLng(Val(Mid$(nm, Len(PRAEFIX) + 2, 2)))
239     Planen "EinheitSetzen"
240 End Sub
241
242 Public Sub EinheitSetzen()
243     Dim wert As String
244     If IsEmpty(einheiten) Then Exit Sub
245     If klickNr < 0 Or klickNr > UBound(einheiten) Then Exit Sub
246     wert = CStr(einheiten(klickNr))
247     modTagesablauf.KartenEntfernen
248     ThisWorkbook.Worksheets("Filter").Range("L16").Value = wert
249     EinheitKnopfBeschriften
250     Application.Calculate
251     modCockpit.DiagrammeAnpassen True
252 End Sub
253
254 ' Text des Cockpit-Knopfs "Einheit: …" nach Filter!L16 (nur Dateien vor 12.70; fehlt der Knopf, nur SichtFaerben)
255 Public Sub EinheitKnopfBeschriften()
256     Dim wert As String
257     wert = CStr(ThisWorkbook.Worksheets("Filter").Range("L16").Value)
258     If Len(wert) = 0 Then wert = "Gesamt"
259     On Error Resume Next
260     ThisWorkbook.Worksheets("Cockpit").Shapes("BTN_Einheit").TextFrame2.TextRange.Text = "Einheit: " & wert
261     On Error GoTo 0
262     SichtFaerben
263 End Sub
264
265 ' ============================================================================ Sicht Sparte | Einheit (Bereich MAKLER)
266 Public Sub SichtSparte()
```

```
    SichtSetzen "Sparte"
End Sub

Public Sub SichtEinheit()
    SichtSetzen "Einheit"
End Sub

Private Sub SichtSetzen(wert As String)
    If modLaden.LadenLaeuft() Then Exit Sub
    ThisWorkbook.Worksheets("Filter").Range("L19").Value = wert
    SichtFaerben
    Application.Calculate
    modCockpit.DiagrammeAnpassen True
End Sub

' Chips wie die Bookmark-Chips: aktiv = weiss mit Schrift in der Kopffarbe (OPAG rot), inaktiv = Kopffarbe mit weisser Schrift
Public Sub SichtFaerben()
    Dim einheit As Boolean, nm As Variant, aktiv As Boolean, shp As Shape
    einheit = (CStr(ThisWorkbook.Worksheets("Filter").Range("L19").Value) = "Einheit")
    On Error Resume Next
    For Each nm In Array("BWS_Sparte", "BWS_Einheit")
        Set shp = Nothing
        Set shp = ThisWorkbook.Worksheets("Cockpit").Shapes(CStr(nm))
        If Not shp Is Nothing Then
            aktiv = (einheit = (nm = "BWS_Einheit"))
            shp.Fill.Visible = msoTrue
            shp.Fill.Solid
            shp.Line.Visible = msoTrue
            shp.Line.ForeColor.RGB = RGB(255, 255, 255)
            shp.Fill.ForeColor.RGB = IIf(aktiv, RGB(255, 255, 255), RGB(142, 15, 30))
            shp.TextFrame2.TextRange.Font.Fill.ForeColor.RGB = IIf(aktiv, RGB(142, 15, 30), RGB(255, 255, 255))
            shp.TextFrame2.TextRange.Font.Bold = IIf(aktiv, msoTrue, msoFalse)
        End If
    Next nm
    On Error GoTo 0
End Sub

' ============================================================================ Rohdaten löschen
Public Sub RohdatenZeigen()
    Dim i As Long, n As Long
    If Not Bereit() Then Exit Sub
    dateien = modLaden.RohdateienListe()
    n = UBound(dateien) - LBound(dateien) + 1
    If n < 0 Then n = 0
    If n > 0 Then
        ReDim gewaehlt(0 To n - 1)
        ReDim geladen(0 To n - 1)
        For i = 0 To n - 1
            geladen(i) = modLaden.RohdateiGeladen(CStr(dateien(i)))
        Next i
    End If
    seiteNr = 0
    RohdatenZeichnen
End Sub

Private Function DateiAnzahl() As Long
    If IsEmpty(dateien) Then Exit Function
    DateiAnzahl = UBound(dateien) - LBound(dateien) + 1
    If DateiAnzahl < 0 Then DateiAnzahl = 0
End Function

Private Function AnzahlGewaehlt() As Long
    Dim i As Long
    For i = 0 To DateiAnzahl() - 1
        If gewaehlt(i) Then AnzahlGewaehlt = AnzahlGewaehlt + 1
    Next i
End Function

Private Sub RohdatenZeichnen()
    Dim ws As Worksheet, n As Long, seiten As Long, i As Long, y As Single
    modTagesablauf.KartenEntfernen
    Set ws = ActiveSheet
    n = DateiAnzahl()
    seiten = Int((n + SEITE - 1) / SEITE)
    If seiten < 1 Then seiten = 1
    If seiteNr > seiten - 1 Then seiteNr = seiten - 1
    kb = 640: kh = 196 + SEITE * 32
    modTagesablauf.KarteModal ws, kb, kh, kx, ky
    Ueberschrift ws, "DATEN  ·  ROHDATEN", "Rohdateien löschen"
    modTagesablauf.Beschriftung ws, "Text", kx + 28, ky + 70, kb - 56, 28, _
        "Dateien im Ordner Rohdaten und in der Monatsablage, neueste oben. Klick wählt eine Datei aus oder ab. „geladen“ = aus der Datei ist ein Stand im Tool.", _
        9, False, C_GRAU, True
    If n = 0 Then
        modTagesablauf.Beschriftung ws, "Leer", kx + 28, ky + 112, kb - 56, 16, "Keine Excel-Dateien im Ordner Rohdaten.", 10, False, C_GRAU
    End If
    For i = seiteNr * SEITE To seiteNr * SEITE + SEITE - 1
        If i > n - 1 Then Exit For
        y = ky + 106 + (i - seiteNr * SEITE) * 32
        DateiZeile ws, i, y
    Next i
    If seiten > 1 Then
        modTagesablauf.Knopf ws, "Zurueck", kx + 28, ky + kh - 88, 70, 26, "‹ zurück", 2, "modWerkzeuge.KlickSeiteZurueck"
```

```
359          modTagesablauf.Beschriftung ws, "Seite", kx + 106, ky + kh - 82, 120, 14, "Seite " & (seiteNr + 1) & " von " &
      seiten & " · " & n & " Dateien", 8.5, False, C_GRAU
360          modTagesablauf.Knopf ws, "Vor", kx + 234, ky + kh - 88, 70, 26, "vor ›", 2, "modWerkzeuge.KlickSeiteVor"
361      End If
362      modTagesablauf.Beschriftung ws, "Zahl", kx + kb - 28 - 220, ky + kh - 82, 220, 14, AnzahlGewaehlt() & " ausgewählt
      ", 9, True, IIf(AnzahlGewaehlt() > 0, C_ROT, C_GRAU)
363      ws.Shapes(PRAEFIX & "Zahl").TextFrame2.TextRange.ParagraphFormat.Alignment = msoAlignRight
364      modTagesablauf.Knopf ws, "Loeschen", kx + kb - 28 - 190, ky + kh - 50, 190, 30, "Ausgewählte löschen …", 4, "modWe
     rkzeuge.KlickLoeschenFragen"
365      modTagesablauf.Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 90, 30, "Abbrechen", 3, "modWerkzeuge.KlickSchliessen
     "
366  End Sub
367
368  Private Sub DateiZeile(ws As Worksheet, i As Long, y As Single)
369      Dim rel As String, nm As String, ordner As String, gr As Double, info As String
370      rel = CStr(dateien(i))
371      nm = Mid$(rel, InStrRev(rel, Application.PathSeparator) + 1)
372      If InStr(rel, Application.PathSeparator) > 0 Then ordner = Left$(rel, InStrRev(rel, Application.PathSeparator) - 1
     ) Else ordner = "Rohdaten (neu)"
373      If Len(nm) > 52 Then nm = Left$(nm, 50) & "…"
374      Zeile ws, "D" & Format(i, "0000"), kx + 28, y, kb - 56, 28, CStr(IIf(gewaehlt(i), "[x]  ", "[  ]  ")) & nm, gewaeh
     lt(i), "modWerkzeuge.KlickDatei"
375      gr = modLaden.RohdateiGroesse(rel)
376      info = ordner & IIf(gr >= 0, " · " & modLaden.ZahlDE(gr / 1024) & " KB", "")
377      modTagesablauf.Beschriftung ws, "I" & Format(i, "0000"), kx + kb - 28 - 250, y + 7, 160, 14, info, 8, False, IIf(g
     ewaehlt(i), RGB(255, 255, 255), C_GRAU)
378      ws.Shapes(PRAEFIX & "I" & Format(i, "0000")).TextFrame2.TextRange.ParagraphFormat.Alignment = msoAlignRight
379      ws.Shapes(PRAEFIX & "I" & Format(i, "0000")).OnAction = "modWerkzeuge.KlickDatei"
380      If geladen(i) Then
381          modTagesablauf.Pille ws, "G" & Format(i, "0000"), kx + kb - 28 - 78, y + 5, 70, 18, "geladen", C_GELBH, C_GELB
     D
382          With ws.Shapes(PRAEFIX & "G" & Format(i, "0000")).TextFrame2
383              .MarginLeft = 2: .MarginRight = 2: .MarginTop = 0: .MarginBottom = 0
384              .TextRange.Font.Size = 8
385              .TextRange.Font.Bold = msoTrue
386              .TextRange.ParagraphFormat.Alignment = msoAlignCenter
387          End With
388          ws.Shapes(PRAEFIX & "G" & Format(i, "0000")).OnAction = "modWerkzeuge.KlickDatei"
389      End If
390  End Sub
391
392  Public Sub KlickDatei()
393      Dim nm As String
394      On Error Resume Next
395      nm = CStr(Application.Caller)
396      On Error GoTo 0
397      If Len(nm) < Len(PRAEFIX) + 5 Then Exit Sub
398      klickNr = CLng(Val(Right$(nm, 4)))
399      Planen "DateiUmschalten"
400  End Sub
401
402  Public Sub DateiUmschalten()
403      If klickNr < 0 Or klickNr > DateiAnzahl() - 1 Then Exit Sub
404      gewaehlt(klickNr) = Not gewaehlt(klickNr)
405      Application.ScreenUpdating = False
406      RohdatenZeichnen
407      Application.ScreenUpdating = True
408  End Sub
409
410  Public Sub KlickSeiteZurueck()
411      If seiteNr > 0 Then seiteNr = seiteNr - 1
412      Planen "RohdatenNeu"
413  End Sub
414
415  Public Sub KlickSeiteVor()
416      seiteNr = seiteNr + 1
417      Planen "RohdatenNeu"
418  End Sub
419
420  Public Sub RohdatenNeu()
421      Application.ScreenUpdating = False
422      RohdatenZeichnen
423      Application.ScreenUpdating = True
424  End Sub
425
426  Public Sub KlickLoeschenFragen()
427      If AnzahlGewaehlt() = 0 Then Exit Sub                   ' nichts gewaehlt: der Zaehler zeigt es
428      Planen "LoeschenFragen"
429  End Sub
430
431  Public Sub LoeschenFragen()
432      Dim ws As Worksheet, i As Long, n As Long, g As Long, liste As String, k As Long
433      modTagesablauf.KartenEntfernen
434      Set ws = ActiveSheet
435      For i = 0 To DateiAnzahl() - 1
436          If gewaehlt(i) Then
437              n = n + 1
438              If geladen(i) Then g = g + 1
439              If k < 6 Then
440                  liste = liste & IIf(Len(liste) > 0, vbLf, "") & Mid$(CStr(dateien(i)), InStrRev(CStr(dateien(i)), Appl
     ication.PathSeparator) + 1)
441                  k = k + 1
442              End If
443          End If
444      Next i
445      If n > k Then liste = liste & vbLf & "… und " & (n - k) & " weitere"
```

```
    kb = 560: kh = 300 + 14 * k
    modTagesablauf.KarteModal ws, kb, kh, kx, ky
    Ueberschrift ws, "DATEN  ·  ROHDATEN", CStr(IIf(n = 1, "1 Datei endgültig löschen?", n & " Dateien endgültig löschen?"))
    modTagesablauf.Pille ws, "Liste", kx + 28, ky + 76, kb - 56, 20 + 14 * (k + IIf(n > k, 1, 0)), liste, C_GRAUXL, C_SW
    If g > 0 Then
        modTagesablauf.Pille ws, "Warnung", kx + 28, ky + kh - 150, kb - 56, 70, _
            IIf(g = 1, "Aus 1 Datei", "Aus " & g & " Dateien") & " ist ein Stand geladen. Nur löschen: die Stände bleiben im Tool. " & _
            "Löschen und neu aufbauen: das Tool wird geleert und aus den übrigen Rohdateien neu geladen (Anwesenheit bleibt).", C_ROTH, C_ROT
        modTagesablauf.Knopf ws, "Neu", kx + kb - 28 - 220, ky + kh - 50, 220, 30, "Löschen und neu aufbauen", 4, "modWerkzeuge.KlickLoeschenNeu"
        modTagesablauf.Knopf ws, "Nur", kx + kb - 28 - 220 - 10 - 120, ky + kh - 50, 120, 30, "Nur löschen", 2, "modWerkzeuge.KlickLoeschenNur"
    Else
        modTagesablauf.Beschriftung ws, "Hinweis", kx + 28, ky + kh - 120, kb - 56, 30, _
            "Aus keiner dieser Dateien ist ein Stand geladen. Gelöschte Dateien lassen sich nicht wiederherstellen.", 9, False, C_GRAU, True
        modTagesablauf.Knopf ws, "Nur", kx + kb - 28 - 160, ky + kh - 50, 160, 30, "Endgültig löschen", 4, "modWerkzeuge.KlickLoeschenNur"
    End If
    modTagesablauf.Knopf ws, "Zurueck", kx + 20, ky + kh - 50, 90, 30, "Zurück", 3, "modWerkzeuge.KlickZurAuswahl"
End Sub

Public Sub KlickZurAuswahl()
    Planen "RohdatenNeu"
End Sub

Public Sub KlickLoeschenNur()
    Planen "LoeschenNur"
End Sub

Public Sub KlickLoeschenNeu()
    Planen "LoeschenNeu"
End Sub

Public Sub LoeschenNur()
    Dim t As String
    If BeladungAnderer() Then Exit Sub
    t = Loeschen()
    modTagesablauf.KartenEntfernen
    modFortschritt.HinweisZeigen "Rohdaten gelöscht", t, (InStr(t, "nicht gelöscht") > 0)
End Sub

Public Sub LoeschenNeu()
    Dim t As String
    If BeladungAnderer() Then Exit Sub                ' vor dem Loeschen und Leeren pruefen, nicht erst im Laden
    t = Loeschen()
    modTagesablauf.KartenEntfernen
    modLaden.ToolLeeren False
    modLaden.LogSchreiben "Neu aufbauen nach dem Löschen: " & t
    modLaden.NeueDatenLaden                           ' Abschlussmeldung kommt vom Laden
End Sub

' Laeuft gerade die Beladung eines Kollegen? Dann Hinweis statt Loeschen oder Leeren – sonst waere das Tool leer und wuerde nicht
' neu aufgebaut, und gespeichert ueberschriebe der leere Stand die Datei, in die dort gerade geladen wird.
Private Function BeladungAnderer() As Boolean
    Dim t As String
    t = modLaden.BeladungAndererText()
    If Len(t) = 0 Then Exit Function
    modTagesablauf.KartenEntfernen
    modFortschritt.HinweisZeigen "Beladung läuft", t, False
    BeladungAnderer = True
End Function

' ausgewaehlte Dateien loeschen; Ergebnis als Meldungstext
Private Function Loeschen() As String
    Dim i As Long, ok As Long, fehler As String, f As String
    For i = 0 To DateiAnzahl() - 1
        If gewaehlt(i) Then
            f = modLaden.RohdateiLoeschen(CStr(dateien(i)))
            If Len(f) = 0 Then
                ok = ok + 1
            Else
                fehler = fehler & vbLf & CStr(dateien(i)) & ": " & f
            End If
        End If
    Next i
    If modLaden.RohdatenAufSharePoint() Then
        ' SharePoint: Excel kann dort nicht loeschen – die Dateien sind im Tool ausgeblendet, der Ordner oeffnet sich im Browser
        Loeschen = IIf(ok = 1, "1 Datei", ok & " Dateien") & " im Tool ausgeblendet: sie werden nicht mehr geladen und nicht mehr angezeigt." & vbLf & _
                   "Die Rohdaten liegen auf SharePoint, dort kann Excel keine Dateien löschen. Endgültig löschen bitte im Browser – der Ordner ist geöffnet."
        On Error Resume Next
        ThisWorkbook.FollowHyperlink modLaden.RohdatenPfad()
        On Error GoTo 0
        Exit Function
    End If
    Loeschen = IIf(ok = 1, "1 Datei gelöscht.", ok & " Dateien gelöscht.")
    If Len(fehler) > 0 Then Loeschen = Loeschen & vbLf & "Diese Dateien wurden nicht gelöscht (geöffnet oder schreibgeschützt?):" & fehler
```

```
End Function

' ============================================================================ Zurücksetzen
Public Sub ZuruecksetzenZeigen()
    If Not Bereit() Then Exit Sub
    mitAnwesenheit = False
    ZuruecksetzenZeichnen
End Sub

Private Sub ZuruecksetzenZeichnen()
    Dim ws As Worksheet
    modTagesablauf.KartenEntfernen
    Set ws = ActiveSheet
    kb = 560: kh = 320
    modTagesablauf.KarteModal ws, kb, kh, kx, ky
    Ueberschrift ws, "DATEN  ·  ZURÜCKSETZEN", "Tool zurücksetzen?"
    modTagesablauf.Pille ws, "Text", kx + 28, ky + 76, kb - 56, 92, _
        "Alle geladenen Stände werden aus dem Tool entfernt: Fakten, Verlauf, Ladeprotokoll, Umbuchungen, Snapshots und die Liste der Themenpostkörbe. " & _
        "Konfiguration, SLA und GeVo-Auswahl bleiben. Die Rohdateien bleiben im Ordner – „Neue Daten laden“ liest sie danach wieder ein.", C_GRAUXL, C_SW
    Zeile ws, "Anw", kx + 28, ky + 184, kb - 56, 30, CStr(IIf(mitAnwesenheit, "[x]  ", "[  ]  ")) & "Auch die Anwesenheit leeren (Mitarbeiter und Kalender)", _
        mitAnwesenheit, "modWerkzeuge.KlickAnwesenheit"
    modTagesablauf.Knopf ws, "Leeren", kx + kb - 28 - 180, ky + kh - 50, 180, 30, "Tool zurücksetzen", 4, "modWerkzeuge.KlickLeeren"
    modTagesablauf.Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 90, 30, "Abbrechen", 3, "modWerkzeuge.KlickSchliessen"
End Sub

Public Sub KlickAnwesenheit()
    mitAnwesenheit = Not mitAnwesenheit
    Planen "ZuruecksetzenNeu"
End Sub

Public Sub ZuruecksetzenNeu()
    Application.ScreenUpdating = False
    ZuruecksetzenZeichnen
    Application.ScreenUpdating = True
End Sub

Public Sub KlickLeeren()
    Planen "Leeren"
End Sub

Public Sub Leeren()
    If BeladungAnderer() Then Exit Sub
    modTagesablauf.KartenEntfernen
    modLaden.ToolLeeren mitAnwesenheit
    EinheitKnopfBeschriften
    modFortschritt.HinweisZeigen "Zurückgesetzt", "Das Tool ist leer" & IIf(mitAnwesenheit, ", auch die Anwesenheit", "") & _
        ". „Neue Daten laden“ liest die Rohdateien wieder ein.", False
End Sub

' ============================================================================ Ältester Vorgang (Knopf „Details“ an der Kachel)
' Je Quelle der Vorgang mit dem hoechsten Rueckstandsalter am letzten Stichtag nach aktivem Filter (wie die Kachel): Ordnungsbegriff,
' LNR, VSS und Sparte, GeVo, Themenpostkorb, Einheit und Rohdatei. „In Rohdatei markieren“ oeffnet die Rohdatei und markiert die
' Zeile gelb (modLaden.VorgangMarkieren). Staende, die vor 12.64 geladen wurden, kennen den Ordnungsbegriff noch nicht – die
' Markierung sucht dann die aelteste Zeile mit VSS, Postkorb und GeVo.
Public Sub AeltesterZeigen()
    Dim ws As Worksheet, q As Variant, v As Variant, y As Single, t() As String, vorgang As String, i As Long
    If Not Bereit() Then Exit Sub
    Set ws = ActiveSheet
    kb = 640: kh = 470
    modTagesablauf.KarteModal ws, kb, kh, kx, ky
    Ueberschrift ws, "KENNZAHL  ·  ÄLTESTER VORGANG", "Ältester Vorgang im Rückstand"
    modTagesablauf.Beschriftung ws, "Text", kx + 28, ky + 70, kb - 56, 28, _
        "Je Quelle der Vorgang mit dem höchsten Rückstandsalter am letzten Stichtag – nach den aktiven Filtern, wie die Kachel.", 9, False, C_GRAU, True
    y = ky + 104
    For Each q In Array("Verbund", "OPAG")
        i = i + 1
        v = modLaden.AeltesterVorgang(CStr(q))
        modTagesablauf.Pille ws, "B" & i, kx + 28, y, kb - 56, 150, "", C_GRAUXL, C_SW
        If Not IsArray(v) Then
            modTagesablauf.Beschriftung ws, "Q" & i, kx + 44, y + 12, kb - 88, 18, UCase$(CStr(q)), 9, True, C_TEAL
            modTagesablauf.Beschriftung ws, "W" & i, kx + 44, y + 36, kb - 88, 18, "Kein Rückstand am letzten Stichtag (nach Filter).", 10, False, C_GRAU
        Else
            modTagesablauf.Beschriftung ws, "Q" & i, kx + 44, y + 12, kb - 88, 16, CStr(UCase$(CStr(q)) & "  ·  Stichtag " & Format$(v(1), "dd.mm.yyyy")), 9, True, C_TEAL
            modTagesablauf.Beschriftung ws, "W" & i, kx + 44, y + 30, kb - 88, 22, CStr(ZahlTage(CDbl(v(2))) & " alt  ·  im Rückstand seit " & _
                Format$(CDate(v(1)) - CDbl(v(2)), "dd.mm.yyyy")), 13, True, C_SW
            If Len(CStr(v(9))) > 0 Then
                t = Split(CStr(v(9)), "|")
                vorgang = "Ordnungsbegriff " & t(1)
                If UBound(t) >= 2 Then vorgang = vorgang & "  ·  LNR " & t(2)
                If UBound(t) >= 4 Then vorgang = vorgang & "  ·  Auftrag " & t(3) & "  ·  Arbeitsgut " & t(4)
            Else
                vorgang = "Ordnungsbegriff erst nach dem nächsten Laden bekannt – „In Rohdatei markieren“ findet ihn s
```

```
chon jetzt"
611             End If
612             Zeilentext ws, "V" & i, y + 58, "Vorgang", vorgang
613             Zeilentext ws, "G" & i, y + 76, "GeVo", CStr(v(6))
614             Zeilentext ws, "P" & i, y + 94, "Postkorb", CStr(CStr(v(5)) & IIf(Len(CStr(v(7))) > 0, "  ·  Einheit " & C
    Str(v(7)), ""))
615             Zeilentext ws, "S" & i, y + 112, "VSS", CStr(CStr(v(4)) & "  ·  " & CStr(v(3)) & IIf(Len(CStr(v(10))) > 0,
     "  ·  Rohdatei " & DateiKurz(CStr(v(10))), ""))
616             modTagesablauf.Knopf ws, "M" & i, kx + kb - 44 - 170, y + 10, 170, 24, "In Rohdatei markieren", 2, CStr(II
    f(CStr(q) = "OPAG", "modWerkzeuge.KlickMarkierenOPAG", "modWerkzeuge.KlickMarkierenVerbund"))
617         End If
618         y = y + 162
619     Next q
620     modTagesablauf.Knopf ws, "Abbrechen", kx + kb - 28 - 120, ky + kh - 50, 120, 30, "Schließen", 1, "modWerkzeuge.Kli
    ckSchliessen"
621 End Sub
622
623 Private Sub Zeilentext(ws As Worksheet, nm As String, y As Single, marke As String, inhalt As String)
624     modTagesablauf.Beschriftung ws, nm & "a", kx + 44, y, 80, 16, marke, 8.5, True, C_GRAU
625     modTagesablauf.Beschriftung ws, nm & "b", kx + 124, y, kb - 168, 16, inhalt, 9, False, C_SW
626 End Sub
627
628 Private Function ZahlTage(n As Double) As String
629     If n = 1 Then ZahlTage = "1 Tag" Else ZahlTage = modLaden.ZahlDE(n) & " Tage"
630 End Function
631
632 Private Function DateiKurz(rel As String) As String
633     DateiKurz = Mid$(rel, InStrRev(Replace(rel, "/", "\"), "\") + 1)
634     If Len(DateiKurz) > 40 Then DateiKurz = Left$(DateiKurz, 38) & "…"
635 End Function
636
637 Public Sub KlickMarkierenVerbund()
638     Planen "MarkierenVerbund"
639 End Sub
640
641 Public Sub KlickMarkierenOPAG()
642     Planen "MarkierenOPAG"
643 End Sub
644
645 Public Sub MarkierenVerbund()
646     Markieren "Verbund"
647 End Sub
648
649 Public Sub MarkierenOPAG()
650     Markieren "OPAG"
651 End Sub
652
653 Private Sub Markieren(quelle As String)
654     Dim t As String
655     modTagesablauf.KartenEntfernen
656     t = modLaden.VorgangMarkieren(quelle)
657     If Len(t) > 0 Then modFortschritt.HinweisZeigen "Ältester Vorgang", t, True
658 End Sub
659
660 ' ============================================================================ Ältere Datei aktualisieren (12.93)
661 ' Karte zu modUpdate: findet die zusaetzlich geoeffnete aeltere Master-Datei, zeigt Datei, Version und Stand und was p
    assiert;
662 ' „Aktualisieren und ersetzen“ uebernimmt die Daten und speichert diese Datei unter dem Namen und am Ort der alten.
663
664 Public Sub UpdateZeigen()
665     Dim ws As Worksheet, info As String, problem As String
666     If Not Bereit() Then Exit Sub
667     updateName = ""
668     problem = modUpdate.AlteDateiPruefen(updateName, info)
669     If Len(problem) > 0 Then
670         modFortschritt.HinweisZeigen "Ältere Datei aktualisieren", problem, True
671         Exit Sub
672     End If
673     Set ws = ActiveSheet
674     kb = 600: kh = 380
675     modTagesablauf.KarteModal ws, kb, kh, kx, ky
676     Ueberschrift ws, "WERKZEUG  ·  AKTUALISIEREN", "Ältere Datei auf Version " & modUpdate.VersionVon(ThisWorkbook) &
    " bringen?"
677     modTagesablauf.Pille ws, "Datei", kx + 28, ky + 76, kb - 56, 48, info, C_GRAUXL, C_SW
678     modTagesablauf.Pille ws, "Text", kx + 28, ky + 134, kb - 56, 168, _
679         "Diese Datei übernimmt alle Daten der älteren Datei – Stände, Ladeprotokoll, Anwesenheit, Standard, SLA und Ei
    nstellungen – " & _
680         "und speichert sich danach unter deren Namen an deren Ort. Die ältere Datei wird dabei ersetzt; Verknüpfungen
    anderer Dateien " & _
681         "(z. B. HR Sales Dashboard) gelten weiter, nichts muss neu geladen werden." & vbLf & vbLf & _
682         "Vorher wird die ältere Datei gesichert (mit Dateipfad im Ordner Protokolle\Sicherung daneben, auf SharePoint
    im Versionsverlauf). " & _
683         "Niemand sonst darf sie gerade geöffnet haben.", C_GRAUXL, C_SW
684     modTagesablauf.Knopf ws, "Update", kx + kb - 28 - 210, ky + kh - 50, 210, 30, "Aktualisieren und ersetzen", 1, "mo
    dWerkzeuge.KlickUpdate"
685     modTagesablauf.Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 90, 30, "Abbrechen", 3, "modWerkzeuge.KlickSchliessen
    "
686 End Sub
687
688 Public Sub KlickUpdate()
689     Planen "UpdateAusfuehren"
690 End Sub
691
692 Public Sub UpdateAusfuehren()
693     Dim text As String, fehler As Boolean
694     If Len(updateName) = 0 Then Exit Sub
```

```
695      modTagesablauf.KartenEntfernen
696      Application.StatusBar = "Daten der älteren Datei werden übernommen …"
697      text = modUpdate.Ausfuehren(updateName, fehler)
698      Application.StatusBar = False
699      updateName = ""
700      modFortschritt.HinweisZeigen IIf(fehler, "Aktualisieren nicht abgeschlossen", "Aktualisiert"), text, fehler
701  End Sub
```

modStandard.bas

```
Option Explicit
' ================================================================================
' modStandard – Karte "Was gehört zum Standard?" (Knopf "Standard festlegen …" im Cockpit)
'
' Zeigt alle bekannten Themenpostkoerbe (Filter!B5:B44) und GeVo-Typen (Filter!G5:G18) als Chips; ein Klick schaltet
' "im Standard" an oder ab. Gespeichert wird als ja/nein in der Konfiguration (I11:I50 zeilengleich mit Filter!B5:B44,
' L11:L24 zeilengleich mit Filter!G5:G18). Der Standard gilt fuer den Wochenreport (Rohdatenblatt, Fakten!Q) und fuer
' "Auf Standard zurücksetzen". "Speichern und anwenden" stellt zusaetzlich die Chips im Cockpit auf den Standard.
' Zeichnet mit den Bausteinen aus modTagesablauf (Praefix TAGW_ zum Aufraeumen).
' ================================================================================

Private Const PRAEFIX As String = "TAGW_"
Private Const C_TEAL As Long = 8220427              ' RGB(11, 111, 125)
Private Const C_SW As Long = 1710618                ' RGB(26, 26, 26)
Private Const C_GRAU As Long = 7039851              ' RGB(107, 107, 107)
Private Const C_GRAUL As Long = 14341584            ' RGB(208, 213, 218)

Private pkName() As String, pkZeile() As Long, pkAn() As Boolean, nPk As Long
Private gvName() As String, gvAn() As Boolean, nGv As Long
Private kx As Single, ky As Single, kb As Single, kh As Single
Private yPk As Single, yGv As Single
Private klickArt As String, klickNr As Long
Private nurNeue As Boolean                          ' Karte nach dem Laden: nur die neuen Postkoerbe ausserhalb HS

Public Function StandardVersion() As String
    StandardVersion = "1"
End Function

' ------------------------------------------------------------------ Einstieg
Public Sub Zeigen(Optional neueNachLaden As Boolean = False)
    Dim ws As Worksheet, wsF As Worksheet, wsK As Worksheet, r As Long, zP As Long, zG As Long
    If modLaden.LadenLaeuft() Then Exit Sub
    nurNeue = neueNachLaden
    On Error GoTo Ersatz
    modTagesablauf.KartenEntfernen
    Set wsF = ThisWorkbook.Worksheets("Filter")
    Set wsK = ThisWorkbook.Worksheets("Konfiguration")
    ' Listen und aktuellen Standard lesen
    nPk = 0: ReDim pkName(1 To 40): ReDim pkZeile(1 To 40): ReDim pkAn(1 To 40)
    For r = 0 To 39
        If Len(Trim$(CStr(wsF.Cells(5 + r, 2).Value))) > 0 And (Not nurNeue Or modLaden.IstNeuerPostkorb(CStr(wsF.Cells(5 + r, 2).Value))) Then
            nPk = nPk + 1
            pkName(nPk) = CStr(wsF.Cells(5 + r, 2).Value)
            pkZeile(nPk) = r
            pkAn(nPk) = (LCase$(Trim$(CStr(wsK.Cells(11 + r, 9).Value))) = "ja")
        End If
    Next r
    nGv = 0: ReDim gvName(1 To 14): ReDim gvAn(1 To 14)
    For r = 0 To IIf(nurNeue, -1, 13)
        nGv = nGv + 1
        gvName(nGv) = IIf(r = 13, "Alle übrigen GeVo-Typen", CStr(wsF.Cells(5 + r, 7).Value))
        If Len(gvName(nGv)) = 0 Then gvName(nGv) = "(frei)"
        gvAn(nGv) = (LCase$(Trim$(CStr(wsK.Cells(11 + r, 12).Value))) <> "nein")
    Next r

    ThisWorkbook.Worksheets("Cockpit").Activate
    Set ws = ActiveSheet
    zP = (nPk + 3) \ 4: If zP = 0 Then zP = 1
    zG = (nGv + 2) \ 3
    kb = 660
    kh = 126 + zP * 30 + 30 + zG * 30 + 76
    If nurNeue Then kh = 126 + zP * 30 + 70
    modTagesablauf.KarteModal ws, kb, kh, kx, ky
    yPk = ky + 126
    yGv = yPk + zP * 30 + 30
    modTagesablauf.Beschriftung ws, "Schritt", kx + 28, ky + 20, kb - 56, 14, "STANDARD-AUSWAHL", 8, True, C_TEAL
    If nurNeue Then
        modTagesablauf.Beschriftung ws, "Frage", kx + 28, ky + 40, kb - 56, 26, "Neue Themenpostkörbe in den Rohdaten", 15, True, C_SW
        modTagesablauf.Beschriftung ws, "Text", kx + 28, ky + 68, kb - 56, 32, _
            "Diese Postkörbe gehören nicht zu HS und zählen erst einmal alle zum Standard. Welche sollen nicht betrachtet werden? " & _
            "Anklicken nimmt sie heraus (weiß = nicht im Standard).", 9.5, False, C_GRAU, True
    Else
        modTagesablauf.Beschriftung ws, "Frage", kx + 28, ky + 40, kb - 56, 26, "Was gehört zum Standard?", 15, True, C_SW
        modTagesablauf.Beschriftung ws, "Text", kx + 28, ky + 68, kb - 56, 32, _
            "Der Standard gilt für den Wochenreport (Blatt Rohdatenblatt) und für „Auf Standard zurücksetzen“ im Cockpit. " & _
            "Anklicken schaltet an oder ab; gefüllt = gehört zum Standard.", 9.5, False, C_GRAU, True
    End If
    modTagesablauf.Beschriftung ws, "LabelPK", kx + 28, yPk - 18, kb - 56, 14, "THEMENPOSTKÖRBE", 8, True, C_GRAU
    If Not nurNeue Then modTagesablauf.Beschriftung ws, "LabelGV", kx + 28, yGv - 18, kb - 56, 14, "GEVO-TYPEN", 8, True, C_GRAU
    For r = 1 To nPk
        ChipZeichnen ws, "P", r
    Next r
    For r = 1 To nGv
        ChipZeichnen ws, "G", r
    Next r
    If nPk = 0 Then
        modTagesablauf.Beschriftung ws, "KeinePK", kx + 28, yPk, kb - 56, 24, "Noch keine Themenpostkörbe bekannt – sie erscheinen nach dem ersten Laden.", 9, False, C_GRAU
```

```
    End If
    If nurNeue Then
        modTagesablauf.Knopf ws, "Anwenden", kx + kb - 28 - 190, ky + kh - 50, 190, 30, "Speichern und weiter", 1, "modStandard.KlickAnwenden"
        modTagesablauf.Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 150, 30, "Alle behalten", 3, "modStandard.KlickAbbrechen"
    Else
        modTagesablauf.Knopf ws, "Anwenden", kx + kb - 28 - 190, ky + kh - 50, 190, 30, "Speichern und anwenden", 1, "modStandard.KlickAnwenden"
        modTagesablauf.Knopf ws, "Speichern", kx + kb - 28 - 190 - 12 - 120, ky + kh - 50, 120, 30, "Nur speichern", 2, "modStandard.KlickSpeichern"
        modTagesablauf.Knopf ws, "Abbrechen", kx + 20, ky + kh - 50, 90, 30, "Abbrechen", 3, "modStandard.KlickAbbrechen"
        modTagesablauf.Knopf ws, "AlleGV", kx + 114, ky + kh - 50, 110, 30, "Alle GeVo an", 3, "modStandard.KlickAlleGeVo"
    End If
    Exit Sub
Ersatz:
    On Error Resume Next
    modTagesablauf.KartenEntfernen
    ThisWorkbook.Worksheets("Konfiguration").Activate
    ThisWorkbook.Worksheets("Konfiguration").Range("I11").Select
    MsgBox "Die Standard-Auswahl steht in der Konfiguration, Spalten H bis L: je Themenpostkorb und GeVo-Typ ja oder nein.", vbInformation, "Standard festlegen"
End Sub

' Chip: gefuellt (petrol, weisse Schrift) = im Standard, weiss mit Rand = nicht im Standard
Private Sub ChipZeichnen(ws As Worksheet, art As String, i As Long)
    Dim x As Single, y As Single, b As Single, h As Single, an As Boolean, t As String, nm As String
    h = 24
    If art = "P" Then
        b = (kb - 56 - 3 * 8) / 4
        x = kx + 28 + ((i - 1) Mod 4) * (b + 8)
        y = yPk + ((i - 1) \ 4) * 30
        an = pkAn(i): t = pkName(i)
    Else
        b = (kb - 56 - 2 * 8) / 3
        x = kx + 28 + ((i - 1) Mod 3) * (b + 8)
        y = yGv + ((i - 1) \ 3) * 30
        an = gvAn(i): t = gvName(i)
    End If
    nm = PRAEFIX & "S_" & art & Format(i, "00")
    On Error Resume Next
    ws.Shapes(nm).Delete
    On Error GoTo 0
    With ws.Shapes.AddShape(5, x, y, b, h)
        .Name = nm
        .Adjustments(1) = 0.5
        If an Then
            .Fill.ForeColor.RGB = C_TEAL
            .Line.Visible = msoFalse
        Else
            .Fill.ForeColor.RGB = RGB(255, 255, 255)
            .Line.ForeColor.RGB = C_GRAUL
            .Line.Weight = 1
        End If
        .OnAction = "modStandard.KlickChip"
        With .TextFrame2
            .MarginLeft = 8: .MarginRight = 8: .MarginTop = 0: .MarginBottom = 0
            .WordWrap = msoFalse
            .VerticalAnchor = msoAnchorMiddle
            .TextRange.Text = t
            .TextRange.Font.Name = "Liberation Sans"
            .TextRange.Font.Size = 8.5
            .TextRange.Font.Bold = IIf(an, msoTrue, msoFalse)
            .TextRange.Font.Fill.ForeColor.RGB = IIf(an, RGB(255, 255, 255), C_GRAU)
            .TextRange.ParagraphFormat.Alignment = msoAlignCenter
        End With
    End With
End Sub

' ------------------------------------------------------------- Klicks
Public Sub KlickChip()
    Dim nm As String, p As Long
    On Error Resume Next
    nm = CStr(Application.Caller)
    On Error GoTo 0
    p = InStr(nm, PRAEFIX & "S_")
    If p = 0 Then Exit Sub
    klickArt = Mid$(nm, p + Len(PRAEFIX) + 2, 1)
    klickNr = CLng(Val(Mid$(nm, p + Len(PRAEFIX) + 3, 2)))
    ' die geklickte Form nicht im eigenen Makro loeschen: Neuzeichnen gleich danach
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modStandard.Umschalten"
End Sub

Public Sub Umschalten()
    If klickArt = "P" And klickNr >= 1 And klickNr <= nPk Then
        pkAn(klickNr) = Not pkAn(klickNr)
    ElseIf klickArt = "G" And klickNr >= 1 And klickNr <= nGv Then
        gvAn(klickNr) = Not gvAn(klickNr)
    Else
        Exit Sub
    End If
    Application.ScreenUpdating = False
    ChipZeichnen ActiveSheet, klickArt, klickNr
    Application.ScreenUpdating = True
```

```
End Sub

Public Sub KlickAlleGeVo()
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modStandard.AlleGeVo"
End Sub

Public Sub AlleGeVo()
    Dim i As Long
    Application.ScreenUpdating = False
    For i = 1 To nGv
        gvAn(i) = True
        ChipZeichnen ActiveSheet, "G", i
    Next i
    Application.ScreenUpdating = True
End Sub

Public Sub KlickAbbrechen()
    If nurNeue Then                                 ' "Alle behalten": Standard bleibt "ja", im Tagesablauf weiter
        Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modStandard.NeueFertig"
    Else
        Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modTagesablauf.KartenEntfernen"
    End If
End Sub

Public Sub NeueFertig()
    nurNeue = False
    modLaden.NeuePostkoerbeErledigt
    modTagesablauf.KartenEntfernen
    modTagesablauf.NachStandardWeiter
End Sub

Public Sub KlickSpeichern()
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modStandard.Speichern"
End Sub

Public Sub KlickAnwenden()
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modStandard.SpeichernUndAnwenden"
End Sub

Public Sub Speichern()
    StandardSchreiben
    modTagesablauf.KartenEntfernen
    modFortschritt.HinweisZeigen "Standard gespeichert", "Der Wochenreport (Blatt Rohdatenblatt) rechnet ab jetzt mit
   dieser Auswahl. " & _
        "Die Chips im Cockpit bleiben, wie sie sind – „Auf Standard zurücksetzen“ übernimmt den Standard in die Ansich
   t.", False
End Sub

Public Sub SpeichernUndAnwenden()
    StandardSchreiben
    modTagesablauf.KartenEntfernen
    modCockpit.StandardAnwenden
    If nurNeue Then NeueFertig
End Sub

Private Sub StandardSchreiben()
    Dim wsK As Worksheet, i As Long
    Set wsK = ThisWorkbook.Worksheets("Konfiguration")
    For i = 1 To nPk
        wsK.Cells(11 + pkZeile(i), 9).Value = IIf(pkAn(i), "ja", "nein")
    Next i
    For i = 1 To nGv
        wsK.Cells(10 + i, 12).Value = IIf(gvAn(i), "ja", "nein")
    Next i
    modLaden.LogSchreiben "Standard-Auswahl gespeichert"
End Sub
```

modPPT.bas

```
Option Explicit
' ================================================================================
' modPPT – Kacheln des Blatts "PPT" als Formen auf eine Folie der ERGO-Vorlage kopieren
' Ablauf: Diagramme aktualisieren, PowerPoint starten, Vorlage (Vorlage\ERGO_Vorlage.pptx neben der Master-Datei)
' als unbenannte Kopie oeffnen (sonst leere Praesentation), Folie mit Layout "Nur Überschrift" (oder "Only Title")
' anlegen, Titel und Untertitel aus PPT!BA1:BA2 (Untertitel in die Kopfzeile ueber dem Titel, falls das Layout eine
' hat), je gewaehlter Kachel (Auswahl aus modFolienauswahl, "111111") die Kachel unter dem Titel aufbauen: Textfelder
' in PowerPoint nachbauen, Diagramme einzeln einfuegen (sonst als Bild), Kopie im Ordner "Praesentationen" sichern.
' ================================================================================

Private Const RAND_SEITE As Single = 41        ' Seitenrand der ERGO-Vorlage (Punkte)
Private Const INHALT_OBEN As Single = 108      ' Oberkante der Inhaltsflaeche unter dem Titel
Private Const RAND_UNTEN As Single = 22
Private Const LUECKE As Single = 14            ' Abstand der Kacheln auf der Folie
Private Const KACHEL_B As Single = 283         ' Kachelraster im Blatt PPT (Punkte, wie im Build)
Private Const KACHEL_H As Single = 195
Private Const KACHEL_LUECKE As Single = 14
Private Const KACHEL_KOPF As Single = 30       ' Kopfleiste: Titel, darunter die Kennzahl; darunter das Diagramm
Private Const FUSS_H As Single = 13            ' Fussnote am unteren Rand der Kachel (liegt ueber dem Diagramm)

Public Sub NachPowerPoint(Optional auswahl As String = "111111")
    If modLaden.LadenLaeuft() Then Exit Sub
    Dim app As Object, prs As Object, sld As Object, lay As Object, grp As Object, shp As Object
    Dim ws As Worksheet, vorlage As String, namen As Variant, i As Long, n As Long, basis As String, ziel As String
    Dim schritt As String
    Dim links As Single, oben As Single, breite As Single, hoehe As Single, faktor As Single
    Dim gruppen() As Object, anz As Long, t As Long, k As Long, basisL As Single, basisT As Single
    Dim spalten As Long, reihen As Long, r As Long, c As Long, inReihe As Long, w0 As Single, h0 As Single
    Dim exB(1 To 6) As Single, exH(1 To 6) As Single, minL As Single, minT As Single, maxR As Single, maxB As Single, vorher As Long
    Dim alsBild As Long, kNr(1 To 6) As Long, exL(1 To 6) As Single, exT(1 To 6) As Single, s As Shape, durchgang As Long
    Dim wertText As String, wertGroesse As Single
    Dim zielL As Single, zielT As Single, fl As Single, ft As Single, fw As Single, fh As Single, nDia As Long, nText As Long

    Set ws = ThisWorkbook.Worksheets("PPT")
    schritt = "Vorbereitung"
    On Error Resume Next
    modCockpit.Aktualisieren True                  ' inkl. Diagramme des Blatts PPT
    On Error GoTo 0
    On Error GoTo Fehler
    basis = modLaden.BasisOrdner()                 ' lokaler Ordner der Master-Datei, auf SharePoint ohne OneDrive die https-Adresse
    If Len(basis) = 0 Then Err.Raise vbObjectError + 5, , "Der Ordner der Master-Datei ist nicht bekannt (Konfiguration!B7)."

    On Error Resume Next
    Set app = GetObject(, "PowerPoint.Application")
    If app Is Nothing Then Set app = CreateObject("PowerPoint.Application")
    On Error GoTo 0
    If app Is Nothing Then
        modFortschritt.HinweisZeigen "Fehler", "PowerPoint konnte nicht gestartet werden.", True
        Exit Sub
    End If
    app.Visible = True

    On Error GoTo Fehler
    schritt = "Vorlage öffnen"
    Set prs = Nothing
    If modLaden.IstUrl(basis) Then
        ' SharePoint: Vorlage direkt ueber https oeffnen (PowerPoint nutzt die Office-Anmeldung); fehlt sie, leere Praesentation
        vorlage = basis & "/Vorlage/ERGO_Vorlage.pptx"
        On Error Resume Next
        Set prs = app.Presentations.Open(vorlage, False, True, True)
        On Error GoTo Fehler
    Else
        vorlage = basis & Application.PathSeparator & "Vorlage" & Application.PathSeparator & "ERGO_Vorlage.pptx"
        If Len(Dir(vorlage)) > 0 Then Set prs = app.Presentations.Open(vorlage, False, True, True)     ' ReadOnly=False, Untitled=True (Kopie), WithWindow=True
    End If
    If prs Is Nothing Then Set prs = app.Presentations.Add
    Set lay = Nothing
    For i = 1 To prs.SlideMaster.CustomLayouts.Count
        If LCase$(prs.SlideMaster.CustomLayouts(i).Name) = "only title" Then Set lay = prs.SlideMaster.CustomLayouts(i)
    Next i
    If lay Is Nothing Then
        For i = 1 To prs.SlideMaster.CustomLayouts.Count
            If InStr(1, LCase$(prs.SlideMaster.CustomLayouts(i).Name), "title only") > 0 Or InStr(1, LCase$(prs.SlideMaster.CustomLayouts(i).Name), "berschrift") > 0 Then Set lay = prs.SlideMaster.CustomLayouts(i)
        Next i
    End If
    If lay Is Nothing Then Set lay = prs.SlideMaster.CustomLayouts(1)
    schritt = "Folie anlegen"
    Set sld = prs.Slides.AddSlide(prs.Slides.Count + 1, lay)
    schritt = "Titel setzen"

    ' Titel und Untertitel in die Platzhalter (Datum, Fusszeile, Foliennummer der Vorlage bleiben unberuehrt)
    For Each shp In sld.Shapes
        If shp.Type = 14 Then                                          ' msoPlaceholder
            Select Case shp.PlaceholderFormat.Type
                Case 1, 3                                              ' ppPlaceholderTitle / ppPlaceholderCenterTitle
```

```
                    shp.TextFrame.TextRange.Text = CStr(ws.Range("BA1").Value)
                Case 2, 4                                   ' ppPlaceholderBody / ppPlaceholderSubtitle
                    shp.TextFrame.TextRange.Text = CStr(ws.Range("BA2").Value)
            End Select
        End If
    Next shp

    ' Kacheln sammeln: je gewaehlter Kachel ihre Formen (Kopf, Kennzahl, Diagramm, Fussnote) und ihre Masse in Excel
    schritt = "Kacheln sammeln"
    If Len(auswahl) <> 6 Then auswahl = "111111"
    anz = 0
    For t = 0 To 5
        If Mid$(auswahl, t + 1, 1) = "1" Then
            k = 0
            maxB = -1
            For i = 1 To ws.Shapes.Count
                If KachelVon(ws.Shapes(i), ws) = t Then
                    k = k + 1
                    If Not ws.Shapes(i).HasChart Then               ' unterstes Textfeld = Fussnote
                        If ws.Shapes(i).Top > maxB Then maxB = ws.Shapes(i).Top
                    End If
                End If
            Next i
            If k > 0 Then
                anz = anz + 1
                kNr(anz) = t: exT(anz) = maxB
            End If
        End If
    Next t
    If anz = 0 Then Err.Raise vbObjectError + 7, , "Keine Kachel ausgewählt oder im Blatt PPT keine Formen gefunden."

    ' Anordnen: 1-3 Kacheln in einer Reihe, 4 als 2 x 2, 5-6 in zwei Reihen zu drei; auf die Inhaltsflaeche skalieren
    ' (hoechstens 1,6-fach vergroessern), jede Reihe mittig
    schritt = "Kacheln anordnen"
    breite = prs.PageSetup.SlideWidth - 2 * RAND_SEITE
    hoehe = prs.PageSetup.SlideHeight - INHALT_OBEN - RAND_UNTEN
    If anz <= 3 Then
        spalten = anz
    ElseIf anz = 4 Then
        spalten = 2
    Else
        spalten = 3
    End If
    reihen = (anz + spalten - 1) \ spalten
    ' Masse aus dem Raster des Builds, nicht aus Excel: Excel misst Spaltenbreiten in Zeichen der Standardschrift, die
    ' Formen im Blatt sind dadurch je nach Rechner breiter oder flacher als geplant (Kacheln gestaucht, Etiketten im
    ' Diagramm ueberdeckt). Auf der Folie bekommt jedes Teil seinen festen Platz in der Kachel.
    w0 = KACHEL_B: h0 = KACHEL_H
    faktor = (breite - (spalten - 1) * LUECKE) / (spalten * w0)
    If (hoehe - (reihen - 1) * LUECKE) / (reihen * h0) < faktor Then faktor = (hoehe - (reihen - 1) * LUECKE) / (reihe
n * h0)
    If faktor > 1.6 Then faktor = 1.6

    ' Kacheln auf der Folie aufbauen. Nicht mehr als Formengruppe ueber die Zwischenablage: PowerPoint hat die Gruppe
je
    ' nach Version als eingebettetes Excel-Objekt eingefuegt, das auf der Folie leer bleibt. Stattdessen:
    ' Textfelder werden in PowerPoint neu angelegt (Text, Schrift, Farbe, Fuellung wie in Excel), jedes Diagramm wird
    ' einzeln kopiert und als Diagramm eingefuegt; kommt dabei kein Diagramm an, wird es als Bild (PNG-Export) gesetzt
.
    For t = 1 To anz
        r = (t - 1) \ spalten
        c = (t - 1) Mod spalten
        inReihe = spalten
        If r = reihen - 1 Then inReihe = anz - r * spalten
        zielL = (prs.PageSetup.SlideWidth - (inReihe * w0 * faktor + (inReihe - 1) * LUECKE)) / 2 + c * (w0 * faktor +
LUECKE)
        zielT = INHALT_OBEN + r * (h0 * faktor + LUECKE)
        schritt = "Kachel " & (kNr(t) + 1) & " aufbauen"
        vorher = sld.Shapes.Count
        nDia = 0: nText = 0
        ' Kennzahl der Kopfleiste: kommt als zweite Zeile in das Titelfeld (zwei getrennte Felder lagen in PowerPoint
        ' versetzt, die Kennzahl klebte am unteren Rand der Kopfleiste)
        wertText = "": wertGroesse = 8
        On Error Resume Next
        For i = 1 To ws.Shapes.Count
            Set s = ws.Shapes(i)
            If KachelVon(s, ws) = kNr(t) Then
                If Not s.HasChart And Not s.Fill.Visible And s.Top < exT(t) Then
                    wertText = s.TextFrame2.TextRange.Text
                    wertGroesse = s.TextFrame2.TextRange.Font.Size
                End If
            End If
        Next i
        On Error GoTo Fehler
        ' zuerst die Diagramme (unten), dann die Textfelder (oben), wie im Blatt
        For durchgang = 1 To 2
            For i = 1 To ws.Shapes.Count
                Set s = ws.Shapes(i)
                If KachelVon(s, ws) = kNr(t) Then
                    fl = zielL: fw = KACHEL_B * faktor
                    If s.HasChart Then                          ' Diagramm unter der Kopfleiste
                        ft = zielT + KACHEL_KOPF * faktor: fh = (KACHEL_H - KACHEL_KOPF) * faktor
                    ElseIf s.Fill.Visible Then                  ' Kopfleiste mit Titel
                        ft = zielT: fh = KACHEL_KOPF * faktor
                    ElseIf s.Top >= exT(t) Then                 ' Fussnote
                        ft = zielT + (KACHEL_H - FUSS_H) * faktor: fh = FUSS_H * faktor
```

```
                    Else                                                ' Kennzahl: steht schon im Titelfeld
                        ft = -1
                    End If
                    If durchgang = 1 And s.HasChart Then
                        DiagrammSetzen sld, s, fl, ft, fw, fh, alsBild, basis
                        nDia = nDia + 1
                    ElseIf durchgang = 2 And Not s.HasChart And ft >= 0 Then
                        If s.Fill.Visible Then
                            TextfeldSetzen sld, s, fl, ft, fw, fh, wertText, wertGroesse
                        Else
                            TextfeldSetzen sld, s, fl, ft, fw, fh
                        End If
                        nText = nText + 1
                    End If
                End If
            Next i
        Next durchgang
        If sld.Shapes.Count <= vorher Then Err.Raise vbObjectError + 9, , "Kachel " & (kNr(t) + 1) & ": PowerPoint hat nichts eingefügt."
        modLaden.LogSchreiben "PPT Kachel " & (kNr(t) + 1) & ": " & nDia & " Diagramm(e), " & nText & " Textfelder, " & _
                              Format(w0 * faktor, "0") & " x " & Format(h0 * faktor, "0") & " pt"
    Next t
    Application.CutCopyMode = False
    ' Der Vertraulich-Vermerk kommt aus dem ERGO-Folienmaster, nichts zusaetzlich einfuegen

    ' Diagnose: was liegt auf der Folie? (Ladelog.txt und Abschlussmeldung) – sichtbar, auf der Folie, mit Groesse
    schritt = "Folie prüfen"
    Dim nForm As Long, nSichtbar As Long, info As String
    On Error Resume Next
    For Each shp In sld.Shapes
        If shp.Type <> 14 Then                                          ' ohne Platzhalter
            nForm = nForm + 1
            If shp.Visible And shp.Width > 5 And shp.Height > 5 And shp.Left < prs.PageSetup.SlideWidth And shp.Top < prs.PageSetup.SlideHeight _
               And shp.Left + shp.Width > 0 And shp.Top + shp.Height > 0 Then nSichtbar = nSichtbar + 1
            If shp.Type = 7 Or shp.Type = 10 Then nSichtbar = nSichtbar - 1          ' eingebettetes Excel-Objekt zeigt nichts
            modLaden.LogSchreiben "PPT Form " & shp.Name & ": Typ " & shp.Type & ", sichtbar " & shp.Visible & ", " & Format(shp.Left, "0") & "/" & _
                                  Format(shp.Top, "0") & " " & Format(shp.Width, "0") & " x " & Format(shp.Height, "0")
        End If
    Next shp
    On Error GoTo Fehler
    info = anz & IIf(anz = 1, " Kachel", " Kacheln") & " kopiert, " & nForm & " Formen auf der Folie, davon " & nSichtbar & " sichtbar im Folienbereich." & _
           IIf(alsBild > 0, " " & alsBild & IIf(alsBild = 1, " Diagramm", " Diagramme") & " als Bild eingefügt (PowerPoint nahm es nicht als Diagramm an).", "")
    modLaden.LogSchreiben "PPT Ergebnis: " & info
    If nSichtbar = 0 Then Err.Raise vbObjectError + 11, , "Auf der Folie ist keine Kachel sichtbar (" & info & "). Details in Ladelog.txt."

    ' Kopie sichern (PowerPoint bleibt mit der Praesentation offen)
    schritt = "Kopie sichern"
    ziel = KopieSichern(prs, basis, "BUE_Reporting_" & Format(Date, "yyyymmdd") & "_" & Format(Time, "hhnn") & ".pptx")
    app.Activate
    modFortschritt.HinweisZeigen "Hinweis", "Folie erzeugt und gesichert:" & vbCrLf & ziel & vbCrLf & info, False
    Exit Sub

Fehler:
    Dim fehlertext As String
    fehlertext = "Fehler " & Err.Number & ": " & Err.Description
    modFortschritt.HinweisZeigen "Fehler", "PowerPoint-Export fehlgeschlagen bei Schritt '" & schritt & "': " & fehlertext, True
End Sub

' Kopie der Praesentation sichern: Ordner Praesentationen neben der Master-Datei. Auf SharePoint (https) zuerst dort hinein
' (Ordner muss bestehen), sonst in „Dokumente“ des Benutzers. Ergebnis: Pfad der Kopie.
Private Function KopieSichern(prs As Object, basis As String, nm As String) As String
    Dim ziel As String, dok As String
    If modLaden.IstUrl(basis) Then
        ziel = basis & "/Praesentationen/" & nm
        On Error Resume Next
        prs.SaveCopyAs ziel
        If Err.Number = 0 Then KopieSichern = ziel: Exit Function
        Err.Clear
        dok = CreateObject("WScript.Shell").SpecialFolders("MyDocuments")
        If Len(dok) = 0 Then dok = Environ$("USERPROFILE")
        On Error GoTo 0
        ziel = dok & Application.PathSeparator & nm
    Else
        ziel = basis & Application.PathSeparator & "Praesentationen"
        If Len(Dir(ziel, vbDirectory)) = 0 Then MkDir ziel
        ziel = ziel & Application.PathSeparator & nm
    End If
    prs.SaveCopyAs ziel
    KopieSichern = ziel
End Function

' Kachel (0..5) einer Form im Blatt PPT, -1 = keine Kachel. Jede Kachel hat genau ein Diagramm ("Chart 10" bis "Chart 15",
' Kachel = Nummer - 10). Ein Textfeld gehoert zu dem Diagramm, ueber dem es liegt (Mitte waagerecht im Diagramm) und
' dessen Kopfbereich oder Flaeche es senkrecht trifft. Gemessen an den echten Diagrammen in Excel – nicht am geplanten
```

```
' Raster: Excel verzerrt das Blatt je nach Standardschrift, sonst rutschen Kopfleisten in die Kachel darueber.
Private Function KachelVon(shp As Shape, ws As Worksheet) As Long
    Dim cx As Single, cy As Single, nr As Long, co As ChartObject, kopf As Single
    KachelVon = -1
    On Error Resume Next
    If shp.HasChart Then
        If shp.Name Like "Chart 1#" Then
            nr = CLng(Mid$(shp.Name, 7)) - 10
            If nr >= 0 And nr <= 5 Then KachelVon = nr
        End If
        Exit Function
    End If
    cx = shp.Left + shp.Width / 2
    cy = shp.Top + shp.Height / 2
    For nr = 0 To 5
        Set co = Nothing
        Set co = ws.ChartObjects("Chart " & (10 + nr))
        If Not co Is Nothing Then
            kopf = co.Height * KACHEL_KOPF / (KACHEL_H - KACHEL_KOPF)              ' Kopfleiste im selben Massstab wie
    das Diagramm
            If cx >= co.Left And cx <= co.Left + co.Width And cy >= co.Top - kopf - 2 And cy <= co.Top + co.Height The
   n
                KachelVon = nr
                Exit Function
            End If
        End If
    Next nr
End Function

' Etiketten von Punkten ohne Wert (#NV) abschalten: Excel blendet sie aus, PowerPoint schreibt "#NV"
Private Sub EtikettenLeererPunkte(ch As Object)
    Dim s As Object, v As Variant, i As Long
    On Error Resume Next
    For Each s In ch.SeriesCollection
        If s.HasDataLabels Then
            v = s.Values
            For i = LBound(v) To UBound(v)
                If IsError(v(i)) Or IsEmpty(v(i)) Then s.Points(i).HasDataLabel = False
            Next i
        End If
    Next s
    On Error GoTo 0
End Sub

' Ein Diagramm des Blatts PPT einzeln kopieren und als PowerPoint-Diagramm einfuegen (#NV-Etiketten weg, Verknuepfung
' zur Master-Datei geloest). Kommt kein Diagramm an (Bild, eingebettetes Objekt), das Diagramm als PNG exportieren und
' als Bild setzen – dann ist es nicht bearbeitbar, aber sicher sichtbar.
Private Sub DiagrammSetzen(sld As Object, s As Shape, l As Single, t As Single, w As Single, h As Single, ByRef alsBil
   d As Long, basis As String)
    Dim rng As Object, x As Object, datei As String, ok As Boolean
    On Error Resume Next
    Err.Clear
    s.Chart.ChartArea.Copy
    If Err.Number <> 0 Then Err.Clear: s.Copy
    DoEvents
    Set rng = Nothing
    Set rng = sld.Shapes.Paste
    If rng Is Nothing Then Set rng = sld.Shapes.PasteSpecial(0)
    If Not rng Is Nothing Then
        Set x = rng.Item(1)
        ok = (rng.Count = 1 And x.HasChart)
        If Not ok Then
            modLaden.LogSchreiben "PPT " & s.Name & ": kam als Typ " & x.Type & " an, nicht als Diagramm – als Bild ge
    setzt"
            rng.Delete
            Set x = Nothing
        End If
    Else
        modLaden.LogSchreiben "PPT " & s.Name & ": Einfügen fehlgeschlagen – als Bild gesetzt"
    End If
    If x Is Nothing Then
        datei = Environ$("TEMP")
        If Len(datei) = 0 Then datei = basis
        datei = datei & Application.PathSeparator & "BUE_PPT_" & Replace(s.Name, " ", "_") & ".png"
        Err.Clear
        s.Chart.Export datei, "PNG"
        If Err.Number = 0 And Len(Dir(datei)) > 0 Then
            Set x = sld.Shapes.AddPicture(datei, False, True, l, t, w, h)
            Kill datei
            alsBild = alsBild + 1
        End If
        On Error GoTo 0
        Exit Sub
    End If
    With x
        .LockAspectRatio = msoFalse
        .Left = l: .Top = t: .Width = w: .Height = h
    End With
    EtikettenLeererPunkte x.Chart
    x.Chart.ChartData.BreakLink
    On Error GoTo 0
End Sub

' Ein Textfeld des Blatts PPT in PowerPoint nachbauen: Text, Schrift (Name, Groesse, fett, kursiv, Farbe), Fuellung,
' Ausrichtung, Innenraender. zeile2: zweite Zeile (Kennzahl der Kopfleiste), normal statt fett, beide Zeilen mittig
Private Sub TextfeldSetzen(sld As Object, s As Shape, l As Single, t As Single, w As Single, h As Single, _
```

```
                          Optional zeile2 As String = "", Optional groesse2 As Single = 8)
    Dim x As Object, txt As String
    On Error Resume Next
    txt = s.TextFrame2.TextRange.Text
    Set x = sld.Shapes.AddTextbox(1, l, t, w, h)                      ' msoTextOrientationHorizontal
    With x
        .Name = s.Name
        If s.Fill.Visible Then
            .Fill.Visible = msoTrue
            .Fill.Solid
            .Fill.ForeColor.RGB = s.Fill.ForeColor.RGB
        Else
            .Fill.Visible = msoFalse
        End If
        .Line.Visible = msoFalse
        With .TextFrame2
            .AutoSize = 0                                             ' msoAutoSizeNone
            .WordWrap = msoTrue
            .MarginLeft = s.TextFrame2.MarginLeft
            .MarginRight = s.TextFrame2.MarginRight
            .MarginTop = s.TextFrame2.MarginTop
            .MarginBottom = s.TextFrame2.MarginBottom
            .VerticalAnchor = s.TextFrame2.VerticalAnchor
            .TextRange.Text = txt
            .TextRange.ParagraphFormat.Alignment = s.TextFrame2.TextRange.ParagraphFormat.Alignment
            With .TextRange.Font
                .Name = s.TextFrame2.TextRange.Font.Name
                .Size = s.TextFrame2.TextRange.Font.Size
                .Bold = s.TextFrame2.TextRange.Font.Bold
                .Italic = s.TextFrame2.TextRange.Font.Italic
                .Fill.ForeColor.RGB = s.TextFrame2.TextRange.Font.Fill.ForeColor.RGB
            End With
        End With
        If Len(zeile2) > 0 Then
            With .TextFrame2
                .TextRange.Text = txt & vbCr & zeile2
                .MarginLeft = 7: .MarginRight = 7: .MarginTop = 2: .MarginBottom = 2
                .VerticalAnchor = 3                                    ' msoAnchorMiddle
                .TextRange.ParagraphFormat.SpaceBefore = 0
                .TextRange.ParagraphFormat.SpaceAfter = 0
                .TextRange.Paragraphs(2).ParagraphFormat.SpaceBefore = 1
                With .TextRange.Paragraphs(2).Font
                    .Bold = msoFalse
                    .Size = groesse2
                End With
            End With
        End If
        .Left = l: .Top = t: .Width = w: .Height = h                  ' AddTextbox passt die Hoehe sonst an
    End With
    On Error GoTo 0
End Sub

Public Function PPTModulVersion() As String
    PPTModulVersion = "11"
End Function
```

modFolienauswahl.bas

```
Option Explicit
' ================================================================================
' modFolienauswahl – Karte "Welche Kacheln sollen auf die Folie?" vor dem PowerPoint-Export
'
' Die sechs Kacheln des Blatts PPT stehen als Kaestchen in derselben Anordnung wie auf dem Blatt (3 x 2); ein Klick
' schaltet eine Kachel an oder ab. Die Auswahl steht als "111111" in Filter!O46 und gilt beim naechsten Mal wieder.
' "Folie erzeugen" ruft modPPT.NachPowerPoint mit der Auswahl auf; dort werden nur diese Kacheln kopiert und auf der
' Folie neu angeordnet (1-3 in einer Reihe, 4 als 2 x 2, 5-6 in zwei Reihen).
' Zeichnet mit den Bausteinen aus modTagesablauf (gleicher Stil, gleiches Praefix TAGW_ zum Aufraeumen).
' ================================================================================

Private Const PRAEFIX As String = "TAGW_"
Private Const C_TEAL As Long = 8220427             ' RGB(11, 111, 125)
Private Const C_TEALT As Long = 16250606           ' RGB(238, 246, 247)
Private Const C_SW As Long = 1710618               ' RGB(26, 26, 26)
Private Const C_GRAU As Long = 7039851             ' RGB(107, 107, 107)
Private Const C_GRAUM As Long = 11379610           ' RGB(154, 163, 173)
Private Const C_GRAUL As Long = 14341584           ' RGB(208, 213, 218)
Private Const C_ROTD As Long = 1970062             ' RGB(142, 15, 30)
Private Const C_GRAUD As Long = 4868682            ' RGB(74, 74, 74)

Private auswahl As String                          ' "111111": je Kachel 1 = auf die Folie
Private kx As Single, ky As Single, kb As Single, kh As Single
Private umschaltNr As Long

Public Function FolienauswahlVersion() As String
    FolienauswahlVersion = "1"
End Function

Private Function Titel(i As Long) As String
    Titel = Choose(i + 1, "Rückstand OPAG", "Rückstand Verbund", "Kapazität (MAK) je KW", _
                   "Eingänge/Abgänge OPAG", "Eingänge/Abgänge Verbund", "Altersstruktur")
End Function

Private Function Farbe(i As Long) As Long
    Select Case i
        Case 0, 3: Farbe = C_ROTD
        Case 1, 4: Farbe = C_TEAL
        Case Else: Farbe = C_GRAUD
    End Select
End Function

Public Function GespeicherteAuswahl() As String
    Dim t As String
    On Error Resume Next
    t = Trim$(CStr(ThisWorkbook.Worksheets("Filter").Range("O46").Value))
    On Error GoTo 0
    If Len(t) <> 6 Or Len(Replace(Replace(t, "0", ""), "1", "")) > 0 Then t = "111111"
    GespeicherteAuswahl = t
End Function

' ------------------------------------------------------------------ Einstieg (Knopf "PowerPoint-Folie erzeugen")
Public Sub Zeigen()
    Dim ws As Worksheet
    If modLaden.LadenLaeuft() Then Exit Sub
    On Error GoTo Ersatz
    modTagesablauf.KartenEntfernen
    ThisWorkbook.Worksheets("Cockpit").Activate
    Set ws = ActiveSheet
    auswahl = GespeicherteAuswahl()
    kb = 580: kh = 360
    modTagesablauf.KarteModal ws, kb, kh, kx, ky
    modTagesablauf.Beschriftung ws, "Schritt", kx + 28, ky + 20, kb - 56, 14, "POWERPOINT-FOLIE", 8, True, C_TEAL
    modTagesablauf.Beschriftung ws, "Frage", kx + 28, ky + 40, kb - 56, 26, "Welche Kacheln sollen auf die Folie?", 15
    , True, C_SW
    modTagesablauf.Beschriftung ws, "Text", kx + 28, ky + 68, kb - 56, 18, _
         "Anklicken schaltet eine Kachel an oder ab. Die Auswahl wird für das nächste Mal gemerkt.", 9.5, False, C_GRA
    U
    KachelnZeichnen ws
    modTagesablauf.Knopf ws, "Folie", kx + kb - 28 - 150, ky + kh - 50, 150, 30, "Folie erzeugen", 4, "modFolienauswah
    l.KlickErzeugen"
    modTagesablauf.Knopf ws, "Abbrechen", kx + kb - 28 - 150 - 12 - 110, ky + kh - 50, 110, 30, "Abbrechen", 2, "modFo
    lienauswahl.KlickAbbrechen"
    modTagesablauf.Knopf ws, "Alle", kx + 20, ky + kh - 50, 100, 30, "Alle auswählen", 3, "modFolienauswahl.KlickAlle"
    Exit Sub
Ersatz:
    On Error Resume Next
    modTagesablauf.KartenEntfernen
    modPPT.NachPowerPoint GespeicherteAuswahl()
End Sub

' Kaestchen je Kachel (3 x 2 wie auf dem Blatt PPT) und Zaehler darunter
Private Sub KachelnZeichnen(ws As Worksheet)
    Dim i As Long, n As Long
    For i = 0 To 5
        KachelZeichnen ws, i
        If Mid$(auswahl, i + 1, 1) = "1" Then n = n + 1
    Next i
    On Error Resume Next
    ws.Shapes(PRAEFIX & "Zaehler").Delete
    On Error GoTo 0
    modTagesablauf.Beschriftung ws, "Zaehler", kx + 28, ky + 274, kb - 56, 16, _
        IIf(n = 0, "Keine Kachel ausgewählt", n & " von 6 Kacheln · " & Anordnung(n)), 8.5, False, IIf(n = 0, RGB(191,
    21, 40), C_GRAU)
```

```
End Sub

Private Function Anordnung(n As Long) As String
    Select Case n
        Case 1: Anordnung = "eine Kachel, vergrößert"
        Case 2, 3: Anordnung = "nebeneinander in einer Reihe"
        Case 4: Anordnung = "zwei Reihen zu je zwei"
        Case Else: Anordnung = "zwei Reihen zu je drei"
    End Select
End Function

Private Sub KachelZeichnen(ws As Worksheet, i As Long)
    Dim x As Single, y As Single, b As Single, h As Single, an As Boolean, nm As String, k As Long
    Dim ff As Object
    b = (kb - 56 - 2 * 12) / 3: h = 78
    x = kx + 28 + (i Mod 3) * (b + 12)
    y = ky + 98 + (i \ 3) * (h + 12)
    an = (Mid$(auswahl, i + 1, 1) = "1")
    nm = PRAEFIX & "T" & i & "_"
    ' alte Formen dieser Kachel entfernen (beim Umschalten)
    On Error Resume Next
    For k = ws.Shapes.Count To 1 Step -1
        If Left$(ws.Shapes(k).Name, Len(nm)) = nm Then ws.Shapes(k).Delete
    Next k
    On Error GoTo 0
    With ws.Shapes.AddShape(5, x, y, b, h)
        .Name = nm & "Feld"
        .Adjustments(1) = 0.1
        .Fill.ForeColor.RGB = IIf(an, C_TEALT, RGB(255, 255, 255))
        .Line.ForeColor.RGB = IIf(an, C_TEAL, C_GRAUL)
        .Line.Weight = IIf(an, 2, 1)
        .OnAction = "modFolienauswahl.KlickKachel"
    End With
    ' Kopfleiste in der Kachelfarbe (wie der Kachelkopf auf der Folie)
    With ws.Shapes.AddShape(5, x + 10, y + 10, 28, 5)
        .Name = nm & "Farbe"
        .Adjustments(1) = 0.5
        .Fill.ForeColor.RGB = Farbe(i)
        .Line.Visible = msoFalse
        If Not an Then .Fill.Transparency = 0.6
        .OnAction = "modFolienauswahl.KlickKachel"
    End With
    With ws.Shapes.AddTextbox(msoTextOrientationHorizontal, x + 10, y + 30, b - 20, 36)
        .Name = nm & "Titel"
        .Fill.Visible = msoFalse
        .Line.Visible = msoFalse
        .OnAction = "modFolienauswahl.KlickKachel"
        .TextFrame2.MarginLeft = 0: .TextFrame2.MarginRight = 0: .TextFrame2.MarginTop = 0: .TextFrame2.MarginBottom =
    0
        .TextFrame2.WordWrap = msoTrue
        .TextFrame2.VerticalAnchor = msoAnchorTop
        With .TextFrame2.TextRange
            .Text = Titel(i) & vbLf & IIf(an, "auf der Folie", "nicht auf der Folie")
            .Font.Name = "Liberation Sans"
            .Font.Size = 8
            .Font.Fill.ForeColor.RGB = IIf(an, C_TEAL, C_GRAUM)
            .Paragraphs(1).Font.Size = 9
            .Paragraphs(1).Font.Bold = msoTrue
            .Paragraphs(1).Font.Fill.ForeColor.RGB = IIf(an, C_SW, C_GRAU)
        End With
    End With
    ' Auswahlkreis oben rechts: gefuellt mit Haken (an) bzw. leerer Ring (aus)
    With ws.Shapes.AddShape(9, x + b - 28, y + 8, 18, 18)
        .Name = nm & "Kreis"
        If an Then
            .Fill.ForeColor.RGB = C_TEAL
            .Line.Visible = msoFalse
        Else
            .Fill.ForeColor.RGB = RGB(255, 255, 255)
            .Line.ForeColor.RGB = C_GRAUL
            .Line.Weight = 1.25
        End If
        .OnAction = "modFolienauswahl.KlickKachel"
    End With
    If an Then
        On Error Resume Next
        Set ff = ws.Shapes.BuildFreeform(msoEditingAuto, x + b - 23.5, y + 17.5)
        ff.AddNodes msoSegmentLine, msoEditingAuto, x + b - 20.5, y + 20.5
        ff.AddNodes msoSegmentLine, msoEditingAuto, x + b - 14.5, y + 13.5
        With ff.ConvertToShape
            .Name = nm & "Haken"
            .Fill.Visible = msoFalse
            .Line.ForeColor.RGB = RGB(255, 255, 255)
            .Line.Weight = 1.75
            .OnAction = "modFolienauswahl.KlickKachel"
        End With
        On Error GoTo 0
    End If
End Sub

' ------------------------------------------------------------- Klicks
Public Sub KlickKachel()
    Dim nm As String, p As Long
    On Error Resume Next
    nm = CStr(Application.Caller)
    On Error GoTo 0
```

```
    p = InStr(nm, PRAEFIX & "T")
    If p = 0 Then Exit Sub
    umschaltNr = CLng(Val(Mid$(nm, p + Len(PRAEFIX) + 1, 1)))
    ' die geklickte Form nicht im eigenen Makro loeschen: Neuzeichnen gleich danach
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modFolienauswahl.Umschalten"
End Sub

Public Sub Umschalten()
    Dim ws As Worksheet
    If umschaltNr < 0 Or umschaltNr > 5 Then Exit Sub
    If Len(auswahl) <> 6 Then auswahl = GespeicherteAuswahl()
    Mid$(auswahl, umschaltNr + 1, 1) = IIf(Mid$(auswahl, umschaltNr + 1, 1) = "1", "0", "1")
    Set ws = ActiveSheet
    Application.ScreenUpdating = False
    KachelnZeichnen ws
    Application.ScreenUpdating = True
End Sub

Public Sub KlickAlle()
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modFolienauswahl.AlleAn"
End Sub

Public Sub AlleAn()
    auswahl = "111111"
    Application.ScreenUpdating = False
    KachelnZeichnen ActiveSheet
    Application.ScreenUpdating = True
End Sub

Public Sub KlickAbbrechen()
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modTagesablauf.KartenEntfernen"
End Sub

Public Sub KlickErzeugen()
    If InStr(auswahl, "1") = 0 Then Exit Sub                ' nichts gewaehlt: der Zaehler zeigt es rot
    Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modFolienauswahl.Erzeugen"
End Sub

Public Sub Erzeugen()
    On Error Resume Next
    ThisWorkbook.Worksheets("Filter").Range("O46").Value = "'" & auswahl
    modTagesablauf.KartenEntfernen
    On Error GoTo 0
    modPPT.NachPowerPoint auswahl
End Sub
```

modFortschritt.bas

```
Option Explicit
' ================================================================================
' modFortschritt – Lade-Overlay mit Fortschrittsbalken (ohne UserForm: Formen auf dem aktiven Blatt)
' FortschrittZeigen "Titel"  -> abgedunkelte Flaeche, weisse Karte, Titel, Statuszeile, Balken, Prozent
' FortschrittSetzen 0.42, "Text"[, "Detail"] -> Balken, Statuszeile und kleine Zeile darunter (z. B. Dateiname) nachzi
   ehen
'                              (zeichnet auch bei ScreenUpdating = False); zu lange Texte werden auf die Breite gekue
   rzt
' FortschrittSchritt "Schritt 1 von 2 · …" -> Zeile ueber dem Status (Abschnitt des Ablaufs), leer = keine
' FortschrittSchliessen        -> alle Formen entfernen
' ================================================================================

Private Const PRAEFIX As String = "LADE_"
Private Const HINWEIS As String = "HINW_"
Private Const KARTE_B As Single = 420
Private Const KARTE_H As Single = 154
Private letztesUpdate As Double
Private aktiv As Boolean
Private blatt As Worksheet
Private hinwTitel As String
Private hinwText As String
Private hinwFehler As Boolean

Public Sub FortschrittZeigen(titel As String)
    Dim sichtbar As Range, links As Single, oben As Single, breite As Single, hoehe As Single
    Dim kx As Single, ky As Single
    On Error Resume Next
    FortschrittSchliessen
    Set blatt = ActiveSheet
    If blatt Is Nothing Then Exit Sub
    Set sichtbar = ActiveWindow.VisibleRange
    links = sichtbar.Left: oben = sichtbar.Top: breite = sichtbar.Width: hoehe = sichtbar.Height
    kx = links + (breite - KARTE_B) / 2
    ky = oben + (hoehe - KARTE_H) / 2 - 20
    ' Abdunkelung
    With blatt.Shapes.AddShape(1, links, oben, breite, hoehe)
        .Name = PRAEFIX & "Schleier"
        .Fill.ForeColor.RGB = RGB(26, 26, 26)
        .Fill.Transparency = 0.55
        .Line.Visible = msoFalse
    End With
    ' Karte
    With blatt.Shapes.AddShape(5, kx, ky, KARTE_B, KARTE_H)
        .Name = PRAEFIX & "Karte"
        .Adjustments(1) = 0.08
        .Fill.ForeColor.RGB = RGB(255, 255, 255)
        .Line.Visible = msoFalse
        .Shadow.Visible = msoTrue
        .Shadow.Blur = 18
        .Shadow.OffsetX = 0
        .Shadow.OffsetY = 6
        .Shadow.Transparency = 0.7
        .Shadow.ForeColor.RGB = RGB(0, 0, 0)
    End With
    ' Farbbalken links als Akzent
    With blatt.Shapes.AddShape(1, kx, ky + 22, 4, KARTE_H - 44)
        .Name = PRAEFIX & "Akzent"
        .Fill.ForeColor.RGB = RGB(191, 21, 40)
        .Line.Visible = msoFalse
    End With
    Textfeld PRAEFIX & "Titel", kx + 24, ky + 18, KARTE_B - 48, 24, titel, 13, True, RGB(26, 26, 26)
    Textfeld PRAEFIX & "Schritt", kx + 24, ky + 46, KARTE_B - 48, 16, " ", 8, True, RGB(11, 111, 125)
    Textfeld PRAEFIX & "Status", kx + 24, ky + 64, KARTE_B - 110, 20, "Vorbereitung …", 9, False, RGB(107, 107, 107)
    Textfeld PRAEFIX & "Prozent", kx + KARTE_B - 90, ky + 64, 66, 20, "0 %", 9, True, RGB(26, 26, 26)
    Textfeld PRAEFIX & "Detail", kx + 24, ky + 84, KARTE_B - 48, 14, " ", 8, False, RGB(154, 163, 173)
    With blatt.Shapes(PRAEFIX & "Prozent").TextFrame2.TextRange.ParagraphFormat
        .Alignment = msoAlignRight
    End With
    ' Balken: Spur und Fuellung
    With blatt.Shapes.AddShape(5, kx + 24, ky + 106, KARTE_B - 48, 10)
        .Name = PRAEFIX & "Spur"
        .Adjustments(1) = 0.5
        .Fill.ForeColor.RGB = RGB(232, 234, 237)
        .Line.Visible = msoFalse
    End With
    With blatt.Shapes.AddShape(5, kx + 24, ky + 106, 6, 10)
        .Name = PRAEFIX & "Balken"
        .Adjustments(1) = 0.5
        .Fill.ForeColor.RGB = RGB(191, 21, 40)
        .Line.Visible = msoFalse
    End With
    Textfeld PRAEFIX & "Hinweis", kx + 24, ky + 122, KARTE_B - 48, 18, "Bitte warten, bis der Balken durch ist. Klicks
    werden währenddessen nicht ausgeführt.", 8, False, RGB(154, 163, 173)
    aktiv = True
    letztesUpdate = 0
    Zeichnen
    On Error GoTo 0
End Sub

Public Sub FortschrittSetzen(anteil As Double, status As String, Optional detail As String = "")
    Dim spur As Shape
    If Not aktiv Then Exit Sub
    On Error Resume Next
    If anteil < 0 Then anteil = 0
```

```
    If anteil > 1 Then anteil = 1
    Set spur = blatt.Shapes(PRAEFIX & "Spur")
    blatt.Shapes(PRAEFIX & "Balken").Width = IIf(anteil * spur.Width < 6, 6, anteil * spur.Width)
    blatt.Shapes(PRAEFIX & "Prozent").TextFrame2.TextRange.Text = Format(anteil * 100, "0") & " %"
    If Len(status) > 0 Then TextPassend blatt.Shapes(PRAEFIX & "Status"), status, False
    TextPassend blatt.Shapes(PRAEFIX & "Detail"), detail, True
    ' hoechstens alle 0,15 s neu zeichnen, damit das Laden nicht ausgebremst wird
    If Timer - letztesUpdate > 0.15 Or Timer < letztesUpdate Or anteil >= 1 Then
        Zeichnen
        letztesUpdate = Timer
    End If
    On Error GoTo 0
End Sub

' Abschnitt des Ablaufs ueber der Statuszeile, z. B. "SCHRITT 1 VON 2 · DATEIEN PRÜFEN"
Public Sub FortschrittSchritt(text As String)
    If Not aktiv Then Exit Sub
    On Error Resume Next
    TextPassend blatt.Shapes(PRAEFIX & "Schritt"), UCase$(text), False
    Zeichnen
    letztesUpdate = Timer
    On Error GoTo 0
End Sub

' Text in ein Feld ohne Umbruch setzen und so weit kuerzen, dass er in die Breite des Felds passt (gemessen an der ech
   ten
' Textbreite). mitte = True kuerzt in der Mitte (Dateinamen: Anfang und Endung bleiben sichtbar), sonst am Ende.
Private Sub TextPassend(shp As Shape, text As String, mitte As Boolean)
    Dim t As String, b As Single, n As Long, versuch As Long
    On Error Resume Next
    If Len(text) = 0 Then shp.TextFrame2.TextRange.Text = " ": Exit Sub
    shp.TextFrame2.TextRange.Text = text
    b = shp.TextFrame2.TextRange.BoundWidth
    If Err.Number <> 0 Or b <= 0 Then                        ' ohne Messung: grobe Schaetzung ueber die Schriftgroess
   e
        Err.Clear
        n = Int(shp.Width / (shp.TextFrame2.TextRange.Font.Size * 0.55))
        If Len(text) > n And n > 8 Then shp.TextFrame2.TextRange.Text = Gekuerzt(text, n, mitte)
        Exit Sub
    End If
    If b <= shp.Width Then Exit Sub
    n = Int(Len(text) * shp.Width / b)                       ' erste Schaetzung, dann schrittweise nachziehen
    Do
        t = Gekuerzt(text, n, mitte)
        shp.TextFrame2.TextRange.Text = t
        If shp.TextFrame2.TextRange.BoundWidth <= shp.Width Or n <= 8 Then Exit Do
        n = n - 2
        versuch = versuch + 1
    Loop While versuch < 40
End Sub

Private Function Gekuerzt(text As String, n As Long, mitte As Boolean) As String
    Dim vorne As Long
    If Len(text) <= n Then Gekuerzt = text: Exit Function
    If mitte Then
        vorne = Int((n - 1) * 0.6)
        Gekuerzt = Left$(text, vorne) & "…" & Right$(text, n - 1 - vorne)
    Else
        Gekuerzt = Left$(text, n - 1) & "…"
    End If
End Function

Public Sub FortschrittSchliessen()
    Dim i As Long, ws As Worksheet
    On Error Resume Next
    aktiv = False
    If blatt Is Nothing Then
        For Each ws In ThisWorkbook.Worksheets            ' Reste nach einem Abbruch auf jedem Blatt entfernen
            For i = ws.Shapes.Count To 1 Step -1
                If Left$(ws.Shapes(i).Name, Len(PRAEFIX)) = PRAEFIX Then ws.Shapes(i).Delete
            Next i
        Next ws
    Else
        For i = blatt.Shapes.Count To 1 Step -1
            If Left$(blatt.Shapes(i).Name, Len(PRAEFIX)) = PRAEFIX Then blatt.Shapes(i).Delete
        Next i
    End If
    Set blatt = Nothing
    On Error GoTo 0
End Sub

Private Sub Zeichnen()
    Dim war As Boolean
    war = Application.ScreenUpdating
    Application.ScreenUpdating = True
    DoEvents
    Application.ScreenUpdating = war
End Sub

Private Sub Textfeld(nm As String, x As Single, y As Single, b As Single, h As Single, text As String, groesse As Sing
   le, fett As Boolean, farbe As Long)
    With blatt.Shapes.AddTextbox(msoTextOrientationHorizontal, x, y, b, h)
        .Name = nm
        .Fill.Visible = msoFalse
        .Line.Visible = msoFalse
        .TextFrame2.MarginLeft = 0: .TextFrame2.MarginRight = 0: .TextFrame2.MarginTop = 0: .TextFrame2.MarginBottom =
```

```
        0
185         .TextFrame2.WordWrap = msoFalse
186         .TextFrame2.VerticalAnchor = msoAnchorMiddle
187         With .TextFrame2.TextRange
188             .Text = text
189             .Font.Name = "Liberation Sans"
190             .Font.Size = groesse
191             .Font.Bold = fett
192             .Font.Fill.ForeColor.RGB = farbe
193         End With
194     End With
195 End Sub
196
197 ' ------------------------------------------------------------- Hinweis-Karte (Ersatz fuer MsgBox): Titel, Text, OK
    -Knopf
198 Public Sub HinweisZeigen(titel As String, text As String, Optional fehler As Boolean = False)
199     ' Die Karte wird nicht sofort gezeichnet, sondern per OnTime, nachdem die aufrufende Prozedur beendet ist und
200     ' Excel Neuberechnung und Bildaufbau abgeschlossen hat. Konfiguration!B9 = NEIN schaltet auf das klassische Fenste
    r.
201     Dim karte As String
202     ' Laden aus dem Tagesablauf: Meldung geht an modTagesablauf, das mit Schritt 2 weitermacht
203     If modTagesablauf.WartetAufLaden() Then
204         modTagesablauf.LadeErgebnis titel, text, fehler
205         Exit Sub
206     End If
207     On Error Resume Next
208     karte = UCase$(Trim$(CStr(ThisWorkbook.Worksheets("Konfiguration").Range("B9").Value)))
209     On Error GoTo 0
210     If karte = "NEIN" Then
211         MsgBox text, IIf(fehler, vbCritical, vbInformation), titel
212         Exit Sub
213     End If
214     hinwTitel = titel: hinwText = text: hinwFehler = fehler
215     On Error GoTo Ersatz
216     modLaden.LogSchreiben "Hinweis geplant: " & titel
217     Application.OnTime Now + TimeSerial(0, 0, 1), "'" & ThisWorkbook.Name & "'!modFortschritt.HinweisZeichnen"
218     Exit Sub
219 Ersatz:
220     MsgBox text, IIf(fehler, vbCritical, vbInformation), titel
221 End Sub
222
223 Public Sub HinweisZeichnen()
224     Dim ws As Worksheet, sichtbar As Range, links As Single, oben As Single, breite As Single, hoehe As Single
225     Dim kx As Single, ky As Single, kb As Single, kh As Single, zeilen As Long, i As Long, teile() As String, akzent A
    s Long
226     Dim titel As String, text As String, fehler As Boolean
227     titel = hinwTitel: text = hinwText: fehler = hinwFehler
228     On Error GoTo Ersatz
229     modLaden.LogSchreiben "Hinweis zeichnen"
230     HinweisEntfernen
231     Set ws = ActiveSheet
232     If ws Is Nothing Then GoTo Ersatz
233     Set sichtbar = ActiveWindow.VisibleRange
234     links = sichtbar.Left: oben = sichtbar.Top: breite = sichtbar.Width: hoehe = sichtbar.Height
235     ' Hoehe nach Textmenge: je Zeile ca. 16 pt, umbrochene Zeilen mitzaehlen
236     teile = Split(text, vbCrLf)
237     zeilen = 0
238     For i = 0 To UBound(teile)
239         zeilen = zeilen + 1 + Int(Len(teile(i)) / 78)
240     Next i
241     kb = 520
242     kh = 96 + zeilen * 16
243     If kh > hoehe - 40 Then kh = hoehe - 40
244     kx = links + (breite - kb) / 2
245     ky = oben + (hoehe - kh) / 2 - 20
246     akzent = IIf(fehler, RGB(191, 21, 40), RGB(15, 148, 167))
247     Set blatt = ws
248     Application.ScreenUpdating = False
249     With ws.Shapes.AddShape(1, links, oben, breite, hoehe)
250         .Name = HINWEIS & "Schleier"
251         .Fill.ForeColor.RGB = RGB(26, 26, 26)
252         .Fill.Transparency = 0.55
253         .Line.Visible = msoFalse
254         .OnAction = "modFortschritt.HinweisSchliessen"
255     End With
256     With ws.Shapes.AddShape(5, kx, ky, kb, kh)
257         .Name = HINWEIS & "Karte"
258         .Adjustments(1) = 0.06
259         .Fill.ForeColor.RGB = RGB(255, 255, 255)
260         .Line.Visible = msoFalse
261         .Shadow.Visible = msoTrue
262         .Shadow.Blur = 18
263         .Shadow.OffsetX = 0
264         .Shadow.OffsetY = 6
265         .Shadow.Transparency = 0.7
266         .Shadow.ForeColor.RGB = RGB(0, 0, 0)
267     End With
268     With ws.Shapes.AddShape(1, kx, ky + 22, 4, kh - 44)
269         .Name = HINWEIS & "Akzent"
270         .Fill.ForeColor.RGB = akzent
271         .Line.Visible = msoFalse
272     End With
273     Textfeld HINWEIS & "Titel", kx + 24, ky + 18, kb - 48, 24, titel, 13, True, RGB(26, 26, 26)
274     Textfeld HINWEIS & "Text", kx + 24, ky + 48, kb - 48, kh - 100, text, 9.5, False, RGB(74, 74, 74)
275     With ws.Shapes(HINWEIS & "Text").TextFrame2
276         .WordWrap = msoTrue
```

```
277          .VerticalAnchor = msoAnchorTop
278      End With
279      With ws.Shapes.AddShape(5, kx + kb - 24 - 96, ky + kh - 40, 96, 26)
280          .Name = HINWEIS & "OK"
281          .Adjustments(1) = 0.3
282          .Fill.ForeColor.RGB = akzent
283          .Line.Visible = msoFalse
284          .OnAction = "modFortschritt.HinweisSchliessen"
285          With .TextFrame2
286              .TextRange.Text = "OK"
287              .TextRange.Font.Name = "Liberation Sans"
288              .TextRange.Font.Size = 10
289              .TextRange.Font.Bold = msoTrue
290              .TextRange.Font.Fill.ForeColor.RGB = RGB(255, 255, 255)
291              .TextRange.ParagraphFormat.Alignment = msoAlignCenter
292              .VerticalAnchor = msoAnchorMiddle
293              .MarginTop = 0: .MarginBottom = 0
294          End With
295      End With
296      Application.ScreenUpdating = True
297      modLaden.LogSchreiben "Hinweis gezeichnet"
298      Exit Sub
299  Ersatz:
300      On Error Resume Next
301      Application.ScreenUpdating = True
302      HinweisEntfernen
303      MsgBox text, IIf(fehler, vbCritical, vbInformation), titel
304  End Sub
305
306  ' OK-Knopf und Schleier: nicht im eigenen Klick loeschen, sondern gleich danach per OnTime
307  Public Sub HinweisSchliessen()
308      On Error Resume Next
309      modLaden.LogSchreiben "Hinweis schliessen"
310      Application.OnTime Now, "'" & ThisWorkbook.Name & "'!modFortschritt.HinweisEntfernen"
311      On Error GoTo 0
312  End Sub
313
314  Public Sub HinweisEntfernen()
315      Dim i As Long, ws As Worksheet
316      On Error Resume Next
317      For Each ws In ThisWorkbook.Worksheets
318          For i = ws.Shapes.Count To 1 Step -1
319              If Left$(ws.Shapes(i).Name, Len(HINWEIS)) = HINWEIS Then ws.Shapes(i).Delete
320          Next i
321      Next ws
322      On Error GoTo 0
323  End Sub
```

Cockpit.cls

```
Option Explicit
' Blattmodul Cockpit: Der Hyperlink eines Chips zeigt auf Spalte A (oder H) der Chipzeile; dort steht in Spalte A die
' Chip-Nummer (1..99 Themenpostkoerbe, 101..114 GeVo-Typen, 201..204 Sparten, 301..306 Einheiten). Der Auswahlwechsel
   schaltet den Chip, danach wandert die
' Auswahl per OnTime auf H1, damit der naechste Klick wieder ein Auswahlwechsel ist.
Private Sub Worksheet_SelectionChange(ByVal Target As Range)
    Dim nr As Long, col As Long
    On Error GoTo Ende
    col = Target.Cells(1, 1).Column
    If col <> 1 And col <> 8 Then Exit Sub
    nr = CLng(Val(Me.Cells(Target.Cells(1, 1).Row, 1).Value))
    If nr <= 0 Then Exit Sub
    modCockpit.ChipSchalten nr
    modCockpit.AuswahlWegPlanen
Ende:
End Sub

Private Sub Worksheet_Change(ByVal Target As Range)
    ' Bezugstag in L12 per Auswahlliste geaendert: Diagramme nachziehen
    On Error GoTo Ende
    If Not Intersect(Target, Me.Range("L12")) Is Nothing Then
        ' geleert (Entf): wieder "neuester Stand" als Formel
        If IsEmpty(Me.Range("L12").Value) Then modCockpit.BezugFormelSetzen
        modCockpit.AktualisierungPlanen
    End If
Ende:
End Sub
```

Kalender.cls

```
 1  Option Explicit
 2  ' Blattmodul Kalender: beim ersten Aufruf je Sitzung zum heutigen Tag springen. Danach bleibt die Position, wie der
 3  ' Nutzer sie verlaesst ("Zu heute" springt jederzeit zurueck).
 4  Private gesprungen As Boolean
 5
 6  Private Sub Worksheet_Activate()
 7      If gesprungen Then Exit Sub
 8      On Error Resume Next
 9      modCockpit.KalenderZuHeute
10      gesprungen = True
11  End Sub
```

Rohdatenblatt.cls

```
Option Explicit
' Blattmodul Rohdatenblatt: beim ersten Aufruf je Sitzung zur aktuellen Woche springen (Vorwoche links, damit das Delta zur
' aktuellen Woche sichtbar ist). Danach bleibt die Position, wie der Nutzer sie verlaesst.
Private gesprungen As Boolean

Private Sub Worksheet_Activate()
    Dim c As Long, letzte As Long, ziel As Long
    If gesprungen Then Exit Sub
    On Error GoTo Ende
    letzte = Me.Cells(1, Me.Columns.Count).End(xlToLeft).Column
    ziel = 0
    For c = 9 To letzte Step 2
        If IsDate(Me.Cells(1, c).Value) Then
            If CDate(Me.Cells(1, c).Value) >= Date - 6 Then ziel = c: Exit For
        End If
    Next c
    If ziel > 0 Then ActiveWindow.ScrollColumn = IIf(ziel - 2 < 9, 9, ziel - 2)
    gesprungen = True
Ende:
End Sub
```

D. Anhang: Python-Spiegel der Ladelogik

verarbeitung.py rechnet dasselbe wie modLaden (Erkennung, Schlüssel, Rückstand, Eingänge, Abgänge, Altersklassen) und dient im Projekt als Prüfreferenz. Für den Celonis-Nachbau ist es die kompakteste Beschreibung der Rechnung.

```
# -*- coding: utf-8 -*-
"""Verarbeitung der Rueckstandsabzuege (OPAG, Verbund) – Python-Spiegel der VBA-Logik in modLaden.

Liest Rohdaten-Blaetter (Kopf in den ersten Zeilen: "Rückstandsstichtag = Mittwoch, 23. September 2026",
Spaltenkoepfe in der Zeile mit "Themenpostkorb"), bildet je Stichtag und Quelle:
- Rueckstand je (Sparte, VSS, Themenpostkorb, GEVO_SL) mit Altersklassen,
- Eingaenge (Schluessel neu gegenueber dem Vortagsstand) und Abgaenge (Schluessel nicht mehr vorhanden,
  zugeordnet nach den Merkmalen des Vortags).
Der Schluessel ist bei Verbund (VSS, Ordnungsbegriff, VORG_VERBU_LNR), bei OPAG zusaetzlich OPS_AUFTR_ID und OPS_ARBGUT_LNR.
"""
import datetime as dt
import re
from collections import OrderedDict

MONATE = ['januar', 'februar', 'märz', 'april', 'mai', 'juni', 'juli', 'august', 'september', 'oktober', 'november', 'dezember']
SPARTE_STANDARD = OrderedDict([
    ('KV', 'KV'), ('LV', 'Leben'), ('LF', 'Leben'), ('LG', 'Leben'),
    ('SV', 'Komposit'), ('KR', 'Komposit'), ('DA', 'Komposit'), ('HA', 'Komposit'), ('AS', 'Komposit'),
    ('UV', 'Komposit'), ('GV', 'Komposit'), ('TR', 'Komposit'), ('IT', 'Komposit'),
])
OHNE_POSTKORB = '(ohne Postkorb)'


def stichtag_aus_text(text):
    """'Rückstandsstichtag = Mittwoch, 23. September 2026' -> date"""
    m = re.search(r'(\d{1,2})\.\s*([A-Za-zäöüÄÖÜ]+)\s+(\d{4})', str(text or ''))
    if not m:
        return None
    monat = m.group(2).lower()
    if monat not in MONATE:
        return None
    return dt.date(int(m.group(3)), MONATE.index(monat) + 1, int(m.group(1)))


def postkorb_norm(wert):
    t = re.sub(r'\s+', ' ', str(wert or '')).strip()
    return t if t else OHNE_POSTKORB


def gevo_typ(gevo_sl):
    """'SV 108 Bestandsübertragung' -> ('108 Bestandsübertragung', 108); '. Ohne Zuordnung' -> ('Ohne Zuordnung', 0)"""
    t = re.sub(r'\s+', ' ', str(gevo_sl or '')).strip()
    if not t:
        return ('(ohne GeVo)', 0)
    teile = t.split(' ', 1)
    if len(teile) == 2 and re.fullmatch(r'[A-Z]{2}', teile[0]):
        t = teile[1]
    t = t.lstrip('. ').strip()
    m = re.match(r'(\d+)\b', t)
    return (t, int(m.group(1)) if m else 0)


def alter_klasse(tage):
    try:
        t = int(tage)
    except (TypeError, ValueError):
        t = 0
    if t <= 7:
        return 0
    if t <= 30:
        return 1
    if t <= 90:
        return 2
    return 3


def blatt_lesen(zeilen):
    """zeilen: Liste von Zeilen (Listen) eines Rohdatenblatts. Liefert (stichtag, quelle, saetze) mit
    saetze = Liste von dict(key, sparte_vss, postkorb, gevo, alter)."""
    stichtag = None
    kopf_i = None
    for i, z in enumerate(zeilen[:15]):
        if stichtag is None:
            for c in z:
                if c is not None and 'stichtag' in str(c).lower():
                    stichtag = stichtag_aus_text(c)
        if any(str(c).strip() == 'Themenpostkorb' for c in z if c is not None):
            kopf_i = i
            break
    if kopf_i is None or stichtag is None:
        return None
    kopf = [str(c).strip() if c is not None else '' for c in zeilen[kopf_i]]
    ix = {h: i for i, h in enumerate(kopf)}
    quelle = 'OPAG' if 'OPS_AUFTR_ID' in ix else 'Verbund'
```

```
    n = {k.lower(): i for k, i in ix.items()}
    c_vss = n.get('verwaltungssystemschlüssel')
    c_ob, c_lnr, c_pk, c_alt, c_gevo = n['ordnungsbegriff'], n['vorg_verbu_lnr'], n['themenpostkorb'], n['rückstandsal
    ter_tage'], n['gevo_sl']
    c_auf, c_arb = n.get('ops_auftr_id'), n.get('ops_arbgut_lnr')
    saetze = []
    for z in zeilen[kopf_i + 1:]:
        if c_vss is None or c_vss >= len(z) or z[c_vss] is None or str(z[c_vss]).strip() == '':
            continue
        vss = str(z[c_vss]).strip()
        key = '%s|%s|%s' % (vss, str(z[c_ob]).strip(), str(z[c_lnr]).strip())
        if quelle == 'OPAG':
            key += '|%s|%s' % (str(z[c_auf]).strip(), str(z[c_arb]).strip())
        saetze.append({'key': key, 'vss': vss, 'postkorb': postkorb_norm(z[c_pk]),
                       'gevo': re.sub(r'\s+', ' ', str(z[c_gevo] or '')).strip(), 'alter': alter_klasse(z[c_alt]),
                       'tage': int(z[c_alt]) if isinstance(z[c_alt], (int, float)) else 0})
    return stichtag, quelle, saetze


def verarbeiten(stichtag, quelle, saetze, snapshot, sparten=SPARTE_STANDARD):
    """snapshot: dict key -> (sparte, vss, postkorb, gevo, gevo_typ, gevo_nr) des Vortagsstands oder None.
    Liefert (fakten_zeilen, neuer_snapshot, kennzahlen)."""
    agg = OrderedDict()
    wechsel = OrderedDict()      # Schluessel in beiden Staenden, anderer Themenpostkorb oder GeVo (Blatt Umbuchungen)

    def zeile(sparte, vss, pk, gevo):
        k = (sparte, vss, pk, gevo)
        if k not in agg:
            typ, nr = gevo_typ(gevo)
            agg[k] = [stichtag, quelle, sparte, vss, pk, gevo, typ, nr, 0, 0, 0, 0, 0, 0, 0, 0, 0]    # [15] hoechstes
    Alter, [16] Summe der Alter (Tage)
        return agg[k]
    neu = {}
    for s in saetze:
        sparte = sparten.get(s['vss'], 'Sonstige')
        z = zeile(sparte, s['vss'], s['postkorb'], s['gevo'])
        z[8] += 1
        z[11 + s['alter']] += 1
        z[15] = max(z[15], s.get('tage', 0))
        z[16] += s.get('tage', 0)
        if snapshot is not None:
            m = snapshot.get(s['key'])
            if m is None:
                z[9] += 1
            elif (m[2], m[3]) != (s['postkorb'], s['gevo']):
                wk = (sparte, m[2], m[3], s['postkorb'], s['gevo'])
                wechsel[wk] = wechsel.get(wk, 0) + 1
        typ, nr = gevo_typ(s['gevo'])
        neu[s['key']] = (sparte, s['vss'], s['postkorb'], s['gevo'], typ, nr)
    if snapshot is not None:
        for key, m in snapshot.items():
            if key not in neu:
                zeile(m[0], m[1], m[2], m[3])[10] += 1
    fakten = list(agg.values())
    kenn = {'zeilen': len(saetze), 'rueckstand': sum(z[8] for z in fakten),
            'eingang': sum(z[9] for z in fakten), 'abgang': sum(z[10] for z in fakten),
            'umbuchungen': [(stichtag, quelle, sp, pv, gevo_typ(gv)[0], gevo_typ(gv)[1], ph, gevo_typ(gh)[0], gevo_typ
    (gh)[1], n)
                            for (sp, pv, gv, ph, gh), n in wechsel.items()]}
    return fakten, neu, kenn


def blaetter_aus_datei(pfad):
    import openpyxl
    wb = openpyxl.load_workbook(pfad, read_only=True, data_only=True)
    aus = []
    for ws in wb.worksheets:
        zeilen = [list(r) for r in ws.iter_rows(values_only=True)]
        erg = blatt_lesen(zeilen)
        if erg:
            aus.append((pfad, ws.title) + erg)
    wb.close()
    return aus


def anwesenheit_lesen(pfad, jahr):
    """Liest die MAK-Liste: Zeile mit 'Mitarbeiter' ist der Kopf; Spalten 'Makler BÜ Gesundheit' / 'Makler BÜ Komposit
    '
    geben die Sparte; Spalten 'NN KW' / 'NN. KW' die Wochenwerte (Zahl, 'Urlaub' oder leer)."""
    import openpyxl
    wb = openpyxl.load_workbook(pfad, read_only=True, data_only=True)
    ws = wb.worksheets[0]
    zeilen = [list(r) for r in ws.iter_rows(values_only=True)]
    kopf_i = next(i for i, z in enumerate(zeilen) if any(c is not None and 'mitarbeiter' in str(c).lower() for c in z)
    )
    kopf = [str(c).replace('\n', ' ').strip() if c is not None else '' for c in zeilen[kopf_i]]
    c_ma = next(i for i, h in enumerate(kopf) if 'mitarbeiter' in h.lower())
    c_einheit = next((i for i, h in enumerate(kopf) if 'einheit' in h.lower()), None)
    c_ges = next((i for i, h in enumerate(kopf) if 'gesundheit' in h.lower()), None)
    c_kom = next((i for i, h in enumerate(kopf) if 'komposit' in h.lower()), None)
    kws = [(i, int(re.match(r'(\d{1,2})', h).group(1))) for i, h in enumerate(kopf) if re.match(r'\d{1,2}\.?\s*KW', h)
    ]
    aus = []
    for z in zeilen[kopf_i + 1:]:
        ma = z[c_ma] if c_ma < len(z) else None
        if ma is None or str(ma).strip() == '' or str(ma).strip().upper() == 'MAK':
```

```
            continue
        einheit = str(z[c_einheit]).strip() if c_einheit is not None and z[c_einheit] is not None else ''
        ges = z[c_ges] if c_ges is not None else None
        sparte = 'Gesundheit' if isinstance(ges, (int, float)) and ges > 0 else 'Komposit'
        for ci, kw in kws:
            w = z[ci] if ci < len(z) else None
            # Wert: Zahl = MAK der Woche, 'U' = Urlaub, 'A' = sonstige Abwesenheit (Text), None = kein Eintrag (zaehlt als Kapazitaet)
            if isinstance(w, (int, float)):
                mak = float(w)
            elif w is None or str(w).strip() == '':
                mak = None
            elif 'urlaub' in str(w).lower():
                mak = 'U'
            else:
                mak = 'A'
            aus.append((einheit, str(ma).strip(), sparte, jahr, kw, mak))
    wb.close()
    return aus


def mitarbeiter_aus_anwesenheit(anwesenheit):
    """Stammdaten je Mitarbeiter aus den Wochenwerten: Kapazitaet = groesster Wochenwert. Liefert
    [(name, einheit, sparte, kapazitaet)] in Reihenfolge des ersten Auftretens und je Mitarbeiter {(jahr, kw): wert}."""
    reihenfolge, daten, wochen = [], {}, {}
    for einheit, name, sparte, jahr, kw, wert in anwesenheit:
        if name not in daten:
            reihenfolge.append(name)
            daten[name] = [name, einheit, sparte, 0.0]
            wochen[name] = {}
        if isinstance(wert, float) and wert > daten[name][3]:
            daten[name][3] = wert
        wochen[name][(jahr, kw)] = wert
    return [tuple(daten[n]) for n in reihenfolge], wochen

```